

Gert v2 Design Document

Gert Team

April 24, 2026

Contents

1	Overview	1
1.1	Problem Statement	1
1.2	Design Rationale	1
1.3	Scope	2
1.4	Design Priorities	2
1.5	Document Roadmap	3
1.6	Success Definition	3
2	Goals and Non-Goals	4
2.1	Goals	4
2.2	Non-Goals	6
3	Architecture	7
3.1	Primary Components	9
3.1.1	Parser	9
3.1.2	Planner	10
3.1.3	Runtime Core	12
3.1.4	Extension Host	14
3.1.5	API / Adapter Layer	15
3.2	Data Flow	17
3.2.1	Phase 1: Parse	17
3.2.2	Phase 2: Plan	17
3.2.3	Phase 3: Run Initialization	17
3.2.4	Phase 4: Step Execution Loop	18
3.2.5	CLI Step Executor Contract	22
3.2.6	Include Step Executor Contract	26
3.2.7	Phase 5: Run Completion	28
3.3	Lifecycle Model	28

3.3.1	Run Start	28
3.3.2	Step Advancement	28
3.3.3	Pause and Resume	28
3.3.4	Wait-for-Event Executor Contract	30
3.3.5	Approve Step Executor Contract	34
3.3.6	Collector Field Validation Contract	39
3.3.7	Cancellation	46
3.3.8	Crash Recovery	47
3.4	Concurrency Model	47
3.4.1	Per-Process Run Isolation	47
3.4.2	Goroutine Model	47
3.4.3	Thread Safety at Component Boundaries	48
3.5	gert serve Integration	48
3.5.1	Architectural Position	48
3.5.2	Event Forwarding	49
3.5.3	Crash Recovery in gert serve	49
3.5.4	Event Dispatcher	49
3.5.5	RPC Method Summary	51
4	Schema vNext	52
4.1	Schema Identity and Versioning	52
4.1.1	apiVersion Values	52
4.1.2	Self-Description via \$schema	53
4.1.3	Versioning Strategy	53
4.2	Core Runbook Fields (v2)	54
4.2.1	Top-Level Field Inventory	54
4.2.2	toolRefs	56
4.2.3	Minimal Valid v2 Runbook	56
4.2.4	Complete Annotated Example	57
4.3	Input and Output Declarations	60
4.3.1	Input Declarations	60
4.3.2	Output Declarations	61
4.3.3	Expression Language	61
4.4	Step Types (v2 Inventory)	63
4.4.1	Common Step Fields	65
4.4.2	Step Type: cli	66

4.4.3	Step Type: choice	70
4.4.4	Step Type: decision	72
4.4.5	Step Type: collector	73
	Field type: text	78
	Field type: number	81
	Field type: integer	82
	Field type: date	82
	Field type: datetime	83
	Field type: boolean	83
	Field type: select	83
	Field type: autocomplete	85
4.4.6	Step Type: approve	91
4.4.7	Step Type: tool	95
4.4.8	Step Type: wait_for_event	97
4.4.9	Step Type: include	100
4.4.10	Step Type: branch	102
4.4.11	Step Type: iterate	103
4.4.12	Step Type: parallel	106
4.4.13	Step Type: assert	108
4.4.14	Step Type: compensate	109
4.4.15	Step Type: end	110
4.5	Tool Definition Schema (v2)	111
4.5.1	Tool Definition Field Inventory	111
4.6	Provider Definition Schema (v2)	112
4.7	Namespace Convention for Extensions	113
4.7.1	The Extension Field Problem	113
4.7.2	Convention: x-<namespace> : at the Field Level	113
4.7.3	Extension Validation	114
4.7.4	Extension Registration Format	115
4.8	Structural vs. Semantic Validation	115
4.8.1	Phase 1: Structural Validation	115
4.8.2	Phase 2: Semantic Validation	116
4.9	Schema Artifacts	118
4.9.1	Shipped Artifacts	118
4.9.2	gert schema export	118
4.9.3	VS Code Integration	118

4.9.4	Schema Version Manifest	119
5	Domain Kit Model	120
5.1	Purpose and Scope	120
5.2	Architectural Narrative	121
5.3	The Clean Kernel Principle	122
5.4	Layer Relationships	123
5.5	Kit Manifest and Compatibility	124
5.6	Extension Runtime Audit	126
5.7	Domain Kit Examples	128
5.8	Non-Goals	129
6	Extension Runtime	131
6.1	Architecture Overview	131
6.2	Capability Taxonomy	132
6.3	Extension Manifest Format	132
6.4	Extension Discovery	133
6.5	Handshake and Lifecycle Protocol	134
6.5.1	extension/initialize (host → extension)	134
6.5.2	contributions/list (host → extension)	136
6.5.3	extension/ping (host → extension)	136
6.5.4	extension/shutdown (host → extension)	137
6.5.5	Lifecycle State Machine	137
6.6	Versioning and Compatibility	137
6.7	Sandboxing and Process Isolation	138
6.7.1	Process Isolation	138
6.7.2	Capability Enforcement	138
6.7.3	Timeout and Crash Handling	139
7	Tool Runtime	141
7.1	Tool Discovery	141
7.1.1	Name Resolution Order	142
7.2	Tool Definition Schema (v2)	142
7.2.1	Transport-Specific Fields	143
7.3	Transport Layer	144
7.3.1	stdio Transport	144

7.3.2	stdio-jsonrpc Transport	145
7.3.3	MCP Transport	146
7.4	Invocation Context	147
7.5	Capability Gate	147
7.6	Output Capture	147
7.7	MCP Tool Integration	148
7.7.1	MCP Server Declaration	148
7.7.2	MCP Discovery	148
7.7.3	MCP Tool Invocation	149
7.7.4	MCP vs stdio-jsonrpc Differences	149
7.8	Cancellation and Timeout	150
7.9	Tool Versioning	151
8	Runtime Events and Determinism	152
8.1	Event System Design	152
8.1.1	Role of Events	152
8.1.2	Delivery Semantics	153
8.2	Event Envelope	153
8.3	Event Catalog	155
8.3.1	Run Lifecycle Events	155
	run/started	155
	run/completed	156
	run/cancelled	156
8.3.2	Step Lifecycle Events	157
	step/started	157
	step/completed	157
	step/failed	158
	step/skipped	158
	step/retrying	159
8.3.3	Governance Events	159
	governance/command_checked	159
	governance/approval_requested	159
	governance/approval_received	160
	governance/redaction_applied	160
8.3.4	Extension and Tool Events	161
	extension/loaded	161

	extension/unloaded	161
	tool/invoked	161
	tool/completed	162
8.3.5	Human-in-the-Loop Events	162
	input/prompted	162
	input/received	162
8.3.6	Saga and Compensation Events (v2.1+)	163
	saga/compensation_triggered	163
	saga/compensation_completed	163
8.4	Event Channels	163
8.4.1	In-Process Event Bus	164
8.4.2	WebSocket Broadcast (HTTP Adapter)	164
8.4.3	JSONL Trace File	165
8.5	Event Subscriptions	165
8.5.1	Programmatic Subscription API	165
8.6	Wire Format	166
8.6.1	Concrete Example: <code>step/started</code>	166
8.6.2	Concrete Example: <code>governance/approval_requested</code>	166
8.7	Required vs. Optional Events	167
8.8	Determinism and Replay Semantics	168
8.8.1	Replay Mode	168
8.8.2	Non-determinism Boundaries	168
9	Security and Trust	169
9.1	Threat Model	169
9.2	Extension Trust Model	171
9.2.1	Trust Levels	171
9.2.2	Extension Signature Format	171
9.2.3	Extension Verification Command	172
9.3	Process Isolation	173
9.3.1	Isolation Mechanisms	173
9.4	Credential and Secret Handling	174
9.4.1	Secret Resolution and Redaction	174
9.4.2	Secret Storage Policy	175
9.5	Transport Security	176
9.5.1	TLS Requirements	176

9.5.2	Client Authentication	176
9.5.3	WebSocket Security	177
9.5.4	Extension JSON-RPC Transport	177
9.6	RBAC for <code>gert serve</code>	178
9.6.1	Actor Identity	178
9.6.2	Roles	178
9.6.3	Role Assignment	179
9.6.4	Approval Authorizer Role	179
9.7	Audit Non-Repudiation	180
9.7.1	Governance Decision Recording	181
9.7.2	Trace File Integrity (HMAC Chaining)	181
9.7.3	SHA256 Evidence Capture	182
9.8	Supply Chain Security	182
9.8.1	Tool Binary Checksums	182
9.8.2	<code>gert verify</code> Command	183
9.8.3	Extension Signature Verification	183
9.9	Security Recommendations	184
10	Testing and Acceptance	185
10.1	Test Framework	185
10.1.1	Standard Library and testify	185
10.1.2	Test Packages	186
10.2	The <code>gert test</code> Feature	186
10.2.1	Scenario Replay Testing	186
10.2.2	Test Assertions (<code>test.yaml</code>)	187
10.2.3	v1 Scenario Compatibility	187
10.3	Contract Tests	188
10.3.1	Core Runtime Contract	188
10.3.2	Schema Contract Tests	188
10.3.3	Extension Manifest and Handshake Tests	189
10.3.4	Tool Invocation Envelope Tests	189
10.3.5	Event Bus Determinism Tests	189
10.4	Integration Test Strategy	190
10.4.1	End-to-End Fixture Runs	190
10.4.2	Fixture Runbook Corpus	190
10.4.3	Trace Assertion Helpers	191

10.5 Regression Testing	191
10.5.1 v1 Runbook Corpus as Fixtures	191
10.5.2 Golden Trace Comparisons	192
10.6 Performance Acceptance Criteria	192
10.7 Extension Test Harness	193
10.7.1 Overview	193
10.7.2 Harness Capabilities	193
10.7.3 Mocking the Extension Transport	194
10.8 Acceptance Criteria	194
10.9 CI/CD Integration	195
10.9.1 PR Gate (every pull request)	195
10.9.2 Integration Gate (merge to main)	195
10.9.3 Nightly Gate	195
11 Open Questions	196
12 Migration and Compatibility	202
12.1 v1 → v2 Compatibility Matrix	202
12.2 Schema Migration: Runbook YAML	204
12.2.1 Breaking Changes Summary	204
12.2.2 Before and After Example	204
12.2.3 Field Mapping Table	206
12.3 Automated Migration Tool	207
12.3.1 Command Specification	207
12.3.2 Flags	207
12.3.3 Migration Algorithm	207
12.3.4 Diff Report	208
12.3.5 Error Handling	209
12.4 Provider and Tool Migration	209
12.4.1 Tool Definitions (<code>.tool.yaml</code>)	209
12.4.2 Provider Definitions (<code>.provider.yaml</code>)	210
12.5 Extension Migration	211
12.5.1 Handshake Protocol Changes	211
12.5.2 Extension Compatibility Shim	212
12.5.3 Extension Author Checklist	212
12.6 Trace Format Migration	212

12.6.1	Breaking Changes in JSONL Trace	212
12.6.2	Trace Replay Compatibility	213
12.6.3	Trace Archive Migration Tool	213
12.7	Rollout Strategy	213
12.7.1	Recommended Adoption Path	213
12.7.2	Parallel Operation	215
12.7.3	Timeline	215
12.8	v1 Compatibility Shim	215
12.8.1	Decision: Provide a v1 Compatibility Shim for 6 Months	215
12.8.2	Shim Behavior	216
12.8.3	Shim Removal	216
12.9	Open Questions and Risks	216
12.9.1	Open Questions	216
12.9.2	Migration Risks	217
12.10	Summary	217
13	Governance and Policy	218
13.1	Governance Primitives	218
13.1.1	Command Allowlist (<code>allowed_commands</code>)	218
13.1.2	Command Denylist (<code>denied_commands</code>)	219
13.1.3	Environment Variable Blocking (<code>deny_env_vars</code>)	219
13.1.4	Output Redaction (<code>redact</code>)	220
13.1.5	Approval Gates	220
13.2	Policy Evaluation Model	221
13.3	Approval Gate Design	221
13.3.1	Schema Declaration	221
13.3.2	UX Contract	222
13.3.3	Approval Decision Protocol	222
13.3.4	Approval Recording in the Trace	223
13.3.5	Timeout and Escalation	224
13.4	Policy-as-Code Direction	225
13.4.1	v2.0: Structured YAML Policy Blocks	225
13.4.2	v2.1+: OPA Integration Hooks	225
13.5	RBAC Model	226
13.5.1	Identity Establishment	226
13.5.2	Runbook-Level Access Control	227

13.5.3	Step-Level Role Restriction	227
13.5.4	Separation of Duties	227
13.6	Redaction	228
13.6.1	Pattern-Based Redaction	228
13.6.2	Explicit Field Marking (v2.1)	228
13.6.3	Redaction Guarantees	228
13.7	Audit Trail Requirements	228
13.7.1	Trace Integrity	229
13.7.2	Compliance Alignment	229
13.8	Extension-Contributed Policy	230
13.8.1	Extension Policy Rule Schema	230
14	Evidence, Tracing, and Resumption	232
14.1	Trace File Format	232
14.1.1	File Location and Naming	232
14.1.2	JSONL Envelope	233
14.2	Implementation Notes for Go	249
15	Adapter Contracts	251
15.1	What is an Adapter Contract?	251
15.2	Contract Categories	252
15.2.1	JSON-RPC Execution Contract (exec/v2)	252
15.2.2	WebSocket Event Stream Contract (events/v2)	254
15.2.3	Tool Invocation Contract (tool/v2)	256
15.2.4	Extension Handshake Contract (extension/v2)	259
15.3	Contract Versioning	261
15.3.1	Version Declaration	261
15.3.2	Breaking vs. Non-Breaking Changes	262
15.3.3	Deprecation Policy	263
15.3.4	Forward and Backward Compatibility	264
15.4	Error Codes	264
15.5	Contract Test Suite	265
15.5.1	Test Suite Structure	265
15.5.2	Running Contract Tests	266
15.5.3	Contract Test CI Integration	266
15.6	Reference Implementations	267

15.7	Contract Stability Guarantees	267
15.8	Contract Documentation and Tooling	268
15.8.1	OpenAPI Specifications	268
15.8.2	JSON Schema Validation	268
15.8.3	Client Library Generation	268
16	Input Provider Framework	270
16.1	Design Goals	270
16.2	Provider Definition Schema	270
16.3	Resolution Protocol	272
16.3.1	Resolution Flow	272
16.3.2	Resolution Request Envelope	273
16.3.3	Resolution Response Envelope	274
16.3.4	Resolution Error Handling	275
16.3.5	Resolution Caching	275
16.4	Built-in Providers	275
16.4.1	env	276
16.4.2	file	276
16.4.3	prompt	276
16.4.4	workspace	277
16.5	Interactive Step Type Contracts	277
16.5.1	Choice Step Contract	277
16.5.2	Decision (Router) Step Contract	279
16.5.3	Collector Step Contract	281
16.5.4	Dynamic Options Protocol	285
16.5.5	Autocomplete Search Protocol	288
16.5.6	Provider Capability Matrix	290
16.6	Provider Composition	291
16.7	Provider Lifecycle	291
17	Observability and Diagnostics	293
17.1	Observability Model	293
17.1.1	Metrics: Aggregated Performance Indicators	293
17.1.2	Distributed Tracing: Execution Flow Visualization	294
17.1.3	Structured Logging: Detailed Contextual Events	295
17.2	OpenTelemetry Integration	295

17.2.1	Span Hierarchy and Attributes	295
	Root Span: <code>gert.run</code>	296
	Child Span: <code>gert.step</code>	296
	Grandchild Span: <code>gert.tool.invoke</code>	297
	Grandchild Span: <code>gert.input.resolve</code>	297
	Grandchild Span: <code>gert.governance.check</code>	297
17.2.2	Span Status and Error Handling	298
17.2.3	Context Propagation	298
	W3C Trace Context Headers	299
	Baggage	299
17.2.4	OTLP Exporter Configuration	299
	OTLP Export (Production)	299
	Console Export (Development)	300
17.2.5	Sampling Strategy	300
17.3	Metrics Catalog	300
17.3.1	Run Metrics	301
17.3.2	Step Metrics	301
17.3.3	Tool Invocation Metrics	301
17.3.4	Approval Metrics	302
17.3.5	Extension Metrics	302
17.3.6	Metrics Endpoint	302
17.3.7	Push-Based Metrics (OTLP)	303
17.4	Structured Logging	303
17.4.1	Log Line Format	303
17.4.2	Required Fields	304
17.4.3	Contextual Fields (When Applicable)	304
17.4.4	Log Levels	304
17.4.5	Sensitive Data Handling	305
17.4.6	Log Output Configuration	305
17.4.7	Log Aggregation and Correlation	306
17.5	Health and Readiness Endpoints	306
17.5.1	Liveness Endpoint	306
17.5.2	Readiness Endpoint	307
17.5.3	Kubernetes Probe Configuration	307
17.6	Diagnostics Commands	308
17.6.1	<code>gert diagnose</code>	308

17.6.2	<code>gert trace show</code>	309
17.6.3	<code>gert trace export</code>	310
17.6.4	<code>gert run list</code>	310
17.7	Integration Recipes	311
17.7.1	Prometheus + Grafana	311
17.7.2	Jaeger (Distributed Tracing)	312
17.7.3	Loki + Grafana (Log Aggregation)	312
17.7.4	Datadog / New Relic (OTLP-Compatible SaaS)	313
17.8	Observability for CI/CD Pipelines	313
17.8.1	JSON Output Mode	314
17.8.2	GitHub Actions Example	314
17.8.3	GitLab CI Example	315
17.9	Summary	315

Chapter 1

Overview

1.1 Problem Statement

Operational procedures are the connective tissue of production engineering. When a service degrades, an incident fires, or a change must be applied under pressure, engineers rely on runbooks—step-by-step guides that encode institutional knowledge about how to respond. The research literature is unambiguous: *unstructured runbooks fail*. Static documents drift from reality, skip governance checks, and produce no audit evidence. Teams that operate at scale cannot afford a response that depends entirely on whether the on-call engineer remembers to follow the wiki [5, 17].

The core operational problem is threefold. First, runbooks must be **executable**: a document that cannot be run is a suggestion, not a procedure. Second, runbooks must be **governed**: every action that touches infrastructure must be bounded by policy—command allowlists, environment variable controls, output redaction, and role-based approval gates. Third, runbooks must be **traceable**: every execution must produce an immutable, auditable record that satisfies SOC 2, ISO 27001, and HIPAA audit requirements [4, 7, 28] without manual effort. Without all three properties, organizations face audit failures, runbook drift, and uncontrolled blast radius during incidents.

Gert is designed to solve exactly this problem: a governed, executable, debuggable runbook engine that produces full traceability and evidence by construction.

1.2 Design Rationale

Gert is designed around three hard constraints: governance must be built in (not added on), traceability must be complete (every action leaves an immutable audit trail), and the component model must be stable (engine, schema, executors, and extensions evolve

independently under a documented contract). These constraints rule out ad-hoc scripting, general-purpose workflow engines, and cloud-execution services as the right foundation. Gert occupies a specific niche: a local-first, governed runbook execution engine with a stable component boundary and complete observability.

1.3 Scope

Gert is: a local-first, governed runbook execution engine with a stable component architecture, versioned extension contracts, a first-class governance and policy layer, full traceability by construction, and defined adapter interfaces for TUI, VS Code, and web clients. It ships as a single Go binary with an optional out-of-process extension runtime.

Gert is not: a managed cloud execution service, a general-purpose workflow orchestrator, a CI/CD pipeline system, a SOAR platform, or a replacement for systems like Temporal, Argo, or PagerDuty Runbook Automation [12, 23, 27]. It does not provide a central policy server, a shared execution registry, or multi-tenant execution isolation.

1.4 Design Priorities

- **A core runtime isolated from presentation adapters.** The execution engine—parser, planner, state machine, governance enforcer, trace writer—must have no dependency on any presentation layer. TUI, VS Code, and web clients attach via a stable event subscription API; they observe execution but never own it. This isolation is the prerequisite for every other architectural goal.
- **Explicit and versioned contracts for schema, tools, and events.** Every interface that crosses a component boundary must be named, versioned, and documented. The runbook schema carries an `apiVersion` field with formal compatibility guarantees. Tool definitions carry a version. The runtime event stream has a defined schema with ordered delivery guarantees. Breaking a contract requires a version increment, not a silent edit.
- **A secure extension runtime using out-of-process plugins.** Extensions run in isolated processes and are granted capabilities explicitly. No extension receives ambient authority. The capability model—what an extension may register, observe, or invoke—is a named, versioned list that the host enforces at admission time. This pattern follows the principle of least privilege endorsed by NIST SP 800-53 [1] and

applied in production by systems such as Kubernetes admission controllers [16] and OPA Gatekeeper [22].

1.5 Document Roadmap

This document is organized as follows. §2 defines measurable goals and explicit non-goals, establishing scope boundaries for implementors. §3 describes the five primary components and the dependency rules between them. §4 specifies the runbook schema, including versioning strategy and field inventory. §6 defines the extension runtime: capability taxonomy, handshake protocol, and trust model. §7 specifies the tool runtime: discovery, invocation, transport contracts, and governance integration. §8 defines the runtime event model: event schema, ordering guarantees, delivery semantics, and replay contract. §9 specifies the security and trust model: threat model, policy enforcement, supply chain controls, and audit requirements. §10 defines the testing and acceptance strategy: contract test categories, coverage requirements, and acceptance criteria. §11 documents open architectural questions with options and resolution criteria.

1.6 Success Definition

Gert is complete when: (1) a runbook executes end-to-end with governance checks, input resolution, tool invocation, and trace emission all functioning correctly; (2) the TUI, VS Code extension, and a third-party adapter can all be built against the published event and API contracts without importing internal packages; (3) the governance layer—allowlists, denylists, env blocking, redaction, approval gates—is enforced for every execution mode including extension-invoked runs; (4) every execution produces an append-only, crash-safe JSONL trace that is structurally valid against the v2 trace schema; and (5) `gert test` passes a scenario suite covering all step types, governance rules, and evidence capture cases.

Chapter 2

Goals and Non-Goals

2.1 Goals

The following goals are the binding commitments of the v2 design. Where possible, each goal is stated as a verifiable criterion rather than an aspiration.

- G1. Preserve and formalize the governance layer.** Every capability that made v1 operationally useful—command allowlists and denylists, environment variable blocking, output redaction with regex patterns, and per-step role-based approval gates—must be present in v2 with identical or stronger semantics. Governance is gert’s core differentiator [8, 23]: approval gates integrated directly into step definitions. v2 must additionally support RBAC scoping on runbook execution: which identities may run which runbooks, and under what conditions.
- G2. Preserve the append-only, crash-safe, tamper-evident trace.** The v2 execution trace must be an append-only JSONL file. Each line is a self-describing JSON object with a unique event ID (UUID), ISO 8601 UTC timestamp [19], actor identity, step ID, and structured payload. The file must be crash-safe (partial writes must not corrupt prior entries) and must support optional cryptographic signing to satisfy SOC 2 and ISO 27001 (A.12.4) audit requirements [4, 7]. This is a launch blocker: no trace, no compliance.
- G3. Preserve evidence capture, deterministic replay, TUI, VS Code extension, and gert test.** The following v1 capabilities must be functionally preserved in v2: evidence capture with text, checklists, and SHA256-hashed attachments; scenario-based deterministic replay (`-mode replay`); the Bubble Tea terminal UI [14]; the VS Code extension powered by the JSON-RPC server [18, 20]; and the `gert test` scenario

runner with its assertion engine. These features represent validated operational value. Their absence in v2 is a regression, not a simplification.

- G4. Establish stable, versioned extension contracts.** Tool definitions (`.tool.yaml`), input providers (`.provider.yaml`), and out-of-process extensions must bind to named, versioned contracts. A host version bump that breaks an extension contract requires a major version increment and a migration path. The capability taxonomy—what an extension may register, observe, and invoke—must be a published, enumerated list, not an implicit set of permissions inferred from code. This pattern is established by VS Code’s extension API [20] and Kubernetes admission controller design [16].
- G5. Introduce saga and compensation for multi-step rollback.** v2 must support explicit compensation registration: when a forward step completes, the runbook may declare a compensating action to execute in reverse order on failure. This is the industry-standard saga pattern, implemented in production by Temporal [27] and AWS Step Functions [9]. Without compensation, partially-executed runbooks that modify infrastructure leave systems in undefined intermediate states.
- G6. Integrate OpenTelemetry for distributed tracing.** Every runbook execution must emit OpenTelemetry spans [6]: one root span per run, one child span per step, with W3C Trace Context [2] (`traceparent` / `tracestate`) propagated to tool invocations. Correlation IDs must appear in both the JSONL trace and the OTel span attributes, so that run traces can be joined with infrastructure observability data in Jaeger [15], Zipkin [30], or any OTLP-compatible backend. This is the current industry standard for operational visibility.
- G7. Decouple adapters from core via a typed event API.** The TUI, VS Code extension, and any future web adapter must attach to the core exclusively via a typed event subscription interface. No adapter may import internal runtime packages or call runtime methods directly. The event schema must be versioned: adapters declare which schema version they consume, and the host rejects incompatible connections. This constraint is necessary for gert to support concurrent adapters (e.g., TUI and VS Code running simultaneously against a single run).
- G8. Establish migration compatibility for v1 runbooks.** The command `gert migrate` (v2 edition) must produce a valid `runbook/v2` document from any well-formed `runbook/v0` or `runbook/v1` input with no manual fixup required for standard runbooks. The v2 runtime should additionally support a compatibility execution

mode that accepts v1 runbooks without prior migration, so that existing users are not blocked from adopting v2 incrementally.

2.2 Non-Goals

The following are explicitly outside the scope of gert v2. These boundaries are feature-scoped, not process-scoped, and are intended to prevent scope creep during implementation.

- NG1. Gert v2 does not provide a managed cloud execution service.** There is no gert-as-a-service, no central execution registry, and no SaaS offering. Gert v2 is a local-first binary. Cloud scheduling, multi-tenant isolation, and remote execution orchestration are concerns for a hypothetical v3 or a separate product.
- NG2. Gert v2 does not implement a general-purpose workflow orchestrator.** Gert is not Temporal, Argo, or Prefect [12, 25, 27]. It does not support distributed workers, fan-out parallelism across remote nodes, or arbitrary DAG execution. Parallel execution within a single local run is a stretch goal; cross-host parallelism is explicitly out of scope.
- NG3. Gert v2 does not ship a built-in policy server.** While v2 defines governance semantics and may integrate policy-as-code hooks [21], it does not bundle or operate an OPA server. Policy evaluation happens at the host process level. Central policy distribution, policy bundles over HTTP, and multi-host policy synchronization are out of scope.
- NG4. Gert v2 does not define an extension marketplace or signing authority.** v2 will define an extension signing *mechanism* (a verifiable public key model), but it will not operate a registry, curate extensions, or act as a certificate authority. Extension distribution is the responsibility of the extension author.
- NG5. Gert v2 does not guarantee binary backward compatibility with v1 internal packages.** v2 is a clean-break architecture. The internal Go package structure, private interfaces, and runtime internals are not subject to backward compatibility guarantees. Only the documented public contracts—the runbook schema, the tool schema, the JSON-RPC API surface, the extension protocol, and the JSONL trace schema—carry compatibility commitments.

Chapter 3

Architecture

This chapter defines the v2 system architecture: the five primary components, their interfaces and invariants, the complete data flow from runbook invocation to trace archive, the execution lifecycle, the concurrency model, and how the `gert serve` JSON-RPC layer integrates as a first-class adapter.

The single most important architectural rule is: **dependencies flow inward toward the Core Domain**. The Runtime Core knows nothing about TUI, VS Code, or JSON-RPC. Adapters know nothing about execution semantics. Every component boundary is enforced by a Go interface; no component imports a package that is “above” it in the dependency graph.

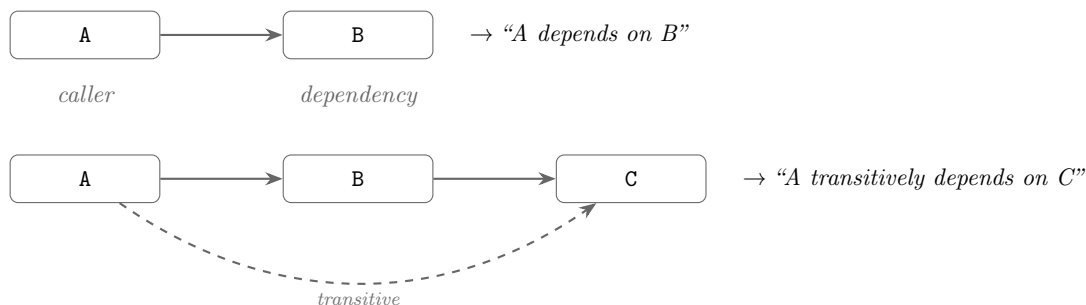


FIGURE 3.2: Dependency direction convention: an arrow from A to B means *A depends on B* (A imports/uses B). Arrows flow *toward* the depended-upon component.

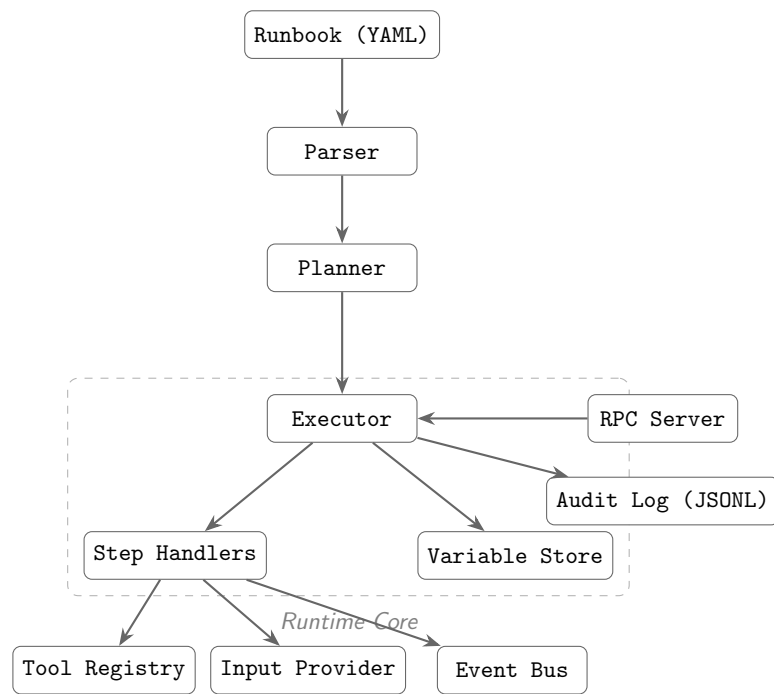


FIGURE 3.1: gert v2 architecture — component dependency overview.

3.1 Primary Components

3.1.1 Parser

Responsibility. The Parser owns the transformation of a raw YAML document into a validated, semantically-resolved `ParsedRunbook` value. It does *not* own execution, planning, tool discovery, or input resolution. It validates schema structure and surface-level semantic rules (duplicate step IDs, unknown step types, malformed Go template expressions) but intentionally defers deep semantic validation (e.g., whether a `from:` binding can actually be resolved) to later pipeline stages.

Interface.

```
// Parser transforms raw YAML bytes into a validated ParsedRunbook.
type Parser interface {
    // Parse reads YAML from r, validates against the v2 JSON Schema,
    // applies structural semantic checks, and returns a ParsedRunbook.
    // Errors carry structured diagnostics (file, line, column, message).
    Parse(ctx context.Context, r io.Reader, opts ParseOptions) (*ParsedRunbook,
        ↪ error)
}

type ParseOptions struct {
    // SchemaVersion overrides the apiVersion field for compatibility shims.
    SchemaVersion string
    // StrictMode causes deprecation warnings to be surfaced as errors.
    StrictMode bool
}

// ParsedRunbook is an immutable, fully-validated representation of a runbook.
// All fields have been structurally validated. Templates are not yet evaluated.
type ParsedRunbook struct {
    Meta          RunbookMeta
    Governance GovernancePolicy // nil if no governance block
    Steps         []ParsedStep
    Imports       map[string]string // alias -> path (not yet resolved)
    Tools         []string          // tool names (not yet discovered)
    Schema        SchemaRef          // version and $schema URL
}
```

```
| }
```

Dependencies. The Parser depends only on the JSON Schema artifact (a static embedded file) and the expression evaluator (for template syntax validation). It has no runtime dependencies.

Key Invariants.

- A `ParsedRunbook` returned without error is structurally valid against the v2 JSON Schema.
- All step IDs are unique within a `ParsedRunbook`.
- No I/O occurs during parsing (file imports are recorded as paths, not resolved).
- Parsing is pure and deterministic: the same input always produces the same output.

3.1.2 Planner

Responsibility. The Planner transforms a `ParsedRunbook` into an `ExecutionPlan`: a resolved, ordered, dependency-annotated representation of every step the runtime will execute. The Planner resolves imports (recursively parsing and inlining included runbooks), discovers and validates tool definitions, resolves input provider bindings to their declared sources, and produces the static step ordering. It does *not* evaluate dynamic conditions (branch predicates, iterate count expressions) — those are runtime concerns.

What is an `ExecutionPlan`? An `ExecutionPlan` is a *flat, ordered list* of resolved steps with pre-computed metadata, not a DAG. Branch logic and iteration are left as runtime decisions because they depend on captured variables evaluated during execution. The plan encodes *which steps exist* and *what preconditions they carry*, not which steps will actually run. This design keeps the planner pure and the runtime authoritative over control flow.

```
type Planner interface {
    // Plan resolves imports, discovers tools, validates bindings,
    // and returns a fully-resolved ExecutionPlan ready for the runtime.
    Plan(ctx context.Context, rb *ParsedRunbook, opts PlanOptions)
    ↪ (*ExecutionPlan, error)
}
```



```

type PlanOptions struct {
    BaseDir      string           // working directory for import resolution
    ToolDirs     []string          // directories to search for .tool.yaml files
    ProviderDir  string           // directory for .provider.yaml files
    Vars         map[string]string // CLI-supplied variable overrides
}

// ExecutionPlan is an immutable, fully-resolved plan for a single run.
type ExecutionPlan struct {
    RunID        string
    RunbookPath  string
    Steps        []ResolvedStep // ordered; includes inlined include steps
    Tools        map[string]*ToolDef // name -> resolved tool definition
    Providers    map[string]*ProviderDef // binding name -> provider
    Governance   *GovernancePolicy
    Metadata     PlanMetadata
}

// ResolvedStep is a single step in the plan, fully resolved.
type ResolvedStep struct {
    ID        string
    Kind      StepKind // cli, choice, decision, collector, tool, include,
    ↪ branch, iterate
    Spec      StepSpec // kind-specific parameters (resolved, not templated)
    Contract  *contract.Contract //
    ↪ effects/reads/writes/idempotent/deterministic
    Depth     int // chain depth (0 = root runbook)
    Origin    string // source runbook path (for inlined include steps)
}

```

Dependencies. The Planner depends on the Parser (for import resolution), the Tool Registry (for `.tool.yaml` discovery), and the Provider Registry (for `.provider.yaml` discovery). It must not depend on the Runtime Core, Extension Host, or any adapter.

Key Invariants.

- An `ExecutionPlan` returned without error contains only resolvable steps.

- All tool names referenced by steps appear in `ExecutionPlan.Tools`.
- Import graphs are acyclic; the Planner detects and rejects cycles.
- All `from:` bindings that declare a provider appear in `ExecutionPlan.Providers`.
- The Planner does not execute or side-effect; it is safe to plan without running.

3.1.3 Runtime Core

Responsibility. The Runtime Core owns the execution loop: advancing through an `ExecutionPlan` step by step, evaluating templates and branch predicates, dispatching tool invocations, collecting evidence, writing the append-only trace, enforcing governance pre-flight checks, emitting structured events, and managing the mutable run state. It does *not* own user presentation, serialization formats, or network transport.

Interface.

```

type Runtime interface {
    // Start initializes a run from a plan and returns a RunHandle.
    // The run does not advance until Next is called.
    Start(ctx context.Context, plan *ExecutionPlan, opts RunOptions) (RunHandle,
        ↪ error)

    // Resume restores a previously checkpointed run from the run store.
    Resume(ctx context.Context, runID string, opts RunOptions) (RunHandle,
        ↪ error)
}

type RunHandle interface {
    // Next advances the run by one step. Blocks until the step completes
    // (or is paused waiting for evidence/approval).
    // Returns io.EOF when the run has no more steps.
    Next(ctx context.Context) (*StepResult, error)

    // Approve records approval for the current step's approval gate.
    Approve(ctx context.Context, decision ApprovalDecision) error
}

```

```

// SubmitEvidence records evidence for interactive steps (choice, decision,
↪ collector).
// For choice: stores selected option. For decision: stores route selection.
// For collector: stores multi-field form data and artifact attachments.
SubmitEvidence(ctx context.Context, stepID string, ev
↪ map[string]*EvidenceValue) error

// Cancel requests a graceful stop of the run.
Cancel(ctx context.Context, reason string) error

// State returns the current mutable run state (read-only snapshot).
State() RunState

// Events returns a channel of structured events emitted during execution.
Events() <-chan Event
}

type RunOptions struct {
    Mode      RunMode      // real, dry-run, replay
    Actor     string       // --as identity
    Vars      map[string]string // variable overrides
    ScenarioDir string      // replay scenario directory
    Store     RunStore     // durable store for checkpointing
    OnEvent   func(Event)  // synchronous event hook (nil = use Events
↪ channel)
}

```

Dependencies. The Runtime Core depends on the Governance Engine, Tool Runtime (for tool invocation), Expression Evaluator (for templates and branch predicates), Evidence store, and Trace Writer. It does not depend on the Extension Host, API/Adapter Layer, or any adapter.

Key Invariants.

- The trace is append-only: no event is ever deleted or modified after being written.
- Governance pre-flight is enforced before every `cli` and `tool` step dispatch.

- Redaction is applied to all captured output before it is written to the trace.
- The run state is checkpointed after every step completion.
- Events are emitted in a total order that matches step execution order.
- The runtime is re-entrant per run: multiple callers may call `Next` sequentially (but not concurrently) on the same `RunHandle`.

3.1.4 Extension Host

Responsibility. The Extension Host manages the lifecycle of out-of-process extensions: discovery, launch, capability negotiation, RPC dispatch, sandboxing, and graceful shutdown. It exposes extension-contributed tools and providers to the Tool Runtime and Provider Registry. It does *not* execute runbook steps directly and does not own the extension communication protocol (that belongs to the extension RPC layer, defined in §04).

Interface.

```
type ExtensionHost interface {
    // Load discovers and launches all extensions declared in the project
    ↪ manifest.
    Load(ctx context.Context, manifest *ProjectManifest) error

    // Shutdown gracefully stops all running extensions.
    Shutdown(ctx context.Context) error

    // ContributedTools returns all tools contributed by loaded extensions.
    ContributedTools() []*ToolDef

    // ContributedProviders returns all providers contributed by loaded
    ↪ extensions.
    ContributedProviders() []*ProviderDef

    // ContributedPolicyRules returns governance rules contributed by
    ↪ extensions.
    // These compose with host policy; host policy takes precedence on conflict.
    ContributedPolicyRules() []PolicyRule
}
```

Dependencies. The Extension Host depends on the extension manifest schema and the extension RPC transport (stdio JSON-RPC, as specified in §04). It does not depend on the Runtime Core or any adapter.

Key Invariants.

- No extension operates without an explicit capability grant from the host.
- Extension crashes do not crash the host process.
- The Extension Host enforces timeout and cancellation on all RPC calls to extensions.
- Extension-contributed policy rules are additive; they can only *restrict*, not *relax*, host-defined policy.

3.1.5 API / Adapter Layer

Responsibility. The API/Adapter Layer is the boundary between the Runtime Core and all external consumers (TUI, VS Code extension, web clients, CI pipelines). It translates the runtime’s event stream and `RunHandle` contract into transport-specific protocols. Each adapter is a thin, stateless renderer: it subscribes to the event stream, presents state to the user, and forwards user input (approvals, evidence, cancellation) back to the `RunHandle`. Adapters own *no* execution logic.

Interface.

```
// Adapter is the interface every consumer of the Runtime Core must implement.
type Adapter interface {
    // Attach binds the adapter to a RunHandle before the first Next call.
    Attach(handle RunHandle) error

    // Run blocks until the run completes or is cancelled. It is responsible
    // for draining Events(), rendering output, and forwarding user input.
    Run(ctx context.Context) error
}

// The gert serve JSON-RPC layer implements Adapter over stdio.
// The TUI (Bubble Tea) implements Adapter over a terminal.
// Future: a web adapter implements Adapter over WebSocket.
```

Dependencies. Adapters depend on the `RunHandle` and `Event` types from the Runtime Core. They must not import any package from the Parser, Planner, or Extension Host.

Key Invariants.

- An adapter may not call `Next` more than once concurrently.
- An adapter may not modify run state directly; all state changes go through `RunHandle`.
- Adapters must handle the case where the event channel is closed (run ended) without blocking.
- An adapter failure must not corrupt the run trace or run state.

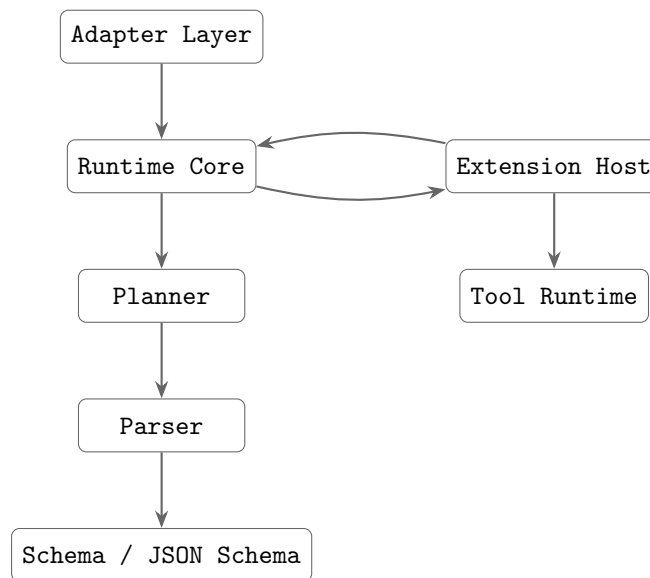


FIGURE 3.3: Component dependency direction. Arrows point from dependent to dependency. No upward arrows exist.

Dependency Direction Summary. No upward arrow exists. The Parser knows nothing about the Runtime. The Runtime knows nothing about adapters. The Extension Host is a peer of the Runtime Core, contributing tools and policy rules but not owning execution.

3.2 Data Flow

This section traces the complete path from runbook invocation to trace archive, identifying exactly where each responsibility is exercised.

3.2.1 Phase 1: Parse

The user invokes `gert exec runbook.yaml -as eng@corp.com`. The CLI layer reads the YAML file and calls `Parser.Parse()`. The Parser validates the document against the embedded v2 JSON Schema, checks structural semantic rules, and returns a `ParsedRunbook`. If validation fails, structured diagnostics are returned and execution does not proceed.

3.2.2 Phase 2: Plan

The `ParsedRunbook` is passed to `Planner.Plan()`. The Planner:

1. Resolves all `imports`: recursively (parse $\rightarrow$ plan each imported runbook, inline its steps with incremented `Depth`).
2. Discovers `.tool.yaml` files for each named tool.
3. Resolves `from`: input bindings to their declared providers.
4. Validates that all step references (branch targets, iterate bodies) are valid step IDs.
5. Constructs and returns an `ExecutionPlan`.

Governance policy from the runbook's `meta.governance` block is extracted and embedded in the `ExecutionPlan` for runtime enforcement.

3.2.3 Phase 3: Run Initialization

`Runtime.Start(plan, opts)` is called. The runtime:

1. Generates a `RunID` (format: `YYYYMMDDTHHmss-xxxxxxx`).
2. Creates the run directory: `.runbook/runs/<runID>/`.
3. Opens the append-only trace file: `.runbook/runs/<runID>/trace.jsonl`.
4. Writes the `run/started` trace event (runbook path, actor, mode, plan hash).

5. Initializes the `GovernanceEngine` from the plan's governance policy.
6. Compiles redaction rules from the governance policy.
7. Returns a `RunHandle`; the run is now paused at step 0.

3.2.4 Phase 4: Step Execution Loop

The adapter calls `RunHandle.Next(ctx)` in a loop until `io.EOF`. For each step, the runtime executes the following sequence:

Step Pre-flight (all step types)

1. Write `step/started` trace event.
2. Emit `StepStarted` runtime event to the event channel.
3. Evaluate template expressions in the step spec against current run variables.

Governance Pre-flight (cli and tool steps)

1. **Command check:** `GovernanceEngine.CheckCommand(argv[0])` evaluates the denylist (deny takes precedence) and then the allowlist. A violation writes a `governance/commandBlocked` trace event and returns an error; the step fails.
2. **Env var check:** `GovernanceEngine.FilterEnvVars()` removes variables matching `deny_env_vars` patterns. Blocked variable names are written to the `governance/envVarBlocked` trace event.
3. **Contract check:** The step's `Contract.Risk()` is evaluated. Steps with `RiskCritical` that do not have an explicit approval gate declaration in the plan emit a `governance/contractWarning` event (not a hard failure in v2.0; upgraded to error in strict mode).
4. **Approval gate:** If the step declares `approvals.min > 0`, the runtime pauses and emits an `ApprovalRequired` event. Execution blocks at this step until `RunHandle.Approve()` is called with sufficient approvals. See §13.3 (Governance chapter) for the full approval protocol.

Dispatch (step-type-specific)

- **cli:** Fork a subprocess. Capture stdout/stderr. Apply redaction rules to captured output before storing in run variables or writing to trace.
- **tool:** Call `ToolRuntime.Invoke(toolName, input)`. The tool runtime dispatches to the appropriate transport (stdio, JSON-RPC, MCP). Redaction applied to returned output.
- **choice:** Emit `ChoiceRequired` event with option list. Block until `SubmitEvidence()` is called with selected option. Store result as named variable. No artifacts involved.
- **decision:** Emit `DecisionRequired` event with route map. Block until `SubmitEvidence()` is called with route selection. Alter execution flow by jumping to the selected runbook or label. Optionally store selection in variable for audit trail.
- **collector:** Emit `CollectorRequired` event with field schema. Block until `SubmitEvidence()` is called with multi-field form data. Validate each submitted value against its declared field type and constraints (see §4.4.5). SHA256-hash each `file` or `image` attachment. Store typed values as run variables: `number` and `integer` as JSON numbers, `boolean` as JSON boolean, multi-select `select` and `choice` as JSON arrays of strings, single-select `select` as a JSON string, `date` and `datetime` as ISO 8601 strings, `text` and `url` as JSON strings. Store `file` and `image` artifacts in the evidence store (not in run variables).
- **include:** Resolve the referenced runbook. Validate the include graph for cycles (load-time DFS — fail fast if cyclic). Evaluate `include.when` if present; skip all steps if false. Apply `include.with` overrides to the current variable scope. Inline-expand the referenced runbook's steps into the current execution queue at `Depth+1`. The include step itself does not appear in the run trace; expanded steps run as if defined inline, each carrying an `Origin` breadcrumb: `[included from: <path>]`.
- **branch:** Evaluate branch predicates (expression evaluator) against current variables. Select the matching branch arm. Write `event/branchResolved` trace event. Advance step cursor to the first step of the resolved arm.
- **iterate:** Evaluate the iteration expression. Write `event/iteratePassStart`. Execute body steps. Write `event/iteratePassEnd`. Repeat until condition is false.

- **wait_for_event:** Serialise run state to the run store. Register an event listener with the Event Dispatcher (source, event ID, filter predicate, and timeout). Set run state to `WAITING` and return to caller — execution is suspended. When the matching event arrives, the Event Dispatcher rehydrates the run, validates the payload against `event.payload_schema`, applies `event.filter` conditions, writes captured variables via `capture` mappings, and resumes at the next step. If `timeout` elapses before an event arrives the `on_timeout` policy governs the outcome (see §3.3.4). This dispatch path is only available in `gert serve` mode; `gert run` MUST reject the step with error: `"step type 'wait_for_event' requires gert serve mode"`.

Step Type Classification

All step types fall into one of three categories based on runtime behavior:

Category	Step Types	Runtime Behavior
Execution	cli, tool	Execute external commands or tools. Require governance pre-flight (allowlist, denylist, redaction). Capture output and exit codes.
Interactive	choice, decision, collector	Block execution, emit event requiring user input. choice stores selected option as variable. decision alters control flow by routing to a runbook or label. collector captures multi-field form data and artifacts (files, images). All three write evidence to trace.
Control Flow	branch, iterate, include	Alter execution path or loop body. branch evaluates predicates at runtime. iterate repeats a body until a condition is false. include resolves and inline-expands a referenced runbook's steps into the current execution queue; the include step itself is not emitted to the trace.
Synchronisation	wait_for_event	Suspend the run until an external event arrives via webhook, message queue, OS signal, or named in-process channel. Persists full run state to durable storage; resumes when the Event Dispatcher delivers a matching, validated event. Valid only in gert serve mode — rejected at runtime in gert run (local CLI) mode.
Approval Gate	approve	Pause execution pending sign-off from one or more authorised identities. Supports all/any/quorum modes with an explicit approver pool. Optionally enforces a business-day timeout (GAP-1) and M-of-N quorum tracking (GAP-2). See §3.3.5.

3.2.5 CLI Step Executor Contract

A **type**: `cli` step executes a shell command or script on the host. Two execution forms are supported:

- **Exec form** (`command/args`) — bypasses the shell entirely. The runtime forks the binary directly, which is both faster and injection-safe.
- **Shell-string form** (`run: "<string>"`) — passes a script to a shell interpreter. Supports multi-line scripts and shell built-ins at the cost of increased injection risk.
- **Multi-shell map form** (`run: {bash: "...", cmd: "...", ...}`) — provides per-shell scripts; the executor selects the first available shell at run time. Designed for cross-platform runbooks.

The fields `command` and `run` are mutually exclusive. If both are present the step fails validation with an error (see validation table below).

Shell Resolution — String Form

When `run` is a string the executor resolves the shell before forking.

Automatic detection (`shell: auto, the default`). The executor inspects the OS at plan-time and selects the platform-native shell:

- **Linux / macOS**: `/bin/sh` — POSIX-guaranteed to exist.
- **Windows**: `pwsh` if `exec.LookPath("pwsh")` succeeds; otherwise `cmd.exe`.

Explicit shell (`shell: bash | sh | pwsh | cmd`). The executor calls `exec.LookPath(shell)` at *plan time*. If the binary is not found the plan fails immediately with:

Shell 'bash' not found on this host (exec.LookPath returned empty)

This is a plan-time error, not a run-time error — the runbook is rejected before execution begins.

Invocation patterns. Once the shell is resolved the script is passed using the shell's standard non-interactive flag:

Shell	Invocation
bash	["bash", "-c", script]
sh	["sh", "-c", script]
pwsh	["pwsh", "-NonInteractive", "-Command", script]
cmd	["cmd.exe", "/c", script]

Template expansion. Template expansion (Sprig/Go template) is applied to the `run` string *before* the result is handed to the shell. The shell receives the already-expanded string. Shell interpretation therefore operates on the final expanded text, not on the template source.

Security note — shell injection. Shell-string form is **not** immune to shell injection. If a template variable contains shell metacharacters (`$`, `'`, `;`, `|`, `&`, parentheses, etc.) they will be interpreted by the shell after template expansion. *Authors are responsible for sanitizing template values used in shell-string steps.* For untrusted input, prefer the `exec` form (`command/args`), which bypasses the shell entirely.

Shell Resolution — Multi-Shell Map Form

When `run` is a YAML mapping (e.g. `run: {bash: "...", pwsh: "..."}`) the executor performs a two-phase resolution.

Phase 1 — Plan-time key validation. The executor reads all map keys and verifies each against the set of recognized shell names: `bash`, `sh`, `pwsh`, `cmd`. Any unrecognized key is a load-time validation error:

```
Unknown shell 'zsh' in run map. Valid values: bash, sh, pwsh, cmd
```

The plan is rejected before execution begins.

Phase 2 — Run-time shell selection. At the moment the step is dispatched the executor walks the map in *insertion order* and calls `exec.LookPath(key)` for each entry:

1. The **first key whose shell resolves successfully** is selected.

2. **Platform-family fallback:** if the `bash` key is present but `bash` is not found, the executor tries `sh` as a fallback before moving on to the next map key.
3. If **no key resolves** the step fails with:

```
CLI step '<id>': no shell from map {bash, pwsh} available on this host.  
Install one of: bash, pwsh.
```

The `shell` field is ignored when `run` is a map. Setting it produces a warning (see validation table).

Audit record. The JSONL trace records which map entry was selected. A `shell_selected` field is written alongside the step's `step/started` event:

```
{ "kind": "step/started", "...", "payload": {  
  "step_id": "deploy-linux",  
  "shell_selected": "bash",  
  ...  
}
```

This field appears in the `step/started` payload whenever a shell-string or map-form CLI step begins. For exec-form steps the field is omitted.

Template expansion. The same expansion-before-invocation rule applies: template expansion is applied to the selected map value before it is passed to the shell.

Platform Variable Injection

The runtime injects a `gert.platform` namespace into the variable scope at run start. These variables are available to all template expressions throughout the run, including `branch` predicates.

Variable	Type	Values
<code>gert.platform.os</code>	string	"linux" "darwin" "windows"
<code>gert.platform.arch</code>	string	"amd64" "arm64" "arm"
<code>gert.platform.shell</code>	string	Shell name selected for the current step (e.g. "bash"). Set after shell resolution; available in <code>capture</code> output and downstream steps. Not usable inside the <code>run</code> script of the same step — resolution happens before template expansion of that field is complete.

`gert.platform.os` is particularly useful for complementing the map form with `branch` steps in cases requiring more complex platform-conditional logic than the map form alone can express.

Validation Rules

Rule	Severity	Message
<code>command</code> and <code>run</code> both present	Error	Fields <code>`command`</code> and <code>`run`</code> are mutually exclusive on type: <code>cli</code>
<code>run</code> is a map and <code>shell</code> is set	Warning	<code>`shell`</code> is ignored when <code>`run`</code> is a map - shell is determined by map key resolution
<code>run</code> map has unknown key	Error	Unknown shell <code>'{key}'</code> in run map. Valid values: <code>bash</code> , <code>sh</code> , <code>pwsh</code> , <code>cmd</code>
<code>shell</code> is explicit but not found on host	Plan-time Error	Shell <code>'{shell}'</code> not found on this host (<code>exec.LookPath</code> returned empty)
<code>run</code> map, no key resolves at run time	Run-time Error	No shell from map <code>{keys}</code> available on this host

3.2.6 Include Step Executor Contract

The **include** step is the v2.0 inline composition primitive. It expands a referenced runbook's steps directly into the caller's execution queue — no new run context, no isolated variable scope, no separate audit entry.

Executor Algorithm

When the executor reaches a **type: include** step it performs the following in order:

1. **Resolve** the referenced runbook by relative path or registry ID.
2. **Cycle check** — validate the include graph (see below). Fail with a hard error if any cycle is detected. This check must have already passed at load time; the executor re-asserts it as a defence-in-depth guard.
3. **Evaluate include.when** — if the condition is present and evaluates to **false**, skip: emit no steps and advance past the include step.
4. **Apply include.with overrides** to the current variable scope. These bindings are visible to all steps that follow (included or caller-defined).
5. **Inline expand** — replace the include step in the execution queue with the referenced runbook's steps, each at **Depth+1** with their **Origin** field set to the referenced runbook path.
6. **Continue execution** — the expanded steps execute as if they had been defined inline in the caller runbook.

Key Architectural Properties

- **No new run context.** The included steps execute within the caller's run; **RunID**, variable scope, and audit log are all shared.
- **No separate audit entry for the include step itself.** Expanded steps appear in the caller's audit log each carrying an **Origin** breadcrumb of the form: **[included from: ./shared/db-preflight.runbook.yaml]**.
- **Shared variable scope.** Variable mutations made by included steps are visible to all subsequent steps in the caller, and vice versa.

- **Include step invisible in trace.** Only the expanded steps appear in the run trace. The include step itself is not emitted as a trace event.

Cycle Detection

Load-time validation. When a runbook is loaded, the runtime builds a directed include graph over the full transitive closure of all `type: include` references. A depth-first search (DFS) with a visited set detects back-edges. A cycle found at any depth is a hard error:

```
cyclic include: A.yaml -> B.yaml -> A.yaml
```

Cyclic include graphs are rejected before execution begins. This check covers both direct and transitive includes.

Runtime note. Because cycles are validated at load time, the executor never needs to guard against infinite include loops at execution time. The executor-level re-assertion in step 2 above is a defence-in-depth check, not the primary enforcement point.

Future: Sub-Procedure Call. `type: include` is the v2.0 composition primitive: inline expansion into a shared variable scope. A future `type: call` step type would provide isolated scope with explicit input/output mapping, analogous to a function call. This pattern is deferred post-v2.0; `type: include` covers the primary composition use case for the v2.0 release.

Step Post-flight (all step types)

1. Apply outcome evaluation: test `when:` expressions; record `OutcomeRecord`.
2. Write captured variables to run state.
3. Checkpoint run state to `.runbook/runs/<runID>/snapshots/<stepID>.json`.
4. Write `step/completed` trace event (outcome, captures, elapsed time).
5. Emit `StepCompleted` runtime event.

3.2.7 Phase 5: Run Completion

When `Next` returns `io.EOF`:

1. Write `run/completed` trace event (final outcome, step counts, elapsed time).
2. Flush and sync the trace file.
3. Compute SHA256 of the trace file; write `run/evidenceHash` record.
4. Optionally archive the run directory (compress to `.runbook/archive/`).
5. Close the event channel.

3.3 Lifecycle Model

3.3.1 Run Start

A run starts in one of two ways:

1. **Direct invocation:** The CLI or test harness calls `Runtime.Start()` directly. This is the path for `gert exec`, `gert tui`, `gert debug`, and `gert test`.
2. **RPC invocation:** A client sends `exec/start` to the JSON-RPC server (`gert serve`). The server calls `Runtime.Start()` on behalf of the client and returns the `RunID` in the response. Subsequent step advancement is driven by `exec/next` RPC calls.

3.3.2 Step Advancement

Step advancement is *always client-driven*. The runtime does not auto-advance. After `Start`, the client calls `Next` to execute each step in sequence. This applies to all adapters, including headless CI mode. The only exception is the `-auto` flag, which causes the CLI adapter to call `Next` in a tight loop, producing the appearance of automatic advancement while retaining the single-step contract.

3.3.3 Pause and Resume

A run is *paused* when `Next` returns a non-terminal result that requires external input: interactive steps (`choice`, `decision`, `collector`) or approval gates. The run remains

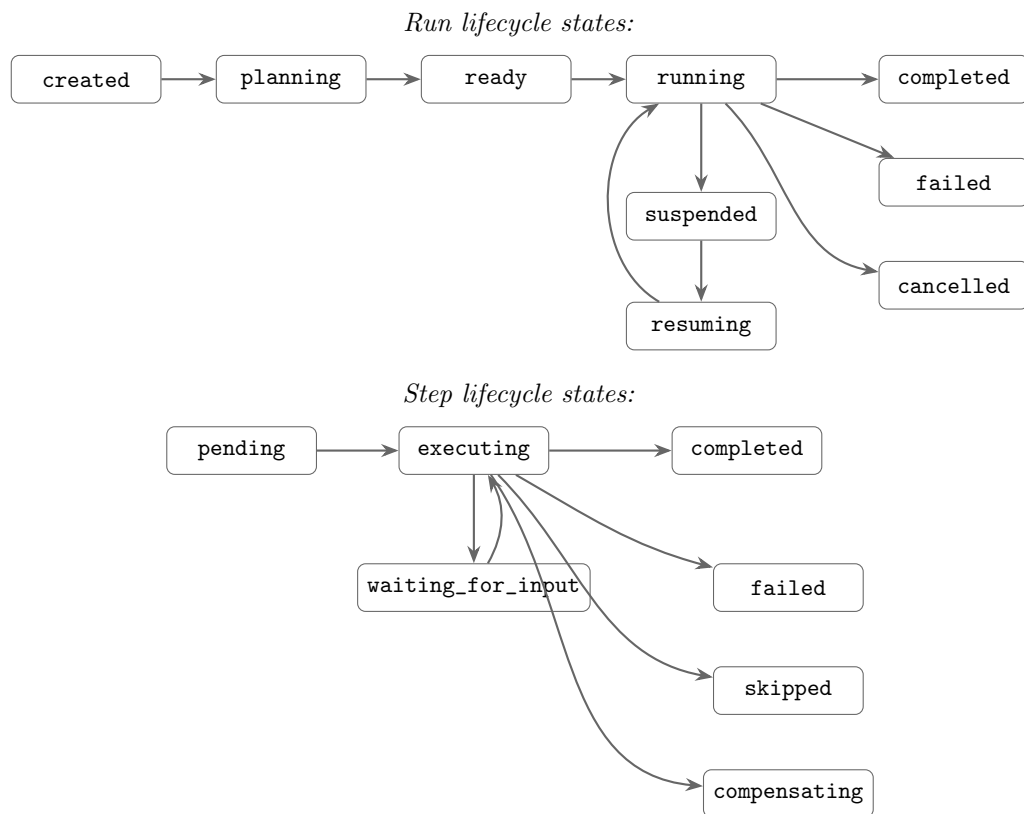


FIGURE 3.4: Execution lifecycle — run states (top) and step states (bottom).

paused until the required input is provided via `SubmitEvidence` or `Approve`. The adapter is responsible for presenting the pause to the user.

The checkpoint written after each completed step is sufficient to resume a run after a process restart. The checkpoint records: completed step IDs, current variable bindings, current evidence values, and the trace sequence number. Resumption is initiated by calling `Runtime.Resume(runID)` from any adapter.

3.3.4 Wait-for-Event Executor Contract

WAITING Run State

The `wait_for_event` step introduces a new **WAITING** run state that is distinct from the existing user-initiated **PAUSED** state:

State	Trigger	Exit Condition
RUNNING	<code>exec/start</code> or resume	Active step execution loop.
PAUSED	User-initiated pause; interactive / approval step	<code>SubmitEvidence</code> or <code>Approve</code> RPC call from adapter.
WAITING	<code>wait_for_event</code> step reached	Event Dispatcher delivers a matching event (<code>WAITING→RUNNING</code>); or timeout expires with <code>on_timeout: fail</code> (<code>WAITING→FAILED</code>) or timeout expires with <code>on_timeout branch:<id></code> (<code>WAITING→RUNNING</code> , jump to branch target).
COMPLETED	All steps finished	Terminal.
FAILED	Unhandled error or <code>on_timeout: fail</code>	Terminal (unless compensated in v2.1).

WAITING is a *system-driven* suspension: the run is blocked on an external signal, not on user input. Clients **MUST** differentiate the two states in UX (e.g. “awaiting webhook” vs. “awaiting approval”) and **MUST NOT** send `exec/next` while the run is in **WAITING** state.

Pause / Resume Protocol

When the step execution loop reaches a `wait_for_event` step it **MUST**:

1. **Serialise run state** — write the full run snapshot (step index, all variable bindings, call stack, active listener registration) to the run store (embedded BoltDB / SQLite, same store used for checkpoint snapshots).

2. **Register event listener** — call `EventDispatcher.Register(runID, eventID, source, filter, payloadSchema, timeout)`. The Dispatcher returns a registration token.
3. **Suspend** — set `run.state = WAITING`, write a `step/waiting` trace event (includes `event.source`, `event.id`, timeout deadline), and return from the execution goroutine. The run goroutine is now idle; no `Next` call is outstanding.

When the Event Dispatcher delivers a matching event to a `WAITING` run it **MUST**:

1. Load the serialised run state from the run store via `store.Load(runID)`.
2. Validate the payload against `step.event.payload_schema` (JSON Schema Draft 2020-12). Validation failure → `FAILED` with schema violation detail.
3. Apply `step.event.filter` key-value predicates against the payload. Mismatch is treated as a non-matching event (listener remains registered; the event is discarded).
4. Apply `step.capture` mappings: extract `json.<path>` fields from the payload and write them into `run.variables`.
5. Write a `step/eventReceived` trace event (event source, sanitised payload summary, captured variable names).
6. Set `run.state = RUNNING` and call `ExecuteNextStep(run)`.

Executor Pseudocode

```
// Suspend path
ExecuteStep(step: WaitForEvent, run: Run):
    store.Persist(run)                // 1. durable checkpoint
    token = dispatcher.Register(      // 2. register listener
        run.id, step.event.id, step.event.source,
        step.event.filter, step.event.payload_schema,
        step.timeout)
    run.variables["gert.event."+step.event.id+".token"] = token // HMAC token
    run.state = WAITING
    emit(step/waiting, {event_id: step.event.id, deadline: now()+step.timeout})
    return (suspended)                // 3. goroutine exits
```

```

// Resume path
OnEventArrived(run_id, event_id, payload):
    run = store.Load(run_id)                // 1. rehydrate
    step = run.currentStep()                // (WaitForEvent step)
    ValidatePayload(payload, step.event.payload_schema) // 2. schema check
    if not MatchFilter(payload, step.event.filter):    // 3. filter
        return // discard; listener stays registered
    ApplyCapture(payload, step.capture, run.variables) // 4. capture
    emit(step/eventReceived, {run_id, event_id})      // 5. trace
    run.state = RUNNING
    ExecuteNextStep(run)                          // 6. advance

// Timeout path
OnTimeout(run_id, event_id):
    run = store.Load(run_id)
    step = run.currentStep()
    switch step.on_timeout:
        case "fail":
            run.state = FAILED
            emit(step/failed, {reason: "wait_for_event timeout"})
        case "continue":
            run.state = RUNNING
            ExecuteNextStep(run)
        case "branch:<step-id>":
            run.state = RUNNING
            run.stepCursor = resolve(step-id)
            ExecuteNextStep(run)

```

Run Persistence Requirement

Runs in `WAITING` state MUST survive a `gert serve` process restart. The serialised run snapshot written in step 1 above MUST include:

- Current step index and depth.
- All variable bindings (including `gert.event.*` tokens).
- Call stack (for `include`-inlined steps).

- The active event listener registration (run ID, event ID, source, filter, timeout deadline as an absolute timestamp).

On `gert serve` restart, the server MUST re-register all persisted `WAITING` listener records with the Event Dispatcher before accepting new connections. Listeners whose timeout deadline has already passed are immediately moved to the timeout resolution path.

Serve-Only Constraint

`wait_for_event` is only valid when running under `gert serve`. If a runbook containing a `wait_for_event` step is executed via `gert run` (local CLI, non-server mode), the Runtime Core MUST fail at dispatch time with:

```
step type 'wait_for_event' requires gert serve mode
```

The Planner MAY emit a validation warning for `wait_for_event` steps in non-serve context (aiding the user before execution begins), but the enforcement point is the Runtime Core dispatch.

Security: HMAC Token for Webhook Transport

When `event.source: webhook`, each `wait_for_event` registration generates a cryptographically random one-time HMAC signing secret. The Event Dispatcher:

1. Generates 32 random bytes via `crypto/rand`.
2. Stores the secret alongside the listener registration in the run store.
3. Exposes the HTTP endpoint `POST /events/{run-id}/{event-id}` and validates incoming requests using HMAC-SHA256 over the raw request body (header: `X-Gert-Signature: sha256=<hex>`).
4. Injects the secret into the run variable `gert.event.{event-id}.token` so the runbook author can communicate it to the external system (e.g. via a preceding `cli` step that calls `curl -data-urlencode`).
5. Rejects requests with invalid or missing signatures with HTTP 401.
6. Invalidates the secret after the first accepted delivery (one-time use).

The token is intentionally injected as a run variable (not hard-coded in the runbook YAML) so it is unique per run and never committed to source control.

3.3.5 Approve Step Executor Contract

The **approve** step is a first-class step type that pauses the run until one or more authorised identities submit a sign-off decision via the Input Provider interface. Two new capabilities — **business-day timeouts** (GAP-1) and **M-of-N quorum tracking** (GAP-2) — extend the base approval gate without changing its fundamental PAUSED-state semantics.

GAP-1: Business Calendar Engine

Motivation. Enterprise approval runbooks express deadlines in *business days* (“respond within 3 business days”), not wall-clock durations. The existing `timeout` field is wall-clock only and cannot satisfy this requirement.

Component responsibilities. The **Business Calendar Engine** is a stateless utility component within the Runtime Core (no external process, no network calls for built-in calendars). Its responsibilities are:

- Determine whether a given instant is a business moment (Monday–Friday, non-holiday in the specified calendar and timezone).
- Calculate the wall-clock deadline that is exactly N business days from a given start instant.
- Enumerate elapsed business days between two instants (for reporting and audit).
- Load and cache named business calendars; built-in calendars are compiled into the gert binary. Custom calendar definitions are deferred to v2.1.

Go interface.

```
type BusinessCalendar interface {
    // IsBusinessDay reports whether t falls on a business day (Mon-Fri,
    ↪ non-holiday)
    // in the given timezone.
    IsBusinessDay(t time.Time, tz *time.Location) bool

    // AddBusinessDays returns the instant that is exactly n business days after
    ↪ t
}
```



```

// in the given timezone. n must be >= 0.
AddBusinessDays(t time.Time, days int, tz *time.Location) time.Time

// ElapsedBusinessDays counts completed business days between start and end
// in the given timezone. Returns 0 if end <= start.
ElapsedBusinessDays(start, end time.Time, tz *time.Location) int
}

```

Built-in calendars for v2.0.

Calendar ID	Definition
default	Monday–Friday, no holidays. Appropriate for organisations that define their own holiday schedule externally.
us-federal	Monday–Friday, US federal public holidays (New Year’s Day, Martin Luther King Jr. Day, Presidents’ Day, Memorial Day, Juneteenth, Independence Day, Labor Day, Columbus Day, Veterans Day, Thanksgiving, Christmas).
uk-banking	Monday–Friday, UK bank holidays for England and Wales (New Year’s Day, Good Friday, Easter Monday, Early May Bank Holiday, Spring Bank Holiday, Summer Bank Holiday, Christmas, Boxing Day).

Custom calendar definitions (YAML-declared holiday lists referenced by a named ID) are **deferred to v2.1** as a future-work item.

Timeout calculation at activation. The key architectural decision for business-day timeouts is a **one-time conversion at step activation**: when the runtime dispatches an **approve** step that carries `timeout_business_days`, it immediately calls:

```
deadline = calendar.AddBusinessDays(now, step.timeout_business_days, tz)
```

The resulting **deadline** is a plain `time.Time` value stored in the run state snapshot alongside the step. The existing timeout enforcement mechanism (the Event Dispatcher’s min-heap scheduler or the polling goroutine) then checks `now >= deadline` at regular intervals — *no business-calendar logic is needed at check time*. This design means the Business Calendar Engine is invoked exactly once per activate step and is entirely absent from the hot-polling path.

- **timezone** is resolved from the step's `timezone` field (IANA timezone string, e.g. `America/New_York`). If absent, the operator's local timezone is used; runbooks that require determinism SHOULD specify an explicit timezone.
- **calendar** is resolved from the step's `business_calendar` field. Defaults to `default` if absent.
- If `timeout_business_days` and `timeout` (wall-clock) are both present on the same step, the *earlier* of the two computed deadlines applies. A Planner warning is emitted to flag the redundancy.

GAP-2: Quorum Approval Tracker

Motivation. Financial and regulatory runbooks require “3 of 5 department heads must approve” semantics. The previous `approvals.min` field was a bare integer with no explicit approver pool, making identity validation and audit reconstruction impossible.

Approval state per step instance. The runtime maintains an `ApprovalRecord` for each active approve step instance:

```
ApprovalRecord {
  step_id:    string
  run_id:     string
  mode:       all | any | quorum
  pool:       []string // eligible approver role names
  required:   int       // 0 = all (for mode: all / any)
  approvals:  []Approval // received approvals
  rejections: []Approval // received rejections
}

Approval {
  approver_id: string // verified identity of the submitter
  role:        string // which pool role the approver satisfied
  timestamp:   time.Time
  notes:       string // optional free-text from the approver
}
```

The `ApprovalRecord` is persisted to the run store at each update (same durable store as run snapshots) so it survives a process restart.

Progression rules.

Mode	Proceed when
all	<code>len(approvals) == len(pool)</code> — every pool member has approved.
any	<code>len(approvals) >= 1</code> — at least one pool member has approved.
quorum	<code>len(approvals) >= required</code> — the required count is reached.

Deduplication: each pool member may submit **exactly one** approval. If the same `approver_id` submits a second approval for the same step instance, the runtime rejects the submission with `ErrDuplicateApproval` and returns an error to the caller — it is not silently ignored.

Rejections are recorded in the audit trail and do *not* block progression by themselves, *unless* quorum becomes mathematically impossible (see Deadlock Detection below).

Deadlock detection. After *every* approval or rejection event, the runtime evaluates:

```
remaining_possible = len(pool) - len(rejections) - len(approvals)
if (len(approvals) + remaining_possible) < required:
    // quorum is impossible - fail immediately
    step.state = FAILED
    emit(step/failed, {reason: "QuorumImpossible", ...})
```

For mode: `all`, `required` is treated as `len(pool)` for this check. For mode: `any`, the check is skipped (one approval always suffices unless the pool is empty, which is a schema validation error). If the check fires, the step fails immediately with error code `QuorumImpossible`; the `on_timeout` policy does *not* apply.

Audit trail. Every approval and rejection is appended to the run's append-only trace as a `approve/decision` trace event containing: `approver_id`, `role`, `decision` (`approve` or `reject`), `timestamp` (RFC 3339), and `notes`.

This satisfies electronic-signature audit requirements including SOC 2 Type II and FDA 21 CFR Part 11.

Input Provider integration. Approval decisions are delivered via the Input Provider JSON-RPC interface. The `approve` step executor registers a pending input request of type `input/submitApproval` with the fields:

```
{
  "run_id":  string,           // identifies the run
  "step_id":  string,           // identifies the approve step
  "decision": "approve"|"reject",
  "notes":    string           // optional
}
```

Before recording the decision, the runtime:

1. Resolves the caller's `approver_id` from the authenticated session (or the `-as` flag in CLI mode).
2. Verifies that `approver_id` maps to at least one role in `approvals.pool`. Unknown identities receive `ErrNotAuthorised`.
3. Checks for duplicate submission (`ErrDuplicateApproval` if already recorded).
4. Appends the `Approval` or rejection record to the `ApprovalRecord`.
5. Re-evaluates the progression rule and the deadlock-detection invariant.
6. Persists the updated `ApprovalRecord` to the run store.

Composition: Business-Day Timeout with Quorum Approval

Both GAP-1 and GAP-2 **compose freely**. A step may declare `timeout_business_days`, `timezone`, `business_calendar`, and `approvals (mode: quorum)` simultaneously. At step activation:

1. The Business Calendar Engine computes the wall-clock **deadline**.
2. The Quorum Tracker initialises an empty `ApprovalRecord`.
3. The step enters PAUSED state; the timeout min-heap is armed with the deadline.

The two mechanisms then run **independently**: the Quorum Tracker processes each incoming approval/rejection without consulting the calendar, and the timeout scheduler checks `now >= deadline` without consulting the quorum state. Whichever condition fires first wins: quorum reached $\rightarrow$ step proceeds; deadline fired $\rightarrow$ `on_timeout` policy governs the outcome.

3.3.6 Collector Field Validation Contract

When the executor receives evidence from a `SubmitEvidence` call for a `collector` step, it validates each field value against the field’s declared type and constraints *before* writing any value to run state or the trace.

Validation Algorithm

For each field in submission order:

1. **Required check** — if the field is `required: true` and the submitted value is absent or empty, record a validation error.
2. **Type coercion and format check** — attempt to parse or coerce the value to the declared type (see per-type rules below). A parse failure is a validation error.
3. **Constraint check** — evaluate `validation.min`, `validation.max`, `validation.step`, and option-list membership as applicable. A constraint violation is a validation error.
4. **Re-prompt** — if any field produced a validation error, the executor sends a new `CollectorRequired` event back to the input provider with an `errors` map populated (field name $\rightarrow$ human-readable error message). The provider **SHOULD** re-render the form highlighting the invalid fields. The re-prompt counts as one retry attempt.
5. **Max retries** — if validation still fails after **3** re-prompt attempts, the executor fails the step: `step.state = FAILED`, writes a `step/failed` trace event, and populates the `gert.error` reserved run variable with structured error detail (see below).

Per-Type Validation and Storage Rules

Type	Validation	Variable Storage
text	Required check; then optional length and pattern constraints: <code>validation.min_length</code> , <code>validation.max_length</code> (character count); <code>validation.pattern</code> (RE2 regex — see below); <code>validation.pattern_hint</code> (shown on pattern failure). <code>multiline: true</code> is a UI hint only; no effect on validation or storage. If <code>format: yaml</code> or <code>format: json</code> is declared, structural parse validation is applied <i>after</i> the pattern check (see below).	JSON string (raw; parsed form not stored)
number	Parse as IEEE 754 float64 (reject non-numeric input). Check <code>validation.min</code> and <code>validation.max</code> (both inclusive). Check <code>validation.step</code> : $(\text{value} - \text{min}) \% \text{step} < \text{epsilon}$.	JSON number (float64)
integer	Parse as int64; reject fractional input. Same <code>min</code> / <code>max</code> / <code>step</code> checks as <code>number</code> .	JSON number (int64)
date	Must match YYYY-MM-DD (RFC 3339 full-date). Validate <code>validation.min</code> / <code>max</code> as date strings (lexicographic comparison is correct for ISO dates). Wall clock at submission time; timezone UTC.	JSON string (YYYY-MM-DD)
datetime	Must be valid RFC 3339 (ISO 8601 with time offset). Validate <code>validation.min</code> / <code>max</code> as RFC 3339 strings; normalise to UTC before comparison. Wall clock at submission time.	JSON string (RFC 3339, e.g. 2026-04-18T14:00:00Z)
boolean	Truthy/falsy coercion: <code>true</code> , <code>"true"</code> , <code>1</code> , <code>"yes"</code> → <code>true</code> ; <code>false</code> , <code>"false"</code> , <code>0</code> , <code>"no"</code> → <code>false</code> . Any other value is a validation error. No <code>min</code> / <code>max</code> constraints.	JSON boolean
select (multiple: false)	Submitted value must exactly match one of the declared <code>options[*].value</code> strings. If <code>options_from</code> was used, validation is against the dynamically fetched list (§16.5.4).	JSON string
select (multiple: true)	All submitted values must be members of the	JSON array of strings

`multiline: true` **Hint**

The `multiline: true` flag on a `text` field is a **UI hint only**. The executor treats the submitted value as a plain string regardless of this flag. Input providers SHOULD render a multi-line textarea (or open `$EDITOR`) rather than a single-line input box when `multiline: true` is set.

`text` Field Pattern Validation

`text` fields support optional length, pattern, and structural-format constraints applied *after* the required check. The full sequence for a `text` field is:

1. **Required check** — same as all field types (step 1 of the general algorithm above).
2. **Min-length check** — if `validation.min_length` is declared, fail if `len(value) < min_length` (character count, not bytes).
3. **Max-length check** — if `validation.max_length` is declared, fail if `len(value) > max_length`.
4. **Pattern check** — if `validation.pattern` is declared, compile and test the full value against the RE2 regex. On failure, the re-prompt error message is `validation.pattern_hint` if present; otherwise the generic message “*Value does not match required format*” is used.
5. **Format check** — if `format: yaml` is declared, attempt `yaml.Unmarshal([]byte(value), &interface{})`. On error, re-prompt with “*Value must be valid YAML.*”. If `format: json` is declared, attempt `json.Unmarshal([]byte(value), &interface{})`. On error, re-prompt with “*Value must be valid JSON.*”. Both checks are **parse-only**: the parsed result is discarded; the value remains a raw string in the run variable. Downstream steps use the `fromYAML` and `fromJSON` template functions for traversal (see §4.3.3). If both `validation.pattern` and `format` are declared, both must pass; the pattern check (step 4) runs first.
6. **Re-prompt** — any failure at steps 2–5 triggers a re-prompt. The 3-attempt limit is a **combined total** across all validation steps: if the operator fails step 2 once, step 4 once, and step 5 once, the step fails on the third attempt regardless of which check failed each time.

RE2 vs ECMA 262. The gert schema specification uses the term *ECMA 262 regex* for compatibility with JSON Schema tooling and browser-based UI validators. At runtime, gert uses Go's `regexp` package, which implements RE2 semantics — not a full ECMA 262 engine.

Patterns **must** be valid RE2 expressions. ECMA 262 features absent from RE2 (lookahead, lookbehind, backreferences) are **rejected at schema load time** with a validation error naming the offending field:

```
error: step "gather-details", field "ticket_ref":  
  validation.pattern uses lookahead (?=...) which is not supported in RE2;  
  replace with an anchored RE2 equivalent
```

Runbook authors targeting both the VS Code extension (which uses a JavaScript regex engine) and the gert runtime (RE2) should restrict patterns to the safe common subset: character classes, quantifiers, anchors, alternation, and non-capturing groups.

Conditional Field Evaluation Contract

`collector` fields MAY declare a **when** expression. The executor evaluates **when** before rendering each field, determining whether the field is shown, prompted, and bound to a variable.

Evaluation model. Fields are evaluated left-to-right in array order:

1. Before rendering field *i*, the executor evaluates its **when** expression against:
 - **Collected values so far** — the values returned by the input provider for fields $0 \dots i - 1$ in the current step (those fields whose **when** evaluated to **true** and were rendered).
 - **Global variable scope** — the run's accumulated variable bindings from all previously completed steps.
2. If **when** evaluates to **false**: the field is **not rendered, not prompted, and its variable binding is skipped entirely**. The field's `id` is NOT added to the step's variable scope for this run.
3. If **when** evaluates to **true**, or if **when** is absent (unconditionally visible), the field is rendered and validated normally.

This is a **pull model**: the executor drives field-by-field evaluation in order; the input provider renders one field at a time, or a full form with some fields conditionally hidden — both are valid provider strategies.

Input provider integration. Two delivery strategies are both valid; providers declare their preference via capability flags (see §16.5.3):

- **Full-form delivery** (recommended for rich UI providers): the executor sends the complete `fields` array in a single `provider/collect` request. Each field carries a pre-evaluated `visible: bool` flag reflecting the current `when` result at dispatch time. Rich UI providers SHOULD implement dynamic client-side show/hide as the user fills in earlier fields, re-evaluating `when` locally against live form state so the form responds without a round-trip to the executor.
- **Streaming delivery** (recommended for CLI providers): the executor sends only the currently-visible fields, evaluating each `when` expression in real time as previous field values are received. This is the natural mode for the built-in `prompt` provider, which renders fields sequentially at the terminal. Fields where `visible: false` are simply never sent.

Variable binding on skip. When a field is skipped (`when = false`):

- Its `id` is NOT added to the step's variable bindings.
- Template expressions in subsequent steps that reference the skipped field's variable resolve to the *zero value*: `"` for string, `0` for number, `false` for boolean.
- If the skipped variable is referenced with `.require` in a downstream template expression, the executor produces a template error at that step.
- Runbook authors should guard downstream template expressions that may depend on conditionally-bound fields:

```
{{/* Safe: test for empty before using a conditionally-bound variable */}}
{{ if .ticket_ref }}JIRA: {{ .ticket_ref }}{{ else }}(no ticket){{ end }}
```

Load-time validation — forward reference check. At schema load time, the parser performs a forward-reference check for every field that declares **when**:

1. Parse the **when** expression and extract all variable references.
2. For each referenced variable, determine whether it is bound by a field declared *earlier* in the same **fields** array (by array index).
3. If any referenced variable is bound by a field declared *later* in the same **fields** array, **reject the runbook at load time** with a validation error:

```
error: step "gather-details", field "region" (index 2):
  when expression references "country" which is bound by field at index 4;
  forward references in when expressions are not allowed
```

Variables from the global scope (prior steps) and the run context are always valid references in **when** expressions; only same-step forward references are rejected.

Variable Storage — Downstream Expression Semantics

Typed storage ensures that template comparison operators in downstream steps work without manual coercion:

```
{{/* number/integer: .amount is float64, not a string */}}
{{ if gt .amount 50000.0 }}HIGH_VALUE{{ end }}

{{/* boolean: .confirmed is bool, not "yes"/"no" */}}
{{ if .confirmed }}PROCEED{{ end }}

{{/* select/multiple or choice/multiple: .systems is []string */}}
{{ if has .systems "payments-api" }}CRITICAL{{ end }}

{{/* date/datetime: .start_date is ISO 8601 string; parseable by time.Parse */}}
{{ if lt .start_date "2026-07-01" }}BEFORE_CUTOVER{{ end }}
```

Error Variable Schema

When a collector step exhausts its re-prompt attempts and fails, the executor populates the **gert.error** reserved run variable:

```
{
  "step_id": "gather-approval-details",
  "kind": "collector.validation_failed",
  "attempts": 3,
  "fields": {
    "amount": "value 49000 is below minimum 50000",
    "start_date": "2025-12-01 is before minimum 2026-01-01",
    "department": "\"legal\" is not a valid option"
  }
}
```

The `gert.error` variable is a reserved JSON object populated on any step failure. A subsequent `branch` step may inspect it to route to an error-handling path. Each new step failure overwrites the previous value.

Ephemeral Field Handling

A collector field may declare `ephemeral: true` to indicate that its value is sensitive and must never be written to any durable store.

Audit redaction. After a field value passes all validation checks, the executor writes the step's output to the JSONL trace file. For fields where `ephemeral: true`:

- The JSONL audit record stores the string literal "[REDACTED]" as the field value — not the actual value.
- The *actual* value IS placed in the in-memory `VariableScope` for the duration of the run and is available to downstream steps via normal template expressions.
- The actual value is **NEVER written to disk** in any log, trace, snapshot, or checkpoint file. This includes the run trace (`trace.jsonl`), the per-step checkpoint (`snapshots/<stepID>.json`), and the evidence hash record.

Go interface contract. The in-memory `VariableScope` type MUST expose the method:

```
IsEphemeral(key string) bool
```

The JSONL trace writer calls `IsEphemeral` for every variable before serialising a `step/completed` or `collector/submitted` event. If the method returns `true`, the writer substitutes "[REDACTED]" for the value in the emitted JSON. No other layer in the stack applies this substitution — the redaction responsibility belongs exclusively to the trace writer.

Tool step compatibility. Ephemeral values MAY be passed as arguments to `type: tool` steps. The tool receives the plaintext value. This is intentional: `ephemeral: true` is an *audit redaction* feature, not end-to-end encryption. Runbook authors are responsible for using ephemeral fields only with tools that handle the value securely.

Suspension/resume constraint. When a run is suspended (e.g., a `wait_for_event` or `approve` step puts the run into `WAITING` state), the in-memory `VariableScope` is not persisted. Because ephemeral values exist only in memory, they are **dropped** on suspension. On resume, any downstream step that requires an ephemeral variable will find it absent from the reconstructed scope.

Constraint: ephemeral variables and suspension

Steps that depend on ephemeral variables **MUST** appear before any `wait_for_event` or `approve` step that could suspend the run. The schema validator **SHOULD** emit a load-time warning (not a hard error in v2.0) if an ephemeral variable is referenced in a step that follows a suspendable step in the same runbook's execution path.

3.3.7 Cancellation

`RunHandle.Cancel(ctx, reason)` initiates graceful cancellation:

1. The currently-executing subprocess (if any) receives `SIGTERM` followed by `SIGKILL` after a 5-second grace period.
2. The runtime writes a `run/cancelled` trace event with the reason and actor.
3. Subsequent `Next` calls return `ErrRunCancelled`.
4. The run state is checkpointed as `cancelled` (resumable).

The cancellation contract does not invoke compensation handlers in v2.0. Saga/compensation (stepping backward through a compensation chain on failure) is deferred to v2.1 as specified in §09 (Open Questions).

3.3.8 Crash Recovery

If the gert process crashes mid-run, the trace and checkpoint files remain on disk in a consistent state due to the append-only, sync-after-write trace discipline. On restart, the user (or the VS Code extension) can call `Runtime.Resume(runID)` to continue from the last completed step. The runtime reads the checkpoint to reconstruct run state. Steps already recorded as completed in the trace are not re-executed.

3.4 Concurrency Model

3.4.1 Per-Process Run Isolation

The v2 runtime is **single-run-per-process**. Each `gert exec` invocation is a separate OS process. `gert serve` is the exception: it is a long-running process that may host multiple sequential runs (one at a time) for a single VS Code extension session. Concurrent runs within a single `gert serve` instance are explicitly out of scope for v2.0 (see §09).

Rationale. Governance enforcement, file-level trace writes, and subprocess management are dramatically simpler with one run per process. Process isolation also provides a natural crash boundary: a failed run cannot corrupt the state of a concurrent run.

3.4.2 Goroutine Model

Within a single run, the concurrency model is:

- **One goroutine per run:** the step execution loop runs on a single goroutine. Steps execute sequentially; there is no goroutine-per-step parallelism in v2.0.
- **Event fan-out goroutine:** a second goroutine reads from an internal event buffer and writes to the `Events()` channel. This decouples event emission (which must be synchronous with step execution) from event consumption (which may be slow, e.g., a WebSocket write).

- **Subprocess goroutine:** when a `cli` step spawns a subprocess, a goroutine is used to drain `stdout/stderr` concurrently, preventing deadlock on full pipe buffers.
- **Tool RPC goroutine:** tool invocations over JSON-RPC or MCP use a goroutine per call, managed by the Tool Runtime's connection pool.

3.4.3 Thread Safety at Component Boundaries

- `RunHandle` is **not safe for concurrent use**. The adapter must serialize all calls. `gert serve` enforces this with a per-session mutex.
- The `Events()` channel is safe for a single consumer goroutine.
- The `GovernanceEngine` is **read-only after construction** and therefore safe for concurrent read access (relevant if future versions support parallel steps).
- The Trace Writer is protected by an internal mutex; write calls are safe to issue from any goroutine within the run.

3.5 gert serve Integration

`gert serve` is the JSON-RPC 2.0 server that powers the VS Code extension, the MCP tools layer, and any future web client. In the v2 architecture it is classified as an **adapter in the API/Adapter Layer**.

3.5.1 Architectural Position

`gert serve` is a process that:

1. Starts a long-lived stdio JSON-RPC 2.0 listener (newline-delimited JSON over `stdin/stdout`).
2. On receiving `exec/start`, calls `Runtime.Start()` and stores the resulting `RunHandle` in a session map (keyed by `RunID`).
3. On receiving `exec/next`, calls `RunHandle.Next()` and returns the `StepResult` as the RPC response.
4. Drains the `Events()` channel in a background goroutine and forwards each event as a JSON-RPC notification to the client.

5. On receiving `exec/approve`, `exec/submitEvidence`, or `exec/cancel`, dispatches to the corresponding `RunHandle` method.

3.5.2 Event Forwarding

Every event emitted by the Runtime Core is forwarded to the client as a JSON-RPC *notification* (no `id` field) with method `event/eventType`. The payload is the event's JSON representation. The VS Code extension subscribes to these notifications to update its UI without polling.

Events forwarded over WebSocket (for a future web adapter) follow the same schema; only the transport changes.

3.5.3 Crash Recovery in `gert serve`

If the VS Code extension reconnects after a crash (e.g., the `gert` process was killed), `gert serve` exposes `exec/list` and `exec/resume` methods. The extension discovers the interrupted run, calls `exec/resume`, and receives a new `RunHandle` over the same session. The trace and checkpoint on disk ensure no steps are lost.

3.5.4 Event Dispatcher

The **Event Dispatcher** is a new runtime component that lives exclusively inside the `gert serve` process. It is not instantiated by `gert run`.

Responsibilities

- **Listener lifecycle** — maintain an in-memory registry of active event listeners (indexed by `run_id + event_id`). Each entry records: source type, filter predicates, payload schema URL, timeout deadline, and HMAC secret (webhook source only). Persist the registry to the run store so it survives process restart.
- **Webhook transport** — expose `POST /events/{run-id}/{event-id}` authenticated with HMAC-SHA256 (see security note in §3.3.4). Parse the request body as JSON and dispatch to the matching listener. Return `HTTP 202 Accepted` on success; `404` if no listener is registered; `401` on signature failure; `410 Gone` if the run is no longer in `WAITING` state.

- **Channel transport** — maintain an in-process `map[string]chan Payload` keyed by `event.id`. Any component within the same `gert serve` process can post to a named channel via `EventDispatcher.Send(eventID, payload)`.
- **Message broker transport** — delegate to a pluggable `MessageBroker` interface (implementation-defined for v2.0; a no-op stub ships by default). The interface follows the same pattern as the `ToolTransport` plug-in model: a registered factory function keyed by broker type string.
- **Signal transport** — register `os/signal` handlers scoped to a specific run. Signal handling is multiplexed: the same OS signal may be registered by multiple active listeners (e.g. `SIGUSR1` on two concurrent waiting runs). Each listener receives its own notification; signal handlers are deregistered on listener expiry or run completion.
- **Timeout enforcement** — maintain a min-heap of timeout deadlines. A background goroutine fires on the earliest deadline and invokes `OnTimeout(runID, eventID)`.

Go Interface

```

type EventDispatcher interface {
    // Register creates a listener for an inbound event on a waiting run.
    // Returns a one-time HMAC token (non-empty for webhook source only).
    Register(ctx context.Context, reg ListenerRegistration) (token string, err
        ↪ error)

    // Deregister removes a listener (called on run completion or cancellation).
    Deregister(runID, eventID string) error

    // Send posts a payload to a named in-process channel listener.
    Send(eventID string, payload json.RawMessage) error

    // RestoreFromStore re-registers all WAITING listeners after process
    ↪ restart.
    RestoreFromStore(store RunStore) error
}

type ListenerRegistration struct {
    RunID      string

```


Chapter 4

Schema vNext

This chapter is the normative specification for all gert data structures: runbook documents, tool definitions, and provider definitions. It defines the contracts that Brian’s parser, Ken’s runtime, and the VS Code extension must implement.

The design philosophy is *minimal core, extensible periphery*: the core language fields that the runtime must understand are small, typed, and stable; everything else is declared as an extension with an explicit namespace. Gert draws a hard boundary and enforces it at parse time.

4.1 Schema Identity and Versioning

4.1.1 apiVersion Values

Every gert document carries a mandatory **apiVersion** field that identifies both the document kind and the schema version. The current values are:

Document kind	apiVersion
Runbook	runbook/v2
Tool definition	tool/v2
Provider definition	provider/v2

The format is <kind>/<major>. Minor and patch changes do not alter the **apiVersion** string; they are tracked by the schema artifact version (see §4.9).

4.1.2 Self-Description via \$schema

Following JSON Schema [29] best practice and the pattern used by GitHub Actions, Argo Workflows, and Kubernetes CRDs, every v2 document *should* carry a `$schema` field pointing to the canonical JSON Schema URL:

```
$schema: "https://schemas.gert.dev/runbook/v2.json"
apiVersion: runbook/v2
id: service-health
name: Service Health Check
...
```

The `$schema` field is optional at runtime (the runtime uses `apiVersion` for dispatch) but is *strongly recommended* because:

- VS Code and JetBrains IDEs provide schema-backed auto-completion and inline validation when `$schema` is present.
- The schema URL carries an implicit version contract: updating the URL updates the validation rules visible to tooling.
- It enables offline validation with any JSON Schema Draft 2020-12 validator (ajv, jsonschema, check-jsonschema).

Schema URLs for each document kind:

```
https://schemas.gert.dev/runbook/v2.json
https://schemas.gert.dev/tool/v2.json
https://schemas.gert.dev/provider/v2.json
```

4.1.3 Versioning Strategy

gert schema versioning follows a simplified semantic versioning model applied to the `apiVersion` major component:

- **Major increment** (e.g. `runbook/v1` → `runbook/v2`): A breaking change. The parser *must not* silently accept the old format. Automated migration is provided via `gert migrate`.

- **Minor/patch changes within a major:** Additive, backward-compatible. New optional fields may be added. Existing fields may not be removed or renamed. The JSON Schema artifact version tracks these with a semver suffix (e.g. `runbook/v2.json` at artifact version `2.1.0`).

What constitutes a breaking change A breaking change is any of:

- Renaming or removing a required field.
- Narrowing an existing enum (removing a valid value).
- Changing a field type (e.g. `string` $\rightarrow$ `object`).
- Changing the meaning of a field such that existing runbooks produce different runtime behavior.
- Removing a step type.

Adding new optional fields, adding new enum values, and loosening existing constraints are *not* breaking changes.

4.2 Core Runbook Fields (v2)

4.2.1 Top-Level Field Inventory

The runbook document has the following top-level fields:

Field	Type	Required	Description
<code>\$schema</code>	string	No	JSON Schema URL (recommended)
<code>apiVersion</code>	string	Yes	Must be <code>runbook/v2</code>
<code>id</code>	string	Yes	Unique machine identifier (slug)
<code>name</code>	string	Yes	Human-readable display name
<code>kind</code>	enum	No	<code>mitigation</code> <code>reference</code> <code>composable</code> <code>rca</code>
<code>description</code>	string	No	Short prose summary
<code>vars</code>	map	No	Default variable values
<code>inputs</code>	map	No	Input declarations (§4.3)
<code>outputs</code>	map	No	Output declarations (§4.3)
<code>toolRefs</code>	array	No	Tool references (§4.2.2)
<code>imports</code>	map	No	Named runbook imports
<code>defaults</code>	object	No	Default step settings
<code>governance</code>	object	No	Governance policy
<code>prose</code>	object	No	Human-readable TSG sections
<code>metadata</code>	map	No	Arbitrary key-value annotations
<code>flow</code>	array	Yes	Execution tree (TreeNode array)

id field The `id` is a slug: lowercase alphanumeric with hyphens, matching `^[a-z][a-z0-9-]*[a-z0-9]$`. It uniquely identifies a runbook within a project and is used by the `include` step and the trace record. It must be stable across renames; changing it is a breaking change for any runbook that includes it by `id`.

kind values

- **procedure** — Workflow procedure; governance defaults to `require-approval` for destructive effects.
- **reference** — Documentation-first runbook; dry-run by default.
- **composable** — Sub-runbook intended for include from other runbooks; not directly executable from CLI.
- **rca** — Root-cause analysis template.

4.2.2 toolRefs

`toolRefs` is a typed array that allows aliasing and explicit path overrides:

```
toolRefs:
- name: curl
- name: nslookup
- name: k8s
  path: ./tools/k8s.tool.yaml    # override default discovery
  alias: kubect1-wrapper        # optional local alias
```

The default discovery path for `name: foo` is `tools/foo.tool.yaml` relative to the runbook file. The `path` field overrides this. The `alias` field lets the runbook reference the tool by a local name that differs from the tool's declared `meta.name`.

4.2.3 Minimal Valid v2 Runbook

```
$schema: "https://schemas.gert.dev/runbook/v2.json"
apiVersion: runbook/v2
id: hello-world
name: Hello World

flow:
- step:
  id: greet
  type: cli
  title: Print greeting
  command: echo
  args: ["Hello, world!"]
  capture:
    output: stdout
- step:
  id: done
  type: end
  title: Complete
  outcome:
    category: resolved
    code: success
```

4.2.4 Complete Annotated Example

```
$schema: "https://schemas.gert.dev/runbook/v2.json"
apiVersion: runbook/v2
id: service-health-check
name: Service Health Check
kind: mitigation
description: |
  Validate DNS resolution and HTTP endpoint reachability
  for a named service. Escalates on failure.

vars:
  default_timeout: "30s"

inputs:
  hostname:
    type: string
    required: true
    description: "Hostname to check"
    from: prompt
  request_id:
    type: string
    required: false
    description: "Request tracking ID"
    from: tracking.id      # provider-resolved

outputs:
  health_status:
    type: string
    description: "Final health assessment"
    from: captures.http_result

toolRefs:
- name: curl
- name: nslookup

governance:
  rules:
```

```
- effects: ["network.probe"]
  action: allow
- effects: ["service.restart"]
  action: require-approval
  min_approvers: 2
redact:
- pattern: "(?i)Authorization:\\s*\\S+"
  replace: "Authorization: <redacted>"

defaults:
  timeout: "60s"

prose:
  safety: |
    Run this runbook only when a request is active. Do not
    loop - each execution creates a trace record.
  prerequisites: "Requires curl >= 7.80 and nslookup on PATH."

metadata:
  team: platform-sre
  runbook-url: "https://wiki.example.com/runbooks/service-health"
  severity: P2

flow:
- step:
  id: check_dns
  type: tool
  title: "Resolve DNS: {{ .hostname }}"
  tool:
    name: nslookup
    action: lookup
    args:
      host: "{{ .hostname }}"
  capture:
    dns_result: stdout
  contract:
    effects: ["network.probe"]
    reads: ["hostname"]
```



```
    writes: ["dns_result"]

- step:
  id: check_http
  type: tool
  title: "HTTP health check: {{ .hostname }}"
  tool:
    name: curl
    action: head
    args:
      url: "https://{{ .hostname }}/healthz"
  capture:
    http_result: stdout
    timeout: "{{ .default_timeout }}"

- step:
  id: evaluate
  type: branch
  title: Evaluate health results
  branches:
    - condition: 'contains(http_result, "200")'
      label: Healthy
      steps:
        - step:
            id: resolved
            type: end
            title: Service is healthy
            outcome:
              category: resolved
              code: http_ok
    - condition: '!contains(http_result, "200")'
      label: Unhealthy
      steps:
        - step:
            id: escalate
            type: end
            title: Escalate to on-call
            outcome:
```

```
category: escalated
code: http_fail
```

4.3 Input and Output Declarations

4.3.1 Input Declarations

Inputs declare variables that must be resolved before execution begins. In v2 the `inputs` map is a top-level field (promoted from `meta.inputs`).

```
inputs:
  <name>:
    type: string | number | boolean | secret
    required: true | false           # default: true
    description: "..."/>

```

type field The `type` field declares the value's type. Supported values:

- **string** — Default. UTF-8 text.
- **number** — Integer or float, passed as string in templates.
- **boolean** — `true` or `false`.
- **secret** — String, redacted in traces and log output.

from: binding syntax The `from` field specifies how the value is resolved. Binding forms:

Binding	Behavior
<code>prompt</code>	Ask the operator interactively at runtime
<code>env.<NAME></code>	Read from environment variable NAME
<code>file.<path></code>	Read first line from a local file
<code><provider>.<field></code>	Delegate to a registered input provider (e.g. <code>tracking.id</code>)
<code>var.<name></code>	Copy from another variable or capture

If `from` is omitted, the field must have a `default` value or the runtime will reject the runbook at semantic validation time.

4.3.2 Output Declarations

Outputs declare values that a composable runbook exposes to its callers. With `type: include`, the caller's variable space is shared directly; explicit output declarations are reserved for the future `type: call` step type (post-v2.0 scope, isolated scope with explicit I/O mapping).

```
outputs:
  <name>:
    type: string | number | boolean | secret
    description: "..."/>

```

The `from: captures.*` binding is the only supported form in v2. The named capture must be written by at least one step in the runbook; the semantic validator checks this.

4.3.3 Expression Language

Gert uses `expr-lang/expr` [11] as the expression language for all conditional fields (`when`, `condition`, `until`), providing natural, Python-like infix expressions. The choice of `expr` is grounded in three facts:

1. Infix notation (`a == "x" && b > 1`) is significantly more readable and familiar to operators than prefix notation (`{{ and (eq .a "x") (gt .b 1) }}`).
2. `expr` provides a safe, sandboxed evaluation environment with no access to Go internals, improving security posture.
3. Expressions are localized to `when/condition` fields; shell command interpolation (`args`, `title`) continues to use Go templates.

Syntax overview Expressions operate on the *run variable map*: all `vars`, resolved `inputs`, and accumulated `captures` merged into a single flat map. Variables are referenced directly by name (`hostname`), without prefix notation.

Operators:

- Comparison: ==, !=, <, >, <=, >=
- Logical: &&, ||, !
- Grouping: (expr)

Built-in functions:

Function	Example
contains(s, substr)	contains(status, "ok")
startsWith(s, prefix)	startsWith(hostname, "prod-")
endsWith(s, suffix)	endsWith(hostname, ".local")
matches(s, pattern)	matches(version, "^v[0-9]+")
len(s)	len(output) > 0
trim(s)	trim(message) != ""
lower(s)	lower(env) == "prod"
upper(s)	upper(region) == "US-WEST"

Examples:

```
# Equality check
when: severity == "critical"

# Logical AND
when: environment == "prod" && count > 1

# Logical OR with grouping
when: (severity == "medium" || severity == "critical") && customer_impact == true

# Function call
condition: contains(http_result, "200")

# Negation
condition: !contains(http_result, "200")

# Numeric comparison
until: operation_status == "Running"
```

String interpolation: still Go templates Shell command arguments (`args`), `title`, `instructions`, and other non-conditional fields *still use Go templates* for variable interpolation. This is intentional: the full power of Go template pipelines, custom function maps, and `fromJSON`/`fromYAML` parsing is necessary for constructing command lines and text output.

Access variables by dot notation: `{{ .hostname }}`.

fromYAML and fromJSON (template-only) `fromYAML` and `fromJSON` parse a raw string variable into a structured `interface{}` value. They are available in Go template contexts (`args`, `title`, etc.) but *not* in `expr` conditions. They are the primary mechanism for traversing values collected from `text` fields that declare `format: yaml` or `format: json` (see §4.4.5). The raw string is *always* what is stored in the run variable; these functions perform the parse on each template evaluation. A parse error in a template expression is a hard step failure.

```
{{/* Access a key from a JSON-typed text field */}}
{{ $cfg := fromJSON .service_config }}
{{ $cfg.region }}
```

```
{{/* Access a nested key from a YAML-typed text field */}}
{{ $spec := fromYAML .operation_spec }}
{{ $spec.replicas }}
```

Expression evaluation errors An evaluation error in a `condition`, `when`, or `until` field is a hard step failure (not a soft skip). The step transitions to `failed` and execution halts unless `continue_on_fail: true` is set on the step.

4.4 Step Types (v2 Inventory)

Each step in flow is a `TreeNode`: either a `step` object (with optional `branches` and `iterate` siblings) or a standalone `iterate` block.

v2 defines fourteen step types:

Type	Purpose
<code>cli</code>	Execute a local command (§4.4.2)
<code>choice</code>	Operator selects from predefined options (§4.4.3)
<code>decision</code>	Operator selects an execution path (§4.4.4)
<code>collector</code>	Operator provides structured input across 9 field types (§4.4.5)
<code>approve</code>	Standalone approval gate with quorum and business-day timeout (§4.4.6)
<code>tool</code>	Invoke a declared tool action (§4.4.7)
<code>wait_for_event</code>	Pause until an inbound event arrives (§4.4.8)
<code>include</code>	Expand another runbook inline into the current run path (§4.4.9)
<code>branch</code>	Conditional fan-out to labelled step sequences (§4.4.10)
<code>iterate</code>	Loop over a collection (§4.4.11)
<code>parallel</code>	Fan-out concurrent branches with join semantics (§4.4.12)
<code>assert</code>	Evaluate a guard expression and halt on failure (§4.4.13)
<code>compensate</code>	Declare a saga compensation action (§4.4.14)
<code>end</code>	Declare a terminal outcome (§4.4.15)

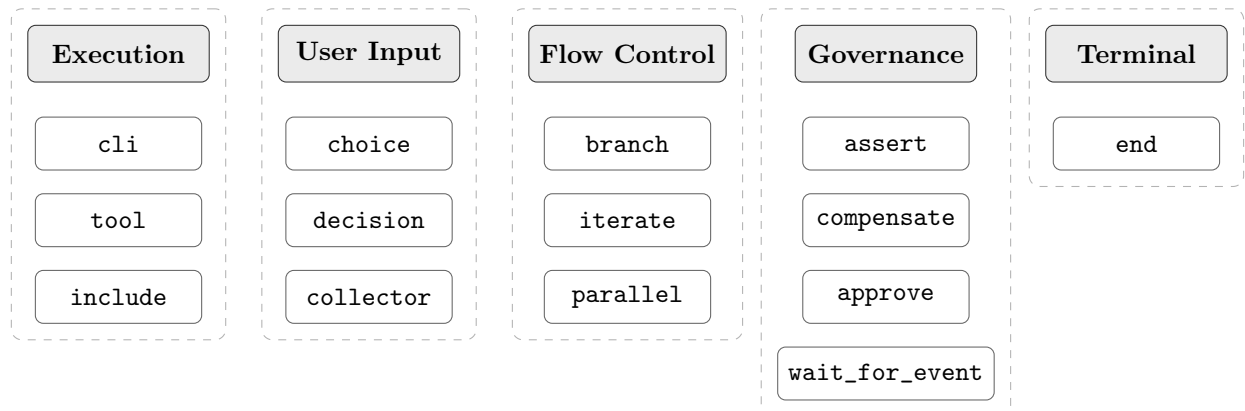


FIGURE 4.1: Step type taxonomy — 14 types organized by category.

4.4.1 Common Step Fields

The following fields are available on every step type:

Field	Type	Required	Description
id	string	Yes	Unique within the runbook
type	enum	Yes	Step type (see below)
title	string	No	Template; displayed in TUI/trace
subtitle	string	No	Secondary template annotation
when	string	No	Guard expression; skips step if false
timeout	string	No	Duration (e.g. 30s); overrides default
delay	string	No	Pre-execution pause (e.g. 5s)
retry	object	No	Retry config (§4.4.1)
continue_on_fail	bool	No	Continue execution on failure
scope	string	No	Named scope for variable isolation
export	string array	No	Variables to export from scope
capture	map	No	Output capture bindings
contract	object	No	Behavioral contract for governance

Retry configuration

```

retry:
  max: 3           # maximum retry attempts (required)
  interval: "5s"   # initial wait between attempts
  backoff: linear  # linear / exponential (default: linear)

```

Step contract The `contract` field annotates a step with its behavioral properties for governance policy evaluation:

```

contract:
  effects:
    - network.probe
    - service.restart
  reads:  [hostname, port]

```

```
writes: [http_result]
deterministic: true
idempotent: false
secrets: [auth_token]
```

Effects are free-form strings matched against `governance.rules[].effects`. The `secrets` list names variables whose values must be redacted in traces.

4.4.2 Step Type: cli

Executes a local command directly. In the v2 runtime `cli` is the canonical command step; the v1 `with.argv` form is accepted in compatibility mode and rewritten to `cli` by the migration tool.

`type:` `cli` supports two mutually exclusive input modes:

- **Exec form** (`command` + `args`): the executor invokes the binary directly without a shell. No shell expansion occurs; arguments are passed verbatim (after template substitution).
- **Run form** (`run`): a shell-script string or a map of per-shell scripts is passed to a shell process. The `shell` field controls which shell is used when `run` is a string.

Using both `command/args` and `run` in the same step is a **schema validation error**.

Field	Type	Required	Description
<code>command</code>	string	See note	Executable name or full path. Mutually exclusive with <code>run</code> .
<code>args</code>	string array	No	Arguments (templates expanded). Only valid with <code>command</code> .
<code>run</code>	string map	See note	Shell-string or per-shell map. Mutually exclusive with <code>command</code> .
<code>shell</code>	string	No	Shell to use with <code>run</code> string form. Enum: <code>auto</code> <code>bash</code> <code>sh</code> <code>pwsh</code> <code>cmd</code> . Default: <code>auto</code> . Ignored when <code>run</code> is a map (validation warning if both are present).
<code>env</code>	map	No	Additional environment variables
<code>workdir</code>	string	No	Working directory
<code>stdin</code>	string	No	Template string piped to stdin
<code>capture</code>	map	No	<code>stdout</code> <code>stderr</code> <code>exit_code</code> → var

Note: Exactly one of `command` or `run` must be present.

Governance checks against the `allowed_commands` / `denied_commands` lists and any matching `governance.rules[].effects` are evaluated before the command executes. If the rule action is `require-approval`, the step pauses for human approval before proceeding.

Run form: string When `run` is a string, the executor spawns the shell specified by the `shell` field and passes the string as a single script argument (equivalent to `shell -c "script"`). When `shell: auto` (the default), the executor selects the platform-native shell: `bash` on Linux/macOS, `pwsh` on Windows (falling back to `cmd` if PowerShell is absent).

Run form: map When `run` is a map, the keys are shell names (`bash`, `sh`, `pwsh`, `cmd`) and the values are shell-script strings. The executor selects the entry that matches the shell available on the current host.

Map key resolution order

1. **Exact match:** the host's configured or detected shell matches a map key (e.g. host has `bash` → use `bash` entry).
2. **Platform-family fallback:** if the exact match fails, fall back according to the table below.
3. **No match:** the step **fails** with the error: *"No matching shell entry for current platform. Available entries: {keys}. Platform shell: {detected}."*

Requested shell	Falls back to
<code>bash</code>	<code>sh</code>
<code>sh</code>	(none — fail)
<code>pwsh</code>	<code>cmd</code> (Windows only)
<code>cmd</code>	(none — fail)

When `run` is a map and a `shell` field is also present, the `shell` field is **ignored** and the runtime emits a validation warning: *"shell field is ignored when run is a map. Remove shell or convert run to a string."*

Example: exec form (unchanged)

```
- step:
  id: tail_logs
  type: cli
  title: "Tail last 100 lines from {{ .log_file }}"
  command: tail
  args: ["-n", "100", "{{ .log_file }}"]
  capture:
    output: stdout
    exit: exit_code
  contract:
    effects: [filesystem.read]
    reads: [log_file]
    writes: [output]
```

Example: run string form with explicit shell

```
- step:
  id: run_script
  type: cli
  shell: bash
  run: |
    if [ "{{ .env }}" = "prod" ]; then
      echo "Production deploy"
    fi
  capture:
    stdout: script_output
```

Example: run string form with auto shell

```
- step:
  id: echo_env
  type: cli
  run: echo "Environment is {{ .env }}"
  capture:
    stdout: echo_result
```

Example: run map form (multi-shell)

```
- step:
  id: clear_cache
  type: cli
  run:
    bash: rm -rf /tmp/cache/{{ .service }}
    cmd: rd /s /q C:\tmp\cache\{{ .service }}
    pwsh: Remove-Item -Recurse -Force C:\tmp\cache\{{ .service }}
  capture:
    exit_code: clear_result
```

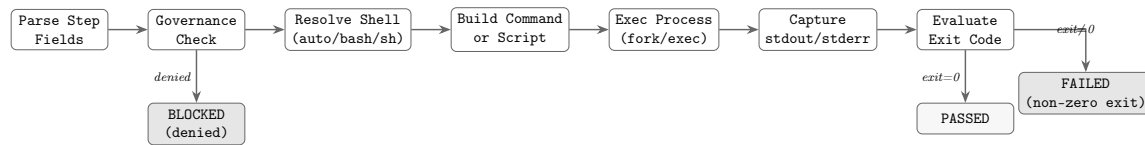


FIGURE 4.2: `type: cli` step execution flow: from field parsing through governance check, shell resolution, command building, process execution, output capture, and exit-code evaluation.

4.4.3 Step Type: choice

Prompts the user to select from a set of predefined options and stores the result in a variable. This is a value-capture primitive, analogous to a form field or radio-button group. The selected value is available to subsequent steps via variable substitution.

Field	Type	Required	Description
<code>prompt</code>	string	Yes	Question or prompt displayed to user
<code>options</code>	array	Yes	List of option objects
<code>variable</code>	string	Yes	Variable name to store selected value
<code>default</code>	string	No	Default option value if user skips
<code>multiple</code>	bool	No	Allow selecting more than one option (default: false)
<code>min_selections</code>	integer	No	Minimum number of selections when <code>multiple: true</code>
<code>max_selections</code>	integer	No	Maximum number of selections when <code>multiple: true</code>

When `multiple: true` the stored variable becomes an array of selected values rather than a single string. The first selected value is used when a downstream template expression expects a scalar.

Option structure Each option is an object with:

Field	Type	Required	Description
label	string	Yes	Display text shown to user
value	string	Yes	Value stored in variable when selected
hint	string	No	Additional context or help text

Constraints

- At least 2 options must be defined.
- All option.value entries must be unique within a step.
- If default is provided, it must match one of the option values.
- The variable name follows standard identifier rules (alphanumeric + underscore).
- When multiple: true, min_selections and max_selections are optional but recommended to constrain the selection count.
- min_selections must be ≥ 1 ; max_selections must be $\leq$ the total number of options and $\geq$ min_selections.

Example: Environment selection

```
- step:
  id: select_environment
  type: choice
  title: Select operation target
  prompt: Which environment are you provisioning to?
  options:
    - label: Staging
      value: staging
      hint: Non-production environment for validation
    - label: Production
      value: production
      hint: Live customer-facing environment
    - label: Canary (10%)
      value: canary
      hint: Gradual rollout to 10% of production traffic
```

```
variable: operation_env
default: staging
```

The value `staging`, `production`, or `canary` is stored in `{{ .operation_env }}` and can be referenced by subsequent steps.

4.4.4 Step Type: decision

Presents the user with 2 or more execution paths and routes the flow to the selected one. This is a control-flow primitive: the user picks a branch, and execution forks to a different runbook or labeled section. Unlike `choice`, which merely stores a value, `decision` directs the execution path itself.

Also acceptable name: `router` (alias for `decision`).

Field	Type	Required	Description
<code>prompt</code>	string	Yes	Question or scenario description
<code>routes</code>	array	Yes	List of route objects (minimum 2)
<code>variable</code>	string	No	Variable to store selected route label (audit)

Route structure Each route is an object with:

Field	Type	Required	Description
<code>label</code>	string	Yes	Display text shown to user
<code>runbook</code>	string	No	Runbook ID to invoke (mutually exclusive with <code>goto</code>)
<code>goto</code>	string	No	Step ID to jump to within current runbook
<code>hint</code>	string	No	Additional context or help text

Constraints

- At least 2 routes must be defined.
- Each route must specify exactly one of `runbook` or `goto`, not both.

- All `route.label` entries must be unique within a step.
- If `variable` is provided, it stores the selected route's `label` for audit trails.

Example: Workflow priority fork

```
- step:
  id: priority_decision
  type: decision
  title: Choose workflow path
  prompt: |
    Based on the impact level and stakeholder requirements, which
    path should we take?
  routes:
    - label: Standard procedure
      runbook: workflow-standard-procedure
      hint: Impact < 5%, no stakeholder escalations
    - label: Emergency cleanup
      runbook: workflow-emergency-cleanup
      hint: Impact > 10% or critical stakeholder concern
    - label: Escalate for review
      goto: escalate_for_review
      hint: Unclear priority; needs senior review
  variable: chosen_path
```

After the user selects a route, execution continues at the target runbook or step. The `chosen_path` variable contains the selected label ("Standard procedure", "Emergency cleanup", or "Escalate for review") for audit purposes.

4.4.5 Step Type: collector

Prompts the user to provide structured input across a rich set of field types: free-form text, numbers, dates, booleans, dropdowns, file attachments, and more. All fields in a single `collector` step are presented and submitted atomically — the operator fills them all in and confirms once. The collected values are stored as named variables or artifacts for downstream steps and evidence capture. This replaces the overly-generic `manual` step type from early v2 drafts.

Field	Type	Required	Description
<code>prompt</code>	string	Yes	Markdown instructions displayed to user
<code>fields</code>	array	Yes	List of field objects to collect (minimum 1)
<code>approvals</code>	object	No	Approval gate config (§4.4.5)

Field structure Each field is an object with:

Field	Type	Required	Description
<code>name</code>	string	Yes	Variable name to store collected value
<code>type</code>	enum	Yes	Field type — see field type inventory below
<code>label</code>	string	Yes	Prompt or label text shown to user
<code>required</code>	bool	No	If true, user must provide a value (default: false)
<code>when</code>	template	No	Go template — field shown and prompted only when truthy; variable binding skipped when falsy
<code>hint</code>	string	No	Additional context or validation help
<code>default</code>	any	No	Pre-filled default value (must match field type)
<code>validation</code>	object	No	Type-specific validation constraints
<code>options</code>	array	No	Option list for <code>select</code> type
<code>options_from</code>	object	No	Dynamic option list provider for <code>select</code> and <code>autocomplete</code> types
<code>multiple</code>	bool	No	Allow multiple selections for <code>select</code> and <code>autocomplete</code> types (default: false)
<code>multiline</code>	bool	No	Enable multi-line textarea for <code>text</code> type (default: false)
<code>ephemeral</code>	bool	No	Value is never written to the audit log and never persisted; in-memory only for the duration of the run (default: false)

Ephemeral fields When a field carries `ephemeral: true`, gert applies the following constraints to the collected value:

- **Variable binding:** The value *is* bound into the runbook variable space and may be used in downstream template expressions (e.g. `{{ .deploy_token }}`) for the duration of the current run.
- **Audit log:** The audit record for this field writes "[REDACTED]" instead of the actual value. The field name, type, and timestamp are still recorded; only the value is suppressed.
- **Persistence:** The value is *never* written to any store — not the JSONL trace, not the evidence artifact store, not any variable snapshot. It exists only in the in-memory execution context.
- **Run suspension:** If the run is suspended after collection (e.g. after a `wait_for_event` step) and then resumed, ephemeral field values are gone. The `collector` step must be re-executed and the operator must re-supply the values.

`ephemeral: true` is valid on any field type, but is semantically meaningful on `text`, `select`, and `boolean` fields. Setting it on `file` or `image` fields is a validation *warning* (not an error) — file content is large and a [REDACTED] placeholder in the audit trail is still useful; however, the *file payload* will not be persisted to the artifact store.

Security note: Ephemeral fields reduce audit footprint for secrets. They do *not* prevent the value from being transmitted to tool steps or sub-processes. Authors are responsible for ensuring ephemeral values are only passed to secure execution contexts.

```
- id: collect_credentials
  type: collector
  label: "Deploy credentials"
  fields:
    - id: operation_user
      type: text
      label: "Service account username"
      required: true
    - id: operation_token
```

```
    type: text
    label: "Deploy token"
    required: true
    ephemeral: true      # never written to audit log or persisted
- id: target_env
  type: select
  label: "Target environment"
  required: true
  options:
    - { value: "staging",    label: "Staging" }
    - { value: "production", label: "Production" }
```

Dynamic Forms When one or more fields carry a **when** expression, the **collector** step renders as a *dynamic form*: fields are shown, hidden, and re-evaluated as the operator fills in earlier fields. **gert** enforces the following evaluation order guarantees:

1. Fields are evaluated in the order they appear in the **fields** array.
2. A field's **when** expression may reference only variables that are declared *earlier* in the same **fields** array, or variables already present in the global runbook variable space (variables set by prior steps, runbook inputs, or **gert.*** built-ins). A forward reference — referencing a field declared *later* in the same **fields** array — is a schema validation error detected at parse time.
3. When a field's **when** expression evaluates to falsy: the field is hidden in the UI, not prompted in a TUI, and its variable binding is *skipped*. The variable does not appear in the runbook variable space; it is not set to **null** or empty string.
4. In interactive UIs (VS Code, browser), if an earlier field value changes after later fields have been filled, all downstream **when** expressions are re-evaluated and dependent fields are shown or hidden accordingly.
5. A **required: true** field with a **when** expression is required only when its **when** condition is truthy. When the condition is falsy, the **required** constraint does not apply.

Field type inventory **collector** supports ten field types:

Type	Description	Notes
<code>text</code>	Single-line or multi-line text input	Set <code>multiline: true</code> for textarea; <code>format</code> for structured validation
<code>number</code>	Integer or float numeric input	Supports <code>validation.min</code> , <code>max</code> , <code>step</code>
<code>integer</code>	Integer-only numeric input	Shorthand for <code>number</code> with <code>step: 1</code>
<code>date</code>	ISO 8601 date (YYYY-MM-DD)	UI hint: date picker
<code>datetime</code>	RFC 3339 datetime with timezone	UI hint: datetime picker
<code>boolean</code>	True/false checkbox	Stored as boolean in variable space
<code>select</code>	Dropdown or option list	Set <code>multiple: true</code> for multi-select
<code>autocomplete</code>	Live search text field	Options fetched from provider as user types; degrades to <code>select</code> in CLI
<code>file</code>	File attachment	Stored as artifact with SHA256 hash
<code>image</code>	Image/screenshot attachment	Stored as artifact

Field type: `text`

Single-line text input by default. Set `multiline: true` to render a multi-line textarea — appropriate for descriptions, reproduction steps, or any field where newlines are meaningful.

Text field validation constraints The optional validation object on a `text` field supports five sub-fields:

Validation field	Type	Required	Description
pattern	string	No	ECMA 262 regular expression; field value must match
pattern_hint	string	No	Human-readable error shown when pattern validation fails
min_length	integer	No	Minimum character count (≥ 0)
max_length	integer	No	Maximum character count (> 0 ; must exceed <code>min_length</code> when both are present)
format	enum	No	Structural parse validation for <code>multiline: true</code> fields: <code>yaml</code> , <code>json</code> , or <code>toml</code>

```

# Basic multiline with hint
- name: description
  type: text
  label: Incident description
  multiline: true
  required: true
  hint: Describe what happened, when, and which services were affected

# Pattern validation - IPv4 address
- name: target_ip
  type: text
  label: Target IP address
  required: true
  validation:
    pattern:
      ↪ "^(25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)\\.\\.\\.\\{3\\}(25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)$"
    pattern_hint: "Must be a valid IPv4 address (e.g. 192.168.1.1)"

# Length constraints
- name: summary
  type: text
  label: Executive summary
  required: true

```

```
validation:
  min_length: 20
  max_length: 500
```

Structured multiline format validation When a text field has `multiline: true`, the optional `validation.format` field declares that the submitted value must be parseable as a specific structured data format:

- `format: yaml` — value must parse as valid YAML.
- `format: json` — value must parse as valid JSON.
- `format: toml` — value must parse as valid TOML v2.1. *Note: TOML format validation is deferred to a future release; declaring it is a validation warning in v2.0.*
- No `format` (default) — plain text; no structural validation.

Parse validation occurs *after submission*. On failure, the operator is re-prompted with the message: "Value must be valid {FORMAT}. Please correct and resubmit." On success, the value is stored as a raw string — the original text, not a parsed object. Parsing is validation only.

Note: The stored value is always a string. Downstream template expressions that need to traverse the structure must use gert's `fromYAML`/`fromJSON` template functions.

```
# YAML-format multiline field
- id: config_patch
  type: text
  label: "Configuration patch (YAML)"
  multiline: true
  required: true
  validation:
    format: yaml
    min_length: 10

# JSON-format multiline field
- id: event_payload
```

```

type: text
label: "Event payload (JSON)"
multiline: true
required: true
validation:
  format: json

```

`validation.format` is silently ignored when `multiline` is `false` or absent — a validation warning is emitted at schema load time.

Field type: number

Integer or floating-point numeric input. The optional `validation` object constrains acceptable values:

Validation field	Type	Required	Description
<code>min</code>	number	No	Minimum acceptable value (inclusive)
<code>max</code>	number	No	Maximum acceptable value (inclusive)
<code>step</code>	number	No	Granularity; set to 1 to enforce integers

```

- name: amount_usd
  type: number
  label: Amount (USD)
  required: true
  validation:
    min: 0
    max: 10000000
- name: port_number
  type: number
  label: Port number
  required: true
  validation:
    min: 1
    max: 65535
    step: 1

```

```
- name: severity_score
  type: number
  label: Severity score (1.0 - 10.0)
  required: true
  validation:
    min: 1.0
    max: 10.0
    step: 0.1
```

Field type: integer

Convenience shorthand for `number` with `step: 1`. The stored variable value is always an integer. Accepts the same `validation.min` and `validation.max` fields as `number`.

```
- name: instance_count
  type: integer
  label: Number of instances
  required: true
  validation:
    min: 1
    max: 100
```

Field type: date

ISO 8601 date input (YYYY-MM-DD). The UI renders a date picker. The stored variable value is the ISO date string.

Validation field	Type	Required	Description
min	ISO date string	No	Earliest acceptable date (inclusive)
max	ISO date string	No	Latest acceptable date (inclusive)

```
- name: start_date
  type: date
  label: Start date
  required: true
  validation:
    min: "2024-01-01"
```


Field type: datetime

RFC 3339 datetime input with explicit timezone offset or Z for UTC (e.g. 2026-04-19T14:30:00-05:00). The UI renders a datetime picker.

Validation field	Type	Required	Description
min	RFC 3339 string	No	Earliest acceptable datetime (inclusive)
max	RFC 3339 string	No	Latest acceptable datetime (inclusive)

```
- name: event_detected_at
  type: datetime
  label: When was the event first detected?
  required: true
```

Field type: boolean

True/false checkbox. The stored variable value is a boolean (**true** or **false**), not a string. No additional validation fields are defined.

```
- name: background_check_completed
  type: boolean
  label: Background check completed?
  required: true
- name: no_disqualifying_issues
  type: boolean
  label: No disqualifying issues found?
  required: true
```

The value is available in templates as a boolean: `{{ if .background_check_completed }}`...`{{ end }}`.

Field type: select

Dropdown or option list. Options are defined inline via **options** (static list) or loaded at runtime via **options_from** (dynamic list from a provider). Exactly one of **options** or **options_from** must be present.

options — static list

Field	Type	Required	Description
value	string	Yes	Value stored in variable when selected
label	string	Yes	Display text shown to user

options_from — dynamic provider

Field	Type	Required	Description
provider	string	Yes	Name of the input provider to query
field	string	No	Provider response field to use as option list

`options` and `options_from` are mutually exclusive. When `multiple: true` is set, the operator may select more than one option and the stored variable becomes an array of selected values.

```
# Static options
- name: department
  type: select
  label: Department
  required: true
  options:
    - value: engineering
      label: Engineering
    - value: sales
      label: Sales
    - value: marketing
      label: Marketing
    - value: finance
      label: Finance

# Dynamic options from provider
- name: budget_line_item
  type: select
  label: Budget line item
```

```
required: true
options_from:
  provider: finance-system
  field: budget_line_items

# Multi-select
- name: affected_services
  type: select
  label: Affected services
  required: true
  multiple: true
  options:
    - value: api-gateway
      label: API Gateway
    - value: auth-service
      label: Auth Service
    - value: payments
      label: Payments
```

Field type: autocomplete

A text field with live search: as the operator types, the configured `options_from` provider is queried and matching options are returned in real time. Useful for searching service names, users, ticket IDs, configuration keys, and any other dataset too large to enumerate statically.

`autocomplete` behaves like `select` in its storage and validation semantics — the selected value is stored as a string (or an array of strings when `multiple: true`). It differs in that options are fetched dynamically on each keystroke rather than declared inline.

The `options_from` object for an `autocomplete` field extends the `select` variant with two additional search-control fields:

options_from field	Type	Required	Description
provider	string	Yes	Input provider ID that handles search queries
field	string	Yes	Provider input field that receives the search query string
min_chars	integer	No	Minimum characters typed before search fires (default: 2)
debounce_ms	integer	No	Debounce delay in milliseconds (default: 300)

CLI / non-interactive fallback: Providers that do not support live search *must* gracefully degrade to a **select** by calling the provider once with an empty query string to fetch all available options. The resulting list is then rendered as a standard dropdown.

```
# Live service search
- name: affected_service
  type: autocomplete
  label: "Affected service"
  required: true
  options_from:
    provider: service-registry
    field: query
    min_chars: 2
    debounce_ms: 300

# Multi-select user lookup
- name: reviewers
  type: autocomplete
  label: "Reviewers"
  multiple: true
  options_from:
    provider: hr-directory
    field: query
    min_chars: 2
```

Approval gate If the `approvals` field is present, the step requires explicit approval from designated roles before proceeding.

Field	Type	Required	Description
<code>min</code>	integer	Yes	Minimum approvals required
<code>roles</code>	array	Yes	List of role names authorized to approve
<code>timeout</code>	string	No	Duration before timeout (e.g. 4h)
<code>on_timeout</code>	enum	No	<code>escalate</code> , <code>fail</code> , <code>skip</code>
<code>escalate_to</code>	array	No	Roles to escalate to if timeout occurs

The `timeout` and `on_timeout` fields are new in v2, addressing the SLA enforcement gap identified in the research brief.

Example: Incident evidence collection

```
- step:
  id: collect_evidence
  type: collector
  title: Collect workflow evidence
  prompt: |
    Gather all relevant evidence for the workflow record.
    Include screenshots, logs, and a summary of impact.
  fields:
    - name: event_summary
      type: text
      label: Event summary
      multiline: true
      required: true
      hint: Describe what happened, when, and what services were affected
    - name: error_log
      type: file
      label: Error log file
      required: true
```

```

    hint: Attach the relevant error log from the affected service
  - name: dashboard_screenshot
    type: image
    label: Monitoring dashboard screenshot
    required: false
    hint: Screenshot showing metrics during the event
  - name: related_ticket
    type: url
    label: Related ticket or alert URL
    required: false
  approvals:
    min: 1
    roles: [approver, responder]
    timeout: 30m
    on_timeout: fail

```

After the user provides the requested evidence and approval is granted, the collected values are available as `{{ .event_summary }}`, `{{ .error_log }}`, etc. File and image attachments are stored as artifacts in the execution trace with SHA256 hashes for tamper-evidence.

Questionnaire Pattern A `collector` step with multiple `fields` is the canonical way to present a questionnaire in gert v2. All fields are presented and submitted atomically: the operator fills in the entire form and confirms once, rather than answering questions in sequential individual steps. This pattern is preferred over a series of single-field `collector` steps when the questions are logically related and should be answered together.

Benefits of the questionnaire pattern:

- Single approval gate covers all related data.
- Atomic submission — partial forms cannot leave variables in inconsistent state.
- Cleaner audit trail: one evidence entry per logical form.
- Better UX: operator sees the full context before filling in any field.

Example: Request details form (4-field form with conditional logic using `when`)

```

- step:

```

```
id: collect_request_details
type: collector
title: Request Details
prompt: |
  Complete the request details form.
fields:
  - name: severity
    type: select
    label: Severity
    required: true
    options:
      - value: low
        label: Low
      - value: medium
        label: Medium
      - value: critical
        label: Critical
    # Shown only when severity = critical (forward-reference guard)
  - name: escalation_reason
    type: text
    label: Escalation reason
    multiline: true
    required: true
    when: "severity == 'critical'"
  - name: affected_systems
    type: text
    label: Affected systems
    required: true
    # Shown for medium and critical; hidden for low
  - name: customer_impact
    type: boolean
    label: Customer-facing impact?
    when: "severity == 'medium' || severity == 'critical'"
```

Validation rules for collector

- A collector step **MUST** have at least one field in `fields`. An empty `fields` array is a schema validation error. Structural validation enforces `minItems: 1` on the `fields`

array.

- Each field **name** must be a valid identifier (alphanumeric and underscore, must start with a letter). Field names must be unique within a step.
- For **select** fields, exactly one of **options** or **options_from** must be present.
- For **number** fields, if both **validation.min** and **validation.max** are present, **min** must be $\leq$ **max**.
- For **date** and **datetime** fields, if both **validation.min** and **validation.max** are present, **min** must be chronologically $\leq$ **max**.
- A field **when** expression **MUST NOT** reference a field declared later in the same **fields** array (no forward references). Forward references are detected at parse time and are a schema validation error.
- A **required: true** field with a **when** expression is only required when the **when** condition is truthy. When **when** evaluates to falsy, **required** does not apply and the variable binding is skipped entirely.
- For **text** fields, if both **validation.min_length** and **validation.max_length** are present, **max_length** **MUST** be strictly greater than **min_length**.
- For **text** fields, **validation.pattern** **MUST** be a valid ECMA 262 regular expression; it is validated at schema load time and will cause a structural validation error if invalid.
- For **text** fields, **validation.pattern_hint** is silently ignored when **validation.pattern** is absent (warning, not error).
- For **autocomplete** fields, **options_from** **MUST** be present and **MUST** include both **provider** and **field**. **options** (static list) is not valid on **autocomplete** fields and is a schema validation error.
- For **autocomplete** fields, **options_from.min_chars** **MUST** be ≥ 0 when specified (default 2). **options_from.debounce_ms** **MUST** be ≥ 0 (default 300).
- **validation.format** is only meaningful when **multiline: true** is also set. If **format** is declared on a non-multiline field, a validation warning is emitted and the constraint is ignored at runtime.

- **validation.format:** `toml` is a validation warning in v2.0 (TOML parse validation is deferred); the runbook is accepted and the format constraint is not enforced at runtime.
- A field with **ephemeral:** `true` and **type:** `file` or **type:** `image` is a validation *warning* (not error). File payloads are large; writing `[REDACTED]` in the audit trail is acceptable, but authors should consider whether attachment evidence is appropriate alongside an ephemeral field.

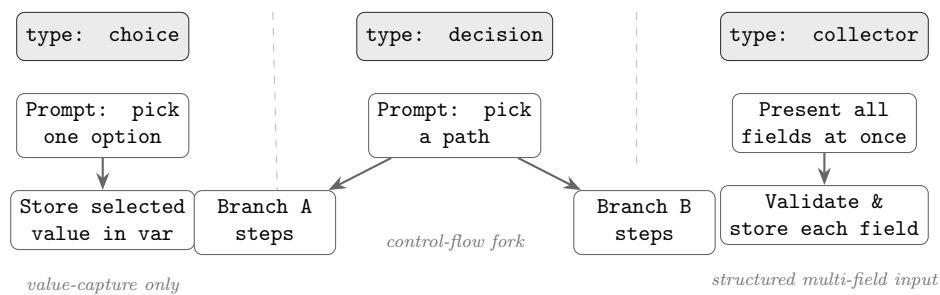


FIGURE 4.3: Human-input step triad: **choice** captures a single selection as a value; **decision** forks execution to the chosen branch; **collector** presents multiple typed fields atomically and stores all captured values.

4.4.6 Step Type: approve

An **approve** step is a dedicated, standalone approval gate that suspends execution until one or more authorized roles explicitly approve (or until the configured timeout fires). Unlike the inline **approvals** block available on **collector** and **choice** steps, **approve** carries no data-collection fields — its sole purpose is to capture an authoritative go/no-go decision and record it in the run audit trail.

Field	Type	Required	Description
<code>title</code>	string	No	Human-readable approval prompt
<code>approvals</code>	object	Yes	Approval gate config (see below)
<code>timeout</code>	string	No	Wall-clock duration (e.g. 48h); mutually exclusive with <code>timeout_business_days</code>
<code>timeout_business_days</code>	integer	No	Timeout in business days; mutually exclusive with <code>timeout</code> ; requires <code>timezone</code>
<code>timezone</code>	string	No	IANA timezone (e.g. <code>America/New_York</code>); required when <code>timeout_business_days</code> is set
<code>business_calendar</code>	string	No	Named calendar ID (e.g. <code>us-federal</code> , <code>uk-banking</code>); defaults to Mon–Fri with no holidays
<code>on_timeout</code>	string	No	<code>fail</code> , <code>continue</code> , or <code>branch:<id></code>
<code>contract</code>	object	No	Behavioral contract for governance

approvals block The `approvals` object controls who may approve and the quorum semantics:

Field	Type	Required	Description
<code>mode</code>	string	No	<code>all</code> (default) — all listed roles must approve; <code>any</code> — one approval suffices; <code>quorum</code> — M-of-N from <code>pool</code>
<code>roles</code>	string[]	No	Authorized role names; used with <code>mode: all</code> or <code>mode: any</code> ; mutually exclusive with <code>pool</code>
<code>pool</code>	string[]	No	Eligible role names for <code>quorum</code> ; used with <code>mode: quorum</code> ; mutually exclusive with <code>roles</code>
<code>required</code>	integer	No	Number of approvals required from <code>pool</code> ; used with <code>mode: quorum</code> ; must be ≥ 1 and $\leq \text{len}(\text{pool})$

Business-Day Timeout `timeout_business_days: 3` means the approval must be granted within 3 working days from the moment the `approve` step became active. The runtime counts elapsed business days using the following rules:

- The clock starts when the step enters the `waiting` state.
- Weekends (Saturday and Sunday) do not count toward the timeout.
- Holidays defined in the named `business_calendar` do not count.
- If `business_calendar` is omitted, the calendar defaults to Monday–Friday with no holidays.
- `timezone` determines the midnight boundary used for day counting; it is required whenever `timeout_business_days` is set.

If both `timeout` and `timeout_business_days` are present in the same step, schema validation **MUST** reject the runbook with a structural error before execution begins. The `on_timeout` semantics remain unchanged: `fail` terminates the run, `continue` advances to the next step, and `branch:<id>` jumps to the named branch.

```
- step:
  id: legal_review
  type: approve
  title: "Legal team approval"
  approvals:
    roles: [legal-counsel]
  timeout_business_days: 5
  timezone: "America/New_York"
  business_calendar: us-federal
  on_timeout: branch:escalate_legal
  contract:
    effects: [approval.legal]
    reads: []
    writes: []
```

Quorum Approval Setting mode: `quorum` together with `pool` and `required` enables M-of-N approval semantics:

- The runtime accepts approvals from any role listed in `pool`.
- Once the number of distinct approvals reaches `required`, the step proceeds immediately without waiting for remaining pool members.
- Each pool member may approve only once; their identity (role + principal) is recorded in the run audit trail.
- If a pool member explicitly rejects, the rejection is recorded in the audit trail. A rejection does not by itself block the step — the step is blocked only if so many members have rejected that the remaining approvers can no longer reach `required`. (For example, if `required: 2` and `pool` has 3 members and 2 reject, the step immediately fails because quorum is no longer reachable.)

```
- step:
  id: security_quorum
  type: approve
  title: "Security committee approval (2 of 5)"
  approvals:
    mode: quorum
```

```
pool: [security-lead, ciso, security-eng-1,
       security-eng-2, security-eng-3]
required: 2
timeout_business_days: 3
timezone: "Europe/London"
on_timeout: fail
contract:
  effects: [approval.security]
  reads: []
  writes: []
```

Validation Rules The following conditions are schema validation errors that **MUST** be reported before execution:

1. `timeout` and `timeout_business_days` are mutually exclusive; both present is an error.
2. `timeout_business_days` requires `timezone`; omitting `timezone` when `timeout_business_days` is set is an error.
3. `approvals.mode: quorum` requires both `approvals.pool` and `approvals.required`; either missing is an error.
4. `approvals.required` must satisfy $1 \leq \text{required} \leq \text{len}(\text{pool})$; violating this range is an error.
5. `approvals.roles` and `approvals.pool` are mutually exclusive; both present is an error.
6. `approvals.mode: all` or `mode: any` requires `approvals.roles` (not `pool`); using `pool` with these modes is an error.

4.4.7 Step Type: tool

Invokes a named tool action. The tool must be declared in `toolRefs`. The runtime dispatches to the tool's configured transport (stdio, jsonrpc, or mcp).

Field	Type	Required	Description
<code>tool.name</code>	string	Yes	Tool name (must match tool-Refs entry)
<code>tool.action</code>	string	Yes	Action declared in tool definition
<code>tool.args</code>	map	No	Argument key-value map (templates expanded)
<code>tool.version</code>	string	No	Optional version pin
<code>capture</code>	map	No	<code>stdout</code> <code>stderr</code> <code>json.<path> → var</code>

Output capture in v2 adds a `json.<path>` form for JSON-structured tool outputs, eliminating the need for follow-up `cli` steps with `jq`:

```
capture:
  pod_name: json.items[0].metadata.name
  status:  json.items[0].status.phase

- step:
  id: list_pods
  type: tool
  title: "List pods in {{ .namespace }}"
  tool:
    name: kubectl
    action: get-pods
    args:
      namespace: "{{ .namespace }}"
    output: json
  capture:
    pod_count: json.items | length
    first_pod: json.items[0].metadata.name
  contract:
    effects: [k8s.read]
    reads: [namespace]
    writes: [pod_count, first_pod]
```

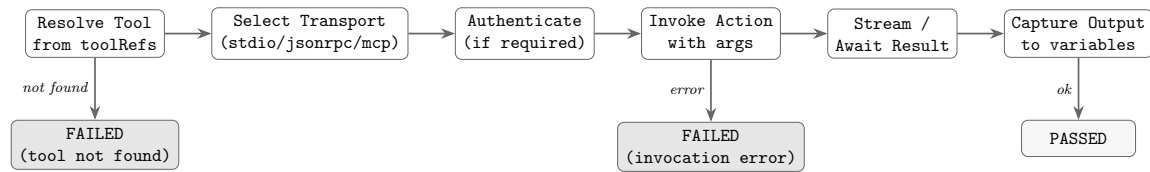


FIGURE 4.4: **type: tool** invocation flow: tool resolution, transport selection, authentication, action invocation, result streaming, and output capture.

A missing tool or invocation error terminates the step as FAILED.

4.4.8 Step Type: `wait_for_event`

Pauses runbook execution until an inbound event arrives on a declared source. Use this step type whenever a runbook must suspend and wait for an external signal before proceeding — a Prometheus alert webhook, an approval callback from a ticketing system, a SIEM event, a message-queue message, or an OS/process signal.

When the runtime reaches a `wait_for_event` step it:

1. Opens the declared inbound endpoint (HTTP listener, queue consumer, or internal channel) for the lifetime of the wait.
2. Suspends the run record in state `waiting_for_event`.
3. When a matching event arrives and passes any declared `event.filter` and `event.payload_schema` checks, it captures payload fields into run variables (via `capture`) and resumes execution at the next step.
4. If the `timeout` expires before a matching event arrives, the step behaves according to `on_timeout: fail` (default), `continue` (skip and proceed), or `branch:<step-id>` (jump to the named step).

Field	Type	Required	Description
<code>event.source</code>	string	Yes	Event source type: <code>webhook</code> , <code>message</code> , <code>signal</code> , or <code>channel</code>
<code>event.id</code>	string	No	Logical event name or channel (e.g. <code>prometheus.alert.firing</code>); templates expanded
<code>event.filter</code>	map	No	Key-value pairs that must match in the incoming event payload
<code>event.payload_schema</code>	string	No	JSON Schema URL to validate the incoming payload before capture
<code>capture</code>	map	No	<code>json.<path></code> → variable; maps event payload fields to run variables
<code>timeout</code>	string	No	Wall-clock duration (e.g. <code>30m</code> , <code>2h</code>); default: no timeout
<code>on_timeout</code>	string	No	<code>fail</code> (default), <code>continue</code> , or <code>branch:<step-id></code>

Example 1 — Prometheus alert webhook

```
- step:
  id: wait_prometheus_alert
  type: wait_for_event
  title: "Wait for Prometheus alert to fire"
  event:
    source: webhook
    id: prometheus.alert.firing
    filter:
      alertname: "HighErrorRate"
      severity: critical
  capture:
    alert_value: json.commonAnnotations.value
    alert_labels: json.commonLabels
  timeout: 30m
```



```
on_timeout: fail
contract:
  effects: []
  reads: []
  writes: [alert_value, alert_labels]
```

Example 2 — Named channel / approval callback

```
- step:
  id: wait_approval_callback
  type: wait_for_event
  title: "Wait for external approval system callback"
  event:
    source: channel
    id: "approval.{{ .request_id }}"
  capture:
    approved_by: json.approver
    decision: json.decision
  timeout: 48h
  on_timeout: branch:escalate_approval
  contract:
    effects: []
    reads: [request_id]
    writes: [approved_by, decision]
```

Transport Note The runtime is responsible for exposing the inbound endpoint; the runbook schema only declares intent. The specific endpoint URL or address is runtime-determined and available to subsequent steps as `{{ .gert.event.<id>.endpoint }}`. Event source semantics:

- **webhook** — runtime opens an HTTP POST listener; the sender POSTs a JSON body.
- **message** — runtime subscribes to a message-queue topic or queue name given by `event.id`.
- **signal** — runtime registers a named OS or process signal handler.
- **channel** — runtime uses a gert-internal named channel, enabling runbook-to-runbook or tool-to-runbook event delivery.

Security Note Webhook endpoints are authenticated per run. The runtime generates a one-time HMAC token when the endpoint is opened; the token is available as `{{ .gert.event.<id>.token }}` for use in configuring the sending system (e.g. embedding in a Prometheus alertmanager receiver URL). The token is single-use and expires when the endpoint closes (on resume or timeout).

4.4.9 Step Type: include

Expands another runbook's steps *inline* into the current run path at the point of the **include** step. This is **not** a sub-procedure call: the included runbook shares the caller's variable space entirely — no isolated scope, no input/output mapping. All included steps appear in the caller's execution trace and audit log as if they were defined inline.

The **include.with** map injects variable overrides into the shared scope before expansion; these values are immediately visible to the included steps and to any steps that follow in the caller.

Field	Type	Required	Description
<code>include.runbook</code>	string	Yes	Path or ID of the runbook to include. Relative paths resolve from the including runbook's location.
<code>include.with</code>	map	No	Variable overrides injected into the shared scope before expansion (key: variable name, value: expression). Applied in order, visible to included steps.
<code>include.when</code>	string	No	Boolean expression. If false, the include is skipped entirely (no steps expanded).

```
- step:
  id: run_db_checks
  type: include
  title: "Run database pre-flight checks"
  include:
    runbook: ./shared/db-preflight.runbook.yaml
    with:
      db_host: "{{ .target_db }}"
```

```

    check_mode: read-only
contract:
  effects: [db.read]
  reads: [target_db]
  writes: []

```

Cycle Detection The runtime builds a runbook dependency graph at load time by resolving all `include.runbook` references transitively. A cycle exists when a runbook directly or transitively includes itself. Cyclic includes are a hard validation error; the runtime rejects the runbook set and reports the full include chain:

```
cyclic include detected: A -> B -> A
```

The validator reports every edge in the offending cycle so the operator can identify the exact chain.

Difference from Sub-Procedure Call `type: include` shares the caller’s variable space; there is no input mapping and no output extraction. The included steps appear inline in the run trace — there is no sub-run, no separate audit entry, and no scope boundary.

If isolation *is* needed (separate variable scope, explicit input/output mapping), that is the responsibility of a future step type (`type: call` or `type: delegate`). That capability is **not in v2.0 scope**.

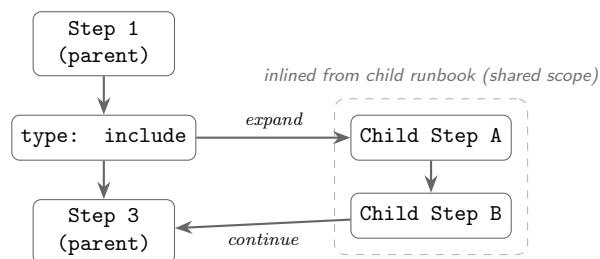


FIGURE 4.5: `type: include` inlines the child runbook’s steps into the parent’s execution path. All included steps share the parent variable scope; no sub-run boundary is created.

4.4.10 Step Type: branch

Evaluates a list of conditions and executes the first matching arm. In v2, **branch** is an explicit first-class step type (as well as the inline **branches:** sibling on any step node).

Field	Type	Required	Description
branches	array	Yes	List of Branch objects
branches[].condition	string	Yes	Go template evaluating to true/false
branches[].label	string	No	Display label for diagram/trace
branches[].steps	array	No	Steps to execute if condition matches

At most one branch arm executes. If no arm matches and no arm has an **else: true** marker, execution continues past the branch node with no steps run. The **else: true** marker (new in v2) designates a catch-all arm:

```
- step:
  id: route_on_env
  type: branch
  title: Route by environment
  branches:
    - condition: 'environment == "production"'
      label: Production path
      steps:
        - step:
            id: require_approval
            type: manual
            title: Require approver confirmation
            approvals:
              min: 1
              roles: [approver]
    - condition: 'environment == "staging"'
      label: Staging path
      steps:
        - step:
            id: log_staging
```

```

    type: cli
    title: Log staging deploy
    command: echo
    args: ["Deploying to staging"]
- else: true
  label: Unknown environment - abort
  steps:
    - step:
      id: abort
      type: end
      title: Unknown environment
      outcome:
        category: escalated
        code: unknown_env

```

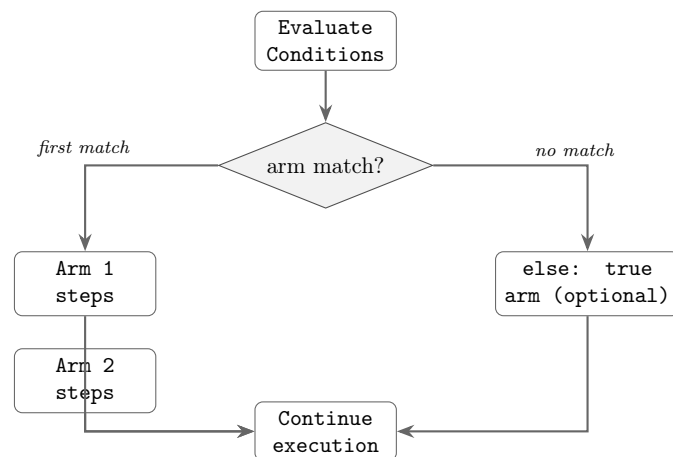


FIGURE 4.6: `type: branch` evaluates each arm's condition in order and executes the first matching arm. At most one arm runs. If no arm matches and no `else` arm is defined, execution continues with no steps run.

4.4.11 Step Type: `iterate`

Repeats a block of steps over a collection or until a convergence condition is met. In v2 `iterate` is a `TreeNode` sibling (not a step type in the `step.type` enum) — it appears at

the tree level alongside `step` nodes.

Field	Type	Required	Description
<code>iterate.over</code>	string	*	Template resolving to comma-separated list
<code>iterate.as</code>	string	No	Loop variable name (default: <code>item</code>)
<code>iterate.max</code>	int	*	Max iterations (convergence mode)
<code>iterate.until</code>	string	*	Convergence condition expression
<code>iterate.collect</code>	map	No	Accumulate captures across passes
<code>iterate.steps</code>	array	Yes	Steps to execute each pass

* `over` or (`max` + `until`) required; mutually exclusive.

The loop variable `{{ .iteration }}` (0-indexed) is always available. In list mode, `{{ .item }}` (or the `as` name) holds the current element.

```
# List iteration
- iterate:
  id: check_all_services
  over: "{{ .service_list }}" # e.g. "api,worker,scheduler"
  as: svc
  collect:
    results: "{{ .svc }}:{{ .http_result }}"
  steps:
    - step:
      id: probe
      type: tool
      title: "Check {{ .svc }}"
      tool:
        name: curl
        action: head
        args:
          url: "https://{{ .svc }}.internal/healthz"
      capture:
```

```
        http_result: stdout

# Convergence iteration
- iterate:
  id: wait_for_deploy
  max: 10
  until: 'operation_status == "Running"'
  steps:
    - step:
      id: poll_deploy
      type: tool
      title: "Poll operation (pass {{ .iteration }})"
      tool:
        name: kubectl
        action: get-deploy-status
        args:
          name: "{{ .operation }}"
      capture:
        operation_status: json.status.phase
    - step:
      id: wait
      type: cli
      title: Wait 15s
      command: sleep
      args: ["15"]
```

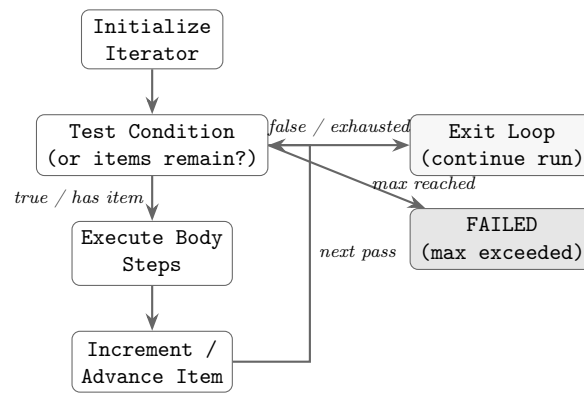


FIGURE 4.7: **type: iterate** loop: the condition (items remaining or `until` expression) is tested before each pass. The body executes, the iterator advances, and the loop repeats until the condition is false, items are exhausted, or `max` iterations are reached.

4.4.12 Step Type: parallel

Executes a set of independent step groups concurrently, then joins. New in v2, addressing the fan-out/fan-in gap identified in Dennis's research brief.

Field	Type	Required	Description
<code>branches</code>	array	Yes	Each branch is an independent group
<code>join.wait_for</code>	enum	No	<code>all</code> <code>any</code> <code>majority</code> (default: <code>all</code>)
<code>join.on_failure</code>	enum	No	<code>fail</code> <code>continue</code> (default: <code>fail</code>)

Parallel branches are isolated from each other's variable writes during execution. After the join, all `capture` values from all branches are merged into the caller's variable map. Write conflicts (two branches writing the same variable) are resolved by last-writer-wins and a warning is emitted.

```

- step:
  id: parallel_checks
  type: parallel

```



```
title: Run DNS and HTTP checks concurrently
branches:
  - label: DNS check
    steps:
      - step:
          id: dns_check
          type: tool
          tool:
            name: nslookup
            action: lookup
            args:
              host: "{{ .hostname }}"
          capture:
            dns_result: stdout
  - label: HTTP check
    steps:
      - step:
          id: http_check
          type: tool
          tool:
            name: curl
            action: head
            args:
              url: "https://{{ .hostname }}/healthz"
          capture:
            http_result: stdout
join:
  wait_for: all
  on_failure: fail
```

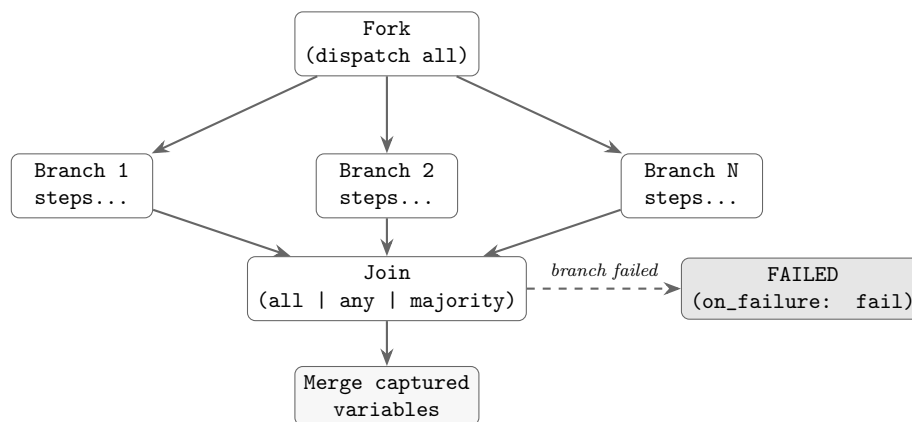


FIGURE 4.8: **type: parallel** fan-out/fan-in: all branches are dispatched concurrently. The join node waits for the configured quorum (**all**, **any**, or **majority**), then merges captured variables. A branch failure triggers the **on_failure** policy.

4.4.13 Step Type: assert

Evaluates a list of assertions against captured variables. Fails the run if any assertion does not pass.

```

- step:
  id: verify_deployment
  type: assert
  title: Verify operation succeeded
  assert:
    - type: contains
      subject: "{{ .deploy_output }}"
      expected: "successfully rolled out"
    - type: eq
      subject: "{{ .exit_code }}"
      expected: "0"
    - type: json_path
      subject: "{{ .api_response }}"
      path: "$.status"
      expected: "active"

```

Assertion types: `contains`, `not_contains`, `matches` (regex), `eq`, `ne`, `lt`, `gt`, `exit_code`, `json_path`.

4.4.14 Step Type: `compensate`

New in v2. Registers a compensation (rollback) action that is executed in reverse order if the run fails or is cancelled after this step. Implements the saga pattern for long-running operational transactions.

Field	Type	Required	Description
<code>compensate.on</code>	enum	No	<code>failure</code> <code>cancel</code> <code>any</code> (default: <code>failure</code>)
<code>compensate.steps</code>	array	Yes	Steps to execute as compensation

The compensation steps execute in LIFO order relative to the sequence of `compensate` nodes encountered during the forward execution pass. Compensation steps share the same variable context as the point at which the `compensate` node was registered.

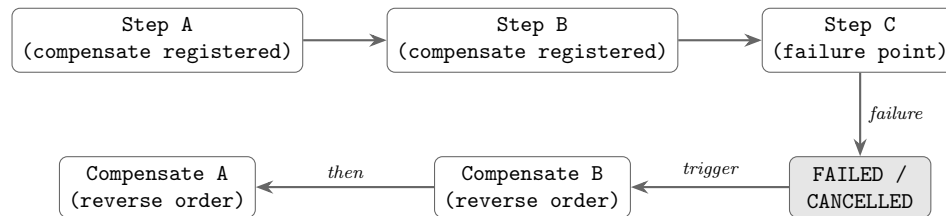
```
- step:
  id: scale_down
  type: tool
  title: Scale down service
  tool:
    name: kubectl
    action: scale
    args:
      operation: "{{ .operation }}"
      replicas: "0"

- step:
  id: register_scale_up
  type: compensate
  title: Compensation - scale back up on failure
  compensate:
    on: failure
    steps:
      - step:
```

```

id: scale_up_comp
type: tool
title: Restore service scale
tool:
  name: kubectl
  action: scale
  args:
    operation: "{{ .operation }}"
    replicas: "{{ .original_replicas }}"

```



saga pattern: undo in reverse registration order

FIGURE 4.9: `type: compensate` saga flow: each compensation step is registered during forward execution. On failure or cancellation, registered compensations fire in reverse order, implementing the saga pattern for long-running operational transactions.

4.4.15 Step Type: end

Declares a terminal outcome and halts execution. Renamed from `router` in v1.

```

- step:
  id: resolved
  type: end
  title: Workflow complete
  outcome:
    category: resolved      # resolved / escalated / no_action / needs_rca
    code: dns_healthy
    meta:
      resolution_time: "{{ .elapsed }}"

```

4.5 Tool Definition Schema (v2)

Tool definitions describe callable tools: their actions, argument schemas, transport, and behavioral contract. In v2 the `tool/v0` version is replaced by `tool/v2`.

4.5.1 Tool Definition Field Inventory

```
$schema: "https://schemas.gert.dev/tool/v2.json"
apiVersion: tool/v2

meta:
  name: kubectl
  version: "1.2"
  description: Kubernetes CLI wrapper
  binary: kubectl

transport:
  mode: stdio          # stdio | jsonrpc | mcp
  # jsonrpc options:
  # start: ["node", "server.js"]
  # port: 3000
  # mcp options:
  # start: ["python", "mcp_server.py"]

actions:
  get-pods:
    description: List pods in a namespace
    argv: ["get", "pods", "-n", "{ .namespace }", "-o", "json"]
    args:
      namespace:
        type: string
        required: true
        description: Kubernetes namespace
    output:
      type: string
      required: false
      default: json
    capture:
```

```
    stdout:
      format: json    # json / text / lines
  contract:
    effects: [k8s.read]
    reads: [namespace]
  governance:
    action: allow
```

New in v2

- `capture.stdout.format` — declares the output format; enables structured `json.<path>` capture in runbook steps.
- `actions.<name>.contract` — per-action behavioral contract automatically merged with any step-level contract override.
- `actions.<name>.governance` — per-action governance policy; overrides the runbook-level rules for this action only.
- `meta.version` — explicit tool version for auditing and pinning.

4.6 Provider Definition Schema (v2)

Provider definitions declare external input resolution services. In v2 the `provider/v0` version is replaced by `provider/v2`.

```
$schema: "https://schemas.gert.dev/provider/v2.json"
apiVersion: provider/v2

meta:
  name: tracking
  version: "1.0"
  description: Issue tracking context provider
  kind: input-provider    # input-provider / enrichment-provider

transport:
  mode: jsonrpc
  start: ["/providers/issue-tracker-provider"]
```

```
fields:
  id:
    type: string
    description: Active request ID
  title:
    type: string
    description: Request title
  severity:
    type: string
    enum: [P1, P2, P3, P4]
  affected_service:
    type: string
    description: Primary impacted service name

capabilities:
  - input-resolution
```

The `fields` map defines the provider's schema. Each field name corresponds to a `from: <provider>.<field>` binding in a runbook input. The semantic validator checks that all `from: <provider>.*` bindings reference a field declared in a loaded provider definition.

4.7 Namespace Convention for Extensions

4.7.1 The Extension Field Problem

Strict JSON Schema validation (`additionalProperties: false`) is a core v2 design constraint: unknown fields in a runbook must be rejected, not silently ignored. At the same time, extensions must be able to attach metadata to steps, runbooks, and tool definitions without forking the core schema.

4.7.2 Convention: `x-<namespace>`: at the Field Level

v2 adopts the `x-<namespace>` prefix convention used by OpenAPI and GitHub Actions. Extension fields must:

1. Start with `x-`.

2. Be followed by a namespace component (lowercase alphanumeric and hyphens, matching `[a-z][a-z0-9-]+`).
3. Be declared in an **extension manifest** registered via `gert extension register` or in `gert.yaml`.

At the runbook level

```
$schema: "https://schemas.gert.dev/runbook/v2.json"
apiVersion: runbook/v2
id: provision-service
name: Provision Service

x-myorg-cost-center: "engineering-ops"
x-myorg-change-id: "CHG-20260418-001"
x-otel-service-name: "payment-service"
```

At the step level

```
- step:
  id: provision
  type: tool
  title: Provision to environment
  x-myorg-risk-level: high
  x-myorg-cleanup-id: "{{ .provision_id }}"
```

4.7.3 Extension Validation

1. **Unregistered extension fields** — The JSON Schema validator accepts any field matching `^x-[a-z][a-z0-9-]+$` without requiring a registered schema. This enables zero-friction annotation. The `gert validate` command emits a **WARNING** for each unregistered extension field.
2. **Registered extension fields** — An extension may ship a JSON Schema fragment that constrains its field values. Once registered, the validator applies the fragment and reports violations as errors, not warnings.
3. **Unknown non-extension fields** — Any field that does not match a core field name and does not start with `x-` is a hard validation error (`additionalProperties: false`).

4.7.4 Extension Registration Format

```
# .gert/extensions/myorg.yaml
apiVersion: extension/v1
namespace: myorg
description: MyOrg internal runbook annotations

fields:
  cost-center:
    type: string
    description: Cost center code for billing
    pattern: "^[a-z]+-[a-z]+"
  risk-level:
    type: string
    enum: [low, medium, high, critical]
  change-id:
    type: string
    pattern: "^CHG-[0-9]{8}-[0-9]{3}$"
  cleanup-id:
    type: string

applies-to: [runbook, step]
```

The extension schema is merged into the live JSON Schema when the extension is registered, enabling IDE auto-completion for extension fields as well.

4.8 Structural vs. Semantic Validation

v2 validation runs in two sequential phases. Both must pass for a runbook to be considered valid.

4.8.1 Phase 1: Structural Validation

Structural validation is pure JSON Schema Draft 2020-12 evaluation. It checks:

- **Required fields** present (id, name, apiVersion, flow).
- **Field types** correct (string vs. integer vs. object vs. array).

- **Enum values** valid (e.g. `kind` must be one of the four defined values).
- **Pattern constraints** satisfied (e.g. `id` must match slug pattern; `timeout` must match `[0-9]+(s|m|h)`).
- **minItems / minLength** constraints met (e.g. `flow` must have at least one node; `choices.options` must have at least two options).
- **additionalProperties** — no unknown core fields present.

Structural validation is performed by the generated JSON Schema artifact (see §4.9). It is stateless: it does not load tool definitions, resolve imports, or evaluate templates.

Implementation note for Brian The Go runtime uses the `invopop/jsonschema` library to generate the JSON Schema from struct tags at startup (no static file required), then validates the parsed YAML document against it. The `yaml.v3 KnownFields(true)` call provides an additional pre-JSON-Schema pass that rejects unknown fields before the struct is even unmarshalled.

4.8.2 Phase 2: Semantic Validation

Semantic validation enforces domain rules that cannot be expressed in JSON Schema. It runs after structural validation and requires the full runbook AST and loaded tool/provider definitions. Semantic checks include:

1. **Step ID uniqueness** — All step IDs in the flat walk of the tree must be unique within the runbook.
2. **Include target resolution** — Every `include.runbook` value must resolve to a relative file path that exists on disk.
3. **Branch condition variable references** — Each `condition` template references only variables that are in scope at that point in the execution tree (declared as `inputs`, `vars`, or captured by an earlier step on a reachable path).
4. **Tool reference resolution** — Every `tool.name` in a tool step must match a declared `toolRefs` entry, and the named `.tool.yaml` file must be loadable and structurally valid.

5. **Tool action existence** — The `tool.action` must be declared in the referenced tool definition.
6. **Provider binding resolution** — Every `inputs.*.from: <provider>.<field>` must resolve to a loaded provider whose field manifest declares that field name.
7. **Cyclic include detection** — The include graph must be a DAG; any cycle is a semantic error reported as `cyclic include detected: A → B → A`.
8. **Capture write-before-read** — A step that reads a captured variable via a template must have a reachable predecessor step that writes it; a warning is emitted if the write is only on some branches.
9. **Governance policy pre-check** — The static governance rules are evaluated against the step contracts declared in the runbook. Any step with an effect that would trigger `deny` is flagged as a semantic error. Steps that would trigger `require-approval` are flagged as warnings.
10. **Output declaration completeness** — Every declared `output` must reference a capture name that is written by at least one step.
11. **Compensate ordering** — Each `compensate` step must appear after the step it is compensating for in tree walk order.

Error reporting Both validation phases produce structured error reports with:

- Error code (e.g. SCH-001, SEM-007)
- File path and YAML line/column (where available)
- Human-readable message
- Suggested fix (where computable)

```
$ gert validate service-health.runbook.yaml
[SEM-003] step "check_http": tool action "head" not declared in
          tools/curl.tool.yaml (declared actions: get, post, download)
          --> service-health.runbook.yaml:34:7
[SEM-007] step "eval": template references variable "dns_result" which
```

```

is only written on branch "DNS check" - may be unbound
--> service-health.runbook.yaml:51:12
hint: add a default value in vars: or guard with {{ if .dns_result }}

```

4.9 Schema Artifacts

4.9.1 Shipped Artifacts

gert v2 ships the following schema artifacts:

Artifact	Description
runbook.v2.schema.json	JSON Schema Draft 2020-12 for runbook documents
tool.v2.schema.json	JSON Schema for tool definitions
provider.v2.schema.json	JSON Schema for provider definitions

All three schemas are generated at runtime from Go struct tags via `invopop/jsonschema` and served by `gert schema export`. They are also embedded in the binary and served at <https://schemas.gert.dev/> for IDE tooling.

4.9.2 gert schema export

```

# Export runbook schema (default)
gert schema export

# Export specific schema
gert schema export --kind runbook    # runbook / tool / provider / bundle

# Export full bundle (all three)
gert schema export --kind bundle > gert-v2-schemas.json

# Validate a document against the exported schema
gert schema export | check-jsonschema --schemafile - runbook.yaml

```

The bundle format is a JSON object with three keys: `runbook`, `tool`, and `provider`.

4.9.3 VS Code Integration

The gert VS Code extension configures schema-backed YAML validation via the `yaml-language-server` extension:

```
# .vscode/settings.json (auto-generated by gert vscode init)
{
  "yaml.schemas": {
    "https://schemas.gert.dev/runbook/v2.json": "**/*.runbook.yaml",
    "https://schemas.gert.dev/tool/v2.json":    "**/*.tool.yaml",
    "https://schemas.gert.dev/provider/v2.json": "**/*.provider.yaml"
  }
}
```

Because the schema URL matches the `$schema` field embedded in each document, IDEs with built-in JSON Schema support (VS Code 1.80+, IntelliJ 2024+) pick up validation and auto-completion without any per-project configuration.

The gert VS Code extension additionally provides:

- Step-type-aware field completion via the `schema/stepFields` JSON-RPC method (returns the field set for a given `type` value).
- Tool argument completion via `schema/toolArgs` (returns the `args` schema for a given tool+action pair from the live tool catalog).
- Inline governance warnings derived from `exec/dryRun` pre-check.

4.9.4 Schema Version Manifest

Each schema artifact carries a version manifest in its `$id` and a `x-gert-version` annotation:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://schemas.gert.dev/runbook/v2.json",
  "x-gert-version": "2.0.0",
  "x-gert-released": "2026-04-18",
  "title": "gert runbook v2",
  ...
}
```

Automated compatibility testing (part of the gert CI pipeline) runs every v1 example runbook through the v2 compatibility parser and asserts no regression, and runs every v2 example through the exported JSON Schema validator.

Chapter 5

Domain Kit Model

5.1 Purpose and Scope

The gert v2 core runtime is deliberately domain-agnostic. It executes steps, enforces governance, captures evidence, and writes traces—but it carries no opinion about what kind of work a runbook describes. This is a feature, not an omission: a domain-agnostic kernel is the prerequisite for a stable, long-lived runtime contract.

Domain Kits are the mechanism by which domain-specific semantics enter the system without contaminating the kernel. A Domain Kit is a higher-level semantic layer built atop core runtime primitives. It provides five things:

1. **Authoring schemas.** A Kit defines a domain-specific YAML vocabulary—specialised step types, field structures, validation constraints—that is natural for authors working in that domain. An workflow Kit might offer a **select-priority** step type with severity and escalation fields; a compliance Kit might offer a **approval-request** type with approver-matrix fields.
2. **Validation rules.** A Kit contributes domain-specific semantic validators that run at parse time or plan time. These validators enforce invariants that the core schema cannot express: “a cleanup step must follow every provision step,” “every runbook must declare an owner,” “escalation timeout must not exceed configured limit.”
3. **Lowering (compilation into core primitives).** Kit-specific step types do not reach the runtime. A Kit compiler transforms (lowers) its domain-specific AST nodes into sequences of core step types (**cli**, **tool**, **manual**, **branch**, **iterate**) before the planner produces an ExecutionPlan. The runtime executes only core primitives; it is never aware that a Kit was involved.

4. **Projections and read models.** A Kit may define read-only projections over run traces. A compliance Kit might project an “audit log” view from the raw trace events; a workflow Kit might project a “timeline” view. Projections are computed after execution; they do not affect runtime behaviour.
5. **Testing contracts.** A Kit defines what “correct” means for its domain: fixture runbooks, expected lowered output, expected validation errors, expected projection shapes. These contracts are the Kit’s acceptance test suite.

Domain Kits do not contribute runtime capabilities. They do not register tools, subscribe to events, or modify execution behaviour. Those are extension concerns (§6). A Kit operates entirely at the authoring and analysis layer—it shapes what authors write and what reviewers see, not what the engine does.

5.2 Architectural Narrative

The v2 architecture draws a hard boundary between three concerns: what the engine executes (core runtime), what capabilities are available at runtime (extensions and tools), and what vocabulary authors use to express intent (Domain Kits). The first two are well-defined in the existing design; the third has been implicit until now.

Without Domain Kits, domain semantics have two bad places to live. They can leak into the core schema, which makes the kernel unstable—every new domain (compliance audits, workflow automation, data migration, system provisioning) adds fields and step types to the core, bloating it into an ever-expanding union type. Or they can be pushed into extensions, which conflates authoring concerns with runtime concerns—an extension that contributes domain-specific validators, authoring schemas, and projections is doing fundamentally different work than an extension that contributes tools or event subscriptions.

Domain Kits solve this by providing a clean, well-defined home for domain semantics. The design principle is:

Core owns execution. The core runtime (§3), governance layer (§??), evidence/tracing system (§??), and event model (§8) are stable, domain-agnostic infrastructure.

Kits own vocabulary. Authoring schemas, domain validation rules, lowering/compilation, projections, and testing contracts belong to Kits. A Kit is the right place to encode “what does a well-formed workflow runbook look like?” or “what fields does an approval-request step require?”

Extensions own capabilities. Tool registration, event subscription, schema extension (namespaced `x-*` fields), policy contribution, and provider registration belong to extensions (§6).

Tools own effects. Executable actions—invoking binaries, calling APIs, performing side-effecting work—belong to the tool runtime (§7).

This separation preserves the kernel’s stability guarantee: the core schema and runtime contract do not change when a new domain is added. The Kit compiles domain-specific vocabulary down to core primitives; the runtime never sees Kit-specific nodes.

5.3 The Clean Kernel Principle

The key architectural invariant is: **the runtime never executes Kit-specific step types**. All domain-specific authoring nodes are lowered (compiled) into core primitives before the planner sees them.

The compilation pipeline is:

Kit YAML → Kit Parser → Kit AST → Kit Compiler → Core YAML → Core Parser → ExecutionPlan

Or equivalently, when integrated into the gert pipeline:

1. The user authors a runbook using Kit vocabulary (e.g., `apiVersion: runbook/v2+ops/v1`).
2. The Kit’s parser validates domain-specific fields and produces a Kit-level AST.
3. The Kit’s compiler lowers the AST into a standard `runbook/v2` document using only core step types.
4. The core parser and planner process the lowered document normally.
5. The runtime executes the plan with no knowledge that a Kit was involved.

This is a compile-time transformation, not a runtime interception. The lowered output is a valid `runbook/v2` document that can be inspected, version-controlled, and executed without the Kit installed. This property is critical for portability: a Kit-authored runbook can always be reduced to its core equivalent.

The Kit compiler must be deterministic: the same Kit input must always produce the same core output. This ensures that `gert test` scenarios written against lowered output remain stable across Kit versions.

5.4 Layer Relationships

The v2 system has four distinct layers. Each layer has a defined responsibility boundary and a defined relationship to the others.

TABLE 5.1: Four-layer responsibility matrix

Layer	Owens	Does NOT own	Runs at
Core Runtime	Execution, state machine, governance enforcement, evidence, event emission, trace writing	Domain vocabulary, tool implementations, UI rendering	Run time
Domain Kits	Authoring schemas, domain validation, lowering/compilation, projections, testing contracts	Execution, runtime capabilities, side effects	Author time / compile time
Extension Runtime	Runtime capabilities: tool registration, event subscription, schema extension (x-*), policy contribution, provider registration	Authoring vocabulary, domain semantics, execution control	Run time (out-of-process)
Tool Runtime	Side-effecting actions: binary execution, API calls, MCP tool invocation	Governance, evidence, schema, authoring vocabulary	Run time (per-invocation or persistent process)

Key distinctions:

Domain Kits are NOT extensions. Extensions contribute runtime capabilities via the capability grant model (§6.2). Kits contribute authoring semantics via compilation. An extension runs as a child process during execution; a Kit runs as a compiler before execution. They serve fundamentally different purposes and operate at different phases.

Domain Kits are NOT tools. Tools perform side-effecting work. Kits are pure transformations with no side effects. A Kit compiler reads YAML and writes YAML; it never invokes binaries, calls APIs, or modifies system state.

Extensions may accompany a Kit but are distinct. A Kit distribution (e.g., a “compliance workflow pack”) may ship both a Domain Kit (authoring schemas, lowering rules) and an extension (workflow-specific tools, PagerDuty providers). These are bundled for convenience but are architecturally separate: the Kit operates at compile time, the extension operates at run time.

5.5 Kit Manifest and Compatibility

A Domain Kit ships a `gert-kit.yaml` manifest alongside its compiler. The manifest declares identity, compatibility, and the Kit’s schema contributions.

```
# gert-kit.yaml - Domain Kit Manifest
apiVersion: kit/v1

meta:
  name: gert.workflow           # Reverse-DNS namespaced identifier
  version: "1.0.0"              # Semver
  description: "Workflow automation authoring kit"
  author: "Gert Project <gert@ormasoft.dev>"

compatibility:
  gert-core: ">=2.0.0 <3.0.0"  # Minimum gert core version (semver range)

schema:
  apiVersion-suffix: "workflow/v1" # Runbooks authored with this Kit use
                                     # apiVersion: runbook/v2+workflow/v1
  json-schema: "./schemas/workflow-v1.json" # Kit-specific JSON Schema overlay

compiler:
  executable: "./bin/gert-kit-workflow" # Kit compiler binary
  args: ["compile"]
  input-format: yaml                    # Kit YAML in, core YAML out
  output-format: yaml

step-types:                            # Domain-specific step types this Kit
→ defines
  - name: approval-request
```

```

    lowers-to: [manual, tool]           # Core types this compiles into
- name: cleanup
  lowers-to: [cli, branch]
- name: select-priority
  lowers-to: [manual, branch]

validators:                             # Domain-specific validation rules
- name: workflow/cleanup-follows-provision
  phase: plan                           # parse / plan
  description: "Every provision step must be followed by a cleanup step"
- name: workflow/owner-required
  phase: parse
  description: "Every workflow runbook must declare an owner in meta.roles"

projections:                            # Read models over trace data
- name: workflow/audit-log
  description: "Workflow timeline derived from trace events"
- name: workflow/evidence-summary
  description: "Audit-ready evidence summary grouped by governance primitive"

```

Compatibility and versioning rules.

- A Kit declares a semver range for the minimum gert core version in `compatibility.gert-core`. If the host version falls outside this range, the Kit is rejected at compile time with a structured error.
- Breaking changes to a Kit's authoring schema require a Kit major version bump. The `apiVersion-suffix` encodes the Kit's major version (e.g., `workflow/v1` → `workflow/v2`). This is independent of the gert core version.
- Additive changes (new optional fields, new step types) within a Kit major version are backward-compatible and do not require a version bump to the suffix.
- The Kit's `json-schema` artifact carries a semver artifact version for minor/patch tracking, following the same pattern as core schema artifacts (§4.1.3).

Local-first constraints. Domain Kits are portable semantic layers. They run on any supported gert host—a local laptop, a CI runner, a self-hosted server—without requiring

network access, cloud services, or central registries. A Kit is a directory containing a manifest, a compiler binary, and schema artifacts. It can be vendored into a repository, distributed as a tarball, or installed from a package manager. There is no Kit registry service, no Kit telemetry, and no phone-home behaviour. This aligns with gert’s local-first design philosophy (§??).

5.6 Extension Runtime Audit

The following table audits concerns currently described in the extension runtime chapter (§6) and evaluates whether each is properly an extension concern, a Domain Kit concern, or shared.

Concern	Current location	Recommendation	Rationale
Domain-specific validators	Implicit in <code>capability/schema-extension</code>	Move to Kit.	Validation rules that enforce domain invariants (e.g., “cleanup must follow provision”) are authoring-time concerns. They belong in the Kit’s validation phase, not in a runtime extension. The extension <code>capability/schema-extension</code> should be reserved for namespaced field contributions (<code>x-*</code>), not for semantic validation of domain workflows.
Authoring schema registration	<code>capability/schema-extension</code>	Clarify boundary.	Extensions contribute namespaced runtime-observable fields (<code>x-acme.*</code>). Kits contribute domain-specific authoring vocabulary (new step types, required field structures). These are different operations. Add a clarifying note to §6.2 that <code>capability/schema-extension</code> is for runtime-observable namespace extensions, not for authoring-vocabulary contributions.

Continued on next page

Concern	Current location	Recommendation	Rationale
Projections / read models	Not currently addressed	Kit concern.	Projections are pure, read-only transformations over trace data. They are computed after execution and do not require runtime capabilities. They belong in the Kit layer. Extensions that need to observe execution should use <code>capability/event-subscription</code> ; post-hoc analysis belongs to Kits.
Compilation hooks	Not currently addressed	Kit concern.	The lowering/compilation step is a pure transformation that happens before the runtime starts. It has no runtime footprint and requires no capabilities. Compilation is entirely within the Kit layer.
Domain testing fixtures	Not currently addressed	Kit concern.	Test fixtures for domain-specific authoring (example runbooks, expected lowered output, expected validation errors) are Kit deliverables. They do not interact with the extension runtime.
Tool registration	<code>capability/tool-registration</code>	Stays in extension runtime.	Tool registration is a runtime capability. A Kit distribution may ship an accompanying extension that registers domain-specific tools, but tool registration itself remains an extension concern.
Policy contribution	<code>capability/policy-contribution</code>	Stays in extension runtime.	Policy rules that are evaluated at run time (pre-step, pre-run) are runtime concerns. Domain-specific validation rules that are evaluated at parse/plan time belong to Kits; runtime policy enforcement belongs to extensions.
Event subscription	<code>capability/event-subscription</code>	Stays in extension runtime.	Event observation is inherently a runtime activity. No change needed.

Summary of changes needed in existing sections.

1. Add a clarifying paragraph to **capability/schema-extension** (§6.2) noting the boundary: extensions contribute namespaced runtime fields; Kits contribute authoring vocabulary and domain step types.
2. Add a forward reference from the extension runtime chapter to the Domain Kit chapter for readers who arrive at the extension spec looking for domain-specific validation or projections.
3. No capabilities need to be moved or removed from the extension taxonomy. The taxonomy is correct for runtime concerns; the issue was that domain/authoring concerns had no defined home. Domain Kits provide that home.

5.7 Domain Kit Examples

The Domain Kit model enables multiple domain-specific vocabularies to coexist without contaminating the gert core. Below are representative examples of how different domains might structure their kits.

Reference implementations. The gert project maintains several reference Domain Kits to demonstrate the model in practice. These are NOT part of gert core, but are distributed separately as example implementations:

gert.workflow — General workflow automation kit with approval-request, cleanup, and select-priority step types. Demonstrates basic Kit compiler structure.

gert.compliance — Compliance and audit-focused kit with evidence collection, audit-log projections, and policy enforcement patterns.

gert.migration — Data migration kit with rollback semantics, checkpoint/restart, and validation step types.

gert.ops — Operations runbook kit implementing the DRI (Directly Responsible Individual) model for incident response and change management. See the *DRI Domain Kit Manual* for details.

The architectural principle remains firm: **gert core contains zero domain-specific vocabulary**. All step types in the core schema (`cli`, `manual`, `tool`, `branch`, `iterate`) are domain-agnostic execution primitives. Domain semantics live exclusively in separately-distributed Kits.

For detailed Kit implementation guides, see the *Domain Kit Development Guide* (separate document).

5.8 Non-Goals

Domain Kits are a precisely scoped mechanism. The following are explicitly not in scope:

Kits are not extensions. A Kit does not contribute runtime capabilities. It does not register tools, subscribe to events, modify execution behaviour, or run as a child process during execution. The extension capability taxonomy (§6.2) does not apply to Kits.

Kits are not tools. A Kit does not perform side-effecting work. It does not invoke binaries, call APIs, or modify system state. The Kit compiler is a pure function: YAML in, YAML out.

Kits are not plugins in a generic sense. Gert does not have a “plugin system.” It has three distinct extension points—Domain Kits (authoring semantics), extensions (runtime capabilities), and tools (effectful actions)—each with a different contract, lifecycle, and trust model. Calling any of these “plugins” obscures the architectural boundaries.

Kits are not templates or examples. A Kit is not a collection of example runbooks. It is a semantic layer: schemas, validators, a compiler, projections, and testing contracts. An example runbook may demonstrate a Kit’s vocabulary, but the Kit itself is the machinery that makes that vocabulary valid and executable.

Kits do not own execution. The core runtime is the sole authority over step advancement, state management, governance enforcement, evidence capture, and trace writing. A Kit may influence what the runtime executes (via lowering), but it never participates in how execution proceeds.

Kits do not replace the core schema. The core `runbook/v2` schema remains the stable contract between authors, the parser, and the runtime. Kit schemas are overlays that compile down to core; they do not bypass or replace it.

Kits are not cloud-native constructs. Consistent with gert’s local-first philosophy, Kits do not require network access, remote registries, or cloud services. A Kit is a local artifact—a directory with a manifest, a compiler, and schemas—that runs wherever gert runs.

Chapter 6

Extension Runtime

The gert v2 extension runtime defines how external packages contribute tools, schema fields, governance policies, and input providers to a running gert host without requiring changes to the host binary. Extensions run as child processes; they communicate with the host over JSON-RPC 2.0 and are governed by an explicit capability grant model.

A normative reference specification is maintained at `specs/002-extension-runtime-v0/spec.md` in the repository. This chapter derives from and supersedes that document for design purposes; in any conflict between the two, the specification takes precedence for implementation.

6.1 Architecture Overview

1. The gert host discovers extension manifests on startup.
2. The host evaluates compatibility and capability policy before launching any process.
3. Each accepted extension is started as a child process; its executable communicates with the host over its declared transport.
4. The host and extension perform a versioned handshake. The host sends the set of capabilities it is willing to grant; the extension confirms receipt.
5. The extension registers its contributions (tools, schema fields, policies, providers).
6. For the remainder of the run the host dispatches invocations to the extension and enforces all capability constraints.
7. At run completion the host sends a graceful shutdown signal.

6.2 Capability Taxonomy

Every operation an extension may perform is gated by a named capability. Capabilities are requested in the manifest and granted (or denied) by host policy before the process is started. The initial capability set is given in Table 6.1.

The capability string format is `capability/<name>`. Future versions may introduce scoped variants using path notation, e.g. `capability/file-read:logs/**`.

6.3 Extension Manifest Format

Each extension ships a `gert-extension.yaml` file alongside its executable. The manifest declares identity, requested capabilities, transport, and compatibility constraints.

```
# gert-extension.yaml - Extension Manifest
apiVersion: extension/v2

meta:
  name: acme.incident-pack           # Reverse-DNS namespaced identifier
  version: "1.2.0"                   # Semver
  description: "Acme incident resolution tools and providers"
  author: "Acme Platform Team <platform@acme.example>"

compatibility:
  api-version: ">=2.0.0 <3.0.0"      # Host API version range (semver)
  protocol-version: "2.0"            # Extension protocol version

entry-point:
  executable: "./bin/acme-extension"
  args: ["--config", "acme.yaml"]

transport: stdio-jsonrpc             # stdio-jsonrpc / grpc / mcp

capabilities:
  - capability/tool-registration
  - capability/provider-registration
  - capability/schema-extension
  - capability/network
```

```
- capability/file-read

# Required when capability/file-read is declared:
file-read-paths:
- "./runbooks/**"
- "./data/**"

# Required when capability/network is declared:
network-hosts:
- "api.pagerduty.com"
- "*.acme.example"

# Required when capability/env-read is declared:
env-read-patterns:
- "ACME_API_*"
- "PD_TOKEN"

platform:                                # Optional OS/arch constraints
  os: ["linux", "darwin", "windows"]
  arch: ["amd64", "arm64"]
```

Manifest validation is performed by the host before the process is started. An extension with an unsatisfied `compatibility.api-version` range is rejected before any subprocess is created.

6.4 Extension Discovery

The host discovers extensions using the following resolution order, with later entries taking precedence:

1. **Built-in extensions:** compiled into the host binary; always available; no manifest required.
2. **Workspace config:** extensions declared in `.gert/extensions.yaml` at the workspace root.
3. **Runbook declaration:** extensions listed in the runbook's top-level `extensions:` field.

4. **CLI override:** `-extension <path>` flags provided at invocation time.

The `.gert/extensions.yaml` workspace config format:

```
# .gert/extensions.yaml
extensions:
- path: "./extensions/acme-pack"    # directory containing gert-extension.yaml
- path: "/opt/gert-extensions/audit-pack"
```

The runbook `extensions:` field:

```
apiVersion: runbook/v2
meta:
  name: incident-triage
extensions:
- path: "./extensions/acme-pack"
```

When the same extension appears in multiple discovery sources, the host deduplicates by `meta.name`. If two distinct extensions share the same name, the one with the higher resolution precedence wins; the lower one is logged as a warning and skipped.

6.5 Handshake and Lifecycle Protocol

All messages use JSON-RPC 2.0. The host is always the initiator; the extension never sends unsolicited requests.

6.5.1 extension/initialize (host → extension)

Sent immediately after the extension process starts. The host communicates the capabilities it is willing to grant and its own version identity.

Request params:

```
{
  "jsonrpc": "2.0",
  "id": "1",
  "method": "extension/initialize",
  "params": {
    "host": {
```

```
    "name": "gert",
    "version": "2.0.0",
    "apiVersion": "2.0"
  },
  "protocolVersion": "2.0",
  "grantedCapabilities": [
    "capability/tool-registration",
    "capability/network"
  ],
  "runContext": {
    "runId": "run-abc123",
    "mode": "real"
  }
}
```

Success response:

```
{
  "jsonrpc": "2.0",
  "id": "1",
  "result": {
    "extension": {
      "name": "acme.incident-pack",
      "version": "1.2.0"
    },
    "protocolVersion": "2.0",
    "acknowledgedCapabilities": [
      "capability/tool-registration",
      "capability/network"
    ]
  }
}
```

Error codes for this method:

- -32001: Protocol version not supported by extension.

- -32002: Required capability not granted (extension considers the capability non-negotiable).
- -32003: Host API version outside extension compatibility range.

6.5.2 contributions/list (host → extension)

Sent after a successful `extension/initialized` notification. The extension returns its contribution manifest, filtered to the granted capability set.

```
{
  "jsonrpc": "2.0",
  "id": "2",
  "method": "contributions/list",
  "params": {}
}
```

Response:

```
{
  "jsonrpc": "2.0",
  "id": "2",
  "result": {
    "tools": [ { "name": "acme.pd-lookup", "... } ],
    "providers": [ { "name": "pd", "prefixes": ["pd."] } ],
    "schemaExtensions": [],
    "policyContributions": []
  }
}
```

6.5.3 extension/ping (host → extension)

Periodic health check. The extension **MUST** respond within the host-configured ping timeout (default 5 seconds). A non-response triggers a crash-recovery cycle (Section 6.7.3).

```
{ "jsonrpc": "2.0", "id": "42", "method": "extension/ping", "params": {} }
```

Response:

```
{ "jsonrpc": "2.0", "id": "42", "result": { "status": "ok" } }
```

6.5.4 extension/shutdown (host → extension)

Graceful termination. The extension SHOULD drain in-flight requests and release resources within the shutdown timeout (default 10 seconds).

```
{ "jsonrpc": "2.0", "id": "99", "method": "extension/shutdown", "params": {} }
```

Response (best-effort; host may not wait):

```
{ "jsonrpc": "2.0", "id": "99", "result": { "status": "ok" } }
```

If the extension does not exit within the shutdown timeout, the host sends SIGTERM followed by SIGKILL after a grace period.

6.5.5 Lifecycle State Machine

1. **discovered**: manifest found and parsed; no process yet.
2. **verified**: manifest validated; capabilities evaluated against policy.
3. **starting**: executable launched; stdin/stdout pipes attached.
4. **initializing**: `extension/initialize` sent; awaiting response.
5. **ready**: `contributions/list` completed; accepting invocations.
6. **draining**: `extension/shutdown` sent; waiting for in-flight completions.
7. **stopped**: process has exited cleanly.
8. **crashed**: process exited unexpectedly (see Section 6.7.3).

6.6 Versioning and Compatibility

The host maintains a monotonically increasing API version string following semver. The extension manifest declares a semver range in `compatibility.api-version`. Compatibility is evaluated before any process is started:

- If the host API version falls outside the declared range, the extension is rejected with a structured error logged to the run trace. The run continues with the remaining available extensions; a warning is surfaced to the operator.

- During `extension/initialize`, the host also sends its `protocolVersion`. If the extension does not support that protocol version it MUST return error code `-32001`.
- If an extension requests a capability that the host version does not implement (e.g., a capability introduced in a later host version), the capability is silently absent from `grantedCapabilities`. The extension SHOULD treat absent non-required capabilities as optional degradation rather than a fatal error.

Host API versions are tracked in `specs/api-versions.md`. Each new capability addition constitutes a minor version bump; breaking changes to existing method contracts constitute a major version bump.

6.7 Sandboxing and Process Isolation

6.7.1 Process Isolation

Extensions always run as child processes of the gert host. They are never loaded as shared libraries or plug-ins executed in-process. The host:

- Sets a restricted environment: only env vars matching `env-read-patterns` from the manifest are forwarded.
- Attaches the extension's stderr to a structured log collector; raw stderr lines are stored in the run trace under `extension.<name>.stderr`.
- Does not forward the host process's file descriptor table beyond stdin/stdout/stderr.

6.7.2 Capability Enforcement

Every RPC method the extension may call is guarded by a capability check. The host maintains a granted-capability set per extension process. Before dispatching any extension-initiated request (e.g., a tool registration during `contributions/list`) the host verifies the required capability is in the granted set. Violations return:

```
{  
  "jsonrpc": "2.0",  
  "id": "...",  
  "error": {
```



```
"code": -32010,  
"message": "capability not granted",  
"data": {  
  "required": "capability/file-write",  
  "granted": ["capability/file-read", "capability/network"]  
}  
}  
}
```

File-path and network-host constraints are enforced by the host at the system call boundary using OS-level mechanisms where available (e.g., `pledge/unveil` on OpenBSD, `seccomp` on Linux) and by host-side validation of all path and URL arguments on other platforms.

6.7.3 Timeout and Crash Handling

- **Per-invocation timeout:** each `tools/invoke` call carries a `timeoutMs` field. If the extension does not respond within that deadline, the host sends a `tools/cancel` request and, if still unresponsive after a grace period, terminates the process.
- **Ping timeout:** if an extension fails to respond to `extension/ping` within 5 seconds it is considered unresponsive. The host logs a structured warning and terminates the process.
- **Unexpected exit:** if the extension process exits with a non-zero status or is killed by a signal, the host marks the extension as **crashed**, emits a `extension.crashed` trace event, and cancels all in-flight invocations with a structured `-32099` (extension crashed) error.
- **Host isolation:** extension crashes never crash the host. A run that loses an extension mid-execution surfaces an error on the affected step and allows the operator to decide whether to abort or continue with a degraded tool set.

Capability name	Allows	Excludes
capability/tool-registration	Register new tool definitions at runtime via <code>contributions/list</code> .	Does not allow the extension to invoke other tools or modify existing tool definitions.
capability/schema-extension	Contribute namespaced additional fields to the runbook schema (e.g. <code>x-acme.*</code> annotations).	Cannot modify or remove existing schema fields; cannot alter core validation rules.
capability/event-subscription	Subscribe to host runtime events (step-started, step-completed, run-ended, etc.) via the <code>events/subscribe</code> method.	Cannot emit synthetic events into the host event bus.
capability/policy-contribution	Register additional governance policy rules evaluated during pre-flight and approval checks.	Cannot override or disable built-in governance rules; cannot grant capabilities to other extensions.
capability/provider-registration	Register new input providers that handle <code>from:</code> prefix bindings.	Cannot intercept resolution requests destined for another registered provider.
capability/file-read	Read files within the path scope declared in the manifest (<code>file-read-paths</code>).	Cannot read files outside declared paths; cannot write or delete files.
capability/file-write	Write or create files within the path scope declared in the manifest (<code>file-write-paths</code>).	Cannot write outside declared paths; does not imply read access.
capability/network	Make outbound network calls to the hosts declared in the manifest (<code>network-hosts</code>).	Cannot connect to undeclared hosts; cannot listen for inbound connections.
capability/env-read	Read environment variables whose names match the declared <code>env-read-patterns</code> list.	Cannot read undeclared variables; cannot write or unset environment variables.
capability/run-state-read	Read current run state (step status, captured variables, trace events).	Cannot modify run state; does not imply the ability to emit output.

TABLE 6.1: Extension capability taxonomy (v2).

Chapter 7

Tool Runtime

The tool runtime is the subsystem responsible for resolving, validating, and invoking tool definitions at runbook execution time. Tools are the primary mechanism by which gert performs side-effecting work: executing binaries, calling JSON-RPC services [18], and delegating to MCP-hosted capabilities [10].

7.1 Tool Discovery

The host resolves tool references in three phases:

1. **Static discovery:** `.tool.yaml` files are located by scanning the following directories in order:
 - (a) The built-in tool registry compiled into the host binary.
 - (b) `<workspace-root>/tools/` — the conventional location for project-level tool definitions (`tools/<name>.tool.yaml`).
 - (c) Directories listed in the workspace config (`.gert/config.yaml`, field `tool-paths`).
 - (d) Required packages declared in the runbook's `requires:` field (each package's `tools/` subdirectory is scanned).
2. **Dynamic registration:** extensions with `capability/tool-registration` may register additional tool definitions during the `contributions/list` phase (see Section 6.5). Dynamically registered tools are merged into the resolved catalog before execution begins.

3. **MCP discovery**: when the runbook declares an MCP server under `tools.mcp-servers`, the host performs a `tools/list` call to that server at startup and registers every tool reported as a dynamically discovered tool (see Section 7.7).

7.1.1 Name Resolution Order

When a runbook step references a tool by name, the host resolves as follows:

1. Exact match in dynamically registered tools (MCP and extension, in registration order).
2. Exact match in project-level `tools/` directory.
3. Exact match in required-package tool directories (in `requires:` declaration order).
4. Exact match in the built-in registry.

If two tools share the same name, the first match wins and a warning is emitted. Tool authors SHOULD use reverse-DNS namespace prefixes to avoid collisions (e.g., `acme.pd-lookup`).

7.2 Tool Definition Schema (v2)

```
# tools/my-service.tool.yaml
apiVersion: tool/v2

meta:
  name: my-service           # Unique identifier within the workspace
  version: "2.0.0"          # Tool definition version (semver)
  description: "Calls the MyService REST API"
  binary: my-service-cli     # Executable name (looked up in PATH)

transport:
  mode: stdio-jsonrpc        # stdio | stdio-jsonrpc | grpc | mcp

# For stdio-jsonrpc and mcp: startup behaviour
startup:
  argv: ["--server-mode"]    # Extra args passed to the binary
  ready-pattern: "ready"     # Regex matched against stdout/stderr to
```

```

                                # confirm the server is accepting requests
    timeout: "10s"
    shutdown-method: "shutdown" # JSON-RPC method to call on graceful stop

governance:
  requires-capabilities: []      # Host capabilities the tool needs, e.g.
                                # ["capability/network"]
  allowed-environments: ["real", "dry-run"]
  requires-approval: false      # Set true to gate invocation on operator OK

actions:
  lookup:
    description: "Look up an entity by ID"
    argv: ["lookup", "--id", "{{ .id }}"] # Only for stdio transport
    timeout: "30s"
    args:
      id:
        type: string
        required: true
        description: "Entity identifier"
        redact: false
    output:
      format: json              # text / json / jsonl
    capture:
      stdout:
        format: json

```

7.2.1 Transport-Specific Fields

stdio The binary is spawned once per invocation. The `argv` array in the action definition is rendered as a Go template with the resolved argument values, then appended to the binary name to form the complete command. The process's stdout is captured as the tool output.

stdio-jsonrpc The binary is spawned once per run (persistent process). The host keeps a single process alive across all action calls for that tool. Each action invocation sends

a JSON-RPC request to the process's stdin and reads the response from stdout. The `startup.ready-pattern` confirms the server is accepting connections.

mcp The host spawns the binary and performs the MCP initialization handshake (Section 7.7). Actions map to MCP `tools/call` requests.

7.3 Transport Layer

7.3.1 stdio Transport

Process establishment. The host calls `exec.Command(binary, resolvedArgv...)` with a controlled environment. A per-invocation timeout context wraps the call.

Invocation envelope. For stdio tools the invocation is entirely encoded in the process arguments and environment; there is no envelope. The arguments are rendered from the action's `argv` template.

Response envelope. Stdout is captured in full after process exit. The exit code is checked: non-zero is mapped to a tool error.

Error envelope.

```
{
  "error": {
    "code": "TOOL_NONZERO_EXIT",
    "message": "tool exited with code 1",
    "details": {
      "exitCode": 1,
      "stderr": "connection refused\n",
      "durationMs": 320
    }
  }
}
```

7.3.2 stdio-jsonrpc Transport

Process establishment. A single process is spawned at first use and kept alive for the duration of the run. The host waits for `startup.ready-pattern` on stdout/stderr before sending the first request.

Invocation envelope.

```
{
  "jsonrpc": "2.0",
  "id": "<invocationId>",
  "method": "<actionName>",
  "params": {
    "args": { "id": "INC-12345" },
    "context": {
      "runId": "run-abc123",
      "stepId": "lookup_incident",
      "traceId": "trace-xyz",
      "userId": "ops@acme.example",
      "attempt": 1
    }
  }
}
```

Response envelope.

```
{
  "jsonrpc": "2.0",
  "id": "<invocationId>",
  "result": {
    "output": { "..."},
    "telemetry": {
      "startedAt": "2026-04-18T10:00:00Z",
      "finishedAt": "2026-04-18T10:00:00.320Z",
      "durationMs": 320
    }
  }
}
```

Error envelope.

```
{
  "jsonrpc": "2.0",
  "id": "<invocationId>",
  "error": {
    "code": -32050,
    "message": "entity not found",
    "data": {
      "category": "runtime",
      "retryable": false,
      "details": { "entityId": "INC-12345" }
    }
  }
}
```

Error code ranges:

- -32700 to -32600: JSON-RPC parse and request errors.
- -32050: Application-level tool error (retryable per `data.retryable`).
- -32051: Capability not granted.
- -32052: Input validation failed.
- -32053: Output schema validation failed.
- -32054: Timeout.
- -32055: Cancelled by host.
- -32099: Tool process crashed.

7.3.3 MCP Transport

See Section 7.7 for the full MCP integration specification.

7.4 Invocation Context

Every tool invocation — regardless of transport — receives a structured context object. For `stdio` tools, relevant fields are injected as environment variables prefixed `GERT_`; for `stdio-jsonrpc` and `mcp` tools, the context is transmitted in the invocation envelope.

```
{
  "context": {
    "runId":      "run-abc123",      // Globally unique run identifier
    "stepId":     "lookup_incident", // Step ID within the runbook
    "traceId":    "trace-xyz789",   // Distributed trace correlation ID
    "userId":     "ops@acme.example", // Identity executing the run
    "mode":       "real",           // real | dry-run | replay
    "attempt":    1,                // Retry attempt number (1-based)
    "capabilities": [               // Effective host capability set
      "capability/network",
      "capability/file-read"
    ]
  }
}
```

7.5 Capability Gate

Before dispatching any tool invocation the host checks two conditions:

1. The tool's `governance.requires-capabilities` list is a subset of the current run's granted capability set. If not, the step fails immediately with a `CAPABILITY_DENIED` error; the run does not attempt to start the tool process.
2. The tool's `governance.allowed-environments` list includes the current execution mode. A tool not listed for `dry-run` will be skipped (with a warning) when the run mode is `dry-run`.

7.6 Output Capture

Tool output is mapped to named step captures using the `capture:` declaration in the action definition.

```
capture:
  dns_output: stdout          # Capture stdout as variable dns_output
  error_log:  stderr          # Capture stderr as variable error_log
  exit_code:  exitCode        # Capture numeric exit code
```

For json output format, capture paths may use dot notation to extract nested fields:

```
capture:
  incident_id: "stdout.incident.id"
  severity:   "stdout.incident.severity"
```

Redaction. Before any captured value is stored in the run state or emitted to the trace, the governance redaction pipeline is applied. Redaction rules are declared in the runbook's `meta.governance.redact` block and in the tool action's per-argument `redact: true` flag. Redacted values are replaced with `<redacted>` in all stored and displayed output.

7.7 MCP Tool Integration

MCP (Model Context Protocol) servers are long-lived processes that expose a dynamic catalog of tools. Gert treats an MCP server as a tool source, not as a single tool.

7.7.1 MCP Server Declaration

```
# In runbook meta or .gert/config.yaml
tools:
  mcp-servers:
    - name: acme-mcp
      binary: acme-mcp-server
      args: ["--port", "stdio"]
      transport: mcp
      startup:
        timeout: "15s"
```

7.7.2 MCP Discovery

On run startup the host:

1. Spawns the MCP server binary.

2. Sends the MCP `initialize` request (per MCP specification).
3. After receiving `initialized`, sends `tools/list`.
4. Registers each returned tool under the name `<server-name>/<tool-name>` in the dynamic tool catalog.

7.7.3 MCP Tool Invocation

When a runbook step calls an MCP-backed tool, the host sends:

```
{
  "jsonrpc": "2.0",
  "id": "5",
  "method": "tools/call",
  "params": {
    "name": "lookup_incident",
    "arguments": { "id": "INC-12345" },
    "_gert_context": {
      "runId": "run-abc123",
      "stepId": "triage_step",
      "traceId": "trace-xyz"
    }
  }
}
```

The `_gert_context` field is a gert extension to the MCP envelope. MCP servers that do not recognise it MUST ignore it. The response follows the standard MCP `tools/call` response format.

7.7.4 MCP vs stdio-jsonrpc Differences

- MCP servers expose multiple tools under a single process; stdio-jsonrpc tools are one tool per process.
- MCP uses a distinct initialization handshake (`initialize` / `initialized`) before tool registration.
- MCP tool names are discovered dynamically at runtime; stdio-jsonrpc action names are declared statically in the `.tool.yaml`.

- MCP responses use a `content` array; gert flattens text content to a single string for capture.

7.8 Cancellation and Timeout

Per-tool timeout. Declared in the action definition under `timeout:` (e.g., 30s). The default is the runbook's `meta.defaults.timeout`. The host wraps each invocation in a `context.WithTimeout`; when the deadline elapses the host sends a `tools/cancel` request (JSON-RPC transport) or sends SIGTERM (stdio transport), then forcibly terminates the process after a 5-second grace period.

Cancellation propagation. When a run is cancelled (via `Ctrl-C`, the debugger quit command, or an API cancellation request), the host:

1. Sets the run-level cancel context.
2. Sends `tools/cancel` to all in-flight JSON-RPC and MCP invocations with a `reason: "run-cancelled"` field.
3. After a 5-second drain window, forcibly terminates any unresponsive tool processes.
4. Emits a `run.cancelled` trace event recording which steps were interrupted.

The `tools/cancel` request envelope:

```
{
  "jsonrpc": "2.0",
  "id": "cancel-5",
  "method": "tools/cancel",
  "params": {
    "invocationId": "5",
    "reason": "run-cancelled"
  }
}
```

7.9 Tool Versioning

Tool definitions carry a `meta.version` field. When multiple definitions for the same tool name are present (e.g., from a required package and from a workspace override), the host selects by resolution order (Section 7.1), not by version number. Explicit version pinning is supported via the runbook's `requires:` block:

```
requires:
- package: github.com/acme/gert-tools
  version: ">=1.2.0 <2.0.0"
```

Chapter 8

Runtime Events and Determinism

Gert v2 is an event-driven system. Every state change in the runtime—run initiation, step advancement, governance checks, tool invocations, human interactions—produces a structured event. Events serve three purposes: (1) adapters (TUI, Web, VS Code) subscribe to events to render UI state without coupling to execution logic, (2) events are appended to the JSONL trace file for audit and replay (§12), and (3) events propagate to observability systems for real-time monitoring and alerting.

This section defines the complete event system: the event envelope schema, delivery semantics, the normative event catalog, event channels, subscription API, wire format, and the required vs. optional event classification.

8.1 Event System Design

8.1.1 Role of Events

Events are **fan-out notifications to subscribers**, not a command or control channel. The Runtime Core emits events as a consequence of state transitions; events do not trigger state transitions. This design ensures that execution semantics remain centralized in the Runtime Core and adapters remain passive renderers.

Key principles:

- **One-way flow:** Events flow from the Runtime Core outward to subscribers. Subscribers never write events back to the runtime.
- **Non-blocking emission:** Event emission must not block step execution. The event bus uses a bounded channel with a discard-on-full policy for in-process subscribers to prevent slow consumers from stalling the runtime.

- **Immutable once emitted:** Events are value objects; they cannot be modified after emission.
- **Best-effort delivery to in-process subscribers:** If a subscriber channel is full, the event is dropped. The trace file write (§12) is the authoritative record.

8.1.2 Delivery Semantics

At-least-once delivery. The JSONL trace file receives every event via a synchronous write before the runtime proceeds to the next action. In-process subscribers (TUI, event forwarders) receive events on a best-effort basis; if the subscriber channel is full, the event is discarded without blocking the runtime.

In-order per run. All events within a single run are totally ordered by their **sequence** field. Events from different runs are not ordered relative to each other. Subscribers that process events asynchronously must tolerate out-of-order arrival from concurrent runs.

Replay determinism. Events written to the trace file are sufficient to reconstruct the execution timeline. Replay mode (§12.3) re-emits the same events in the same order, allowing adapters to render a historical run indistinguishably from a live run.

8.2 Event Envelope

Every event carries a mandatory envelope with the following fields. Consumers **MUST** tolerate unknown fields to support additive schema evolution.

Field	Type	Description
<code>event_id</code>	UUID v4	Globally unique event identifier
<code>run_id</code>	UUID	Execution run identifier (stable across pauses/resumes)
<code>runbook_id</code>	string	Runbook name or path from runbook YAML <code>meta.name</code>
<code>timestamp</code>	RFC3339	UTC timestamp with microsecond precision [19]
<code>kind</code>	string	Event type (e.g., <code>step/started</code> , <code>governance/approved</code>)
<code>sequence</code>	int64	Monotonically increasing integer within this run (starting from 0)
<code>payload</code>	object	Event-specific data (schema varies per kind)

Sequence numbers. The `sequence` field provides a total order over events in a single run. It is assigned by the `RunStore.WriteTrace` implementation and persists across pause/resume cycles. Sequence numbers are not globally unique; they are scoped to a single `run_id`.

Timestamps. The `timestamp` field uses RFC3339 format with at least microsecond precision (e.g., `2026-04-18T14:32:05.123456Z`). Timestamps are wall-clock times and must not be used for precise duration calculations (use event-specific `duration_ms` fields instead).

Event IDs. The `event_id` is a UUID v4 assigned at emission time. It is globally unique across all runs and all gert deployments. Event IDs are used for correlation in distributed tracing systems (§15) and for deduplication in event forwarding pipelines.

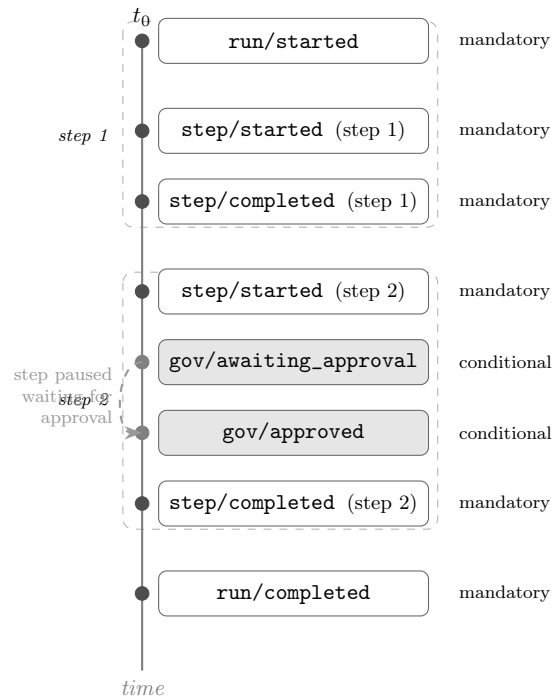


FIGURE 8.1: Typical event sequence for a two-step run where the second step requires governance approval. Mandatory events (solid dots) are emitted for every run; conditional events (hollow region) appear only when the relevant policy is triggered.

8.3 Event Catalog

The following subsections define every normative event kind in v2.0. Each definition includes the payload schema, emission semantics, and whether the event is required or optional.

8.3.1 Run Lifecycle Events

run/started

Emitted: Once, as the first event in every execution. Must be the first line in the trace file.

Payload:

```
{
  "runbook_id": <string>, // from meta.name or file path
  "version": <string>, // runbook schema version (e.g., "runbook/v2")
  "actor": <string>, // identity (from --as flag or adapter session)
  "input_hash": <string>, // SHA256 of sorted input key-value pairs
  "mode": <string>, // "real" | "dry-run" | "replay"
  "client": <string>, // "cli" | "tui" | "web" | "vscode" | "mcp"
  "parent_run_id": <string> // omitted if not an invoke child
}
```

Semantics. The `input_hash` is the SHA256 hex digest of the JSON-serialized input dictionary with keys sorted lexicographically. This hash is used to detect input tampering in replay mode. The `parent_run_id` field is present only if this run was initiated by an `invoke` step in another run.

Required: YES. Every run MUST emit this event.

`run/completed`

Emitted: Once, as the terminal event for a successful run (all steps passed or skipped).

Payload:

```
{
  "outcome": <string>, // "success" | "failure" | "cancelled"
  "step_count": <int>, // total steps in the plan
  "passed": <int>, // steps that completed successfully
  "failed": <int>, // steps that failed
  "skipped": <int>, // steps skipped (branch/iterate)
  "duration_ms": <int64> // wall-clock duration from run/started
}
```

Semantics. The `outcome` field distinguishes clean completion (`success`), error termination (`failure`), and operator-initiated halt (`cancelled`). If `outcome` is `failure`, the last event before this one should be a `step/failed` or `governance/blocked`.

Required: YES. Every run that reaches a terminal state MUST emit this event.

`run/cancelled`

Emitted: When a run is cancelled by operator action (SIGINT, API call, or approval rejection).

Payload:

```
{
  "reason":      <string>, // "user_interrupt" / "approval_rejected" /
    ↪ "timeout"
  "cancelled_by": <string>, // actor identity (omitted if SIGINT)
  "step_id":     <string>  // step that was active at cancellation time
}
```

Semantics. This event is followed immediately by `run/completed` with outcome: `"cancelled"`. The `reason` field disambiguates graceful operator cancellation (`user_interrupt`) from governance-driven rejection (`approval_rejected`).

Required: YES, if a run is cancelled before natural completion.

8.3.2 Step Lifecycle Events

`step/started`

Emitted: Immediately before a step executes (after governance checks pass).

Payload:

```
{
  "step_id": <string>, // stable step identifier from runbook YAML
  "step_type": <string>, // "cli" / "manual" / "tool" / "invoke" / "branch"
    // / "iterate" / "parallel" / "compensate"
  "index": <int> // zero-based position in the ExecutionPlan
}
```

Required: YES. Every step that begins execution MUST emit this event.

`step/completed`

Emitted: After a step exits successfully.

Payload:

```
{
  "step_id": <string>,
  "outcome": <string>, // "success" / "skipped"
  "duration_ms": <int64>,
  "exit_code": <int> // for cli/tool steps; 0 for manual/invoke
}
```

Semantics. An outcome of `skipped` occurs for steps in non-taken branch paths or exhausted iterate loops. Exit code is present only for `cli` and `tool` steps; for other step types it is always 0.

Required: YES, for every step that completes without error.

`step/failed`

Emitted: When a step exits with a non-zero exit code or runtime error.

Payload:

```
{
  "step_id": <string>,
  "error_code": <string>, // "exit_nonzero" / "timeout" / "tool_error"
                        // / "governance_blocked" / "invoke_failed"
  "error_message": <string>, // human-readable (redacted)
  "retryable": <bool>, // true if retry policy allows retry
  "exit_code": <int> // for exit_nonzero; omitted otherwise
}
```

Semantics. The `retryable` field indicates whether the runtime will attempt a retry (requires a retry policy on the step). If `retryable` is true, this event will be followed by `step/retrying`.

Required: YES, for every step that fails.

`step/skipped`

Emitted: When a step is not executed due to control flow.

Payload:

```
{
  "step_id": <string>,
  "reason": <string> // "branch_not_taken" / "iterate_exhausted"
                // / "dry_run" / "cancelled"
}
```

Required: YES, for every step that is planned but not executed.

step/retrying

Emitted: Before each retry attempt (v2.1 feature; requires retry policy).

Payload:

```
{
  "step_id":    <string>,
  "attempt":    <int>,      // 1 = first retry, 2 = second, etc.
  "max_attempts": <int>,
  "backoff_ms": <int64>    // delay before this retry
}
```

Required: YES, if retry is attempted. Optional in v2.0 (retry not yet implemented).

8.3.3 Governance Events

All governance events are normatively defined in §11. The following is a summary for completeness.

governance/command_checked

Emitted: When a command is evaluated against allowlist and denylist.

Payload:

```
{
  "step_id":    <string>,
  "command":    <string>,  // base name (argv[0] stripped of path)
  "verdict":    <string>,  // "allowed" | "denied"
  "rule_matched": <string> // "allowlist" | "denylist" | "default_allow"
}
```

Required: MAY be emitted. Recommended for audit trails but not required for basic operation.

governance/approval_requested

Emitted: When a step's approval gate is entered.

Payload:

```
{
  "step_id": <string>,
  "approver_roles": [<string>], // roles that can approve
  "min_approvals": <int>,
  "timeout_seconds": <int>, // 0 if no timeout
  "message": <string> // prompt text for approvers (redacted)
}
```

Required: YES, when approval gates are used.

governance/approval_received

Emitted: Each time an approver submits a decision.

Payload:

```
{
  "step_id": <string>,
  "approver": <string>, // actor identity
  "role": <string>, // role the approver is acting in
  "decision": <string>, // "approved" | "rejected"
  "comment": <string> // optional justification (redacted)
}
```

Required: YES, for every approval decision (including rejections).

governance/redaction_applied

Emitted: When redaction rules match and modify captured output.

Payload:

```
{
  "step_id": <string>,
  "field_path": <string>, // "stdout" | "stderr" | "capture.var_name"
  "pattern_count": <int> // number of patterns that matched (not the
    ↪ patterns)
}
```

Semantics. The actual redaction patterns are not logged to avoid accidentally revealing what sensitive data looks like. Only the fact that redaction occurred is recorded.

Required: MAY be emitted. Useful for audit compliance.

8.3.4 Extension and Tool Events

extension/loaded

Emitted: When an extension successfully completes the handshake.

Payload:

```
{
  "extension_id": <string>, // from extension manifest
  "version":      <string>, // semver version
  "capabilities": [<string>] // list of capability names granted
}
```

Required: MAY be emitted. Useful for debugging extension issues.

extension/unloaded

Emitted: When an extension process exits or is terminated.

Payload:

```
{
  "extension_id": <string>,
  "reason":      <string> // "clean_exit" / "crash" / "timeout" / "killed"
}
```

Required: MAY be emitted.

tool/invoked

Emitted: When a tool call is dispatched (before the tool executes).

Payload:

```
{
  "step_id":      <string>,
  "tool_name":    <string>,
  "transport":    <string>, // "stdio" / "jsonrpc" / "mcp"
  "correlation_id": <string> // UUID v4, links to tool/completed
}
```

Required: MAY be emitted. Recommended for observability.

tool/completed

Emitted: When a tool call returns.

Payload:

```
{
  "step_id":      <string>,
  "tool_name":    <string>,
  "correlation_id": <string>, // matches tool/invoked
  "exit_code":    <int>,
  "duration_ms":  <int64>
}
```

Required: MAY be emitted.

8.3.5 Human-in-the-Loop Events

input/prompted

Emitted: When a manual step or input provider displays a prompt.

Payload:

```
{
  "step_id":      <string>, // omitted if prompt is from provider
  "prompt_id":    <string>, // UUID v4 for correlation
  "input_type":   <string>, // "text" | "checklist" | "attachment"
  "prompt":       <string>  // prompt text (redacted)
}
```

Required: YES, for manual steps. MAY be emitted for provider prompts.

input/received

Emitted: When the operator submits a response.

Payload:

```
{
  "step_id":      <string>,
  "prompt_id":    <string>, // matches input/prompted
  "redacted":     <bool>    // true if redaction rules matched the response
}
```


Semantics. The actual input value is not included in this event (it is recorded in the trace via `manual/completed` or `step/completed`). The `redacted` flag indicates whether the input passed through the redaction pipeline.

Required: YES, for manual steps.

8.3.6 Saga and Compensation Events (v2.1+)

The following events are reserved for the saga/compensation pattern, which is targeted for v2.1. They are included here for forward compatibility.

`saga/compensation_triggered`

Emitted: When a step fails and has registered compensation handlers.

Payload:

```
{
  "failed_step_id": <string>,
  "compensation_steps": [<string>] // step IDs to execute in reverse order
}
```

Required: YES, if compensation is triggered (v2.1+).

`saga/compensation_completed`

Emitted: After all compensation steps complete.

Payload:

```
{
  "outcome": <string> // "success" | "partial" | "failed"
}
```

Required: YES, if compensation is triggered (v2.1+).

8.4 Event Channels

Events flow through three channels, each with distinct delivery semantics and subscribers.

8.4.1 In-Process Event Bus

Purpose. The in-process event bus delivers events from the Runtime Core to adapters (TUI, `gert serve` session handlers) running in the same process.

Implementation. A Go `chan Event` with a fixed buffer size (default: 100 events). The Runtime Core writes to the channel; adapters read from it. If the channel is full, new events are discarded (the trace file is the authoritative record).

Subscriber API. Adapters call `RunHandle.Events()` to obtain a read-only event channel:

```
type RunHandle interface {  
    // Events returns a read-only channel of events for this run.  
    // The channel is closed when the run enters a terminal state.  
    Events() <-chan Event  
    // ... other methods ...  
}
```

Lifecycle. The event channel is created at `Runtime.Start()` and closed when the run emits `run/completed` or `run/cancelled`. Subscribers must drain the channel to avoid blocking the runtime (though the runtime will discard events if the subscriber is too slow).

8.4.2 WebSocket Broadcast (HTTP Adapter)

Purpose. For `gert serve -http`, events are broadcast to WebSocket clients for real-time UI updates.

Implementation. The HTTP adapter subscribes to `RunHandle.Events()` and forwards events to all connected WebSocket clients for the given `run_id`. The `/events` endpoint uses WebSocket.

Wire format. JSON-encoded event envelopes, one per WebSocket message frame.

Delivery semantics. Best-effort. If a WebSocket client is slow, the adapter may drop events to prevent backpressure. Clients should use the `/runs/:id/trace` HTTP endpoint to fetch the authoritative trace file if they detect gaps.

8.4.3 JSONL Trace File

Purpose. The append-only JSONL trace file (§12) is the durable, authoritative record of all events. It is the only channel with guaranteed at-least-once delivery.

Implementation. The Runtime Core writes every event synchronously to `.runbook/runs/<run-id>/trace.jsonl` before proceeding to the next action. File writes use `O_APPEND` mode to ensure atomicity on Unix systems.

Format. One JSON object per line, with the envelope fields from §6.2. See §12.1 for the complete trace file specification.

8.5 Event Subscriptions

8.5.1 Programmatic Subscription API

Extensions and adapters can subscribe to events programmatically. The subscription API supports filtering by event kind prefix and by `run_id`.

```
// EventSubscriber is the interface for subscribing to runtime events.
type EventSubscriber interface {
    // Subscribe registers a subscription with optional filters.
    // Returns a channel that will receive matching events.
    Subscribe(ctx context.Context, opts SubscribeOptions) (<-chan Event, error)
}

type SubscribeOptions struct {
    // KindPrefix filters events to those matching the prefix.
    // Examples: "step/", "governance/", "run/started"
    KindPrefix string

    // RunID filters events to a specific run. If empty, all runs.
    RunID string

    // BufferSize is the channel buffer size. Default: 100.
    BufferSize int
}
```

Example use cases.

- A TUI adapter subscribes with `KindPrefix: "step/"` to render step progress.
- A governance dashboard subscribes with `KindPrefix: "governance/"` to display policy violations.
- An observability forwarder subscribes with no filters to send all events to an Open-Telemetry collector (§15).

8.6 Wire Format

Events are serialized as JSON. The wire format is identical for the JSONL trace file, WebSocket broadcasts, and JSON-RPC notifications.

8.6.1 Concrete Example: step/started

```
{
  "event_id":    "a3f9c1e2-8d4b-4e1f-9a2c-3d5e6f7a8b9c",
  "run_id":      "20260418T143022-a3f9c1",
  "runbook_id":  "deploy-frontend",
  "timestamp":   "2026-04-18T14:30:25.123456Z",
  "kind":        "step/started",
  "sequence":    3,
  "payload": {
    "step_id":    "build_assets",
    "step_type":  "cli",
    "index":      2
  }
}
```

8.6.2 Concrete Example: governance/approval_requested

```
{
  "event_id":    "b1c2d3e4-5f6a-7b8c-9d0e-1f2a3b4c5d6e",
  "run_id":      "20260418T143022-a3f9c1",
  "runbook_id":  "deploy-frontend",
  "timestamp":   "2026-04-18T14:32:10.456789Z",
```

```

"kind":      "governance/approval_requested",
"sequence":  12,
"payload": {
  "step_id":      "deploy_to_prod",
  "approver_roles": ["SRE", "PM"],
  "min_approvals": 2,
  "timeout_seconds": 3600,
  "message":      "Deploying to production. Confirm change window."
}
}

```

8.7 Required vs. Optional Events

The following table classifies every event as REQUIRED (MUST be emitted) or OPTIONAL (MAY be emitted). Required events form the minimal contract for adapter compatibility; optional events provide observability and debugging enhancements.

Event Kind	Status	Notes
run/started	REQUIRED	First event in every trace
run/completed	REQUIRED	Terminal event
run/cancelled	REQUIRED	If cancelled
step/started	REQUIRED	For every executed step
step/completed	REQUIRED	For every successful step
step/failed	REQUIRED	For every failed step
step/skipped	REQUIRED	For every skipped step
step/retrying	OPTIONAL	v2.1+ retry feature
governance/command_checked	OPTIONAL	Useful for audit trails
governance/approval_requested	REQUIRED	If approval gates used
governance/approval_received	REQUIRED	If approval gates used
governance/redaction_applied	OPTIONAL	Compliance logging
extension/loaded	OPTIONAL	Debugging
extension/unloaded	OPTIONAL	Debugging
tool/invoked	OPTIONAL	Observability
tool/completed	OPTIONAL	Observability
input/prompted	REQUIRED	For manual steps
input/received	REQUIRED	For manual steps
saga/compensation_triggered	REQUIRED	v2.1+ if saga used
saga/compensation_completed	REQUIRED	v2.1+ if saga used

Adapter compatibility. Adapters **MUST** handle all **REQUIRED** events. They **SHOULD** gracefully ignore unknown event kinds to support forward compatibility (e.g., a v2.0 adapter rendering a v2.1 trace with saga events).

8.8 Determinism and Replay Semantics

8.8.1 Replay Mode

In replay mode (§12.3), the Runtime Core reads events from a historical trace file and re-emits them in sequence order, respecting the original timing and causality. Adapters (TUI, Web, VS Code) subscribe to the replayed event stream and render the historical run indistinguishably from a live run.

Replay guarantees.

- Every event in the original trace is re-emitted with the same **sequence**, **timestamp**, **kind**, and **payload**.
- New **event_id** values are assigned (event IDs are not stable across replays).
- Step execution does not occur; the runtime only emits events. No subprocesses are forked, no tools are invoked, no governance checks are evaluated.

8.8.2 Non-determinism Boundaries

The following actions introduce non-determinism and are not captured in events:

- System time (**timestamp** fields use wall-clock time, not logical time).
- External network calls (DNS resolution, HTTP responses, database queries).
- File system state outside **.runbook/**.
- Random UUIDs (**event_id** and **correlation_id** fields are regenerated in replay).

To achieve full deterministic replay (for testing and debugging), use dry-run mode (§8.2) or snapshot the external environment.

Chapter 9

Security and Trust

Gert v2 is designed to operate in regulated, high-trust environments where operational actions must be auditable, accountable, and governed by explicit policy. Security is not an add-on; it is foundational to the system's architecture. This chapter defines the threat model, trust boundaries, process isolation mechanisms, credential handling, transport security, RBAC model, audit non-repudiation, and supply chain security.

9.1 Threat Model

Gert v2 addresses the following threat categories, which are common in operational runbook systems and have real-world precedent in systems like AWS Systems Manager, PagerDuty Runbook Automation, and Rundeck [8, 23, 24].

This threat model is derived from NIST SP 800-53 security controls [1] and ISO/IEC 27001 guidance on privileged access management and audit logging [4].

Threat	Mitigation	Section Ref
Runbook author executes unauthorized commands	Command allowlist/denylist enforced at runtime core	§9.1, §11
Malicious extension injects code into host process	Out-of-process extension isolation; cryptographic signature verification	§9.2, §4
Tool or extension leaks credentials into trace	Mandatory redaction for sensitive fields; <code>sensitive: true</code> input marking	§9.4
Unauthorized user triggers runbook execution	RBAC enforced at <code>gert serve</code> ingress	§9.6
Man-in-the-middle attack on JSON-RPC transport	TLS required for all non-localhost listeners	§9.5
Replay attack on approval decision	Approval tokens signed with Ed25519; <code>approvalId</code> is single-use	§9.6, §11
Compromised tool binary executes untrusted code	Tool binary checksum verification before invocation	§9.8

TABLE 9.1: Threat model and mitigations for gert v2.

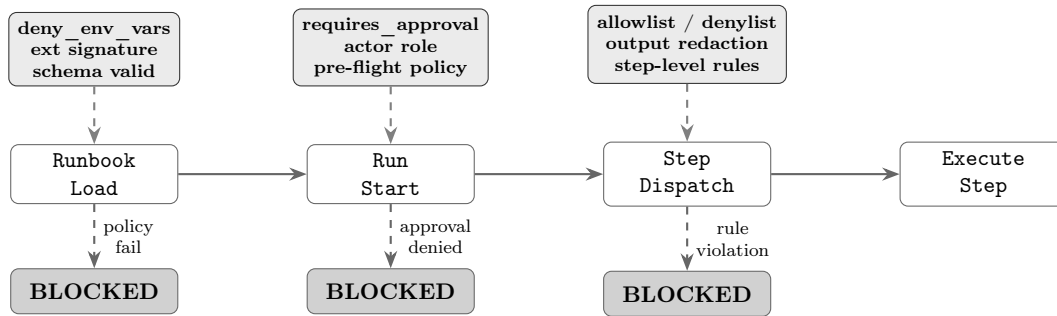


FIGURE 9.1: Governance enforcement points across the gert execution pipeline. Each stage has dedicated policy checks; failure at any checkpoint halts execution and emits a `governance/blocked` trace event.

9.2 Extension Trust Model

Extensions operate under one of three trust levels, declared explicitly in the extension manifest and evaluated by the host before any process is started.

9.2.1 Trust Levels

Local trusted. Installed by the machine owner (via package manager or explicit installation to a system-wide extension directory). These extensions run as child processes of the current user, inherit the user’s capability set (subject to manifest restriction), and have access to all host capabilities declared in their manifest. No signature verification is performed; trust is conferred by the installation act.

Example: an extension installed via `apt-get` or `brew` into `/usr/local/lib/gert-extensions/`.

Project-scoped. Declared in the runbook project directory (e.g., `.gert/extensions.yaml`) or inline in the runbook’s `extensions:` field. These extensions run in a restricted sandbox: only capabilities explicitly granted by the workspace governance policy are forwarded. File and network access are constrained to declared paths and hosts. Signature verification is *optional* but recommended.

Use case: a team-maintained extension for Acme-specific incident tooling, distributed via the runbook repository.

Remote/signed. Extensions distributed via package repository or downloaded from the internet. These extensions *must* carry a cryptographic signature verified against a trust store. The signature covers the extension manifest and binary. If verification fails, the extension is rejected before any subprocess is started.

Signature format is defined in Section 9.2.2.

9.2.2 Extension Signature Format

Remote/signed extensions must include a `signature:` block in `gert-extension.yaml`:

```
# gert-extension.yaml - signed extension
apiVersion: extension/v2

meta:
  name: acme.incident-pack
```

```
version: "1.2.0"

signature:
  algorithm: ed25519
  publicKeyFingerprint: "sha256:abcd1234..."
  signatureData: "base64-encoded-signature"
  signedFields: ["meta", "capabilities", "entry-point", "transport"]

trust_chain:
- fingerprint: "sha256:abcd1234..."
  issuer: "Acme Platform Team CA"
  notBefore: "2024-01-01T00:00:00Z"
  notAfter: "2025-01-01T00:00:00Z"
```

Signature verification algorithm. The signature is computed over the canonical JSON representation of the fields listed in `signedFields`. The host:

1. Extracts the public key fingerprint from the `signature` block.
2. Looks up the corresponding public key in the trust store (`$HOME/.gert/trusted-keys/` or `/etc/gert/trusted-keys/`).
3. Reconstructs the canonical JSON of the signed fields (sorted keys, no whitespace).
4. Verifies the Ed25519 signature over the canonical JSON bytes.
5. Checks that the current time falls within the `notBefore` and `notAfter` window of the trust chain.

If any step fails, the extension is rejected with a `governance/extensionRejected` trace event and reason `signature_verification_failed`.

9.2.3 Extension Verification Command

The `gert extension verify` command allows operators to manually verify an extension before installation:

```
$ gert extension verify ./acme-extension/gert-extension.yaml \
  --trust-store /etc/gert/trusted-keys/
```

```
Extension: acme.incident-pack v1.2.0
Signature: valid (ed25519, expires 2025-01-01)
Issuer: Acme Platform Team CA
Capabilities: tool-registration, network
Trust level: remote/signed
Verification: PASSED
```

9.3 Process Isolation

Extensions and tools always run as separate OS processes. They are never loaded as shared libraries or executed in-process. This isolation boundary ensures that a crash, hang, or malicious extension cannot compromise the host process.

9.3.1 Isolation Mechanisms

Separate OS process. As defined in §4, extensions are forked as child processes with their own address space, file descriptor table, and signal handlers.

Filesystem access control. Extensions with `capability/file-read` or `capability/file-write` declare the set of filesystem paths they may access via `file-read-paths` and `file-write-paths` in the manifest. The host enforces these constraints:

- On OpenBSD: via `unveil(2)` to restrict visible paths.
- On Linux: via `seccomp-bpf` to filter `open(2)` and `openat(2)` syscalls.
- On all platforms: by validating all path arguments in `file-read` and `file-write` RPC methods before dispatch.

Attempts to access undeclared paths return a `capability-denied` error.

Network access control. Extensions with `capability/network` declare the set of hostnames they may connect to via `network-hosts`. The host validates outbound connections:

- Before dispatching an extension RPC that specifies a target URL, the host checks that the hostname matches one of the declared `network-hosts` patterns.
- Extensions may not listen for inbound connections.

- Wildcard patterns are supported: `*.acme.example` permits `api.acme.example`, `prod.acme.example`, etc.

Environment variable isolation. Extensions receive ONLY the environment variables explicitly forwarded by the runbook's `governance.allowed_env` list. Host environment variables that match `governance.deny_env_vars` patterns are never forwarded, even if they appear in `allowed_env`.

The denylist takes precedence: if `AWS_SECRET_ACCESS_KEY` is denied, it is stripped from the extension's environment regardless of other policy.

Crash isolation. If an extension process crashes (exits with non-zero status or is killed by a signal), the host:

1. Emits an `extension.crashed` trace event with exit code and signal number.
2. Marks the extension as unavailable for the remainder of the run.
3. Cancels all in-flight invocations to that extension with error code `-32099` (`extension_crashed`).
4. Continues the run: the host process does not terminate.

9.4 Credential and Secret Handling

Gert v2 does not store secrets. It acts as a pass-through: secrets are resolved from input providers at runtime, used for a single step invocation, and then discarded. Secrets never appear in trace logs or captured output.

9.4.1 Secret Resolution and Redaction

Input provider sensitivity marking. Input provider definitions declare which resolved values are sensitive via `sensitive: true` in the provider schema:

```
# vault.provider.yaml
outputs:
  credentials:
    type: object
    sensitive: true
```

When the host resolves `from: vault.credentials`, it marks the returned value as sensitive in memory. This flag is respected by all downstream consumers.

Tool sensitive inputs. Tool action definitions declare which input parameters accept sensitive data via `sensitive_inputs`:

```
# my-tool.tool.yaml
actions:
  deploy:
    args:
      api_key:
        type: string
        required: true
    sensitive_inputs: [api_key]
```

When the host prepares the tool invocation envelope, it marks the `api_key` field as sensitive. The tool runtime redacts this field in all trace events.

Mandatory redaction. All values marked `sensitive: true` are redacted before being written to the trace or displayed in any adapter UI. Redaction replaces the value with the string `<redacted>`.

In addition, the runbook's `governance.redact` block defines regex patterns applied to all captured output (stdout, stderr, JSON responses) before storage. See §11 for the full redaction specification.

9.4.2 Secret Storage Policy

Gert v2 does NOT:

- Store secrets in the runbook YAML.
- Write secrets to disk (trace files, state snapshots, etc.).
- Cache secrets across steps (unless explicitly declared by the input provider's `cache_scope`).
- Transmit secrets over unencrypted transports.

Secrets must be resolved via input providers that integrate with secret management systems (e.g., HashiCorp Vault, AWS Secrets Manager, 1Password).

9.5 Transport Security

Gert v2 supports two primary transport modes for JSON-RPC communication:

1. **stdio**: JSON-RPC over stdin/stdout of a child process. No network exposure; inherently local. No TLS required.
2. **HTTP/WebSocket**: JSON-RPC over HTTP for `gert serve` mode. Network exposure possible; TLS and authentication required for non-localhost.

9.5.1 TLS Requirements

Localhost exception. If `gert serve` binds to `127.0.0.1` or `localhost`, TLS is optional. Connections are assumed to be local and trusted.

Non-localhost listeners. If `gert serve` binds to `0.0.0.0`, a public IP, or a hostname, TLS is *mandatory*. The host will refuse to start without a valid TLS certificate and private key.

Configuration:

```
$ gert serve --http 0.0.0.0:8443 \  
  --tls-cert /etc/gert/tls/cert.pem \  
  --tls-key /etc/gert/tls/key.pem
```

The host performs standard TLS 1.3 handshake with the client. TLS 1.2 is supported for compatibility but TLS 1.1 and earlier are rejected.

9.5.2 Client Authentication

The host supports two authentication modes for `gert serve -http`:

Bearer token authentication. Clients include an `Authorization: Bearer <token>` header on all HTTP requests. The token is validated against a configured token store (file-based, environment variable, or integration with an IdP).

Example:

```
$ gert serve --http :8443 --auth-mode bearer \  
  --bearer-tokens /etc/gert/tokens.yaml
```

The `tokens.yaml` file maps tokens to actor identities:

```
# tokens.yaml
tokens:
- token: "abc123..."
  actor: "ops@acme.example"
  roles: ["operator", "viewer"]
```

Mutual TLS (mTLS). The host requires clients to present a valid X.509 client certificate signed by a trusted CA. The client certificate's subject CN or SAN is used as the actor identity.

Configuration:

```
$ gert serve --http :8443 --auth-mode mtls \
  --tls-client-ca /etc/gert/tls/client-ca.pem
```

mTLS is the recommended authentication mode for production deployments and enterprise environments.

9.5.3 WebSocket Security

WebSocket connections for the event stream (§13) inherit the security context of the HTTP server:

- If the HTTP server uses TLS, WebSocket connections are encrypted (`wss://`).
- Client authentication applies: bearer tokens or client certificates are validated before the WebSocket upgrade.
- Once upgraded, the WebSocket connection is bound to the authenticated actor identity for the lifetime of the connection.

9.5.4 Extension JSON-RPC Transport

Extensions communicate with the host via stdio JSON-RPC (§4). This transport is not exposed to the network and does not require TLS. The process boundary is the trust boundary: the host trusts that OS-level process isolation prevents interception of stdin/stdout pipes.

9.6 RBAC for gert serve

When gert runs in server mode (`gert serve -http`), it enforces role-based access control (RBAC) on all API operations. Actors are assigned roles, and each role grants a specific set of permissions.

9.6.1 Actor Identity

An **actor** is the authenticated identity making a request. Actor identity is derived from:

- The bearer token (mapped to an actor ID via the token store), or
- The client certificate subject CN/SAN (in mTLS mode).

The actor identity is recorded in all trace events where a human or external system initiates an action (run start, approval submission, cancellation).

9.6.2 Roles

Gert v2 defines three standard roles:

Viewer. Read-only access. Permissions:

- List runs (`GET /runs`)
- Get run details (`GET /runs/:id`)
- Subscribe to event stream (`GET /ws` or `GET /events`)
- Read trace files

Cannot start runs, approve, or cancel.

Operator. Execution permissions. Includes all **viewer** permissions plus:

- Start a run (`POST /exec/start`)
- Cancel a run (`POST /exec/cancel`)
- Submit approvals (`POST /approvals`)

Cannot modify governance policy or manage roles.

Admin. Full control. Includes all **operator** permissions plus:

- Modify workspace governance policy
- Manage role assignments (add/remove actors from roles)
- Configure extension trust store

9.6.3 Role Assignment

Roles are assigned in the workspace config (`gert-project.yaml`):

```
# gert-project.yaml
access:
  roles:
    - role: admin
      actors:
        - "alice@acme.example"
        - "bob@acme.example"
    - role: operator
      actors:
        - "ops-team@acme.example"
        - "oncall@acme.example"
    - role: viewer
      actors:
        - "*@acme.example" # All Acme employees
```

Actor patterns support wildcards for organization-wide grants.

9.6.4 Approval Authorizer Role

The `governance.require_approval` field in the runbook declares the role required to approve a step:

```
steps:
  - id: delete_database
    cli: "kubectl delete database prod-db"
    approvals:
      min: 1
      roles: ["dba", "admin"]
```

When the run reaches this step, it emits an **approval/required** trace event. The step is paused. Only actors with the **dba** or **admin** role may submit a valid approval.

The approval submission includes:

```
POST /approvals
{
  "runId": "run-abc123",
  "stepId": "delete_database",
  "decision": "approve",
  "actorId": "alice@acme.example",
  "signature": "<ed25519-signature-of-approval-payload>",
  "timestamp": "2026-04-18T12:00:00Z"
}
```

The host verifies:

1. The actor is authenticated and has one of the required roles.
2. The signature is valid (Ed25519 over the canonical JSON of **runId**, **stepId**, **decision**, **actorId**, **timestamp**).
3. The **timestamp** is within a 5-minute clock skew window.
4. The **runId** and **stepId** match an active approval gate.
5. This approval has not been submitted before (replay protection).

If all checks pass, the approval is recorded in the trace (**approval/received**) and the step proceeds if the minimum approval count is reached.

9.7 Audit Non-Repudiation

Non-repudiation ensures that governance decisions — especially approvals and rejections — cannot be denied by the actor who made them. This is critical for regulatory compliance (SOC2, ISO27001, HIPAA) [4, 7, 28].

9.7.1 Governance Decision Recording

Every governance decision event (`governance/approval_received`, `governance/approval_rejected`, `governance/commandBlocked`) must record:

- Actor identity (authenticated user or service principal)
- Timestamp (RFC 3339 with timezone) [19]
- Decision payload (what was approved/rejected)
- Signature (Ed25519 over the canonical decision payload)

Example trace event:

```
{
  "eventType": "governance/approval_received",
  "timestamp": "2026-04-18T12:00:00Z",
  "runId": "run-abc123",
  "stepId": "delete_database",
  "actor": "alice@acme.example",
  "decision": "approve",
  "signature": "ed25519:abcd1234...",
  "signatureAlgorithm": "ed25519",
  "signaturePublicKey": "sha256:pubkey-fingerprint"
}
```

The signature covers the fields `runId`, `stepId`, `actor`, `decision`, and `timestamp`. The public key is looked up from the actor's identity record or client certificate.

9.7.2 Trace File Integrity (HMAC Chaining)

The JSONL trace file (§12) is append-only, but this alone does not prevent tampering. To provide cryptographic integrity, the host optionally computes an HMAC chain over the trace:

1. At run start, initialize $H_0 = \text{HMAC-SHA256}(\text{key}, \text{runId})$.
2. After writing each trace event E_i , compute $H_i = \text{HMAC-SHA256}(\text{key}, H_{i-1} || E_i)$.

3. Store `H_i` in the event as `integrityHash`.
4. At run completion, emit a final `run.completed` event with `finalHash: H_N`.

Verification: a verifier recomputes the HMAC chain from `H_0` forward. If any event is modified or deleted, the chain breaks.

The HMAC key is derived from the workspace master key or provided via `-integrity-key`.

9.7.3 SHA256 Evidence Capture

As defined in §12, gert captures SHA256 hashes of all evidence artifacts (attachments, screenshots, CLI output files). These hashes are written to the trace event immediately after the artifact is captured, providing a cryptographic anchor for the artifact's integrity.

The combination of HMAC trace chaining + SHA256 evidence hashes provides end-to-end non-repudiation for the entire run.

9.8 Supply Chain Security

Tools and extensions are executable code distributed via package managers, repositories, or direct download. Supply chain attacks (e.g., compromised binaries, backdoored dependencies) are a real threat in operational environments. Gert v2 provides checksum verification and signature validation to mitigate these risks.

9.8.1 Tool Binary Checksums

Tool definitions may declare a `checksum` field in `.tool.yaml`:

```
# kubectl.tool.yaml
meta:
  name: kubectl
  binary: kubectl

checksum:
  algorithm: sha256
  digest: "a3b4c5d6e7f8..."
```

Before invoking the tool, the host:

1. Resolves the binary path (via `PATH` lookup or absolute path).
2. Computes the SHA256 hash of the binary file.
3. Compares the computed hash to `checksum.digest`.
4. If the hashes do not match, the tool invocation is blocked with a `governance/toolChecksumMismatch` trace event.

9.8.2 gert verify Command

The `gert verify` command validates all tools and extensions in the workspace before execution:

```
$ gert verify --workspace .
Verifying tools...
✓ kubectl: checksum verified (sha256:a3b4c5...)
✓ curl: no checksum declared (skipped)
× jq: checksum mismatch (expected sha256:abc123, got sha256:def456)

Verifying extensions...
✓ acme.incident-pack: signature valid (ed25519, expires 2025-01-01)
× debug-tools: no signature (required for remote extensions)

Verification: FAILED (2 errors)
```

This command should be run in CI/CD pipelines before deploying runbooks to production.

9.8.3 Extension Signature Verification

As defined in §9.2.2, remote extensions must carry a cryptographic signature. The signature verification process ensures that:

- The extension binary and manifest have not been tampered with.
- The extension was signed by a trusted authority (public key in the trust store).
- The signature has not expired (`notAfter` check).

If verification fails, the extension is rejected before any subprocess is started.

9.9 Security Recommendations

For production deployments, the following security practices are recommended:

1. **Enable TLS for all gert serve deployments** — even internal networks can be compromised.
2. **Use mTLS for client authentication** — bearer tokens are vulnerable to theft; client certificates provide stronger identity.
3. **Enable HMAC trace chaining** — provides cryptographic integrity for audit trails.
4. **Declare tool binary checksums** — prevents execution of tampered binaries.
5. **Require signatures for all remote extensions** — do not trust unsigned code from the internet.
6. **Run gert verify in CI** — catch integrity violations before production.
7. **Use least-privilege capability grants** — extensions should request only the capabilities they need; admins should audit and deny excessive requests.
8. **Regularly rotate approval keys** — the Ed25519 keys used for signing approvals should be rotated every 90 days.

These recommendations align with NIST SP 800-53 controls for privileged access management (AC-2, AC-6), audit and accountability (AU-2, AU-9), and system integrity (SI-7) [1].

Chapter 10

Testing and Acceptance

This section specifies the complete testing strategy for gert v2: the test framework, unit and contract test patterns, integration test strategy, the `gert test` feature, regression testing against the v1 runbook corpus, performance acceptance criteria, the extension test harness, and CI/CD gate requirements.

10.1 Test Framework

10.1.1 Standard Library and testify

All gert v2 tests are written in Go using the standard `testing` package as the test runner and github.com/stretchr/testify [26] for assertions:

- `testify/assert` — non-fatal assertions; test continues on failure.
- `testify/require` — fatal assertions; test halts on failure.
- `testify/mock` — interface mocking for unit tests (engine, event bus, trace writer).

Table-driven tests are the preferred pattern for all logic that has well-defined input/output pairs (e.g. expression evaluation, schema validation, governance rule matching):

```
func TestGovernanceAllowlist(t *testing.T) {
    cases := []struct {
        name      string
        cmd       []string
        allowed   bool
    }{
```

```
    {"exact match", []string{"kubectl", "get", "pods"}, true},
    {"partial match", []string{"kubectl", "delete", "ns"}, false},
    {"empty allowlist", []string{"echo", "hi"}, false},
  }
  for _, tc := range cases {
    t.Run(tc.name, func(t *testing.T) {
      result := allowlist.Check(tc.cmd)
      assert.Equal(t, tc.allowed, result)
    })
  }
}
```

10.1.2 Test Packages

Each Go package in the v2 module ships a co-located `_test.go` file (white-box tests) and optionally an `<pkg>_integration_test.go` file (integration tests, gated by a build tag). The naming convention:

```
pkg/engine/engine.go
pkg/engine/engine_test.go           // white-box unit tests
pkg/engine/engine_integration_test.go // build tag: //go:build integration
```

10.2 The gert test Feature

10.2.1 Scenario Replay Testing

`gert test <runbook.yaml>` discovers and executes all scenario tests for a runbook. A scenario test consists of:

- `inputs.yaml` — resolved input values for the run.
- `scenario.yaml` (optional) — pre-recorded command responses and manual evidence.
- `test.yaml` — assertions over the resulting trace and captures.

Scenario directories are located by convention:


```
{runbook-dir}/scenarios/{runbook-name}/{scenario-name}/
  inputs.yaml
  scenario.yaml  # optional; omit for runbooks with no CLI/tool steps
  test.yaml
```

The test runner executes the runbook in replay mode using the scenario file as the command/evidence fixture, then evaluates assertions from `test.yaml`.

10.2.2 Test Assertions (`test.yaml`)

```
assertions:
  outcome: passed          # "passed" / "failed" / "skipped"
  steps_passed: 5
  steps_failed: 0

captures:
  db_host:
    equals: "prod-db.example.com"
  migration_status:
    contains: "complete"

trace:
- type: step/completed
  step_id: run-migration
- type: governance/blocked
  step_id: drop-table
  data.rule: denylist
```

The `trace` assertions match events in order of occurrence. The `data.*` dotted paths dereference into the event's `data` object.

10.2.3 v1 Scenario Compatibility

v1 scenario files (`commands/evidence` structure) are forward-compatible with v2 without modification. The v2 parser accepts both the v1 format and the extended v2 format (which adds the `assertions` block). Runbook authors can migrate incrementally by adding `test.yaml` alongside existing v1 scenarios.

10.3 Contract Tests

Contract tests verify the interface boundaries between v2's five components. Each contract test lives in the *consumer* package and uses a conformance helper that validates any implementation satisfying the interface.

10.3.1 Core Runtime Contract

The runtime `Executor` interface must satisfy:

- Emits a `run/started` event before executing any step.
- Emits a `step/started` before and `step/completed/step/failed` after each step.
- Emits `run/completed` or `run/failed` as the final event.
- Does not emit events after the terminal event.
- Does not call any adapter method directly (events only).

```
// ConformanceTest validates any Runtime implementation.
func ConformanceTest(t *testing.T, rt Runtime) {
    t.Helper()
    bus := newTestEventBus()
    err := rt.Execute(context.Background(), fixtureRunbook, bus)
    require.NoError(t, err)
    events := bus.Events()
    require.Equal(t, "run/started", events[0].Type)
    require.Equal(t, "run/completed", events[len(events)-1].Type)
    assertNoEventsAfterTerminal(t, events)
}
```

10.3.2 Schema Contract Tests

- Every `apiVersion:` `runbook/v2` document in `testdata/` must pass structural validation without errors.
- Every invalid fixture in `testdata/invalid/` must fail validation with a structured error (message + path).

- Structural and semantic validation are tested independently: a structurally valid document may fail semantic validation (e.g. referencing an undefined capture).

10.3.3 Extension Manifest and Handshake Tests

The extension host verifies:

- An extension process that responds to the initialisation handshake within the configured timeout is accepted.
- An extension process that fails to respond is terminated and its capabilities are not registered.
- Capability grants in the manifest are enforced: an extension without `cap:schema.register` cannot register schema.
- A manifest with missing required fields is rejected at load time with a structured error.

10.3.4 Tool Invocation Envelope Tests

- Tool invocations carry the correct `run_id` and `step_id` correlation fields.
- Tool outputs are captured and available as step captures after the step completes.
- A tool that exits with non-zero triggers `step/failed` with `error_type: tool_error`.
- Timeout cancellation: a tool that exceeds its timeout receives a `context.DeadlineExceeded` and emits `step/failed` with `error_type: timeout`.

10.3.5 Event Bus Determinism Tests

- Events delivered to the event bus maintain the `seq` ordering guaranteed by `RunStore.WriteTrace`.
- Slow adapter consumers do not cause the engine's execution loop to block (bounded channel or dropping policy required).
- Event replay (`ReadTraceSince`) returns events in ascending `seq` order.

10.4 Integration Test Strategy

10.4.1 End-to-End Fixture Runs

Integration tests spin up the gert runtime against fixture runbooks stored in `testdata/runbooks/` and assert on the resulting trace files. They are gated by the `integration` build tag and run in CI separately from unit tests.

```
//go:build integration

func TestE2E_BranchingRunbook(t *testing.T) {
    dir := t.TempDir()
    runID, err := exec.RunFixture(dir, "testdata/runbooks/branch.runbook.yaml",
        WithInputs(map[string]string{"env": "prod"}),
        WithScenario("testdata/scenarios/branch-prod.yaml"),
    )
    require.NoError(t, err)
    trace := readTrace(t, dir, runID)
    assertEvent(t, trace, "step/completed", map[string]any{
        "data.step_id": "deploy-prod",
    })
    assertNoEvent(t, trace, "step/completed", map[string]any{
        "data.step_id": "deploy-staging",
    })
}
```

10.4.2 Fixture Runbook Corpus

The integration test corpus includes fixtures covering:

- Linear runbook (all step types: cli, manual, tool, invoke).
- Branching runbook (both branch arms; governance-blocked branch).
- Iterate runbook (list mode, convergence mode, parallel concurrency).
- Nested invoke (parent + two child runbooks).
- Retry policy (transient failure followed by success).

- Resumption (crash after step 2, resume, verify trace continuity).
- Governance: allowlist violation, denylist violation, approval gate.

10.4.3 Trace Assertion Helpers

A shared test package `pkg/testutil` provides helpers:

```
package testutil

// ReadTrace reads and parses all events from a run's trace.jsonl.
func ReadTrace(t *testing.T, runDir, runID string) []engine.DurableEvent

// AssertEvent asserts that at least one event matches type and data predicates.
func AssertEvent(t *testing.T, events []engine.DurableEvent,
    eventType string, predicates map[string]any)

// AssertEventOrder asserts events appear in the given type order.
func AssertEventOrder(t *testing.T, events []engine.DurableEvent,
    types ...string)

// AssertNoEvent asserts no event matches the given type and predicates.
func AssertNoEvent(t *testing.T, events []engine.DurableEvent,
    eventType string, predicates map[string]any)
```

10.5 Regression Testing

10.5.1 v1 Runbook Corpus as Fixtures

The v1 runbook corpus (located in `examples/` and operator-contributed runbooks) serves as a regression suite for v2 compatibility. The regression test runner:

1. Reads each v1 runbook from the corpus.
2. Runs `gert validate` (v2 schema) and asserts it passes (or passes after migration).
3. For runbooks with associated v1 scenario files, runs `gert test` and asserts the outcome matches the v1 golden output.

4. Fails the build if any previously passing corpus runbook regresses.

Migration compatibility is measured as: the percentage of v1 corpus runbooks that pass v2 `gert validate` without modification. The v2 launch target is $\geq 95\%$.

10.5.2 Golden Trace Comparisons

For deterministic scenarios, the expected trace can be stored as a golden file. The golden comparison test:

1. Runs the scenario.
2. Normalises timestamps and run IDs (replaces with fixed values).
3. Compares the normalised JSON-lines output to `testdata/golden/<scenario>.jsonl`.
4. Fails if any event differs. Golden files are updated by running with `-update` flag.

10.6 Performance Acceptance Criteria

The following latency budgets must pass on a reference machine (4-core, 8 GB RAM, SSD) with the integration test suite:

Operation	p50 target	p99 target
<code>gert validate</code> (single runbook, <50 steps)	<50 ms	<200 ms
Step startup (cli step)	<10 ms	<50 ms
Event delivery to adapter (local channel)	<1 ms	<5 ms
Trace event write + fsync	<5 ms	<20 ms
Snapshot write (100 steps history)	<20 ms	<80 ms
<code>gert test</code> (10-step replay scenario)	<500 ms	<1 s
Tool invocation round-trip (stdio, local process)	<50 ms	<200 ms

Performance benchmarks are written using Go's `testing.B` and run in CI weekly (not on every PR). A regression of $>2\times$ on any metric blocks the nightly build.

10.7 Extension Test Harness

10.7.1 Overview

Extension developers can test their extension logic without spinning up the full gert runtime. The `pkg/exttest` package provides a test harness:

```
import "github.com/ormasoftchile/gert/v2/pkg/exttest"

func TestMyExtension(t *testing.T) {
    h := exttest.NewHarness(t,
        exttest.WithManifest("testdata/my-ext.manifest.yaml"),
        exttest.WithCapabilities("cap:schema.register", "cap:tool.register"),
    )
    defer h.Close()

    // Start the extension process
    h.Start("./my-extension")

    // Assert it registered expected tools
    tools := h.RegisteredTools()
    require.Contains(t, tools, "my-tool")

    // Invoke a tool and assert the response
    resp := h.InvokeTool("my-tool", map[string]any{"arg1": "value"})
    assert.Equal(t, 0, resp.ExitCode)
    assert.Contains(t, resp.Stdout, "expected output")
}
```

10.7.2 Harness Capabilities

The test harness provides:

- A mock extension host that implements the full handshake protocol.
- Assertion helpers for registered tools, schema registrations, and event subscriptions.
- A `FakeRunContext` that provides a realistic run context without executing a runbook.
- Automatic process cleanup on test completion.

- Configurable capability grants (to test capability enforcement).

10.7.3 Mocking the Extension Transport

For pure unit tests of extension logic (without spawning a subprocess), the transport layer is mockable:

```
// InProcessTransport runs the extension handler in-process for unit testing.  
type InProcessTransport struct {  
    Handler ExtensionHandler  
}
```

10.8 Acceptance Criteria

The following acceptance criteria must be satisfied before a v2 release is considered shippable:

1. Core runtime has no direct adapter dependencies — verified by the `no_import` contract test.
2. Extensions register schema and tools through declared contracts — verified by the extension handshake conformance suite.
3. Capability violations fail fast with structured errors — verified by the governance contract test for each capability type.
4. Adapters integrate using the same stable core contracts — verified by running each adapter’s integration test against a mock core.
5. Every step type (`cli`, `manual`, `tool`, `invoke`, `branch`, `iterate`) has ≥ 1 end-to-end integration test covering the happy path and at least one failure path.
6. The v1 runbook corpus passes `gert validate` at $\geq 95\%$ without modification.
7. All performance targets in § 8.6 pass on the reference machine.
8. `gert test` successfully replays all scenario fixtures in `testdata/scenarios/`.

10.9 CI/CD Integration

10.9.1 PR Gate (every pull request)

1. **Unit tests:** `go test ./...` (no build tags). Target: <60 s.
2. **Schema validation:** validate all YAML fixtures in `testdata/` against v2 JSON Schema. Target: <10 s.
3. **Contract tests:** `go test -run Contract ./...` Target: <30 s.
4. **Linters:** `golangci-lint` run with the project's `.golangci.yaml` config.
5. **Vet:** `go vet ./...`

All five gates must pass for a PR to be mergeable.

10.9.2 Integration Gate (merge to main)

1. **Integration tests:** `go test -tags=integration ./...` Target: <5 min.
2. **Regression corpus:** `gert test` against v1 corpus. Target: <2 min.
3. **Golden trace comparison:** normalised trace comparison for all deterministic fixtures.

10.9.3 Nightly Gate

1. **Performance benchmarks:** `go test -bench=. -benchtime=10s ./...` — compare against stored baseline; fail on >2× regression.
2. **Race detector:** `go test -race ./...`
3. **Extension conformance suite:** run all built-in extensions through the conformance harness.

Chapter 11

Open Questions

The questions below are unresolved architectural decisions. Each entry states **(a)** why the question matters for the design, **(b)** the options under consideration, and **(c)** what additional information is needed to close it. Decisions must be recorded in `.squad/decisions.md` before core contract freeze.

Q1. Can the v2 runtime execute v1 runbooks without prior migration?

Why it matters: Existing gert users have production runbooks in `runbook/v0` and `runbook/v1` format. If v2 can only execute v2-schema runbooks, adoption requires a migration step that may be blocked by organizational change control. A compatibility execution mode (“v1 shim”) removes this barrier but adds permanent maintenance cost to the parser.

Options: (a) Full compatibility mode: v2 runtime transparently executes v1 runbooks by internally upgrading them to v2 schema at load time, with no schema file written. (b) Migration gate: v2 refuses to run v1 runbooks and provides a clear error pointing to `gert migrate`. (c) Opt-in compatibility flag: `-compat-v1` enables the shim; production is v2-only.

Information needed: A survey of how many v1-format runbooks exist in representative deployments, and whether `gert migrate` can be made fully lossless for all v1 constructs.

Q2. What is the concurrency model for the core runtime?

Why it matters: The v1 runtime is single-run-per-process. Whether v2 supports multiple concurrent runs in a single process affects the state isolation model, the trace writer design, the governance enforcer (is it per-run or global?), and whether `gert serve` can multiplex concurrent VS Code sessions.

Options: (a) Single-run-per-process, matching v1 semantics; each `gert exec` or `gert tui` is a separate OS process. (b) Multiple concurrent runs in-process, with run-scoped state isolation via Go context and per-run goroutine groups. (c) Multiple runs via a long-lived daemon (`gert serve` as the host), with runs as logical sessions.

Information needed: Whether the VS Code extension use case requires concurrent runs in a single `gert serve` process (e.g., multiple open runbook panels), and what the goroutine safety requirements are for the trace writer and governance layer.

Q3. Is the v2 trace format backward-compatible with the v1 JSONL trace, or is it a new format?

Why it matters: Operators and compliance tools may have parsers, SIEM integrations, or dashboards built against the v1 JSONL event schema. A breaking change in the trace format requires those integrations to be updated before v2 can be rolled out. A backward-compatible superset format avoids this but constrains the v2 event schema design.

Options: (a) New format: v2 defines a clean JSONL schema with a required `schema_version` field; v1 trace parsers are unsupported. (b) Backward-compatible superset: v2 trace events include all v1 fields plus new fields; existing parsers continue to work on the subset they understand. (c) Parallel trace output: v2 writes both a v2 trace and an optional v1 compatibility trace (flag-enabled).

Information needed: An inventory of known downstream trace consumers and whether the v1 format has fields that would be structurally incompatible with the v2 event model.

Q4. How do extension-contributed policies compose with host-level governance policies?

Why it matters: The host enforces a governance policy (command allowlist, env blocking, redaction). An extension may also wish to contribute policies—e.g., a security extension that adds additional command denylists or redaction patterns. The composition model determines whether extensions can weaken host policies (prohibited), extend them (likely desired), or operate in a separate policy namespace (safest but most limited).

Options: (a) Additive composition only: extensions may only add restrictions (denylist entries, redaction patterns); they may never remove host-level restrictions. (b) Policy namespacing: each extension has an isolated policy scope; the host applies all active

policies via intersection. (c) Policy delegation: the host can explicitly delegate a policy domain to an extension (e.g., redaction rules owned by a data-classification extension).

Information needed: A threat model analysis of what happens when a malicious or compromised extension contributes a policy that attempts to weaken host restrictions. This should feed into §9.

Q5. Does the `.provider.yaml` input provider framework survive unchanged in v2?

Why it matters: v1's input provider model (`.provider.yaml`, JSON-RPC resolution, `from:` bindings, workspace config) is a complete and working framework. It must either be carried forward as-is, redesigned to fit the v2 component model, or replaced by an extension-based resolution mechanism. The choice affects the schema (§4), the extension runtime (§6), and any operator who has a custom provider binary.

Options: (a) Preserve as-is: `.provider.yaml` format and JSON-RPC protocol are unchanged; providers are a stable first-class concept. (b) Redesign as extension capability: providers become a named capability that extensions may implement; the `.provider.yaml` format is deprecated. (c) Hybrid: `.provider.yaml` providers continue to work as a built-in extension type, with a migration path to the full extension protocol.

Information needed: Whether any v1 provider binaries are in active production use that would be broken by a protocol change, and whether the extension handshake protocol can be made a strict superset of the provider JSON-RPC protocol.

Q6. Is the `gert serve` JSON-RPC contract a versioned breaking change or an evolutionary extension?

Why it matters: The VS Code extension consumes the `gert serve` JSON-RPC 2.0 API over stdio. If v2 changes the method names, parameter shapes, or notification payloads, the extension must be updated simultaneously. A versioned negotiation protocol (client declares supported API version at handshake) allows the extension to evolve independently of the binary.

Options: (a) Hard break: v2 defines a new JSON-RPC method set; the VS Code extension is updated as part of the v2 launch. (b) Versioned negotiation: the `initialize` handshake includes a protocol version field; the host advertises capabilities and the client selects a compatible version. (c) No change: the v1 JSON-RPC surface is preserved unchanged in v2.

Information needed: A complete method inventory of the v1 JSON-RPC API surface (all methods and notification shapes currently used by the VS Code extension), and a determination of which methods require changes to expose v2-specific features (e.g., saga events, OTel trace IDs).

Q7. What is the extension supply chain and signing model?

Why it matters: Extensions run in separate processes but are granted capabilities that include tool registration, schema field contribution, and event subscription. A malicious or tampered extension binary could exfiltrate captured secrets, inject commands, or suppress governance events. An extension signing model (verified publisher key) raises the bar for supply chain attacks, but adds operational friction for local development.

Options: (a) No signing required; extensions are trusted if they are present in the declared extension directory. (b) Optional signing: extensions may carry a detached signature; the host warns but does not block unsigned extensions. (c) Mandatory signing for capability classes above a threshold (e.g., extensions that request `governance.policy` capability must be signed). (d) Keyless signing via Sigstore/cosign for developer convenience.

Information needed: A threat model for the extension trust boundary (see Q4 above), and a survey of whether the target operator community has PKI infrastructure that could issue extension signing keys.

Q8. Is a gRPC transport for the VS Code extension viable for v0, or should it be deferred?

Why it matters: The current JSON-RPC 2.0 over stdio transport works but limits the VS Code extension to a single active connection and requires the extension to manage the gert process lifecycle. A gRPC transport would enable streaming (server-push events without polling), multiple concurrent clients, and connection multiplexing. However, it adds protocol complexity and requires a port allocation model.

Options: (a) Retain JSON-RPC stdio as the sole transport in v2; gRPC deferred to a future version. (b) Add gRPC as an opt-in alternative transport alongside JSON-RPC stdio. (c) Replace JSON-RPC stdio with gRPC as the primary VS Code transport; provide a backward-compatible shim for the transition period.

Information needed: Benchmarking of event delivery latency over JSON-RPC stdio versus gRPC under realistic VS Code workloads (step execution at 100ms/step, TUI

refresh at 60fps equivalent). Also requires a determination of whether VS Code's Node.js extension host supports gRPC client libraries without native module issues.

Q9. What is the parallel execution model for independent steps within a single run?

Why it matters: v1 executes all steps sequentially. Many real runbooks contain independent diagnostic steps that could execute in parallel, reducing total wall-clock time. However, parallelism complicates the trace ordering guarantee, the governance enforcer (can two parallel steps both consume the same approval token?), and the TUI rendering model.

Options: (a) No parallelism in v2; fan-out deferred to a future version. (b) Explicit parallel blocks via a new `parallel:` step type that joins on completion of all children. (c) Implicit parallelism inferred from the step dependency graph (steps with no data dependency on preceding steps execute concurrently).

Information needed: Whether any v1 runbooks in active use have independent steps that would benefit from parallelism, and what the governance implications are for concurrent step execution (e.g., two CLI steps executing simultaneously under the same allowlist).

Q10. How should the saga compensation registry interact with the trace and approval model?

Why it matters: A compensation registered for a step must itself be subject to governance (it may execute privileged commands), must appear in the trace (compensations are audit events), and may require its own approval gate if the original step had one. The interaction between the compensation executor and the governance layer is non-trivial and must be specified before implementation.

Options: (a) Compensations execute with the same governance context as the original step (same allowlist, same approver roles). (b) Compensations are exempt from approval gates (compensations are emergency rollback actions; blocking them on approval would defeat their purpose). (c) Compensation governance is separately configurable per compensation definition.

Information needed: Industry precedent from Temporal (compensation activities run in the same workflow context) and AWS Step Functions [9] (catch/retry blocks have no separate IAM policy). A decision here feeds into §4 (compensation field design) and §9 (governance scope).

Q11. What is the schema namespace convention for extension-contributed fields, and how are they validated?

Why it matters: The v2 schema design (§4) states that extension fields are namespaced, but the namespace convention and validation mechanism are undefined. If two extensions contribute fields under the same namespace, or if the host does not validate extension-contributed fields against a schema, runbooks using those fields will have unpredictable behavior at runtime.

Options: (a) Extension ID as namespace prefix (`x-myext.fieldName`); validated against a JSON Schema fragment published by the extension at registration time. (b) Dot-notation namespace under a reserved top-level key (`extensions.myext.fieldName`); host validates at parse time. (c) Extension fields are opaque to the host; the extension validates its own fields at execution time.

Information needed: Whether JSON Schema Draft 2020-12 supports dynamic `$ref` resolution at validation time, which would allow the host to assemble a composite schema from the host schema plus all registered extension schema fragments.

Q12. What is the timeout and escalation model for human approval steps?

Why it matters: v1 approval gates block indefinitely waiting for a role-holder to approve. In production, a stalled approval can block an incident response indefinitely. SLA enforcement—timeout after N minutes, escalate to the next role, optionally auto-approve or auto-reject—is a standard pattern in ITIL emergency change processes and is implemented by PagerDuty [23] and AWS Step Functions [9] human approval tasks. Without it, gert is not usable for SLA-bound incident response (*Dennis Research Brief §5.2*).

Options: (a) No timeout in v2; approval escalation deferred. (b) Optional `timeout` field on approval steps; on expiry, run is suspended and operator is notified. (c) Full escalation chain: `timeout` triggers escalation to a secondary role list; if that also times out, the step enters a configurable terminal state (auto-approve, auto-reject, or suspend-for-manual-review).

Information needed: Whether the v2 runtime concurrency model (Q2 above) supports timer-based step suspension without blocking the process, and what notification mechanism is available to alert escalation targets (stdout, webhook, or an extension-contributed notifier).

Chapter 12

Migration and Compatibility

Gert v2 is a breaking redesign from v1. This section specifies the migration path from `runbook/v0` and `runbook/v1` to the v2 schema, defines what `gert migrate` (v2 version) must accomplish, and establishes compatibility guarantees for existing runbooks, tools, providers, and integrations.

Migration is a launch blocker for adoption: every operator with a v1 runbook must understand what breaks, what is automatically migrated, and what requires manual effort. This chapter provides a concrete, actionable roadmap for teams moving from v1 to v2.

12.1 v1 → v2 Compatibility Matrix

The following table catalogs every v1 feature, interface, and artifact and specifies its v2 status. A BREAKING change requires manual intervention or automated migration; a COMPATIBLE change preserves the v1 contract.

v1 Feature/Interface	Status	v2 Change	Migration Notes
runbook/v0 schema	BREAKING	Removed	Migrate to v1 first, then v2
runbook/v1 schema	BREAKING	Promoted fields	gert migrate auto-rewrites
provider/v0 schema	BREAKING	New namespace	Manual rewrite required
tool/v0 schema	BREAKING	New fields	gert migrate rewrites
CLI: gert run	COMPATIBLE	New flags added	All v1 flags preserved
CLI: gert test	COMPATIBLE	Unchanged	
CLI: gert serve	COMPATIBLE	New RPC methods	v1 methods preserved
CLI: gert validate	COMPATIBLE	v2 schema aware	
CLI: gert migrate	COMPATIBLE	v0→v1→v2 path	New -to-v2 flag
JSON-RPC: exec/*	COMPATIBLE	Additive	New methods; old methods unchanged
JSON-RPC: runbook/*	COMPATIBLE	Additive	
JSONL trace format	BREAKING	New envelope	See §10.5
Extension handshake	BREAKING	New capability fields	See §10.4
Event stream schema	BREAKING	New event kinds	Old events preserved
VS Code extension protocol	COMPATIBLE	Backward compat	v2 extension reads v1 traces
TUI adapter	COMPATIBLE	Event-based	

Key takeaways:

- All **CLI commands and flags** are backward compatible. Existing scripts using **gert run** continue to work.
- All **JSON-RPC methods** are backward compatible. The v2 **gert serve** daemon accepts v1 client requests.
- The **runbook YAML schema** has breaking changes (field promotions, renames). Automated migration is provided via **gert migrate**.
- The **tool and provider schemas** have breaking changes. Manual updates required.
- The **JSONL trace format** has breaking changes (new envelope fields). Historical trace readers must be updated.
- The **extension handshake** has breaking changes. Extension authors must adopt the v2 protocol.

12.2 Schema Migration: Runbook YAML

12.2.1 Breaking Changes Summary

The v1 → v2 runbook schema migration involves three categories of change:

1. **Field promotions:** Seven `meta.*` fields are promoted to top-level (§3.2.4).
2. **Step type renames:** `collector` → `manual`, `router` → `end`.
3. **New required fields:** `$schema`, `id` (derived from `name` if absent).

All other runbook semantics are preserved. Step execution behavior, governance primitives, evidence capture, and input/output contracts remain unchanged.

12.2.2 Before and After Example

v1 runbook (`simple.runbook.yaml`):

```
apiVersion: runbook/v1

meta:
  name: service-health-check
  kind: operational
  description: Check service health and restart if needed
  vars:
    service_name: "api-server"
  governance:
    allowed_commands: [systemctl, curl, jq]

flow:
  - step:
      id: check_status
      type: cli
      title: Check service status
      with:
        argv: ["systemctl", "status", "{{ .service_name }}"]
      capture:
        exit_code: exit_status
```

```
- step:
  id: prompt_restart
  type: collector
  title: Service is down. Restart?
  evidence:
    - kind: text
      name: decision

- step:
  id: restart_service
  type: cli
  when: "decision == \"yes\""
  with:
    argv: ["systemctl", "restart", "{{ .service_name }}"]
```

v2 runbook (migrated):

```
$schema: "https://schemas.gert.dev/runbook/v2.json"
apiVersion: runbook/v2

id: service-health-check
name: Service Health Check
kind: operational
description: Check service health and restart if needed

vars:
  service_name: "api-server"

governance:
  allowed_commands: [systemctl, curl, jq]

flow:
  - step:
    id: check_status
    type: cli
    title: Check service status
    with:
```

```

    argv: ["systemctl", "status", "{{ .service_name }}"]
  capture:
    exit_code: exit_status

- step:
  id: prompt_restart
  type: manual
  title: Service is down. Restart?
  evidence:
    - kind: text
      name: decision

- step:
  id: restart_service
  type: cli
  when: "decision == \"yes\""
  with:
    argv: ["systemctl", "restart", "{{ .service_name }}"]

```

12.2.3 Field Mapping Table

v1 Path	v2 Path	Notes
meta.name	name	Promoted to top-level; now capitalized (human-readable)
meta.kind	kind	Promoted
meta.description	description	Promoted
meta.vars	vars	Promoted
meta.inputs	inputs	Promoted
meta.governance	governance	Promoted
meta.prose	prose	Promoted
tree	flow	Renamed (tree was v0 legacy)
step.type: collector	step.type: manual	Renamed
step.type: router	step.type: end	Renamed
tools: [string]	toolRefs: [object]	Structured (§3.5)

12.3 Automated Migration Tool

12.3.1 Command Specification

The `gert migrate` command automates runbook, tool, and provider migrations. It is extended in v2 to support v1→v2 migration.

```
# Preview migration (dry-run)
gert migrate --to-v2 --dry-run runbook.yaml

# Migrate a single runbook in-place
gert migrate --to-v2 runbook.yaml

# Migrate all runbooks in a directory tree
gert migrate --to-v2 ./runbooks/

# Output migrated YAML to stdout (for review)
gert migrate --to-v2 --output - runbook.yaml

# Check if migration is needed (exit 1 if needed, 0 if already v2)
gert migrate --to-v2 --check runbook.yaml
```

12.3.2 Flags

Flag	Type	Description
<code>-to-v2</code>	bool	Enable v1→v2 migration mode
<code>-dry-run</code>	bool	Preview changes without writing
<code>-output PATH</code>	string	Write to file or - for stdout
<code>-check</code>	bool	Exit 1 if migration needed, 0 otherwise
<code>-force</code>	bool	Overwrite existing v2 files
<code>-backup</code>	bool	Create <code>.v1.bak</code> backup before rewrite (default: true)

12.3.3 Migration Algorithm

The migration tool performs the following transformations in order:

1. **Parse input:** Read YAML with v1 parser. If `apiVersion` is already `runbook/v2`, exit 0 (no-op).

2. **Version check:** If `apiVersion` is `runbook/v0`, emit error: “v0 must be migrated to v1 first”.
3. **Add `$schema`:** Insert `$schema: "https://schemas.gert.dev/runbook/v2.json"` at document top.
4. **Update `apiVersion`:** Change `runbook/v1` → `runbook/v2`.
5. **Derive `id`:** If no `meta.name`, derive from filename. Convert to kebab-case (e.g., `service_health` → `service-health`).
6. **Promote `meta.*` fields:** Extract `meta.name`, `meta.kind`, etc. and move to top-level. Capitalize `name` for human readability.
7. **Rename step types:** `collector` → `manual`, `router` → `end`.
8. **Rename `tree`:** If `tree:` is present, rename to `flow:`.
9. **Rewrite `tools`:** Convert shorthand `tools: [name]` to structured `toolRefs: [{name, version?}]`.
10. **Validate:** Parse the migrated YAML with the v2 parser. If validation fails, rollback and report error.
11. **Write output:** If `-output` is specified, write to that path; otherwise rewrite in-place.

12.3.4 Diff Report

The migration tool emits a structured diff report showing all changes:

Migration report for `service-health-check.runbook.yaml`:

```
[CHANGE] apiVersion: runbook/v1 → runbook/v2
[ADD]    $schema: "https://schemas.gert.dev/runbook/v2.json"
[ADD]    id: "service-health-check"
[MOVE]   meta.name → name
[MOVE]   meta.kind → kind
[MOVE]   meta.description → description
[MOVE]   meta.vars → vars
[MOVE]   meta.governance → governance
[RENAME] step "prompt_restart": type collector → manual
```

```
[OK]      3 steps validated successfully
```

Summary: 8 changes applied. No errors.

12.3.5 Error Handling

If migration fails (e.g., v2 validation error, ambiguous field conflict), the tool:

1. Rolls back any partial changes (if in-place mode).
2. Emits a structured error message with file, line, and remediation advice.
3. Exits with status 1.

Example error:

```
ERROR: Migration failed for service-health-check.runbook.yaml
Line 14: step "check_status" has invalid v2 field "with.argv"
Suggestion: v2 uses "with.command" for CLI steps. Update manually.
```

12.4 Provider and Tool Migration

12.4.1 Tool Definitions (.tool.yaml)

The v1 tool/v0 schema is replaced by tool/v2. Key changes:

1. **New required field:** `transport` (one of `stdio`, `jsonrpc`, `mcp`).
2. **Renamed field:** `executable` → `command` (aligns with runbook YAML).
3. **New field:** `capabilities` (list of required capability grants; see §4).

v1 tool definition:

```
apiVersion: tool/v0
name: kubectl
executable: /usr/local/bin/kubectl
args_template: "{{ .args }}"
```

v2 tool definition:

```
$schema: "https://schemas.gert.dev/tool/v2.json"
apiVersion: tool/v2

id: kubectl
name: kubectl
transport: stdio
command: /usr/local/bin/kubectl
args_template: "{{ .args }}"
capabilities:
  - exec:command
  - net:egress
```

Migration tool support. The `gert migrate -to-v2` command rewrites tool definitions automatically, but **requires manual review** for the `capabilities` field (the tool cannot infer required capabilities statically).

12.4.2 Provider Definitions (.provider.yaml)

The `v1 provider/v0` schema is replaced by `provider/v2`. Key changes:

1. **Namespace convention:** Provider fields now use a `provider/*` namespace to avoid name collisions with core schema fields.
2. **New resolution protocol:** `v2` providers declare a `resolve_method` (one of `static`, `env`, `prompt`, `jsonrpc`).

v1 provider definition:

```
apiVersion: provider/v0
name: incident-id
source: env
env_var: INCIDENT_ID
```

v2 provider definition:

```
$schema: "https://schemas.gert.dev/provider/v2.json"
apiVersion: provider/v2

id: incident-id
```



```
name: Incident ID
resolve_method: env
provider/env:
  var_name: INCIDENT_ID
  required: true
```

Migration tool support. The `gert migrate -to-v2` command partially rewrites provider definitions but **requires manual namespace updates** for custom provider fields.

12.5 Extension Migration

12.5.1 Handshake Protocol Changes

The v2 extension handshake introduces new capability negotiation (§4.3). Extensions must update their initialization sequence to:

1. Declare supported `protocol_version` (set to "2.0").
2. Declare required `capabilities` (list of capability names).
3. Handle `CapabilityDenied` responses gracefully.

v1 handshake (JSON-RPC):

```
→ {"jsonrpc": "2.0", "method": "initialize", "params": {"version": "1.0"}}
← {"jsonrpc": "2.0", "result": {"ready": true}}
```

v2 handshake (JSON-RPC):

```
→ {"jsonrpc": "2.0", "method": "initialize", "params": {
  "protocol_version": "2.0",
  "capabilities": ["fs:read", "net:egress"]
}}
← {"jsonrpc": "2.0", "result": {
  "granted_capabilities": ["fs:read"],
  "denied_capabilities": ["net:egress"]
}}
```

12.5.2 Extension Compatibility Shim

To ease the transition, the v2 Extension Host provides a **v1 compatibility shim**:

- Extensions that declare `protocol_version: "1.0"` are accepted.
- The shim grants all capabilities by default (insecure, but backward-compatible).
- A deprecation warning is logged: “Extension ‘foo’ uses deprecated v1 protocol. Update to v2 by YYYY-MM-DD.”
- The shim will be removed in v2.1 (approximately 6 months after v2.0 release).

12.5.3 Extension Author Checklist

Extension authors must:

1. Update `protocol_version` to "2.0" in the `initialize` method.
2. Declare all required capabilities in the handshake.
3. Test against the v2 Extension Host in capability-denied scenarios.
4. Update extension manifest to declare minimum gert version (`min_gert_version: "2.0.0"`).
5. Review the capability taxonomy (§4.4) and minimize requested capabilities.

12.6 Trace Format Migration

12.6.1 Breaking Changes in JSONL Trace

The v2 trace format (§12.1) adds mandatory envelope fields. The v1 trace format had:

```
{"seq":1,"type":"run/started","timestamp":"...","data":{...}}
```

The v2 trace format requires:

```
{"event_id":"...","run_id":"...","runbook_id":"...","timestamp":"...",  
  "kind":"run/started","sequence":1,"payload":{...}}
```

Field mapping:

v1 Field	v2 Field	Notes
<code>seq</code>	<code>sequence</code>	Renamed
<code>type</code>	<code>kind</code>	Renamed
<code>data</code>	<code>payload</code>	Renamed
<code>(none)</code>	<code>event_id</code>	New (UUID v4)
<code>(none)</code>	<code>run_id</code>	New (extracted from file path in v1)
<code>(none)</code>	<code>runbook_id</code>	New (from <code>meta.name</code>)

12.6.2 Trace Replay Compatibility

The v2 `gert replay` command can read v1 traces in **compatibility mode**:

1. Detect v1 format by presence of `type` field (v2 uses `kind`).
2. Synthesize missing fields: `event_id` (new UUID), `run_id` (from file path), `runbook_id` (from first event payload).
3. Map `seq` → `sequence`, `type` → `kind`, `data` → `payload`.
4. Replay as normal.

Compatibility mode will be removed in v2.1.

12.6.3 Trace Archive Migration Tool

Teams with historical v1 trace archives can migrate them to v2 format using:

```
gert trace migrate --to-v2 .runbook/runs/*/trace.jsonl
```

This rewrites all trace files in-place, adding the required v2 envelope fields.

12.7 Rollout Strategy

12.7.1 Recommended Adoption Path

Teams should adopt v2 in four phases:

Phase 1: Read-Only Validation (Week 1-2)

1. Install gert v2.0 alongside v1 (both can coexist).
2. Run `gert validate -v2` against existing v1 runbooks.
3. Run `gert migrate -to-v2 -dry-run -check` to identify migration scope.
4. Review migration reports; identify runbooks that require manual fixups.

Phase 2: Schema Migration (Week 3-4)

1. Migrate runbooks in a staging directory: `gert migrate -to-v2 ./runbooks/`.
2. Review migrated YAML for correctness (especially governance blocks, tool refs).
3. Run `gert validate` against migrated runbooks to confirm v2 compliance.
4. Commit migrated runbooks to version control (with `.v1.bak` backups retained).

Phase 3: Extension and Tool Migration (Week 5-6)

1. Update all `.tool.yaml` files: add `transport` and `capabilities` fields.
2. Update all `.provider.yaml` files: namespace custom fields.
3. Update custom extensions: implement v2 handshake, declare capabilities.
4. Test extensions in v2 runtime with governance policies enabled.

Phase 4: Integration Migration (Week 7-8)

1. Update integrations (CI/CD pipelines, monitoring scripts) to use v2 CLI flags.
2. Update JSON-RPC clients (if any) to handle new event kinds (§6).
3. Update trace parsers to handle v2 envelope format (§12).
4. Decommission v1 runtime once all integrations are v2-native.

12.7.2 Parallel Operation

During the migration period, v1 and v2 runtimes can coexist:

- Install v2 as `/usr/local/bin/gert2` or via version manager (`gert02`).
- Run v1 runbooks with `gert run` (v1 binary).
- Run v2 runbooks with `gert2 run` or `gert -version 2 run`.
- The v2 runtime rejects v0 runbooks; v1 runbooks are accepted in compatibility mode.

12.7.3 Timeline

Milestone	Timeline
v2.0 release	T+0
v1 compatibility shim supported	T+0 to T+6 months
v1 handshake deprecated warning	T+3 months
v1 compatibility shim removed	T+6 months (v2.1 release)
v1 runtime end-of-life	T+12 months

12.8 v1 Compatibility Shim

12.8.1 Decision: Provide a v1 Compatibility Shim for 6 Months

The v2 parser and runtime include a **v1 compatibility mode** that accepts `runbook/v1` documents and silently normalizes them to v2 semantics. This shim is enabled by default in v2.0 and will be removed in v2.1 (approximately 6 months after v2.0 release).

Rationale:

- Lowers the migration barrier for large organizations with hundreds of runbooks.
- Allows operators to adopt v2 runtime immediately without waiting for schema migration.
- Provides a grace period for extension authors to update their code.
- Encourages migration by emitting deprecation warnings (`-strict` mode treats warnings as errors).

12.8.2 Shim Behavior

The compatibility shim:

1. Accepts `apiVersion: runbook/v1` documents.
2. Silently applies the same transformations as `gert migrate -to-v2`.
3. Emits deprecation warnings to `stderr`:

```
WARN: runbook 'service-health-check' uses deprecated runbook/v1 schema.  
      Run 'gert migrate --to-v2 runbook.yaml' to upgrade.  
      v1 support will be removed in gert v2.1 (2026-10-18).
```

4. If `-strict` flag is passed, treats deprecation warnings as errors (exit 1).

12.8.3 Shim Removal

The v1 compatibility shim will be removed in v2.1 (target: 6 months post-v2.0 release).
After removal:

- The v2 parser rejects `runbook/v1` documents with error: “v1 schema not supported. Migrate to v2.”
- All v1 runbooks must be migrated using `gert migrate -to-v2`.
- The v1 compatibility shim code is deleted from the codebase.

12.9 Open Questions and Risks

12.9.1 Open Questions

1. **Provider field namespacing:** Should v2 enforce a strict `provider/*` namespace, or allow custom prefixes? (Recommendation: strict namespace, relaxed in v2.1 if needed.)
2. **Migration of tool discovery paths:** v1 searches `tools/` by default. Should v2 preserve this, or require explicit `-tool-dir` flags? (Recommendation: preserve default path for compatibility.)

3. **Trace archive retention:** Should `gert trace migrate` preserve v1 traces as `.v1.jsonl` backups? (Recommendation: yes, with `-no-backup` opt-out.)

12.9.2 Migration Risks

1. **Runbooks with custom `meta.*` fields:** If a v1 runbook has custom fields under `meta.*`, migration may fail. Mitigation: emit detailed error messages with field paths.
2. **Extensions relying on v1-specific behaviors:** Some extensions may depend on v1 event ordering or field names. Mitigation: provide comprehensive v2 migration guide for extension authors.
3. **Historical trace parsing:** Teams with custom trace analysis scripts will need updates. Mitigation: publish a trace format migration guide with field mappings.

12.10 Summary

Gert v2 introduces breaking changes to the runbook, tool, and provider schemas, the JSONL trace format, and the extension handshake protocol. All changes are necessary to support the new capability-based security model (§7), the event-driven architecture (§6), and the governance primitives (§11).

The migration path is designed to minimize disruption:

- **CLI and JSON-RPC contracts are backward-compatible.**
- **Schema migration is automated** via `gert migrate -to-v2`.
- **v1 compatibility shim** provides a 6-month grace period for runbook and extension migration.
- **Phased rollout** allows teams to adopt v2 incrementally over 8 weeks.

Teams should begin migration planning with Phase 1 (read-only validation) and proceed through schema, extension, and integration migration in sequence. The v1 runtime will reach end-of-life 12 months after v2.0 release, giving organizations ample time to transition.

Chapter 13

Governance and Policy

Governance is Gert’s core differentiator: allowlists, denylists, environment variable blocking, output redaction, and approval gates. This section defines the v2 governance model, specifying what policy fields survive from v1, how policy is evaluated at runtime, whether policy is host-enforced or extension-contributed, and how approval gates integrate with the event model.

Governance features are a first-class design requirement and must not be treated as optional or defer-to-extension.

13.1 Governance Primitives

The following primitives form gert’s policy vocabulary. Each is defined precisely below, with its evaluation point in the execution pipeline, its schema representation, and its trace event.

13.1.1 Command Allowlist (`allowed_commands`)

Definition. A whitelist of executable names (`argv[0]`, without path components) that may be invoked by `cli` steps and tool subprocess invocations. If the allowlist is non-empty, any command whose base name is not in the list is blocked before the subprocess is forked.

Semantics. Path-stripping is applied before matching: `/usr/bin/curl` matches the allowlist entry `curl`. Glob patterns are *not* supported in v2.0 (exact match only). The empty list (`[]`) means all commands are permitted (permissive default).

Schema.

```
meta:
  governance:
    allowed_commands: [kubect1, curl, jq, aws]
```


Evaluation point. Runtime Core, immediately before subprocess fork (after template evaluation). See §13.2.

Trace event. `governance/commandBlocked` — written when a command is denied. Fields: `stepId`, `command`, `reason`: `"not_in_allowlist"`.

13.1.2 Command Denylist (`denied_commands`)

Definition. An explicit list of executable names that are unconditionally blocked. The denylist takes precedence over the allowlist: if a command appears in both, it is denied.

Use case. Operators use the denylist to forbid dangerous commands (`rm`, `dd`, `shutdown`) without needing to enumerate all permitted commands.

Schema.

```
meta:
  governance:
    denied_commands: [rm, dd, shutdown, reboot]
```

Evaluation point. Runtime Core, same checkpoint as allowlist. Denylist check runs first.

Trace event. `governance/commandBlocked` — fields: `stepId`, `command`, `reason`: `"in_denylist"`.

13.1.3 Environment Variable Blocking (`deny_env_vars`)

Definition. A list of glob patterns matched against environment variable names. Any environment variable whose name matches a pattern is stripped from the subprocess environment before the command is executed.

Rationale. Prevents accidental or malicious exfiltration of secrets (e.g., `AWS_SECRET_ACCESS_KEY`, `GITHUB_TOKEN`) by commands in the runbook.

Schema.

```
meta:
  governance:
    deny_env_vars:
      - "*_SECRET*"
      - "*_TOKEN"
      - "AWS_*"
      - "GITHUB_*
```

Evaluation point. Runtime Core, immediately before subprocess fork. Applied to the full process environment (not just explicitly passed vars).

Trace event. `governance/envVarBlocked` — fields: `stepId`, `varNames`: `[string]` (names of blocked variables, not values).

13.1.4 Output Redaction (redact)

Definition. A list of redaction rules. Each rule has a `pattern` (a RE2 regular expression) and a `replace` string. After a `cli` or `tool` step produces output, all redaction rules are applied to the captured stdout/stderr before the output is stored in run variables or written to the trace.

Semantics. Rules are applied in declaration order. The replace string may use RE2 backreferences (`$1`). The special replace value `[REDACTED]` is the conventional replacement for secrets.

Schema.

```
meta:
  governance:
    redact:
      - pattern: "token=[A-Za-z0-9_\\-]{20,}"
        replace: "token=[REDACTED]"
      - pattern: "password: .+"
        replace: "password: [REDACTED]"
      - pattern: "(?i)bearer\\s+[A-Za-z0-9._\\-]+"
        replace: "Bearer [REDACTED]"
```

Evaluation point. Runtime Core, immediately after subprocess/tool output is captured, before capture variables are set and before any trace write.

Trace event. `governance/redactionApplied` — fields: `stepId`, `ruleCount` (number of rules that matched, not the patterns themselves, to avoid logging the pattern that matched sensitive data).

13.1.5 Approval Gates

Approval gates are first-class step-level governance primitives. They are defined on any step type and pause execution until a minimum number of approvals from specified roles have been recorded. See §13.3 for the complete approval gate design.

13.2 Policy Evaluation Model

Policy primitives are evaluated at specific, well-defined points in the execution pipeline. The evaluation order within each checkpoint is fixed and must not be reordered by extensions.

Checkpoint	Primitives Evaluated	Phase
Parser validation	Schema structural rules	Parse
Planner validation	Import cycles, tool existence	Plan
Run pre-flight	Extension policy rules	Run init
Step pre-flight	Denylist check	Runtime Core
Step pre-flight	Allowlist check	Runtime Core
Step pre-flight	Approval gate	Runtime Core
Subprocess fork	Env var blocking	Runtime Core
Output captured	Redaction	Runtime Core
Post-step	Contract risk warning	Runtime Core

Order of precedence. Within the pre-flight checkpoint for a `cli` or `tool` step, the evaluation order is:

1. Denylist check (hard block, step fails immediately if matched).
2. Allowlist check (hard block, step fails immediately if not matched and list is non-empty).
3. Approval gate (soft pause; step is held until approvals arrive).
4. Env var blocking (applied at fork time, not a failure condition — silently strips).

A policy violation at any hard-block checkpoint writes the corresponding trace event and returns an error from `RunHandle.Next()`. The run is placed in the `policy_violation` terminal state. The operator must resolve the policy (e.g., add the command to the allowlist) and re-run; resumption from a policy violation is not supported in v2.0.

13.3 Approval Gate Design

13.3.1 Schema Declaration

Approval gates are declared in the step's `approvals` block:

```

steps:
- id: delete_database
  kind: cli
  command: "kubectl delete namespace {{ .targetNamespace }}"
  approvals:
    min: 2
    roles: ["approver", "reviewer"]
    timeout: 24h
    on_timeout: fail    # or: escalate
    escalate_to: ["senior-sre", "director-of-eng"]
    message: "Deleting {{ .targetNamespace }} is irreversible. Confirm change
    ↪ window."

```

13.3.2 UX Contract

When the Runtime Core reaches a step with an `approvals` block, it:

1. Writes `governance/approvalRequired` trace event (stepId, message, roles, min, timeout).
2. Emits an `ApprovalRequired` runtime event on the event channel.
3. Returns from `Next()` with a `StepResult` of kind `PendingApproval`. The step does *not* execute yet.
4. Subsequent calls to `Next()` on the same step return `ErrAwaitingApproval` until the approval quorum is met.

The adapter (VS Code extension, TUI, or `gert serve` client) is responsible for presenting the approval request to authorized users and collecting decisions.

13.3.3 Approval Decision Protocol

An approver submits a decision by calling `RunHandle.Approve(ctx, ApprovalDecision)`:

```

type ApprovalDecision struct {
  StepID  string    // step to approve (must match current paused step)
  Actor   string    // identity of the approver (e.g., "alice@corp.com")
  Role    string    // role the approver is acting in (must be in
  ↪      step.approvals.roles)

```

```

    Approved bool      // true = approved, false = rejected
    Comment  string    // optional justification
    At       time.Time // timestamp of decision
}

```

The runtime validates:

- The Actor identity is established (either from `-as` flag or, in a multi-user context, from the adapter's authenticated session).
- The Role is in the step's declared `approvals.roles`.
- The same actor may not approve the same gate twice.
- If `Approved = false`, the gate is immediately rejected (any single rejection vetos the gate), and the run enters the `rejected` terminal state.

When the approval count reaches `min`, the runtime writes `governance/approvalGranted` to the trace and the next call to `Next()` proceeds with normal step execution.

13.3.4 Approval Recording in the Trace

Every approval decision is written to the trace as an immutable record:

```

// governance/approvalRequired
{
    "sequence": 14,
    "event": "governance/approvalRequired",
    "stepId": "delete_database",
    "message": "Deleting prod-ns is irreversible. Confirm change window.",
    "roles": ["approver", "reviewer"],
    "min": 2,
    "timeout": "24h",
    "at": "2026-04-18T10:00:00Z"
}

// governance/approvalDecision (written once per decision)
{
    "sequence": 15,

```

```

    "event": "governance/approvalDecision",
    "stepId": "delete_database",
    "actor": "alice@corp.com",
    "role": "approver",
    "approved": true,
    "comment": "Change window confirmed with customer.",
    "at": "2026-04-18T10:05:00Z"
}

// governance/approvalGranted (written when quorum is met)
{
    "sequence": 16,
    "event": "governance/approvalGranted",
    "stepId": "delete_database",
    "approvals": ["alice@corp.com", "bob@corp.com"],
    "at": "2026-04-18T10:07:00Z"
}

```

The trace provides a complete, tamper-evident audit record of who approved what and when.

13.3.5 Timeout and Escalation

If `timeout` is set and the approval quorum is not met within the deadline:

- `on_timeout: fail`: The run enters the `timed_out` terminal state. `governance/approvalTimedOut` is written to the trace.
- `on_timeout: escalate`: The runtime emits an `ApprovalEscalated` event. If `escalate_to` roles are declared, a new `ApprovalRequired` event is emitted targeting those roles. The timeout clock restarts. Escalation is attempted once; if the escalated request also times out, the run fails.

Timeout enforcement is implemented via a `context.WithDeadline` passed to the approval wait loop. The adapter (or the `gert serve` session timer) is responsible for surfacing the timeout to the approver.

13.4 Policy-as-Code Direction

13.4.1 v2.0: Structured YAML Policy Blocks

For v2.0, gert uses **structured YAML policy blocks** embedded in the runbook's `meta.governance` section. This is the approach inherited from v1, extended with the richer primitives defined in §13.3.

Rationale for v2.0:

- The policy vocabulary is small and closed (six primitives as of v2.0). A full policy DSL would be over-engineered for this set.
- YAML policy blocks are self-contained: the runbook declares its own governance constraints without requiring external policy files or a running policy server.
- Structured blocks are straightforward to validate with JSON Schema at authoring time, giving operators early feedback in the VS Code extension.
- The existing v1 governance model (allowlists, redaction, env blocking) translates directly to structured blocks with no loss of expressiveness.

13.4.2 v2.1+: OPA Integration Hooks

Dennis's research confirms that Open Policy Agent (OPA) is the CNCF standard for policy-as-code in operational tooling. For **v2.1**, gert will introduce OPA integration at the following evaluation hooks:

```
// OPA evaluation hook interface (v2.1 target)
type PolicyEngine interface {
    // EvaluatePreStep is called before each step pre-flight.
    // The policy engine receives the full step context and returns
    // allow/deny with a reason.
    EvaluatePreStep(ctx context.Context, input PolicyInput) (PolicyDecision,
        ↪ error)

    // EvaluatePreRun is called once after plan construction.
    // Used for runbook-level access control (RBAC).
    EvaluatePreRun(ctx context.Context, input PolicyInput) (PolicyDecision,
        ↪ error)
```

```

}

type PolicyInput struct {
    Runbook    RunbookMeta
    Step       ResolvedStep
    Actor      string
    Vars       map[string]string
    Contract   *contract.Contract
}

type PolicyDecision struct {
    Allow    bool
    Reason   string
    // For approval gates: required roles computed by policy engine
    RequiredApprovers []string
}

```

The `PolicyEngine` interface will have two implementations in v2.1:

1. **Built-in:** evaluates the structured YAML policy blocks (backward-compatible with v2.0 runbooks).
2. **OPA:** calls an OPA bundle loaded from a project policy directory (`.gert/policy/*.rego`).

The two implementations compose: the built-in engine runs first; if it allows, the OPA engine runs second. A denial from either blocks the step.

Recommendation: Do not block the v2.0 launch on OPA integration. Ship the structured YAML model first, then introduce OPA as an opt-in engine in v2.1. This allows the governance interface to be designed correctly without the complexity of Rego authoring in the initial release.

13.5 RBAC Model

13.5.1 Identity Establishment

Actor identity is established via the `-as <identity>` flag passed at invocation time. The identity string is an opaque label (conventionally an email address or a role name) recorded in the trace and used for approval gate matching.

v2.0 limitation: Identity is not authenticated. The `-as` flag is self-asserted; gert does not verify that the caller is who they claim to be. Enforcement of identity authenticity is the responsibility of the surrounding infrastructure (CI/CD pipeline, VS Code extension SSO, MCP server authentication).

v2.1 target: An `IdentityProvider` interface will allow adapters to supply verified identity tokens (e.g., OIDC tokens from VS Code GitHub Copilot authentication), replacing self-asserted identity for governed environments.

13.5.2 Runbook-Level Access Control

Runbooks may declare a `requires_role` field to restrict who may execute them:

```
meta:
  governance:
    requires_role: "responder"
    # or: requires_any_role: ["approver", "responder"]
    # or: requires_all_roles: ["approver", "reviewer"]
```

The Runtime Core evaluates this constraint during run initialization (pre-flight, before any step executes). If the actor's identity does not match, `governance/accessDenied` is written to the trace and the run fails immediately.

13.5.3 Step-Level Role Restriction

Individual steps may declare `requires_role` to restrict who may trigger that step. This is distinct from approval gates (which require external approval) — step role restriction *prevents* a step from executing unless the actor themselves holds the required role.

```
steps:
- id: promote_to_production
  kind: cli
  requires_role: "release-manager"
  command: "kubectl set image deployment/app app={{ .imageTag }}"
```

13.5.4 Separation of Duties

The approval gate model supports separation of duties: the `same-actor-cannot-approve` invariant (defined in §13.3) ensures that a step author cannot self-approve their own gate.

In environments requiring formal approval processes, the actor who initiates the run and the actor(s) who approve high-risk steps must be different.

13.6 Redaction

13.6.1 Pattern-Based Redaction

The primary redaction mechanism is pattern-based: operators declare RE2 regular expressions in `meta.governance.redact`. Redaction is applied eagerly: captured output is redacted *before* any capture variable assignment or trace write.

13.6.2 Explicit Field Marking (v2.1)

For structured tool outputs (JSON objects returned by MCP tools), v2.1 will introduce explicit field marking: a tool definition may declare that a returned field contains sensitive data. The runtime will redact the field value without requiring a regex pattern.

```
# In .tool.yaml (v2.1 target)
outputs:
  - name: api_key
    sensitive: true    # value redacted from trace and captures
```

13.6.3 Redaction Guarantees

- Redaction is applied to all output paths: captured variables, trace events, and event channel payloads.
- Redaction rules are compiled (regex pre-compiled) during run initialization, not re-compiled per step.
- Redaction is applied unconditionally in all run modes (real, dry-run, replay).
- The original (pre-redaction) value is *never* written to disk in any path.

13.7 Audit Trail Requirements

The append-only JSONL trace file is gert's primary audit artifact. Every governance event is a first-class trace entry. The following governance events are guaranteed to be written:

Trace Event	Written When
<code>governance/commandBlocked</code>	A command is blocked by denylist or allowlist
<code>governance/envVarBlocked</code>	Env vars are stripped before subprocess fork
<code>governance/redactionApplied</code>	Redaction rules matched captured output
<code>governance/approvalRequired</code>	A step's approval gate is entered
<code>governance/approvalDecision</code>	An approver submits approve or reject
<code>governance/approvalGranted</code>	Approval quorum is met
<code>governance/approvalRejected</code>	An approver rejects the gate
<code>governance/approvalTimedOut</code>	Approval timeout expires
<code>governance/accessDenied</code>	RBAC check fails (run or step level)
<code>governance/contractWarning</code>	Step contract risk level is critical
<code>governance/policyViolation</code>	Generic policy violation (OPA engine, v2.1)

13.7.1 Trace Integrity

The trace file is protected by:

- **Append-only write discipline:** the trace writer never seeks backward or truncates. Each write is followed by `fsync`.
- **SHA256 evidence hash:** at run completion, the SHA256 of the entire trace file is written as the final record and also stored in a sidecar `trace.sha256` file.
- **Monotonic sequence numbers:** every trace event has a `sequence` field (incrementing integer). Gaps in sequence numbers signal corruption or tampering.

13.7.2 Compliance Alignment

These audit trail guarantees are designed to satisfy the requirements of:

- **SOC 2 Type II** — Approval and authorization controls: every command execution and approval is logged with actor identity and timestamp.
- **HIPAA** — Access controls and audit logging: env var blocking and redaction prevent PHI from appearing in traces; approval gates enforce separation of duties.
- **ITIL Process Management** — Normal/standard/emergency workflow paths can be modeled via `requires_role` and multi-approver gates.

13.8 Extension-Contributed Policy

Extensions may contribute governance rules through the `ContributedPolicyRules()` method of the Extension Host (see §02). These rules compose with host policy under the following invariants:

1. **Additive only.** Extension rules may add restrictions; they may never remove or override host-defined policy. An extension cannot remove a command from the denylist, relax a required role, or suppress a redaction rule.
2. **Host policy takes precedence on conflict.** If an extension rule and a host rule address the same primitive and disagree, the more restrictive interpretation applies.
3. **Extension rules are visible in the trace.** A `governance/extensionPolicyApplied` trace event is written when an extension-contributed rule is evaluated, naming the extension and rule ID.
4. **Extension rules are not trusted with approval authority.** Extensions may not grant approvals on behalf of human approvers. Approval decisions must originate from a human actor via the `RunHandle.Approve()` API.

13.8.1 Extension Policy Rule Schema

```
// PolicyRule is contributed by an extension and evaluated at pre-step
↪ checkpoint.
type PolicyRule struct {
    ID          string      // unique within the extension's namespace
    Description string
    // Evaluate returns Deny if the rule blocks this step, Allow otherwise.
    // Extensions communicate rules as static declarations (v2.0); dynamic
    // rule evaluation via the extension RPC transport is v2.1.
    StepKinds []StepKind // step types this rule applies to (empty = all)
    CommandGlob string    // if set, only applies to commands matching this
    ↪ glob
    Action    PolicyAction // Deny or Warn
    Reason    string      // human-readable reason written to trace event
}
```

Extension-contributed policy is loaded during the Extension Host's `Load()` phase, before the first `Next()` call. Policy rules are static for the lifetime of a run; extensions may not add or remove rules after the run has started.

Chapter 14

Evidence, Tracing, and Resumption

Gert v2 captures a complete, durable record of every execution for audit, compliance, incident response, and operational replay. This section specifies the append-only JSONL trace format, evidence capture model, per-step snapshot contract, run resumption semantics, replay mode, and OpenTelemetry integration. Together these mechanisms ensure that every run produces a self-contained artifact that answers three questions: *what happened*, *what evidence was collected*, and *how can execution be recovered or replicated*.

14.1 Trace File Format

14.1.1 File Location and Naming

Each run produces a single trace file located at:

```
.runbook/runs/<run-id>/trace.jsonl
```

The `<run-id>` is a URL-safe string assigned at run creation time (e.g. `20260418T143022-a3f9c1`). All run artefacts are co-located under the run directory:

```
.runbook/runs/<run-id>/
  trace.jsonl           # append-only JSONL event log
  snapshots/           # per-step state checkpoints
    step-0000.json
    step-0001.json
    ...
  attachments/         # binary evidence referenced from trace events
    <sha256>.bin
```

```

...
run.yaml          # completion manifest (written on terminal event)
annotations.jsonl # operator notes (append-only)
lock              # PID lock (removed on clean exit)

```

14.1.2 JSONL Envelope

Every line in `trace.jsonl` is a self-contained JSON object with a common envelope. Consumers MUST tolerate unknown fields to allow additive schema evolution.

```

{
  "event_id": <string>,    // UUID v4 for correlation
  "sequence": <int64>,     // monotonically increasing within this run
  "kind":     <string>,    // event kind (see §12.1.3)
  "timestamp": <RFC3339>, // UTC timestamp with sub-second precision
  "run_id":   <string>,    // run identifier
  "path":     <array>,    // tree-path to the active node (may be null)
  "payload":  <object>    // kind-specific payload (see below)
}
\end{verbatim}

```

The `\texttt{sequence}` field provides a total order over events in a single run. It is assigned

`\subsection{Event Type Catalogue}`

The following event kinds are normative in v2. All user-supplied string fields (command arguments)

`\subsubsection{\texttt{run/started}}`

Emitted once, as the first event in every trace file.

```

\begin{verbatim}
"payload": {
  "runbook_id": <string>, // runbook name/path
  "user":       <string>, // actor identity
  "mode":       <string>, // "real" | "replay" | "dry-run"
  "inputs":     <object>, // resolved input values (redacted)
}

```

```

    "client":      <string>, // "cli" | "tui" | "web" | "mcp"
    "parent_run_id": <string> // omitted if not an invoke child
  }
\end{verbatim}

```

```
\subsubsection{\texttt{step/started}}
```

Emitted immediately before executing a step.

```

\begin{verbatim}
"payload": {
  "step_id":    <string>, // stable step identifier
  "step_type":  <string>, // "cli" | "manual" | "tool" | "invoke"
  "step_name":  <string>  // human-readable name from runbook YAML
}
\end{verbatim}

```

```
\subsubsection{\texttt{step/completed}}
```

Emitted after a step exits cleanly.

```

\begin{minted}{go}
"payload": {
  "step_id":    <string>,
  "exit_status": <int>,      // exit code for cli steps; 0 for others
  "duration_ms": <int64>,   // wall-clock duration in milliseconds
  "captures":   <object>,   // captured output variables (redacted)
  "evidence":   [           // zero or more evidence items
    {
      "name":    <string>,
      "kind":    "text" | "checklist" | "attachment",
      "value":   <string>, // for kind=text
      "items":   <object>, // for kind=checklist
      "path":    <string>, // relative path for kind=attachment
      "sha256":  <string>, // hex SHA256 for kind=attachment
      "size":    <int64>   // bytes for kind=attachment
    }
  ]
}

```



```

    }
  ]
}
\end{minted}

```

```
\subsubsection{\texttt{step/failed}}
```

Emitted when a step exits with a non-zero status or returns a runtime error.

```

\begin{minted}{json}
"data": {
  "step_id": <string>,
  "error_type": <string>, // "exit_code" | "timeout" | "governance"
                        // | "tool_error" | "invoke_failed"
  "error": <string>, // human-readable message (redacted)
  "exit_status": <int>, // for error_type=exit_code
  "duration_ms": <int64>
}
\end{minted}

```

```
\subsubsection{\texttt{step/skipped}}
```

Emitted when a step is not executed due to branch evaluation or iterate exhaustion.

```

\begin{minted}{json}
"data": {
  "step_id": <string>,
  "reason": "branch_not_taken" | "iterate_exhausted" | "dry_run"
}
\end{minted}

```

```
\subsubsection{\texttt{step/retrying}}
```

Emitted before each retry attempt (requires the retry policy feature).

```
\begin{minted}{json}
```

```
"data": {
  "step_id":      <string>,
  "attempt":      <int>,      // attempt number (1 = first retry)
  "max_attempts": <int>,
  "backoff_ms":   <int64>
}
\end{minted}
```

```
\subsubsection{\texttt{tool/invoked}}
```

Emitted when the tool runtime dispatches a tool call.

```
\begin{minted}{json}
"data": {
  "tool_name":      <string>,
  "transport":      "stdio" | "jsonrpc" | "mcp",
  "correlation_id": <string>, // UUIDv4, links to tool/responded
  "inputs":         <object>  // sanitized inputs
}
\end{minted}
```

```
\subsubsection{\texttt{tool/responded}}
```

Emitted when the tool call returns.

```
\begin{minted}{json}
"data": {
  "correlation_id": <string>,
  "exit_code":      <int>,
  "duration_ms":    <int64>,
  "truncated":      <bool>    // true if output was capped
}
\end{minted}
```

```
\subsubsection{\texttt{manual/prompted}}
```

Emitted when a manual step displays its prompt to the operator.

```
\begin{minted}{json}
"data": {
  "step_id":      <string>,
  "prompt_text": <string>,    // the prompt shown (redacted)
  "sla_seconds": <int>        // 0 if no SLA configured
}
\end{minted}
```

```
\subsubsection{\texttt{manual/completed}}
```

Emitted when the operator submits a response to a manual step.

```
\begin{minted}{json}
"data": {
  "step_id":      <string>,
  "response":     <string>,    // operator's text response (redacted)
  "attestation":  <string>,    // "operator_confirmed" | "skipped"
  "approver":     <string>,    // identity of responding actor
  "duration_ms":  <int64>
}
\end{minted}
```

```
\subsubsection{\texttt{governance/blocked}}
```

Emitted when a governance rule blocks execution.

```
\begin{minted}{json}
"data": {
  "step_id":      <string>,
  "rule":         <string>,    // allowlist | denylist | env_block
                                   // | output_redact | approval_gate
  "action_attempted": <string>, // description of blocked action
  "policy":       <string>     // policy name or path
}
\end{minted}
```

```
\end{minted}
```

```
\subsubsection{\texttt{governance/approved}}
```

Emitted when an approval gate is cleared.

```
\begin{minted}{json}
```

```
"data": {  
  "step_id":      <string>,  
  "approver":     <string>, // identity of approver  
  "approval_method": <string>, // "interactive" | "mcp" | "api"  
  "duration_ms":  <int64>  
}
```

```
\end{minted}
```

```
\subsubsection{\texttt{run/completed}}
```

Terminal event for a successful run.

```
\begin{minted}{json}
```

```
"data": {  
  "final_status": "passed" | "failed" | "skipped",  
  "duration_ms":  <int64>,  
  "step_count":   <int>,  
  "passed":       <int>,  
  "failed":       <int>,  
  "skipped":      <int>  
}
```

```
\end{minted}
```

```
\subsubsection{\texttt{run/failed}}
```

Terminal event for a run that halted due to an unrecoverable error.

```
\begin{minted}{json}
```

```
"data": {
```

```

    "failing_step_id": <string>,
    "error":          <string>,
    "duration_ms":    <int64>
}
\end{minted}

```

```
\subsubsection{\texttt{checkpoint}}
```

Internal event written by `\texttt{RunStore.SaveCheckpoint}` to link a snapshot file to the trace.

```

\begin{minted}{json}
"data": {
  "snapshot_file": <string> // e.g. "step-0003.json"
}
\end{minted}

```

```
\subsection{Crash Safety}
```

The trace file is opened with `\texttt{O_APPEND}` and each event is written as a single `\texttt{write}` call.

The Go implementation:

```

\begin{minted}{go}
// WriteTrace appends one event atomically and fsyncs.
func (d *DirRunStore) WriteTrace(event DurableEvent) error {
    p := filepath.Join(d.baseDir, "trace.jsonl")
    f, err := os.OpenFile(p, os.O_CREATE|os.O_WRONLY|os.O_APPEND, 0644)
    if err != nil {
        return fmt.Errorf("open trace: %w", err)
    }
    defer f.Close()

    data, err := json.Marshal(event)
    if err != nil {
        return fmt.Errorf("marshal event: %w", err)
    }
}

```

```

    data = append(data, '\n') // JSONL delimiter
    if _, err := f.Write(data); err != nil {
        return fmt.Errorf("write trace: %w", err)
    }
    return f.Sync() // durability guarantee
}
\end{minted}

```

On recovery, any line that fails `\texttt{json.Unmarshal}` is silently skipped. The last clean

\subsection{Tamper Evidence}

The v2 trace format supports optional HMAC-SHA256 signing of each event. When a signing key

```

% ----- %
\section{Evidence Capture}
% ----- %

```

\subsection{What Constitutes Evidence}

In v2, ```evidence''` is any artefact that attests to a step's outcome:

```

\begin{itemize}
  \item \textbf{Command output} - stdout/stderr of a \texttt{cli} step, captured and embedded
  \item \textbf{Manual attestations} - operator-provided text, checklist responses, and expl
  \item \textbf{Tool responses} - structured JSON returned by a \texttt{tool} step, captured
  \item \textbf{File attachments} - binary or text files explicitly attached by an operator
\end{itemize}

```

\subsection{SHA256 Hashing}

All file attachments are content-addressed by their SHA256 digest. The `\texttt{evidence}` pack

```

\begin{minted}{go}
// HashFile returns hex SHA256 and file size.
func HashFile(path string) (sha256hex string, size int64, err error)

```

```
// NewAttachmentEvidence creates an EvidenceValue with hash and size.
func NewAttachmentEvidence(path string) (*EvidenceValue, error)
\end{minted}
```

The hash is stored in the `\texttt{step/completed}` event alongside the relative path under `\texttt{step}`.

Text evidence fields (command output, operator responses) are not individually hashed - the `\texttt{step}` event contains the full text.

\subsection{File Attachments}

When a step produces a binary artefact, the runtime:

```
\begin{enumerate}
  \item Copies the file to \texttt{.runbook/runs/<run-id>/attachments/<sha256>.<ext>}.
  \item Emits an \texttt{EvidenceValue} of kind \texttt{attachment} in the \texttt{step/completed} event.
\end{enumerate}
```

Attachments are deduplicated by content: if two steps produce identical file content, they share the same attachment.

\subsection{Evidence Retention}

Evidence is retained for the lifetime of the run directory. The workspace-level retention policy is defined in `\texttt{workspace.yaml}`.

```
\begin{itemize}
  \item \textbf{\texttt{retention.runs}} - number of completed runs to retain (default: 50).
  \item \textbf{\texttt{retention.max\_age\_days}} - maximum age in days (default: 90).
  \item \textbf{\texttt{retention.attachments}} - whether to include attachments in retention.
\end{itemize}
```

The `\texttt{gert gc}` command enforces retention policy by removing the oldest run directories.

```
% ----- %
\section{Run State Snapshots}
% ----- %
```

\subsection{Checkpoint Model}

A checkpoint is a complete serialisation of \texttt{RunState} written to \texttt{snapshots/}

```
\begin{minted}{go}
type RunState struct {
    RunID          string
    RunbookPath    string
    Mode           string          // "real" | "replay" | "dry-run"
    StartedAt      time.Time
    Actor          string
    CurrentStepIndex int          // flat-mode index (legacy compat)
    Path           TreePath      // tree-mode cursor (authoritative)
    Vars           map[string]string // resolved input bindings
    Captures       map[string]string // accumulated step outputs
    History        []*StepResult  // completed step records
    InvokeStack    []InvokeFrame  // nested invoke context stack
    IterateState   map[string]*IterateProgress
    ParallelSlots  map[string][]SlotState
    LastCheckpointSeq int64          // trace seq at checkpoint time
}
\end{minted}
```

\subsection{Atomic Snapshot Write}

Snapshots are written atomically: the runtime marshals the state to a \texttt{.tmp} file, fsyncs it, and then renames it to the final name.

```
\begin{minted}{go}
func (d *DirRunStore) SaveCheckpoint(state *RunState) (string, error) {
    filename := fmt.Sprintf("step-%04d.json", len(state.History))
    finalPath := filepath.Join(d.baseDir, "snapshots", filename)
    tmpPath   := finalPath + ".tmp"

    data, _ := json.MarshalIndent(state, "", " ")
    os.WriteFile(tmpPath, data, 0644) // write to temp
    tf, _ := os.Open(tmpPath); tf.Sync(); tf.Close() // fsync temp
}
```



```

        os.Rename(tmpPath, finalPath)           // atomic rename
        // ... then write checkpoint trace event
    }
\end{minted}

```

```

\subsection{Minimum Resumption State}

```

The state required to resume a run from step $\$N\$$ is exactly the snapshot at `\texttt{step-NNNN}`

```

\begin{itemize}
  \item All resolved input values (\texttt{Vars}).
  \item Outputs captured by completed steps (\texttt{Captures}).
  \item The tree cursor (\texttt{Path}) identifying which node executes next.
  \item Branch decisions already taken (encoded in \texttt{History} step statuses).
  \item Iterate progress counters and parallel slot states.
  \item The invoke stack for nested runbooks.
\end{itemize}

```

The runbook YAML itself is not stored in the snapshot; it is re-read from `\texttt{RunbookPath}`

```

% ----- %
\section{Run Resumption}
% ----- %

```

```

\subsection{The \texttt{gert exec --resume} Contract}

```

```

\begin{minted}{bash}
gert exec --resume <run-id>
\end{minted}

```

The runtime executes the following recovery sequence:

```

\begin{enumerate}
  \item Locate \texttt{.runbook/runs/<run-id>/}.
  \item Attempt to acquire the \texttt{lock} file. Fail if a live process holds it.
  \item Call \texttt{RunStore.LoadLatestCheckpoint()} - scan \texttt{snapshots/} descending,

```

```

\item Re-open \texttt{trace.jsonl} for append.
\item Advance \texttt{CurrentStepIndex} by one (or advance the tree \texttt{Path} cursor past the last step).
\item Re-build the governance engine from the runbook's policy block.
\item Resume the execution loop from the restored \texttt{Path}.
\end{enumerate}

```

The resumption does *not* re-execute any step recorded in \texttt{History}. It continues from the last step.

\subsection{Resumability by Step Type}

```

\begin{itemize}
\item \textbf{\texttt{cli}} steps - safe to resume. The step is re-executed from scratch.
\item \textbf{\texttt{manual}} steps - the operator is re-prompted. The prior response (if any) is ignored.
\item \textbf{\texttt{tool}} steps - resumable if idempotent. If the tool call was in-flight, it is resumed.
\item \textbf{\texttt{invoke}} steps - if the child run completed (its terminal event is in \texttt{History}), it is resumed.
\item \textbf{\texttt{branch}/\texttt{iterate}} - structural nodes; not executed directly.
\end{itemize}

```

\subsection{Idempotency Guidance}

Runbook authors should write resumable runbooks by following these guidelines:

```

\begin{itemize}
\item Use \texttt{kubectl apply} and \texttt{terraform apply} instead of imperative create/delete.
\item Use \texttt{--if-not-exists} or \texttt{--create-or-replace} flags where available.
\item For tool steps, declare \texttt{idempotent: true} in the tool definition when the operation is idempotent.
\item For manual steps, write prompts as questions that remain valid on a second ask (``Has the tool completed?'').
\item Avoid steps whose side effects depend on the current time or a random seed without an explicit seed.
\end{itemize}

```

\subsection{Partial Step Resumption}

If a \texttt{tool/invoked} event has no matching \texttt{tool/responded} event, the tool call is considered orphaned.

```

\begin{enumerate}
\item The runtime logs a warning identifying the orphaned correlation ID.

```

```

\item If \texttt{idempotent: true}: the tool call is re-issued from the beginning.
\item If \texttt{idempotent: false} (default): the step is treated as \textbf{failed} and
\end{enumerate}

```

```

% ----- %
\section{Replay Mode}
% ----- %

```

```

\subsection{Overview}

```

```

\begin{minted}{bash}
gert exec --mode replay --scenario <scenario.yaml>
\end{minted}

```

Replay mode executes a runbook against a set of pre-recorded command responses and evidence v

```

\begin{itemize}
\item \textbf{Testing} - validating that a runbook's branching logic, captures, and governa
\item \textbf{Demonstration} - showing a runbook's execution flow without real infrastru
\item \textbf{Regression} - verifying that a refactored runbook produces the same trace as
\end{itemize}

```

```

\subsection{Scenario File Format}

```

A scenario file is a YAML document that maps commands to recorded responses and step IDs to r

```

\begin{minted}{yaml}
commands:
  - argv: ["kubectl", "get", "pods", "-n", "prod"]
    stdout: "NAME          READY   STATUS    RESTARTS\npod-abc    1/1     Running   0\n"
    stderr: ""
    exit_code: 0

evidence:
  check-pod-health:      # step_id
  observation:          # evidence name

```

```

        kind: text
        value: "pod is healthy, proceeding"
\end{minted}

```

Commands are matched in order of declaration (first match wins). Unmatched commands fail the

```
\subsection{Replay Semantics}
```

In replay mode:

```

\begin{itemize}
  \item \texttt{cli} steps - the command executor returns the pre-recorded stdout/stderr/exit
  \item \texttt{manual} steps - the evidence collector returns the pre-recorded evidence value
  \item \texttt{tool} steps - tool invocations are intercepted; if a matching recorded response
  \item \texttt{invoke} steps - the child runbook is executed in replay mode using the same state
  \item Governance rules - evaluated normally; replay does not bypass governance.
  \item Trace events - written normally; the replay trace can be compared against a golden trace
\end{itemize}

```

```
\subsection{Determinism Requirement}
```

For replay to produce the same outcome across runs, the runbook must be deterministic given the

```

\begin{itemize}
  \item Timestamps in expressions (use captured values instead of \texttt{time.Now()}).
  \item Random identifiers in step names or captures.
  \item File system state that is not reproduced by the scenario.
  \item External API calls in tool steps that are not replayed (tool steps should be declared
\end{itemize}

```

```

% ----- %
\section{OpenTelemetry Integration}
% ----- %

```

```
\subsection{Rationale}
```

The JSONL trace format is optimised for durability, human readability, and offline analysis.

\subsection{Span Hierarchy}

When OTel is enabled, the runtime creates a span hierarchy that mirrors the runbook execution.

```
\begin{minted}{yaml}
Span: gert.run                // root span, covers entire run
  Span: gert.step             // one per step executed
    Span: gert.tool.invoke    // one per tool call within a step
    Span: gert.step           // next step
    ...
\end{minted}
```

The Go implementation uses `go.opentelemetry.io/otel/trace`:

```
\begin{minted}{go}
func (e *Engine) runWithOTel(ctx context.Context) error {
    ctx, runSpan := otel.Tracer("gert").Start(ctx, "gert.run",
        trace.WithAttributes(
            attribute.String("gert.run_id",    e.State.RunID),
            attribute.String("gert.runbook",   e.Runbook.Meta.Name),
            attribute.String("gert.actor",     e.State.Actor),
            attribute.String("gert.mode",      e.State.Mode),
        ),
    )
    defer runSpan.End()
    return e.execute(ctx)
}

func (e *Engine) executeStep(ctx context.Context, step Step) error {
    ctx, stepSpan := otel.Tracer("gert").Start(ctx, "gert.step",
        trace.WithAttributes(
            attribute.String("gert.step_id",    step.ID),
            attribute.String("gert.step_type",  step.Type),
            attribute.String("gert.step_name",   step.Name),
        ),
    )
    defer stepSpan.End()
    return e.executeStep(ctx, step)
}
```

```

    ),
  )
  defer stepSpan.End()
  // ...
}
\end{minted}

```

\subsection{Span Attributes}

The following attributes are set on all spans:

```

\begin{center}
\begin{tabular}{lll}
\hline
\textbf{Attribute} & \textbf{Spans} & \textbf{Description} \\
\hline
\texttt{gert.run\_id} & all & Run identifier \\
\texttt{gert.runbook} & all & Runbook name \\
\texttt{gert.actor} & run & Actor identity \\
\texttt{gert.mode} & run & Execution mode \\
\texttt{gert.step\_id} & step, tool & Step identifier \\
\texttt{gert.step\_type} & step & Step type \\
\texttt{gert.tool\_name} & tool & Tool name \\
\texttt{gert.transport} & tool & Transport (stdio/jsonrpc/mcp) \\
\hline
\end{tabular}
\end{center}

```

Trace context is propagated into tool invocations via the W3C `\texttt{traceparent}` header when

\subsection{Relationship to JSONL Trace}

OTel spans and the JSONL trace are complementary, not redundant:

```

\begin{itemize}
\item The JSONL trace is the authoritative durable record. It contains evidence hashes, gov

```

```

\item OTel spans are ephemeral, push-based, and optimised for latency-sensitive observability.
\item The trace event \texttt{"seq"} is embedded as the \texttt{gert.trace\_seq} span attribute.
\end{itemize}

```

```
\subsection{Configuration}
```

OTel is opt-in and configured via standard OTEL environment variables:

```

\begin{verbatim}
OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:4317
OTEL_SERVICE_NAME=gert
OTEL_TRACES_SAMPLER=always_on

```

When `OTEL_EXPORTER_OTLP_ENDPOINT` is not set, the OTel SDK is initialised with a no-op tracer and zero overhead is incurred. There is no `-otel` flag; configuration is entirely via the environment. This ensures that CI and offline use work without modification.

14.2 Implementation Notes for Go

The key Go interfaces for this subsystem are:

```

// TraceWriter is the interface the engine uses to record events.
// Implementations must be safe for concurrent use from a single goroutine
// (the engine's execution loop); multi-goroutine safety is optional.
type TraceWriter interface {
    Write(event DurableEvent) error
    Close() error
}

// RunStore manages all artefacts for a single run.
type RunStore interface {
    RunID() string
    WriteTrace(event DurableEvent) error
    ReadTrace() ([]DurableEvent, error)
    ReadTraceSince(afterSeq int64) ([]DurableEvent, error)
    SaveCheckpoint(state *RunState) (string, error)
    LoadLatestCheckpoint() (*RunState, int64, error)
}

```

```
    AcquireLock() error
    ReleaseLock() error
    WriteManifest(manifest *RunManifest) error
}

// EvidenceCollector is the interface for gathering evidence at manual steps.
type EvidenceCollector interface {
    Collect(ctx context.Context, stepID string,
        prompts []Prompt) (map[string]*EvidenceValue, error)
}
```

The `DirRunStore` is the production implementation. A `MemRunStore` (storing events in a slice) is provided for unit tests where filesystem I/O is undesirable. The `ReplayEvidenceCollector` returns pre-recorded evidence for scenario tests.

Chapter 15

Adapter Contracts

Gert v2 decouples the core runtime from presentation adapters (TUI, Web, VS Code) and integration systems (CI/CD platforms, observability backends, approval workflow systems) through explicitly versioned contracts. An **adapter contract** is a stable, versioned interface that gert commits to maintain across minor and patch releases. Adapters are consumers of these contracts: they observe, render, or orchestrate gert runs, but they own no execution logic.

This chapter defines the four primary adapter surfaces exposed by gert v2: JSON-RPC execution contract, WebSocket event stream contract, tool invocation contract, and extension handshake contract.

15.1 What is an Adapter Contract?

An **adapter contract** is a versioned API specification that defines:

1. The set of RPC methods, request/response schemas, and error envelopes.
2. The stability guarantee: which fields are required, which are optional, and which may be added in future minor versions.
3. The versioning scheme: how breaking changes are communicated, and how long deprecated versions are supported.
4. The transport layer: HTTP, WebSocket, stdio JSON-RPC, etc.

Adapter contracts are the integration surface of gert. They are designed for external systems to consume. Breaking changes to a contract require a major version bump and a deprecation period.

Example consumers:

- VS Code extension (consumes JSON-RPC execution contract and WebSocket event stream) [20]
- GitLab CI runner (consumes JSON-RPC execution contract to trigger runs)
- Datadog integration (consumes event stream to emit metrics)
- Slack approval bot (consumes event stream, submits approvals via JSON-RPC)

15.2 Contract Categories

Gert v2 exposes four distinct adapter surfaces, each with its own versioning and stability guarantees.

15.2.1 JSON-RPC Execution Contract (exec/v2)

The execution contract is the primary interface for starting, monitoring, and controlling gert runs. It is served over `gert serve -stdio` (stdio JSON-RPC) or `gert serve -http` (HTTP POST to `/rpc`).

Contract version: `exec/v2`

Transport: `stdio` JSON-RPC 2.0 or HTTP POST (JSON-RPC over HTTP)

Methods:

Request schema — `exec/start`:

```
{
  "jsonrpc": "2.0",
  "id": "1",
  "method": "exec/start",
  "params": {
    "runbookPath": "./runbooks/incident-triage.yaml",
    "inputs": {
      "incident_id": "INC-12345",
```

Method	Description
<code>exec/start</code>	Start a new runbook execution. Params: <code>runbookPath</code> , <code>inputs</code> , <code>mode</code> , <code>actorId</code> . Returns: <code>runId</code> , <code>status</code> .
<code>exec/next</code>	Advance the run by one step or until it reaches a pause point (approval, manual, error). Params: <code>runId</code> . Returns: <code>stepId</code> , <code>status</code> , <code>output</code> .
<code>exec/cancel</code>	Cancel an in-progress run. Params: <code>runId</code> , <code>reason</code> . Returns: <code>cancelled: bool</code> .
<code>exec/status</code>	Get the current status of a run. Params: <code>runId</code> . Returns: <code>status</code> , <code>currentStepId</code> , <code>progress</code> .
<code>run/list</code>	List all runs in the workspace. Params: <code>filter</code> , <code>limit</code> . Returns: <code>runs: [RunSummary]</code> .
<code>run/get</code>	Get full details of a run. Params: <code>runId</code> . Returns: <code>run: RunDetails</code> .

TABLE 15.1: JSON-RPC execution contract methods (`exec/v2`).

```

    "severity": "high"
  },
  "mode": "real",           // real / dry-run / replay
  "actorId": "ops@acme.example"
}
}

```

Response schema — `exec/start`:

```

{
  "jsonrpc": "2.0",
  "id": "1",
  "result": {
    "runId": "run-abc123",
    "status": "running",
    "createdAt": "2026-04-18T12:00:00Z",
    "traceFile": "./traces/run-abc123.jsonl"
  }
}

```

Error envelope: All errors follow JSON-RPC 2.0 error format:

```
{
  "jsonrpc": "2.0",
  "id": "1",
  "error": {
    "code": 1001,
    "message": "run not found",
    "data": {
      "runId": "run-xyz789",
      "category": "client_error",
      "retryable": false
    }
  }
}
```

See Section 15.4 for the canonical error code registry.

Stability guarantee: Within the `exec/v2` contract:

- All required fields in request/response schemas are stable.
- Optional fields may be added in minor versions (clients **MUST** ignore unknown fields).
- Required fields will not be removed or renamed (breaking change → `exec/v3`).
- Error codes are stable; new codes may be added but existing codes will not change semantics.

15.2.2 WebSocket Event Stream Contract (`events/v2`)

The event stream contract allows adapters to subscribe to real-time events from a running or completed gert execution. It is served over `gert serve -http` at the `/ws` or `/events` endpoint.

Contract version: `events/v2`

Transport: WebSocket (`wss://` or `ws://`)

Connection lifecycle:

1. Client establishes WebSocket connection to `/ws`.
2. Client sends a `subscribe` message with `runId`.
3. Server streams all events for that run in order.
4. Client may disconnect at any time; server closes gracefully.

Subscribe message:

```
{
  "action": "subscribe",
  "runId": "run-abc123",
  "sinceSequence": 0 // Optional: replay from sequence number
}
```

Event envelope: Events are transmitted as JSON objects, one per WebSocket message. The event envelope is identical to the trace event format defined in §12:

```
{
  "eventType": "step/started",
  "timestamp": "2026-04-18T12:00:00Z",
  "runId": "run-abc123",
  "stepId": "lookup_incident",
  "sequence": 42,
  "payload": {
    "stepType": "tool",
    "toolName": "acme.pd-lookup"
  }
}
```

All events carry:

- `eventType`: canonical event type (e.g., `step/started`, `governance/approval_required`).
- `timestamp`: RFC 3339 timestamp.
- `runId`: globally unique run identifier.

- **sequence**: monotonically increasing event sequence number (starts at 0).
- **payload**: event-specific data.

Reconnection protocol: If the WebSocket connection is lost, the client may reconnect and provide **sinceSequence** to resume from the last received event:

```
{
  "action": "subscribe",
  "runId": "run-abc123",
  "sinceSequence": 42 // Resume from event 43
}
```

The server will replay all events with **sequence** > 42.

Stability guarantee: Within the **events/v2** contract:

- Event envelope fields (**eventType**, **timestamp**, **runId**, **sequence**) are stable.
- New event types may be added in minor versions.
- Existing event types will not change their **eventType** string.
- Payload schemas may add optional fields; clients **MUST** ignore unknown fields.

15.2.3 Tool Invocation Contract (**tool/v2**)

The tool invocation contract defines the stdio JSON-RPC protocol that tools must implement to be invoked by gert. All tools — whether built-in, project-level, or extension-contributed — implement this contract.

Contract version: **tool/v2**

Transport: stdio JSON-RPC 2.0

Invocation request:

```
{
  "jsonrpc": "2.0",
  "id": "invocation-5",
  "method": "invoke",
  "params": {
    "action": "lookup",
    "inputs": {
      "id": "INC-12345"
    },
    "context": {
      "runId": "run-abc123",
      "stepId": "lookup_incident",
      "traceId": "trace-xyz789",
      "userId": "ops@acme.example",
      "mode": "real",
      "attempt": 1
    }
  }
}
```

Success response:

```
{
  "jsonrpc": "2.0",
  "id": "invocation-5",
  "result": {
    "outputs": {
      "incident": {
        "id": "INC-12345",
        "severity": "high",
        "status": "open"
      }
    },
    "exitCode": 0,
    "telemetry": {
      "durationMs": 320,

```

```
        "startedAt": "2026-04-18T12:00:00Z",
        "finishedAt": "2026-04-18T12:00:00.320Z"
    }
}
```

Error response:

```
{
  "jsonrpc": "2.0",
  "id": "invocation-5",
  "error": {
    "code": -32050,
    "message": "incident not found",
    "data": {
      "category": "runtime",
      "retryable": false,
      "details": {
        "incidentId": "INC-12345",
        "statusCode": 404
      }
    }
  }
}
```

Cancellation request:

```
{
  "jsonrpc": "2.0",
  "id": "cancel-5",
  "method": "cancel",
  "params": {
    "invocationId": "invocation-5",
    "reason": "run-cancelled"
  }
}
```


MCP tool adapter: When an MCP server is used as a tool source (§5), the gert host wraps the MCP `tools/call` protocol to satisfy the tool invocation contract. The adapter:

1. Translates the `invoke` request to an MCP `tools/call` request.
2. Forwards the `context` object as `_gert_context` in the MCP request (extension to MCP protocol).
3. Translates the MCP response back to the tool invocation contract response.
4. Flattens MCP `content` arrays to a single string for capture.

Stability guarantee: Within the `tool/v2` contract:

- Request/response envelope fields are stable.
- The `context` object fields are stable; new fields may be added.
- Error code ranges are stable (see Section 15.4).
- Tools MUST ignore unknown fields in requests.

15.2.4 Extension Handshake Contract (`extension/v2`)

The extension handshake contract defines the initialization and capability negotiation protocol between the gert host and an extension process. It is defined in §4 and summarized here for completeness.

Contract version: `extension/v2`

Transport: `stdio JSON-RPC 2.0`

Initialize request:

```
{
  "jsonrpc": "2.0",
  "id": "1",
  "method": "extension/initialize",
  "params": {
```

```
"host": {
  "name": "gert",
  "version": "2.0.0",
  "apiVersion": "2.0"
},
"protocolVersion": "2.0",
"grantedCapabilities": [
  "capability/tool-registration",
  "capability/network"
],
"runContext": {
  "runId": "run-abc123",
  "mode": "real"
}
}
```

Initialize response:

```
{
  "jsonrpc": "2.0",
  "id": "1",
  "result": {
    "extension": {
      "name": "acme.incident-pack",
      "version": "1.2.0"
    },
    "protocolVersion": "2.0",
    "acknowledgedCapabilities": [
      "capability/tool-registration",
      "capability/network"
    ]
  }
}
```

Heartbeat (ping/pong):

Request:

```
{
  "jsonrpc": "2.0",
  "id": "42",
  "method": "extension/ping",
  "params": {}
}
```

Response:

```
{
  "jsonrpc": "2.0",
  "id": "42",
  "result": {
    "status": "ok"
  }
}
```

Stability guarantee: Within the `extension/v2` contract:

- The `extension/initialize` request/response schema is stable.
- New capabilities may be added; extensions **MUST** ignore unknown capabilities in `grantedCapabilities`.
- The `extension/ping` method signature is stable.
- Extension error codes (§4) are stable.

15.3 Contract Versioning

Each adapter contract has a version declared in the initial handshake or HTTP header. The version format is `{surface}/{major}` (e.g., `exec/v2`, `events/v2`). There is no minor version: any breaking change bumps the major version.

15.3.1 Version Declaration

HTTP-based contracts (JSON-RPC, WebSocket): Clients **MUST** send an `X-Gert-Contract` header on the initial HTTP request:

```
GET /ws HTTP/1.1
Host: gert.acme.example:8443
X-Gert-Contract: events/v2
```

The server responds with:

```
HTTP/1.1 200 OK
X-Gert-Contract: events/v2
```

If the client sends an unsupported contract version, the server responds with HTTP 400 and error:

```
{
  "error": {
    "code": 4001,
    "message": "unsupported contract version",
    "data": {
      "requestedVersion": "events/v3",
      "supportedVersions": ["events/v2"]
    }
  }
}
```

stdio-based contracts (tool, extension): The contract version is declared in the first message exchanged (initialize or handshake):

```
{
  "protocolVersion": "2.0"
}
```

If the protocol version is unsupported, the responder returns error code -32001 (protocol version not supported).

15.3.2 Breaking vs. Non-Breaking Changes

Non-breaking changes (minor version, no contract version bump):

- Adding optional fields to requests or responses.

- Adding new methods to the contract.
- Adding new error codes.
- Adding new event types.

Consumers **MUST** implement forward compatibility: ignore unknown fields, ignore unknown methods, ignore unknown event types.

Breaking changes (major version bump):

- Removing or renaming required fields.
- Changing the semantics of existing methods.
- Removing methods.
- Changing error code meanings.
- Incompatible transport changes (e.g., switching from JSON-RPC to gRPC).

Breaking changes require a new contract version (e.g., `exec/v2` → `exec/v3`).

15.3.3 Deprecation Policy

- When a new major contract version is released, the previous version is supported for 12 months after the new version's GA date.
- During the deprecation period, both versions are supported simultaneously.
- After 12 months, the old version is removed. Clients must upgrade.

Example timeline:

- 2026-04-01: `exec/v2` released.
- 2027-06-01: `exec/v3` released; `exec/v2` enters deprecation.
- 2028-06-01: `exec/v2` removed; only `exec/v3` supported.

15.3.4 Forward and Backward Compatibility

Forward compatibility (clients): Clients MUST ignore unknown fields in responses. This allows the server to add optional fields without breaking existing clients.

Example:

```
// Server adds a new optional field in exec/v2
{
  "result": {
    "runId": "run-abc123",
    "status": "running",
    "newOptionalField": "value" // Clients ignore this
  }
}
```

Backward compatibility (servers): Servers MUST accept requests that omit optional fields added in later minor versions. Servers MUST NOT require new optional fields.

Example:

```
// Client sends an old request format (before a new optional field was added)
{
  "method": "exec/start",
  "params": {
    "runbookPath": "./runbook.yaml"
    // Missing new optional field "labels"
  }
}
// Server accepts and defaults the missing field
```

15.4 Error Codes

All adapter contracts use a canonical error code registry. Error codes are integers organized by category. Codes are stable across contract versions.

Error envelope structure:

```
{
```

```
"error": {
  "code": 2001,
  "message": "command denied: not in allowlist",
  "data": {
    "category": "governance",
    "retryable": false,
    "details": {
      "command": "rm",
      "reason": "in_denylist"
    }
  }
}
```

Error code stability:

- Existing error codes will not change their meaning.
- New error codes may be added in minor versions.
- Clients MUST handle unknown error codes gracefully (fallback to generic error handling).

15.5 Contract Test Suite

The gert repository includes a contract test suite that validates adapter implementations against the canonical contracts. Adapter authors SHOULD run these tests to ensure compatibility.

15.5.1 Test Suite Structure

Contract tests are located in `testdata/contracts/`:

```
testdata/contracts/
  exec-v2/
    01-start-run.test.yaml
    02-cancel-run.test.yaml
    03-list-runs.test.yaml
```

```
events-v2/
  01-subscribe.test.yaml
  02-reconnect.test.yaml
tool-v2/
  01-invoke.test.yaml
  02-cancel.test.yaml
extension-v2/
  01-initialize.test.yaml
  02-ping.test.yaml
```

Each test case is a YAML file with:

- **description**: human-readable test case description.
- **request**: the JSON-RPC request or WebSocket message.
- **expectedResponse**: the expected response (or error).
- **preconditions**: setup required before the test (e.g., "a run with id run-abc123 exists").

15.5.2 Running Contract Tests

```
$ gert contract test --adapter=vscode
Running contract tests for adapter: vscode
✓ exec-v2/01-start-run (passed)
✓ exec-v2/02-cancel-run (passed)
× events-v2/01-subscribe (failed: missing 'sequence' field)
```

Contract **test** results: 2 passed, 1 failed

Adapter implementations SHOULD achieve 100% pass rate before release.

15.5.3 Contract Test CI Integration

Contract tests are part of the CI gate for all adapter implementations:

- The VS Code extension CI runs `gert contract test -adapter=vscode`.
- Third-party adapters are encouraged to use the same test suite.
- Failed contract tests block merges to the main branch.

15.6 Reference Implementations

VS Code extension. The official VS Code extension (`vscode/` in the repository) is the reference implementation of the JSON-RPC execution contract (`exec/v2`) and the WebSocket event stream contract (`events/v2`). Adapter authors should study its implementation for best practices.

Built-in tools. The built-in tools (`ext/builtin/tools/` in the repository) are the reference implementation of the tool invocation contract (`tool/v2`). They demonstrate how to handle `invoke`, `cancel`, and error responses.

MCP adapter. The MCP tool adapter (`ext/toolruntime/mcp_adapter.go`) demonstrates how to wrap an external protocol (MCP) to satisfy the tool invocation contract.

15.7 Contract Stability Guarantees

Gert v2 commits to the following stability guarantees for all adapter contracts:

1. **Semantic versioning:** major version bumps indicate breaking changes; minor/patch versions are backward compatible.
2. **12-month deprecation period:** old contract versions are supported for 12 months after a new major version is released.
3. **Additive changes only:** within a major version, only non-breaking changes (adding optional fields, new methods, new event types) are permitted.
4. **Error code stability:** error codes do not change meaning within a major version.
5. **Forward compatibility:** clients must ignore unknown fields; servers must accept omitted optional fields.

These guarantees are designed to minimize adapter churn and allow external systems to depend on gert contracts with confidence.

15.8 Contract Documentation and Tooling

15.8.1 OpenAPI Specifications

All HTTP-based contracts (JSON-RPC over HTTP, WebSocket event stream) are documented as OpenAPI 3.1 specifications [3] in `specs/contracts/`:

- `specs/contracts/exec-v2.openapi.yaml`
- `specs/contracts/events-v2.openapi.yaml`

These specifications are the source of truth for request/response schemas and are used to generate client libraries.

15.8.2 JSON Schema Validation

All JSON-RPC request/response schemas are defined as JSON Schema (Draft 2020-12) [29] in `specs/contracts/schemas/`. These schemas are used for:

- Runtime validation of requests/responses.
- Contract test validation.
- Client library generation (TypeScript, Python, Go).

15.8.3 Client Library Generation

The gert project provides code generation tools to generate client libraries from the OpenAPI and JSON Schema definitions:

```
$ gert codegen client --language=typescript --output=./clients/ts/  
Generated TypeScript client for exec/v2 and events/v2  
Files written to ./clients/ts/
```

Official client libraries are provided for:

- TypeScript (for VS Code and web adapters)
- Python (for CI/CD integrations)
- Go (for extension authors)

Code	Description	Retryable
1xxx — Execution Errors		
1001	Run not found (invalid <code>runId</code>)	No
1002	Step failed (runtime error in step execution)	Maybe
1003	Run cancelled (user or timeout cancellation)	No
1004	Run already exists (duplicate <code>runId</code>)	No
2xxx — Governance Errors		
2001	Command denied (not in allowlist or in denylist)	No
2002	Approval timeout (no approval received in time)	No
2003	Approval rejected (approver rejected the request)	No
2004	Insufficient approvals (min approval count not reached)	No
3xxx — Schema Errors		
3001	Schema validation failed (runbook does not match schema)	No
3002	Unknown step type (unrecognized <code>stepType</code>)	No
3003	Import cycle detected (circular runbook dependencies)	No
4xxx — Integration Errors		
4001	Tool not found (tool name does not resolve)	No
4002	Extension handshake failed (protocol error)	Maybe
4003	Input resolution failed (provider error or missing input)	Maybe
4004	Unsupported contract version	No
5xxx — Server Errors		
5001	Internal error (unexpected host failure)	Maybe
5002	Resource exhausted (rate limit or quota exceeded)	Yes
5003	Service unavailable (host shutting down or overloaded)	Yes

TABLE 15.2: Canonical error code registry for adapter contracts.

Chapter 16

Input Provider Framework

The input provider framework governs how gert resolves dynamic input values at run time. When a runbook declares an input with a `from:` binding, the provider framework locates the appropriate provider, dispatches a resolution request, and returns the concrete value before the first step that needs it begins.

16.1 Design Goals

- **Extensibility:** third-party systems (PagerDuty, ServiceNow, Vault, etc.) integrate as provider plugins without changes to the host binary.
- **Transparency:** every resolved value is recorded in the run trace so the operator can audit where each input came from.
- **Fallback safety:** resolution failures degrade gracefully to an interactive prompt rather than aborting the run silently.
- **Consistency with tools:** providers share the same transport layer as tools (`stdio`, `stdio-jsonrpc`, `mcp`), minimising implementation surface.

16.2 Provider Definition Schema

Each provider is declared in a `.provider.yaml` file. By convention, project-level providers live in `providers/<name>.provider.yaml`.

```
# providers/pagerduty.provider.yaml
apiVersion: provider/v2
```

```
meta:
  name: pd                                # Prefix used in from: bindings (e.g. pd.*)
  version: "1.0.0"
  description: "Resolves inputs from PagerDuty incidents and services"
  binary: pd-provider                     # Executable name (looked up in PATH)

transport:
  mode: stdio-jsonrpc                     # stdio / stdio-jsonrpc / mcp
  startup:
    argv: ["--config", "pd.yaml"]
    ready-pattern: "provider ready"
    timeout: "10s"
    shutdown-method: "shutdown"

resolution:
  method: provider/resolve                 # JSON-RPC method the host calls
  prefixes:
    - "pd."                               # All from: values starting with pd.
                                           # are dispatched to this provider

# Schema for what the provider needs in each resolution request
input-schema:
  type: object
  properties:
    bindings:
      type: object
      description: "Map of input name to from: binding string"
    context:
      type: object
      description: "Execution context (runId, userId, mode)"
  required: ["bindings"]

# Schema for what the provider returns
output-schema:
  type: object
  properties:
    resolved:
```

```
    type: object
    description: "Map of input name to resolved string value"
  warnings:
    type: array
    items:
      type: string
  required: ["resolved"]

  cache:
    enabled: true
    scope: run                # run / step / none
    ttl: "60s"

  governance:
    requires-capabilities:
      - capability/network    # This provider needs outbound network
    env-read-patterns:
      - "PD_API_KEY"
      - "PD_*
```

16.3 Resolution Protocol

16.3.1 Resolution Flow

When the run engine encounters an input with a `from:` binding (e.g., `from: pd.incident.title`), the following steps occur:

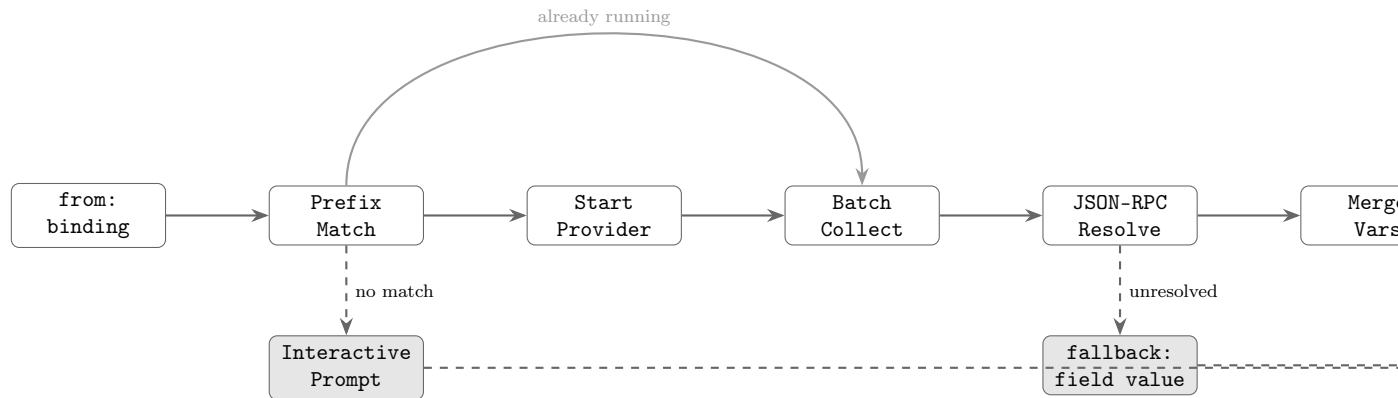


FIGURE 16.1: Input provider resolution pipeline. A `from: binding` triggers a prefix lookup; if a matching provider is found its process is started (or reused), inputs are batched, and a single `provider/resolve` JSON-RPC call returns all values. Unresolved bindings fall back to the `fallback:` field or an interactive prompt.

1. **Prefix match:** the framework scans registered providers for one whose declared prefix matches the start of the binding string. The longest matching prefix wins.
2. **Provider start:** if the matched provider's process is not yet running, the host spawns it and waits for the ready signal (same mechanics as `stdio-jsonrpc` tools; see Section 7.3).
3. **Batch collection:** all inputs whose binding prefix maps to the same provider are grouped into a single resolution request to minimise round-trips.
4. **JSON-RPC call:** the host sends a `provider/resolve` request.
5. **Result merge:** resolved values are merged into the run's variable map.
6. **Fallback:** any binding that was not resolved (absent from the response's `resolved` map, or where the provider returned a warning) falls back according to the input's `fallback:` field (default: interactive prompt).

16.3.2 Resolution Request Envelope

```
{
```

```
"jsonrpc": "2.0",
"id": "prov-1",
"method": "provider/resolve",
"params": {
  "bindings": {
    "incident_title": "pd.incident.title",
    "incident_severity": "pd.incident.severity",
    "service_name": "pd.service.name"
  },
  "context": {
    "runId": "run-abc123",
    "stepId": "__pre-flight__",
    "userId": "ops@acme.example",
    "mode": "real",
    "traceId": "trace-xyz"
  }
}
}
```

16.3.3 Resolution Response Envelope

```
{
  "jsonrpc": "2.0",
  "id": "prov-1",
  "result": {
    "resolved": {
      "incident_title": "Database connection pool exhausted",
      "incident_severity": "P1",
      "service_name": "payments-api"
    },
    "warnings": [],
    "telemetry": {
      "startedAt": "2026-04-18T10:00:00Z",
      "finishedAt": "2026-04-18T10:00:00.080Z",
      "durationMs": 80
    }
  }
}
```


16.3.4 Resolution Error Handling

If the provider returns a JSON-RPC error, or if a binding key is absent from the **resolved** map, the host applies the input's fallback strategy:

```
# In runbook meta.inputs
inputs:
  incident_title:
    from: pd.incident.title
    description: "Active incident title"
    fallback: prompt          # prompt / fail / default
    default: "Unknown incident" # Used when fallback: default
```

- **fallback: prompt** (default): the host issues an interactive prompt for the value. In non-interactive modes (dry-run, CI) this causes the run to fail.
- **fallback: default**: the declared **default:** value is used without prompting. A warning is written to the trace.
- **fallback: fail**: the run aborts immediately with a structured error.

16.3.5 Resolution Caching

Providers may declare a cache policy in their definition. The host respects this policy:

- **scope: run** — the resolved value is reused for all subsequent steps in the same run that reference the same binding. This is the default for external providers.
- **scope: step** — the value is resolved fresh on each step that references it.
- **scope: none** — caching is disabled; every binding triggers a new RPC call.

Cache entries are keyed by (**runId**, **bindingString**). Cached values are stored in the run's in-memory variable map only; they are not persisted across run resumptions.

16.4 Built-in Providers

The following providers are compiled into the gert host binary and are always available without a **.provider.yaml** file.

16.4.1 env

Reads values from environment variables.

```
inputs:
  api_key:
    from: env.ACME_API_KEY
    description: "API key for the Acme service"
```

The `env.` prefix is followed by the exact environment variable name. If the variable is not set, resolution fails and the fallback strategy is applied. The `env` provider never contacts an external process; it reads directly from the host's environment.

16.4.2 file

Reads values from files on the local file system.

```
inputs:
  ssh_key:
    from: "file ~/.ssh/id_rsa"
    description: "SSH private key"
  config_value:
    from: "file ./etc/gert/config.json#/database/host"
    description: "Database host from config file"
```

The path after `file.` is the file path. An optional JSON Pointer fragment (`#/path/to/field`) extracts a nested value from a JSON or YAML file. Without a fragment, the entire file contents are returned as a string.

16.4.3 prompt

The interactive prompt provider is the default resolution strategy. It is also the fallback for any failed provider resolution (unless overridden).

```
inputs:
  server_name:
    from: prompt
    description: "Server to check"
```

In **real** and **debug** modes, the host presents a terminal prompt to the operator. In **dry-run** and **replay** modes, the prompt provider reads from the scenario's `inputs.yaml` instead. In non-interactive environments where no scenario is available, the prompt provider fails with an informative error directing the operator to supply the value via `-var` or an explicit provider.

16.4.4 workspace

Reads values from the workspace configuration file (`.gert/config.yaml`).

```
inputs:
  default_region:
    from: workspace.defaults.region
    description: "AWS region from workspace config"
```

The path after `workspace.` is a dot-notation key into the workspace configuration object. This allows teams to centralise defaults without repeating them in every runbook.

16.5 Interactive Step Type Contracts

Input providers not only resolve `from:` bindings for runbook inputs, they also power the three interactive step types: **choice**, **decision**, and **collector**. When the run engine executes one of these steps, it dispatches a step-specific request to the configured input provider (or falls back to the built-in prompt provider).

16.5.1 Choice Step Contract

A **choice** step presents the operator with a fixed set of labeled options and stores the selected value as a variable. This is analogous to a dropdown menu or radio button group in a graphical form.

Request envelope. When the run engine executes a **choice** step, it sends a `provider/choice` JSON-RPC request to the configured provider:

```
{
  "jsonrpc": "2.0",
  "id": "choice-1",
```

```
"method": "provider/choice",
"params": {
  "stepId": "select-region",
  "prompt": "Select the target AWS region",
  "options": [
    { "value": "us-east-1", "label": "US East (N. Virginia)" },
    { "value": "us-west-2", "label": "US West (Oregon)" },
    { "value": "eu-west-1", "label": "Europe (Ireland)" }
  ],
  "default": "us-east-1",
  "context": {
    "runId": "run-abc123",
    "stepId": "select-region",
    "userId": "ops@acme.example",
    "mode": "real"
  }
}
```

Response envelope. The provider returns the selected value:

```
{
  "jsonrpc": "2.0",
  "id": "choice-1",
  "result": {
    "selected": "us-west-2"
  }
}
```

Contract requirements.

- The `options` array must contain at least two elements.
- Each option must have both `value` (the string stored in the variable) and `label` (the human-readable text displayed to the operator).
- The `default` field is optional. If present, it must match one of the `value` fields in the `options` array.

- The provider's **selected** response must match one of the option values exactly. If the provider returns a value not in the options list, the run engine fails the step with a **choice.invalid_selection** error.
- Providers MAY render options in the order provided, or MAY sort them alphabetically by label for improved usability. The canonical order is declared in the runbook YAML.
- **multiple: true** (optional, default **false**) — when set, the provider MUST allow the operator to select one or more values. The **selected** response field MUST be a JSON array of strings rather than a single string. The run engine stores the result as a JSON array of strings; see §4.4.5 for storage semantics.
- **options_from** (optional) — mutually exclusive with the static **options** array. When present, the run engine calls **inputProvider/getOptions** to fetch options dynamically before dispatching the **provider/choice** request; see §16.5.4.

Built-in prompt provider behaviour. The built-in **prompt** provider presents a numbered list in the terminal and waits for the operator to enter the number or value corresponding to their selection. In **dry-run** mode, it reads the value from the scenario's **inputs.yaml** or uses the **default** if provided.

16.5.2 Decision (Router) Step Contract

A **decision** step presents the operator with a set of routing choices, where each choice directs execution to a different runbook or labeled section. Unlike **choice**, a **decision** is a control-flow primitive: the selected route determines which node the run engine transitions to next.

Request envelope. When the run engine executes a **decision** step, it sends a **provider/decision** JSON-RPC request:

```
{
  "jsonrpc": "2.0",
  "id": "decision-1",
  "method": "provider/decision",
  "params": {
    "stepId": "triage-severity",
    "prompt": "Select the incident severity level",
```

```
{
  "routes": [
    { "route": "p1-critical", "label": "P1: Critical (immediate)" },
    { "route": "p2-high",     "label": "P2: High (1 hour SLA)" },
    { "route": "p3-normal",   "label": "P3: Normal (24 hour SLA)" }
  ],
  "context": {
    "runId": "run-abc123",
    "stepId": "triage-severity",
    "userId": "ops@acme.example",
    "mode": "real"
  }
}
```

Response envelope. The provider returns the selected route label:

```
{
  "jsonrpc": "2.0",
  "id": "decision-1",
  "result": {
    "route": "p2-high"
  }
}
```

Contract requirements.

- The `routes` array must contain at least two elements.
- Each route must have both `route` (the target node label or runbook ID) and `label` (the human-readable text).
- The provider does *not* need to know about the runbook graph structure. It only returns the selected route label. The run engine is responsible for resolving the route label to a target node and validating that the target exists.
- If the provider returns a route label not in the `routes` array, the run engine fails the step with a `decision.invalid_route` error.

- The decision result is *not* stored as a variable. It only controls execution flow. If the operator needs the decision recorded, the runbook author must add an explicit variable assignment in the target node.

Built-in prompt provider behaviour. The built-in `prompt` provider displays the routes as a numbered list and waits for the operator to select one. In `dry-run` mode, the route is read from the scenario file or the run fails if no scenario is provided.

16.5.3 Collector Step Contract

A `collector` step gathers unstructured input from the operator: free-form text, multi-line notes, file attachments, screenshots, URLs, or other evidence artifacts. The collected data is stored as named variables or artifacts in the run's state.

Request envelope. When the run engine executes a `collector` step, it sends a `provider/collect` JSON-RPC request:

```
{
  "jsonrpc": "2.0",
  "id": "collect-1",
  "method": "provider/collect",
  "params": {
    "stepId": "gather-evidence",
    "instructions": "Upload a screenshot of the error and describe the issue",
    "fields": [
      {
        "name": "screenshot",
        "type": "file",
        "label": "Error screenshot",
        "required": true,
        "visible": true,
        "accept": ["image/png", "image/jpeg"],
        "maxSizeBytes": 10485760
      },
      {
        "name": "description",
        "type": "text",
```

```
    "label": "Describe what you observed",
    "required": true,
    "visible": true,
    "multiline": true,
    "validation": {
      "minLength": 10,
      "maxLength": 2000,
      "pattern": "^[\\s\\S]+$",
      "patternHint": "Description must not be empty"
    }
  },
  {
    "name": "log_url",
    "type": "url",
    "label": "Link to related logs (optional)",
    "required": false,
    "visible": true
  }
],
"context": {
  "runId": "run-abc123",
  "stepId": "gather-evidence",
  "userId": "ops@acme.example",
  "mode": "real"
}
}
```

Response envelope. The provider returns the collected values and any uploaded artifacts:

```
{
  "jsonrpc": "2.0",
  "id": "collect-1",
  "result": {
    "values": {
      "description": "Database connection pool exhausted...",
      "log_url": "https://logs.acme.example/trace/xyz"
    }
  },
}
```



```
"artifacts": [  
  {  
    "field": "screenshot",  
    "filename": "error-2026-04-18.png",  
    "contentType": "image/png",  
    "sizeBytes": 245631,  
    "sha256": "a3c7d1f...",  
    "storagePath": "artifacts/run-abc123/screenshot.png"  
  }  
]  
}  
}
```

Contract requirements.

- The `fields` array declares the form structure. Each field has:
 - `name`: the variable name where the value will be stored.
 - `type`: one of `text`, `number`, `integer`, `date`, `datetime`, `boolean`, `select`, `file`, `image`, or `url`. Full per-type validation and variable-storage rules are defined in §4.4.5.
 - `label`: human-readable prompt text.
 - `required`: boolean flag.
 - `visible`: boolean flag, pre-evaluated by the executor from the field's `when` expression before the request is sent. `true` means the field should be rendered; `false` means the field should be hidden or skipped. CLI providers SHOULD skip fields where `visible: false`; rich UI providers SHOULD show or hide the field dynamically as the user fills in earlier fields, re-evaluating `when` client-side so the form responds in real time without a round-trip to the executor. If `when` was absent on the field, `visible` is always `true`. See §3.3.6 for the full conditional field evaluation contract.
 - `validation` (optional object): client-side validation constraints that providers MAY enforce before submitting values, reducing unnecessary round-trips. The executor always re-validates server-side regardless of whether the provider enforced them:

- * `validation.pattern` — RE2 regex that the value must fully match (`text` fields only).
 - * `validation.patternHint` — human-readable message shown when `pattern` fails (e.g. “*Must be a valid ticket reference: ABC-1234*”).
 - * `validation.minLength`, `validation.maxLength` — character-count bounds for `text` fields.
 - * `validation.min`, `validation.max`, `validation.step` — numeric bounds for `number` and `integer` fields.
 - * `validation.min`, `validation.max` — date bounds for `date` and `datetime` fields (ISO 8601 strings; lexicographic comparison is correct for ISO dates).
 - * `options` array or `options_from` binding — required for `select` fields; see §16.5.4.
 - * `multiple: true|false` (default `false`) — enables multi-select for `select` fields.
 - * `validation.min_selections`, `validation.max_selections` — array length bounds for multi-select fields.
 - * `multiline: true|false` — UI hint for `text` fields; no effect on validation or variable storage.
 - * `accept` (MIME types) and `maxSizeBytes` — for `file` and `image` fields.
- Providers MUST return all required fields in the `values` map. If a required field is missing, the run engine fails the step with a `collector.missing_required_field` error.
 - For `type: file` fields, providers MUST upload the file to the artifact store and return metadata in the `artifacts` array. The artifact metadata includes:
 - `field`: the field name from the request.
 - `filename`: the original filename.
 - `contentType`: MIME type.
 - `sizeBytes`: file size in bytes.
 - `sha256`: SHA-256 hash for integrity verification.
 - `storagePath`: path where the artifact is stored (relative to the run’s artifact root).

- File size limits are enforced by the provider. If an uploaded file exceeds `maxSizeBytes`, the provider MUST reject it with a `collector.file_too_large` error before storing it.
- Supported MIME types for file uploads:
 - Images: `image/png`, `image/jpeg`, `image/gif`
 - Documents: `application/pdf`, `text/plain`, `text/markdown`
 - Archives: `application/zip`, `application/x-tar`, `application/gzip`
 - Logs: `text/plain`, `application/json`
- Providers MAY support additional MIME types, but MUST respect the `accept` constraint if declared in the field definition.
- The artifact storage path is provider-specific. The built-in `prompt` provider stores artifacts in `.gert/runs/<runId>/artifacts/`. External providers (e.g., S3, Azure Blob) MAY store artifacts in remote storage and return a signed URL or storage reference.

Built-in prompt provider behaviour. The built-in `prompt` provider displays each field as a terminal prompt. For `type: text` with `multiline: true`, it opens the operator's configured editor (`$EDITOR`). For `type: file`, it prompts for a file path and copies the file to the local artifact store. For `type: select`, it presents a numbered list (single-select) or a checkbox list (multi-select). For `type: boolean`, it prompts with a `[y/N]` confirmation. For `type: number`, `integer`, `date`, and `datetime`, it accepts free-text input and validates the format before accepting. In `dry-run` mode, all values are read from the scenario file.

16.5.4 Dynamic Options Protocol

When a `select` field in a `collector` step, or a `choice` step, declares `options_from.provider`, the run engine must fetch the option list dynamically from that provider before rendering the field or dispatching the interactive step request.

Fetch Sequence

1. **Provider lookup** — the run engine resolves the provider ID from `options_from.provider` in `ExecutionPlan.Providers`. If the provider is not registered, the step fails immediately with `options_from.unknown_provider`.

2. **Provider start** — if the provider process is not yet running, it is started using the normal `stdio-jsonrpc` startup sequence.
3. **Options request** — the run engine sends an `inputProvider/getOptions` JSON-RPC request to the provider, including the current variable scope so the provider can filter or compute options contextually.
4. **Option list received** — the run engine treats the returned `options` array as if it had been declared statically in the runbook. Validation and rendering proceed identically to the static-options path.

Failure to fetch options (provider error, network error, or timeout) fails the step immediately with `options_from.fetch_failed`. There is no silent fallback to an empty options list.

`inputProvider/getOptions` Request

```
{
  "jsonrpc": "2.0",
  "id": "getopts-1",
  "method": "inputProvider/getOptions",
  "params": {
    "providerId": "employee-directory",
    "field":      "managers",
    "stepId":     "assign-manager",
    "variables": {
      "department": "engineering",
      "region":     "us-west"
    },
    "context": {
      "runId": "run-abc123",
      "stepId": "assign-manager",
      "userId": "ops@acme.example",
      "mode": "real"
    }
  }
}
```

Request fields.

- `providerId` — the provider ID declared in `options_from.provider`.
- `field` — the `options_from.field` value; allows a single provider to serve multiple distinct option lists (e.g. `managers`, `engineers`, `directors`).
- `stepId` — the step making the request (for audit and debugging).
- `variables` — the current run variable scope at the time the step is reached. The provider MAY use these to filter or compute options contextually (e.g. return only managers in the operator's department).
- `context` — standard execution context (run ID, actor, mode).

inputProvider/getOptions Response

```
{
  "jsonrpc": "2.0",
  "id": "getopts-1",
  "result": {
    "options": [
      { "value": "alice",    "label": "Alice Chen (Staff Eng)" },
      { "value": "bob",    "label": "Bob Okafor (Principal)" },
      { "value": "charlie", "label": "Charlie Park (Director)",
        "hint": "Manages infrastructure" }
    ],
    "cacheTtlSeconds": 300
  }
}
```

Response fields.

- `options` — array of `{value, label}` objects. Each `value` is the string stored in the run variable on selection; `label` is the human-readable text. An optional `hint` field provides secondary descriptive text.
- `cacheTtlSeconds` (optional) — if non-zero, the run engine caches the returned option list in memory for the remainder of the run (capped at the declared TTL). This avoids repeated RPC calls when the same dynamic field appears in multiple steps of a single run. A value of 0 disables caching for this response.

Capability declaration. Providers that implement `inputProvider/getOptions` MUST declare the capability in their handshake response. If the run engine encounters an `options_from` binding and the matched provider does not advertise `getOptions: true`, the step fails with `options_from.capability_not_supported`.

16.5.5 Autocomplete Search Protocol

The `type: autocomplete` field in a `collector` step requires live-search support from the input provider. The run engine issues `inputProvider/search` RPC calls as the operator types, with debouncing handled by the UI layer.

`inputProvider/search` Request

```
{
  "jsonrpc": "2.0",
  "id": "search-1",
  "method": "inputProvider/search",
  "params": {
    "provider": "service-registry",
    "field": "query",
    "query": "pay",
    "minChars": 2,
    "runId": "20260418T142300-abc12345",
    "variables": {
      "region": "us-west"
    }
  }
}
```

Request fields.

- `provider` — the provider ID from `options_from.provider` (or the field's dedicated `autocomplete.provider` binding).
- `field` — the field ID within the step, allowing a single provider to serve multiple autocomplete fields.
- `query` — the current text typed by the operator. May be an empty string `""` on initial focus or during CLI degradation (see below).

- **minChars** — the minimum character count declared on the field. The run engine does not issue the RPC until `len(query) >= minChars`; this field is informational for the provider.
- **runId** — the current run ID (for audit and logging on the provider side).
- **variables** — the current run variable scope at dispatch time. Providers MAY use this to filter results contextually (e.g., return only services in the operator's selected region).

inputProvider/search Response

```
{
  "jsonrpc": "2.0",
  "id": "search-1",
  "result": {
    "results": [
      { "value": "payments-api", "label": "Payments API" },
      { "value": "payroll-svc", "label": "Payroll Service" }
    ],
    "hasMore": false
  }
}
```

Response fields.

- **results** — array of {value, label} objects. **value** is the string stored in the run variable on selection; **label** is the human-readable text shown in the UI.
- **hasMore** (boolean) — if **true**, the UI SHOULD display a hint such as *“Showing top N results — keep typing to narrow.”*. A **false** value indicates the full matching set is returned.

Timeout contract. Providers MUST return within **500 ms**. If no response is received within this window, the run engine treats the call as timed out, returns an empty results list, and allows the operator to retry by continuing to type. The timeout is measured from the moment the RPC request is sent on the transport.

CLI degradation. The built-in `prompt` provider does not support real-time search. When an `autocomplete` field is dispatched to a CLI provider, the run engine issues a single `inputProvider/search` call with `query: ""` to retrieve all available options, then presents the result as a standard `select` list. Typing ahead is not supported in CLI mode.

Capability declaration. Providers that implement `inputProvider/search` MUST declare `search: true` in their capability advertisement (see matrix below). A provider that does not advertise `search: true` will have autocomplete fields degraded to the CLI path automatically.

16.5.6 Provider Capability Matrix

Not all providers support all interactive step types. The following table shows which step types each built-in provider supports:

Provider	Input Resolution	choice	decision	collector	getOptions	search
<code>prompt</code>	Yes	Yes	Yes	Yes	No	No
<code>env</code>	Yes	No	No	No	No	No
<code>file</code>	Yes	No	No	No	No	No
<code>workspace</code>	Yes	No	No	No	No	No

`search: false` for all built-in providers reflects CLI degradation: autocomplete fields degrade to a standard select when the active provider cannot perform live search. Rich UI providers (e.g., the VS Code extension's built-in provider) SHOULD declare `search: true` to enable real-time autocomplete rendering.

External providers (PagerDuty, ServiceNow, Slack, etc.) MAY implement any combination of these contracts. A provider that implements `provider/choice` MUST declare this in its capability advertisement during the handshake phase:

```
# Example provider capability declaration
{
  "jsonrpc": "2.0",
  "method": "provider/initialize",
  "params": {
    "capabilities": {
      "resolution": true,

```



```
    "choice": true,  
    "decision": false,  
    "collector": true,  
    "getOptions": true,  
    "search": true  
  }  
}  
}
```

If the run engine encounters a `choice`, `decision`, or `collector` step and the configured provider does not advertise the corresponding capability, the engine falls back to the built-in `prompt` provider.

16.6 Provider Composition

A `from:` binding may declare a priority-ordered chain of providers. The host attempts each provider in order and uses the first successful result.

```
inputs:  
  api_token:  
    from:  
      - env.ACME_API_TOKEN           # Try env var first  
      - vault.secret/acme/api-token # Fall back to Vault  
      - prompt                       # Final fallback: ask the operator  
    description: "API token for Acme service"
```

Chaining is evaluated left-to-right. A provider is considered to have failed if it returns an error, if the binding key is absent from the resolved map, or if the resolved value is an empty string (unless the input declares `allow-empty: true`).

16.7 Provider Lifecycle

Process model. Providers that use the `stdio-jsonrpc` or `mcp` transport are *persistent*: the host spawns each provider process once at the start of the pre-flight resolution phase and keeps it alive for the entire run. This avoids repeated startup latency for providers that need to authenticate or maintain session state (e.g., Vault token renewal, PagerDuty OAuth).

Providers using the `stdio` transport are *per-resolution*: the host spawns a new process for each resolution request.

Startup. Provider processes are started lazily on first use, following the same `ready-pattern` / timeout mechanism as `stdio-jsonrpc` tools (Section 7.3).

Shutdown. At run completion (success, failure, or cancellation) the host iterates over all running provider processes and sends the method declared in `transport.startup.shutdown-method` (default: `shutdown`). Providers are given a 5-second grace period to clean up before the host sends `SIGTERM`, then `SIGKILL` after a further 5 seconds.

Crash handling. If a provider process crashes during a resolution call, the host logs a structured `provider.crashed` trace event and applies the input's `fallback:` strategy for all bindings that were in-flight at the time of the crash.

Chapter 17

Observability and Diagnostics

Operational visibility is essential for production systems. This chapter specifies the observability model for gert v2: structured logging, metrics, distributed tracing, health probes, and diagnostics tooling. It is distinct from § 12 (Evidence, Tracing, and Resumption): **§12 specifies immutable audit records for compliance and resumption**; this chapter specifies **operational signals for operators** to answer questions like *is the system healthy?*, *what is slow?*, *where did this run fail?*, and *what is the resource utilization?*

The design follows the three-pillar observability model [13] adapted for workflow execution: metrics for aggregated performance, distributed tracing for execution flow, and structured logging for detailed context. All three are instrumented using industry standards (OpenTelemetry [6], W3C Trace Context [2], RFC 3339 timestamps [19]) to ensure interoperability with observability platforms.

17.1 Observability Model

Gert v2 applies the three-pillar observability model to workflow execution. Each pillar serves a distinct purpose:

17.1.1 Metrics: Aggregated Performance Indicators

Purpose: Track aggregate performance, throughput, error rates, and resource utilization over time.

Use Cases:

- *Alerting:* Trigger alerts when error rates exceed thresholds or latency degrades

- *Capacity Planning*: Measure run throughput and active concurrency to inform scaling decisions
- *SLO Monitoring*: Track percentile latencies (p50, p95, p99) for step execution times

Metric Types:

- **Counter**: Monotonically increasing (total runs, total errors)
- **Gauge**: Point-in-time measurement (active runs, loaded extensions)
- **Histogram**: Distribution of values (run duration, tool invocation latency)

Exposition Format: Prometheus OpenMetrics format exposed via HTTP endpoint (GET /metrics) when running in server mode (`gert serve -http`).

17.1.2 Distributed Tracing: Execution Flow Visualization

Purpose: Capture the hierarchical execution tree with timing and outcome for every run, step, and tool invocation.

Use Cases:

- *Root-Cause Analysis*: Identify which step in a failed run caused the failure
- *Latency Diagnosis*: Find which tool invocations are slow
- *Cross-System Correlation*: Link gert spans to HTTP API call traces via W3C Trace Context propagation

Implementation: OpenTelemetry SDK with OTLP exporter [6]. Spans emitted for every run (root span), step (child spans), and tool invocation (grandchild spans).

Span Hierarchy:

```
gert.run (run_id=...)
+- gert.step (step_id=backup-db, step_type=cli)
| +- gert.tool.invoke (tool=ssh, exit_code=0)
|   L- gert.governance.check (policy=allow-ssh)
+- gert.step (step_id=deploy, step_type=tool)
|   L- gert.tool.invoke (tool=kubectl, exit_code=0)
L- gert.step (step_id=verify, step_type=manual)
    L- gert.input.resolve (provider=approval-gate)
```

Trace Backends: Jaeger [15], Zipkin [30], Datadog, New Relic, AWS X-Ray (all OTLP-compatible).

17.1.3 Structured Logging: Detailed Contextual Events

Purpose: Emit detailed, machine-parseable log lines with run and step context for debugging and incident investigation.

Use Cases:

- *Debugging:* Search logs by `run_id` to see all activity for a specific run
- *Error Investigation:* View full error messages and stack traces in context
- *Compliance Audit:* Query logs for all runs by a specific actor or runbook

Log Format: JSON, one object per line, emitted to `stderr` by default.

Required Fields:

- `timestamp` (RFC 3339 [19])
- `level` (debug, info, warn, error)
- `msg` (human-readable message)
- `run_id` (when in run context)
- `step_id` (when in step context)

Aggregation: Loki, Elasticsearch, Splunk, CloudWatch Logs (any system that can parse JSON logs).

17.2 OpenTelemetry Integration

Gert v2 integrates OpenTelemetry as the standard tracing layer. All spans, metrics, and logs follow the OpenTelemetry specification [6] to ensure portability across observability backends.

17.2.1 Span Hierarchy and Attributes

OpenTelemetry spans form a hierarchical tree representing the execution structure. Each span level has required attributes for filtering and analysis.

Root Span: `gert.run`

The root span represents the entire run from start to completion.

Span Name: `gert.run`

Required Attributes:

```
runbook.id      = "deploy-app-v3"           // runbook identifier
runbook.version = "1.2.3"                   // version (if specified)
actor.id        = "alice@example.com"       // human or service principal
run.id          = "20260418T143022-a3f9c1"  // unique run ID
run.mode        = "real"                    // "real" | "replay" | "dry-run"
client.type     = "cli"                     // "cli" | "tui" | "web" | "mcp"
```

Span Status:

- OK if run outcome is `success`
- ERROR if run outcome is `failure`, `cancelled`, or `governance-blocked`

Child Span: `gert.step`

Each step execution produces a child span under `gert.run`.

Span Name: `gert.step`

Required Attributes:

```
step.id         = "backup-db"               // step identifier from YAML
step.type       = "cli"                     // "cli" | "manual" | "tool" | "invoke"
step.index      = 2                         // zero-based position in sequence
run.id          = "20260418T143022-a3f9c1"  // run correlation
step.name       = "Backup production database" // human-readable name
```

Optional Attributes:

```
step.outcome    = "success"                 // "success" | "failure" | "skipped"
step.retries    = 2                         // number of retry attempts
```

Span Events: Emit span events for key transitions:

- `governance.check.started` — governance evaluation started
- `governance.check.passed` — governance check passed
- `approval.requested` — human approval requested
- `approval.granted` — approval decision logged

Grandchild Span: gert.tool.invoke

Tool invocations create grandchild spans under `gert.step`.

Span Name: `gert.tool.invoke`

Required Attributes:

```
tool.name      = "kubectl"           // tool identifier
tool.transport = "exec"              // "exec" | "ssh" | "http"
tool.exit_code = 0                   // process exit code (exec only)
run.id         = "20260418T143022-a3f9c1" // run correlation
step.id        = "deploy"            // parent step
```

Optional Attributes:

```
tool.args      = ["apply", "-f", "deployment.yaml"] // redacted args
tool.duration_ms = 1234                             // invocation latency
```

Grandchild Span: gert.input.resolve

Input provider resolutions create grandchild spans.

Span Name: `gert.input.resolve`

Required Attributes:

```
input.name      = "db_password"      // input key from runbook
provider.type   = "vault"             // provider type
provider.address = "https://vault:8200" // endpoint (if applicable)
run.id          = "20260418T143022-a3f9c1"
```

Grandchild Span: gert.governance.check

Governance evaluations create grandchild spans.

Span Name: `gert.governance.check`

Required Attributes:

```
policy.name      = "allow-ssh"        // policy identifier
policy.outcome   = "allow"            // "allow" | "deny"
run.id           = "20260418T143022-a3f9c1"
step.id          = "backup-db"
```

17.2.2 Span Status and Error Handling

OpenTelemetry spans have three status codes:

- UNSET — default, no explicit status set
- OK — operation completed successfully
- ERROR — operation failed

Status Mapping:

- `gert.run` span:
 - OK if run outcome is `success`
 - ERROR if run outcome is `failure`, `cancelled`, `governance-blocked`
- `gert.step` span:
 - OK if step outcome is `success` or `skipped`
 - ERROR if step outcome is `failure`
- `gert.tool.invoke` span:
 - OK if `exit_code == 0`
 - ERROR if `exit_code != 0` or invocation crashes

When a span is marked ERROR, the runtime MUST record the error message as a span event or exception:

```
span.RecordError(err, trace.WithAttributes(  
    attribute.String("error.type", "tool_invocation_failed"),  
    attribute.String("error.message", err.Error()),  
))
```

17.2.3 Context Propagation

Gert v2 propagates trace context across system boundaries using W3C Trace Context [2].

W3C Trace Context Headers

When a tool invokes an HTTP API, the runtime **MUST** inject W3C headers:

```
traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01
tracestate: gert=run_id:20260418T143022-a3f9c1
```

The `traceparent` header contains:

- `version` (00)
- `trace-id` (128-bit, globally unique)
- `parent-id` (64-bit span ID)
- `trace-flags` (sampling decision)

The `tracestate` header carries vendor-specific metadata. Gert includes `run_id` as baggage for cross-system correlation.

Baggage

The `run_id` is propagated as OpenTelemetry baggage to all child spans and external systems:

```
| baggage.Set("run_id", runID)
```

This allows downstream systems to tag their own spans with the originating gert run ID, enabling end-to-end tracing across multi-system workflows.

17.2.4 OTLP Exporter Configuration

The runtime supports two trace export modes:

OTLP Export (Production)

Configure via environment variable or CLI flag:

```
export OTEL_EXPORTER_OTLP_ENDPOINT=http://jaeger:4317
gert run deploy.yaml
```

Or:

```
gert run deploy.yaml --otel-endpoint=http://jaeger:4317
```

The runtime uses the OTLP/gRPC exporter by default. For HTTP, use:

```
export OTEL_EXPORTER_OTLP_ENDPOINT=http://jaeger:4318
export OTEL_EXPORTER_OTLP_PROTOCOL=http/protobuf
```

Console Export (Development)

For local development without a trace backend:

```
gert run deploy.yaml --otel-console
```

This prints spans to `stderr` in a human-readable format.

17.2.5 Sampling Strategy

Gert v2 uses **parent-based sampling** with a configurable default rate:

- If an incoming request already has a trace context (e.g., gert invoked via HTTP from another traced system), **inherit the parent's sampling decision**.
- If no parent context exists, sample at the configured rate (default: 100% = always sample).

Configuration:

```
export OTEL_TRACES_SAMPLER=parentbased_traceidratio
export OTEL_TRACES_SAMPLER_ARG=0.1 # sample 10% of traces
```

For production systems with high throughput, consider 0.01 (1%) or 0.001 (0.1%) to reduce backend load.

17.3 Metrics Catalog

Gert v2 exposes the following metrics in Prometheus OpenMetrics format. All metrics are prefixed with `gert_`.

Metric	Type	Labels	Description
<code>gert_runs_total</code>	Counter	<code>outcome</code> , <code>runbook_id</code>	Total run completions by outcome (success , failure , cancelled)
<code>gert_run_duration_seconds</code>	Histogram	<code>outcome</code> , <code>runbook_id</code>	End-to-end run duration (start to completion)
<code>gert_active_runs</code>	Gauge	—	Currently executing runs

TABLE 17.1: Run-level metrics

17.3.1 Run Metrics

Histogram Buckets: `gert_run_duration_seconds` uses exponential buckets: 0.5, 1, 2, 5, 10, 30, 60, 120, 300, 600, 1800, 3600 (seconds).

17.3.2 Step Metrics

Metric	Type	Labels	Description
<code>gert_steps_total</code>	Counter	<code>outcome</code> , <code>step_type</code>	Total step completions by outcome and type
<code>gert_step_duration_seconds</code>	Histogram	<code>step_type</code>	Per-step duration by type (cli , manual , tool , invoke)

TABLE 17.2: Step-level metrics

Histogram Buckets: `gert_step_duration_seconds` uses buckets: 0.1, 0.5, 1, 2, 5, 10, 30, 60, 120, 300 (seconds).

17.3.3 Tool Invocation Metrics

Metric	Type	Labels	Description
<code>gert_tool_invocations_total</code>	Counter	<code>tool_name</code> , <code>outcome</code>	Tool invocation count by tool and outcome (success , failure)
<code>gert_tool_duration_seconds</code>	Histogram	<code>tool_name</code>	Tool invocation latency

TABLE 17.3: Tool invocation metrics

Histogram Buckets: `gert_tool_duration_seconds` uses buckets: 0.01, 0.05, 0.1, 0.5, 1, 2, 5, 10, 30 (seconds).

17.3.4 Approval Metrics

Metric	Type	Labels	Description
<code>gert_approval_wait_seconds</code>	Histogram	<code>step_id</code>	Time from approval request to decision (approved or rejected)
<code>gert_approvals_total</code>	Counter	<code>decision, step_id</code>	Total approval decisions (approved, rejected)

TABLE 17.4: Approval metrics

Histogram Buckets: `gert_approval_wait_seconds` uses buckets: 10, 30, 60, 300, 600, 1800, 3600, 7200, 14400, 28800, 86400 (seconds, up to 24 hours).

17.3.5 Extension Metrics

Metric	Type	Labels	Description
<code>gert_extension_crashes_total</code>	Counter	<code>extension_id</code>	Extension process crashes
<code>gert_extension_loaded</code>	Gauge	<code>extension_id</code>	1 if extension is loaded, 0 otherwise

TABLE 17.5: Extension metrics

17.3.6 Metrics Endpoint

When running in server mode, metrics are exposed at:

GET `http://localhost:8080/metrics`

Configure the server:

```
gert serve --http --addr=:8080
```

The response is in Prometheus text exposition format. Example:

```
# TYPE gert_runs_total counter
gert_runs_total{outcome="success",runbook_id="deploy-app-v3"} 42
gert_runs_total{outcome="failure",runbook_id="deploy-app-v3"} 3

# TYPE gert_run_duration_seconds histogram
gert_run_duration_seconds_bucket{outcome="success",runbook_id="deploy-app-v3",le="1"} 5
gert_run_duration_seconds_bucket{outcome="success",runbook_id="deploy-app-v3",le="5"} 18
gert_run_duration_seconds_bucket{outcome="success",runbook_id="deploy-app-v3",le="+Inf"} 42
gert_run_duration_seconds_sum{outcome="success",runbook_id="deploy-app-v3"} 187.4
gert_run_duration_seconds_count{outcome="success",runbook_id="deploy-app-v3"} 42

# TYPE gert_active_runs gauge
gert_active_runs 3
```

17.3.7 Push-Based Metrics (OTLP)

For environments where scraping is not feasible (e.g., ephemeral CI jobs), use OTLP metrics export:

```
export OTEL_EXPORTER_OTLP_ENDPOINT=http://collector:4317
export OTEL_METRICS_EXPORTER=otlp
gert run deploy.yaml
```

The runtime pushes metrics to the OTLP endpoint at configurable intervals (default: 60 seconds).

17.4 Structured Logging

Gert v2 emits structured JSON logs to `stderr`. Every log line is a complete JSON object.

17.4.1 Log Line Format

```
{
  "timestamp": "2026-04-18T14:30:22.123456Z",
  "level": "info",
  "msg": "step execution started",
  "run_id": "20260418T143022-a3f9c1",
```

```
"step_id": "backup-db",  
"step_type": "cli",  
"actor": "alice@example.com"  
}
```

17.4.2 Required Fields

- **timestamp** — RFC 3339 format [19] with microsecond precision, UTC timezone
- **level** — one of: `debug`, `info`, `warn`, `error`
- **msg** — human-readable message describing the event

17.4.3 Contextual Fields (When Applicable)

- **run_id** — run identifier (present in all logs within run context)
- **step_id** — step identifier (present in all logs within step context)
- **step_type** — step type (`cli`, `manual`, `tool`, `invoke`)
- **actor** — user or service principal executing the run
- **tool_name** — tool identifier (for tool invocation logs)
- **error** — error message (for `error` level logs)
- **exit_code** — process exit code (for CLI step completions)

17.4.4 Log Levels

- **debug** — Detailed diagnostic information (e.g., input resolution, policy evaluation details)
- **info** — Operational milestones (e.g., step started, run completed)
- **warn** — Recoverable issues (e.g., retry attempt, fallback to default)
- **error** — Failures requiring attention (e.g., step failure, governance denial)

Configure via environment variable or CLI flag:

```
export GERT_LOG_LEVEL=debug
gert run deploy.yaml
```

Or:

```
gert run deploy.yaml --log-level=debug
```

Default: info.

17.4.5 Sensitive Data Handling

Log lines MUST NOT contain sensitive input values. The runtime applies the same redaction rules as the trace file (see § 12.4).

Safe:

```
{"msg": "resolved input 'db_password' from provider 'vault'"}
```

Unsafe:

```
{"msg": "resolved input 'db_password' = 'hunter2'"}
```

When logging errors that may contain sensitive data, scrub or omit the error message:

```
{"msg": "input resolution failed", "input_name": "db_password",  
  "provider": "vault", "error": "connection timeout"}
```

17.4.6 Log Output Configuration

By default, logs are written to `stderr`. To write to a file:

```
gert run deploy.yaml --log-output=file:/var/log/gert/gert.log
```

This writes logs to the file *in addition to* `stderr`.

For syslog output:

```
gert run deploy.yaml --log-output=syslog://localhost:514
```

17.4.7 Log Aggregation and Correlation

Every log line in run context includes `run_id`, enabling log aggregation by run:

Loki Query:

```
{job="gert"} | json | run_id="20260418T143022-a3f9c1"
```

Elasticsearch Query:

```
GET /gert-logs-*/_search
{
  "query": {
    "term": {"run_id": "20260418T143022-a3f9c1"}
  }
}
```

This returns all log lines for the specified run, ordered by timestamp.

17.5 Health and Readiness Endpoints

When running in server mode (`gert serve -http`), the runtime exposes HTTP endpoints for health probes. These are designed for Kubernetes liveness and readiness probes.

17.5.1 Liveness Endpoint

`GET /health`

Purpose: Indicates whether the gert process is alive.

Response (200 OK):

```
{
  "status": "ok",
  "version": "v2.0.0",
  "uptime_seconds": 3600
}
```

Use Case: Kubernetes liveness probe to restart crashed pods.

17.5.2 Readiness Endpoint

GET /ready

Purpose: Indicates whether the runtime is ready to accept new runs.

Response (200 OK):

```
{
  "status": "ready",
  "extensions_loaded": 3,
  "extensions_failed": 0
}
```

Response (503 Service Unavailable):

```
{
  "status": "not_ready",
  "reason": "extension 'aws-tools' failed to load",
  "extensions_loaded": 2,
  "extensions_failed": 1
}
```

Use Case: Kubernetes readiness probe to delay traffic until extensions are loaded.

17.5.3 Kubernetes Probe Configuration

```
apiVersion: v1
kind: Pod
metadata:
  name: gert-server
spec:
  containers:
  - name: gert
    image: gert:v2.0.0
    command: ["gert", "serve", "--http", "--addr=:8080"]
    livenessProbe:
      httpGet:
        path: /health
        port: 8080
      initialDelaySeconds: 5
```

```
    periodSeconds: 10
  readinessProbe:
    httpGet:
      path: /ready
      port: 8080
    initialDelaySeconds: 10
    periodSeconds: 5
```

17.6 Diagnostics Commands

Gert v2 provides CLI commands for self-diagnosis and trace inspection.

17.6.1 gert diagnose

Purpose: Run pre-flight checks to verify that the runtime environment is correctly configured.

Usage:

```
gert diagnose [--runbook=<path>]
```

Checks Performed:

1. **Runbook Schema Validation:** Parse and validate the runbook YAML against JSON Schema
2. **Tool Availability:** Verify that all declared tools (`.tool.yaml`) are executable and findable in `PATH` or `.runbook/tools/`
3. **Input Provider Connectivity:** Test that all input providers (`.provider.yaml`) can be reached (e.g., Vault is reachable, API keys are valid)
4. **Extension Loading:** Attempt to load all declared extensions and report any load failures
5. **Governance Policy Syntax:** Validate OPA policies (if policy-as-code is configured)

Output (Success):

- ✓ Runbook schema validation passed
- ✓ Tool 'kubectl' found at /usr/local/bin/kubectl
- ✓ Tool 'aws' found at /usr/local/bin/aws
- ✓ Input provider 'vault' reachable at https://vault:8200
- ✓ Extension 'aws-tools' loaded successfully
- ✓ Extension 'k8s-tools' loaded successfully

Diagnostics: PASS

Output (Failure):

- ✓ Runbook schema validation passed
- × Tool 'kubectl' not found in PATH or .runbook/tools/
- ✓ Tool 'aws' found at /usr/local/bin/aws
- × Input provider 'vault' unreachable: connection refused
- ✓ Extension 'aws-tools' loaded successfully
- × Extension 'k8s-tools' failed to load: missing dependency

Diagnostics: FAIL (3 errors)

Exit Code: 0 if all checks pass, 1 if any check fails.

17.6.2 gert trace show

Purpose: Pretty-print a completed run's trace from the JSONL file.

Usage:

```
gert trace show <run_id>
```

Output:

```
Run: 20260418T143022-a3f9c1
Runbook: deploy-app-v3
Actor: alice@example.com
Started: 2026-04-18T14:30:22Z
Completed: 2026-04-18T14:33:15Z
Duration: 2m53s
Outcome: success
```

Steps:

```
[0] backup-db (cli) - 12.3s - success
    L- tool: ssh - 12.1s - exit_code=0
[1] deploy (tool) - 45.2s - success
    L- tool: kubectl - 44.8s - exit_code=0
[2] verify (manual) - 2m10s - success
    L- approval: alice@example.com - granted
```

17.6.3 gert trace export

Purpose: Convert a JSONL trace to OpenTelemetry OTLP format for import into Jaeger or Zipkin.

Usage:

```
gert trace export <run_id> --format=otlp --output=trace.json
```

Supported Formats:

- `otlp` — OTLP JSON format
- `jaeger` — Jaeger JSON format
- `zipkin` — Zipkin JSON format

Use Case: Retrospectively analyze a run in a trace visualization tool when the runtime was not configured to export traces at execution time.

17.6.4 gert run list

Purpose: List recent runs with status.

Usage:

```
gert run list [--limit=N] [--json]
```

Output (Human-Readable):

RUN_ID	RUNBOOK	ACTOR	OUTCOME	DURATION
20260418T143022-a3f9c1	deploy-app-v3	alice@example.com	success	2m53s
20260418T140015-f3a7b2	rollback-v2	bob@example.com	failure	45s
20260418T135512-c9e8d1	backup-db	cron@system	success	3m12s

Output (JSON, for CI integration):

```
[
  {
    "run_id": "20260418T143022-a3f9c1",
    "runbook_id": "deploy-app-v3",
    "actor": "alice@example.com",
    "outcome": "success",
    "duration_ms": 173000,
    "started_at": "2026-04-18T14:30:22Z",
    "completed_at": "2026-04-18T14:33:15Z"
  },
  ...
]
```

Use Case: CI pipelines can parse JSON output to determine run status.

17.7 Integration Recipes

This section provides practical integration patterns for common observability platforms.

17.7.1 Prometheus + Grafana

Step 1: Start gert server

```
gert serve --http --addr=:8080
```

Step 2: Configure Prometheus scrape target

Add to `prometheus.yml`:

```
scrape_configs:
- job_name: 'gert'
  static_configs:
    - targets: ['localhost:8080']
  metrics_path: '/metrics'
  scrape_interval: 15s
```

Step 3: Import Grafana dashboard

Gert provides a reference Grafana dashboard JSON at `.runbook/observability/grafana-dashboard.json`. Import it into Grafana to visualize:

- Run throughput (runs/minute)
- Success vs. failure rate
- p50, p95, p99 run duration
- Active runs (gauge)
- Step execution time by type
- Tool invocation latency

17.7.2 Jaeger (Distributed Tracing)

Step 1: Start Jaeger (all-in-one)

```
docker run -d --name jaeger \
  -p 16686:16686 \
  -p 4317:4317 \
  jaegertracing/all-in-one:latest
```

Step 2: Configure gert to export traces

```
export OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:4317
gert run deploy.yaml
```

Step 3: View traces in Jaeger UI

Open <http://localhost:16686>, select service `gert`, and search for traces.

17.7.3 Loki + Grafana (Log Aggregation)

Step 1: Configure Promtail to scrape gert logs

Add to `promtail-config.yaml`:

```
scrape_configs:
  - job_name: gert
    static_configs:
      - targets:
          - localhost
        labels:
          job: gert
```

```
    __path__: /var/log/gert/*.log
  pipeline_stages:
    - json:
        expressions:
          timestamp: timestamp
          level: level
          msg: msg
          run_id: run_id
    - labels:
        level:
        run_id:
```

Step 2: Start gert with file logging

```
gert run deploy.yaml --log-output=file:/var/log/gert/gert.log
```

Step 3: Query logs in Grafana Explore

```
{job="gert"} | json | run_id="20260418T143022-a3f9c1"
```

17.7.4 Datadog / New Relic (OTLP-Compatible SaaS)

Both platforms support OpenTelemetry OTLP ingestion.

Datadog:

```
export OTEL_EXPORTER_OTLP_ENDPOINT=https://api.datadoghq.com
export DD_API_KEY=<your-api-key>
gert run deploy.yaml
```

New Relic:

```
export OTEL_EXPORTER_OTLP_ENDPOINT=https://otlp.nr-data.net:4317
export NEW_RELIC_API_KEY=<your-api-key>
gert run deploy.yaml
```

Consult platform documentation for exact endpoint URLs and authentication headers.

17.8 Observability for CI/CD Pipelines

Gert v2 provides machine-readable output for CI/CD integration.

17.8.1 JSON Output Mode

```
gert run deploy.yaml --output=json
```

On completion, emits a JSON summary to `stdout`:

```
{
  "run_id": "20260418T143022-a3f9c1",
  "outcome": "success",
  "step_count": 3,
  "failed_step": null,
  "duration_ms": 173000,
  "started_at": "2026-04-18T14:30:22Z",
  "completed_at": "2026-04-18T14:33:15Z"
}
```

Exit Code:

- 0 if outcome == "success"
- 1 if outcome == "failure"
- 2 if outcome == "governance-blocked"
- 3 if outcome == "cancelled"

17.8.2 GitHub Actions Example

```
- name: Run gert runbook
  id: gert_run
  run: |
    OUTPUT=$(gert run deploy.yaml --output=json)
    echo "$OUTPUT" | jq .
    RUN_ID=$(echo "$OUTPUT" | jq -r .run_id)
    OUTCOME=$(echo "$OUTPUT" | jq -r .outcome)
    echo "run_id=$RUN_ID" >> $GITHUB_OUTPUT
    echo "outcome=$OUTCOME" >> $GITHUB_OUTPUT

- name: Upload trace artifact
  if: always()
```



```
uses: actions/upload-artifact@v3
with:
  name: gert-trace
  path: .runbook/runs/${{ steps.gert_run.outputs.run_id }}/trace.jsonl
```

17.8.3 GitLab CI Example

```
deploy:
  script:
    - gert run deploy.yaml --output=json | tee run-output.json
    - RUN_ID=$(jq -r .run_id run-output.json)
    - echo "Run ID: $RUN_ID"
  artifacts:
    when: always
    paths:
      - .runbook/runs/
  reports:
    junit: .runbook/runs/*/trace.jsonl
```

17.9 Summary

This chapter specifies a production-grade observability model for gert v2 grounded in industry standards:

- **Three-pillar model:** Metrics (Prometheus), distributed tracing (OpenTelemetry), structured logging (JSON)
- **Span hierarchy:** `gert.run` → `gert.step` → `gert.tool.invoke` / `gert.governance.check` / `gert.input.resolve`
- **Context propagation:** W3C Trace Context headers and OpenTelemetry baggage for cross-system correlation
- **Metrics catalog:** 9 metrics covering runs, steps, tools, approvals, and extensions
- **Health endpoints:** `/health` (liveness), `/ready` (readiness), `/metrics` (Prometheus)
- **Diagnostics commands:** `gert diagnose`, `gert trace show`, `gert trace export`, `gert run list`

- **Integration recipes:** Prometheus, Grafana, Jaeger, Loki, Datadog, New Relic
- **CI/CD support:** JSON output mode with exit codes for pipeline integration

Operators gain visibility into run health, performance bottlenecks, and failure root causes. The design ensures portability across observability platforms by adhering to open standards [2, 6, 13].

Bibliography

- [1] NIST special publication 800-53 — security and privacy controls for information systems and organizations. Technical report, National Institute of Standards and Technology, 2020. URL <https://csrc.nist.gov/publications/detail/sp/800-53/rev-5/final>. Catalog of security and privacy controls with principle of least privilege guidance.
- [2] W3C trace context specification. Technical report, World Wide Web Consortium (W3C), 2020. URL <https://www.w3.org/TR/trace-context/>. Standard for distributed trace context propagation via traceparent and tracestate headers.
- [3] OpenAPI specification v3.1.0. Technical report, OpenAPI Initiative, 2021. URL <https://spec.openapis.org/oas/v3.1.0>. Standard for HTTP API description and documentation.
- [4] ISO/IEC 27001:2022 — information security management systems. Technical report, International Organization for Standardization, 2022. International standard for information security management, including audit requirements (A.12.4).
- [5] Runbook engineering and SOP design in high availability environments. *International Journal of Scientific Research in Engineering and Technology (IJSRET)*, 10(6), 2023. Explores principles of runbook engineering for SRE teams with focus on auditability, compliance, and MTTR reduction.
- [6] OpenTelemetry specification. Technical report, Cloud Native Computing Foundation (CNCF), 2023. URL <https://opentelemetry.io/docs/specs/otel/>. Specification for distributed tracing, metrics, and logs with W3C Trace Context propagation.
- [7] SOC 2 — trust services criteria. Technical report, American Institute of Certified Public Accountants (AICPA), 2023. URL <https://www.aicpa.org/interestareas/frc/assuranceadvisoryservices/socforserviceorganizations.html>. Auditing standard for security, availability, processing integrity, confidentiality, and privacy.

-
- [8] Amazon Web Services. AWS systems manager automation runbooks, 2024. URL <https://docs.aws.amazon.com/systems-manager/latest/userguide/automation-documents.html>. YAML/JSON automation documents with approval gates, cross-account execution, and rate controls.
 - [9] Amazon Web Services. AWS step functions developer guide, 2024. URL <https://docs.aws.amazon.com/step-functions/>. State machine-based workflow orchestration with error handlers and human approval tasks.
 - [10] Anthropic. Model context protocol (MCP) specification, 2024. URL <https://modelcontextprotocol.io/>. Protocol for connecting AI models with external data sources and tools.
 - [11] Anton Medvedev. `expr-lang/expr`: Expression language for Go, 2024. URL <https://github.com/expr-lang/expr>. Fast, safe, and intuitive expression evaluation engine with Python-like syntax.
 - [12] Argo Project (CNCF). Argo workflows documentation, 2024. URL <https://argoproj.github.io/workflows/>. Kubernetes-native containerized workflow engine with YAML/JSON CRD-based definitions.
 - [13] Betsy Beyer, Chris Jones, Jennifer Petoff, and Niall Richard Murphy, editors. *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, 2016. Foundational SRE principles including elimination of toil and runbook automation.
 - [14] Charm. Bubble tea — a Go framework for building terminal UIs, 2024. URL <https://github.com/charmbracelet/bubbletea>. The Elm Architecture for building terminal user interfaces in Go.
 - [15] Cloud Native Computing Foundation. Jaeger documentation, 2024. URL <https://www.jaegertracing.io/docs/>. Distributed tracing platform compatible with OpenTelemetry.
 - [16] Cloud Native Computing Foundation. Kubernetes documentation, 2024. URL <https://kubernetes.io/docs/>. Container orchestration platform with admission controllers and RBAC.
 - [17] IBM Corporation. *Delivering Consistency and Automation with Operational Runbooks*. IBM Redbooks, 2022. Deep dive into IBM Runbook Automation service, blending analytics with automation for IT operations.

- [18] JSON-RPC Working Group. JSON-RPC 2.0 specification, 2010. URL <https://www.jsonrpc.org/specification>. Remote procedure call protocol encoded in JSON.
- [19] G. Klyne and C. Newman. RFC 3339 — date and time on the internet: Timestamps. RFC 3339, Internet Engineering Task Force (IETF), 2002. URL <https://www.rfc-editor.org/rfc/rfc3339>.
- [20] Microsoft. Visual Studio Code extension API, 2024. URL <https://code.visualstudio.com/api>. API for building VS Code extensions with versioned contracts.
- [21] Open Policy Agent (CNCF). Open policy agent (OPA) documentation, 2024. URL <https://www.openpolicyagent.org/docs/>. General-purpose policy engine with Rego language for authorization and compliance.
- [22] Open Policy Agent (CNCF). OPA gatekeeper documentation, 2024. URL <https://open-policy-agent.github.io/gatekeeper/>. Policy controller for Kubernetes using OPA to enforce cluster-wide policies.
- [23] PagerDuty. PagerDuty runbook automation, 2024. URL <https://www.pagerduty.com/platform/automation/>. SaaS solution integrating runbook automation with incident response (formerly Rundeck).
- [24] PagerDuty (Rundeck). Rundeck documentation, 2024. URL <https://docs.rundeck.com/>. Open source job scheduling and orchestration with RBAC and audit logging.
- [25] Prefect Technologies. Prefect documentation, 2024. URL <https://docs.prefect.io/>. Python-native workflow orchestration with dynamic DAGs and hybrid deployment flexibility.
- [26] Stretchr. Testify — a toolkit with common assertions and mocks for Go, 2024. URL <https://github.com/stretchr/testify>. Go testing library with assertions, mocking, and test suites.
- [27] Temporal Technologies. Temporal workflow engine documentation, 2024. URL <https://docs.temporal.io/>. Durable execution with saga pattern support, compensation activities, and deterministic replay.
- [28] U.S. Department of Health and Human Services. HIPAA security rule — 45 CFR part 164, subpart c, 2013. URL <https://www.hhs.gov/hipaa/for-professionals/>

- `security/index.html`. Security standards for protected health information (ePHI) including audit controls.
- [29] A. Wright, H. Andrews, B. Hutton, and G. Dennis. JSON schema: A media type for describing JSON documents. Internet draft, JSON Schema, 2020. URL <https://json-schema.org/specification.html>. Draft 2020-12.
- [30] Zipkin. Zipkin documentation, 2024. URL <https://zipkin.io/>. Distributed tracing system for gathering timing data and troubleshooting latency.